

```
Hms.sql
IF NOT EXISTS (SELECT name FROM sys.databases WHERE name = 'HospitalManagementSystem')

BEGIN

    PRINT '✅ Database HospitalManagementSystem created.';

END

ELSE

    PRINT '❗ Database already exists — skipping creation.';

GO
```

```
USE HospitalManagementSystem;

GO
```

```
-- =====
-- MASTER TABLES
-- =====
```

```
IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'Patients')

BEGIN

    CREATE TABLE Patients (

        PatientID          INT IDENTITY PRIMARY KEY,

        PatientName        VARCHAR(100) NOT NULL,

        Email              VARCHAR(100) UNIQUE NOT NULL,

        Gender              VARCHAR(10),

        DateOfBirth         DATE,

        PhoneNumber         VARCHAR(15),

        Address             VARCHAR(255),

        BloodGroup          VARCHAR(5),

        EmergencyContact    VARCHAR(15),

        EmergencyContactName VARCHAR(100),

        CreatedAt           DATETIME DEFAULT GETDATE(),

    )
```

```

        IsActive          BIT DEFAULT 1
    );

    PRINT '✅ Table Patients created.';

END ELSE PRINT '❗ Table Patients already exists.';

GO

```

```

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'Doctors')

```

```

BEGIN

```

```

    CREATE TABLE Doctors (

```

```

        DoctorID          INT IDENTITY PRIMARY KEY,
        DoctorName         VARCHAR(100) NOT NULL,
        Email              VARCHAR(100) UNIQUE NOT NULL,
        Specialty           VARCHAR(100),
        PhoneNumber         VARCHAR(15),
        LicenseNumber       VARCHAR(50),
        YearsOfExperience   INT,
        CreatedAt           DATETIME DEFAULT GETDATE(),
        IsActive            BIT DEFAULT 1
    );

```

```

    PRINT '✅ Table Doctors created.';

```

```

END ELSE PRINT '❗ Table Doctors already exists.';

```

```

GO

```

```

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'Pharmacists')

```

```

BEGIN

```

```

    CREATE TABLE Pharmacists (

```

```

        PharmacistID      INT IDENTITY PRIMARY KEY,
        PharmacistName     VARCHAR(100) NOT NULL,
        Email              VARCHAR(100) UNIQUE NOT NULL,
        PhoneNumber        VARCHAR(15),
        LicenseNumber       VARCHAR(50),

```

```
        CreatedAt    DATETIME DEFAULT GETDATE(),
        IsActive    BIT DEFAULT 1
    );

    PRINT '✅ Table Pharmacists created.';
END ELSE PRINT '❗ Table Pharmacists already exists.';
GO
```

```
IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'Radiologists')
BEGIN
    CREATE TABLE Radiologists (
        RadiologistID INT IDENTITY PRIMARY KEY,
        RadiologistName VARCHAR(100) NOT NULL,
        Email          VARCHAR(100) UNIQUE NOT NULL,
        PhoneNumber    VARCHAR(15),
        LicenseNumber  VARCHAR(50),
        Specialization VARCHAR(100),
        YearsOfExperience INT,
        CreatedAt      DATETIME DEFAULT GETDATE(),
        IsActive       BIT DEFAULT 1
    );

    PRINT '✅ Table Radiologists created.';
END ELSE PRINT '❗ Table Radiologists already exists.';
GO
```

```
-- Users table (with Radiologist patch logic)
IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'Users')
BEGIN
    CREATE TABLE Users (
        UserID      INT IDENTITY PRIMARY KEY,
        Username    VARCHAR(50) UNIQUE NOT NULL,
        PasswordHash VARCHAR(255) NOT NULL,
```

```

Email    VARCHAR(100) UNIQUE NOT NULL,
Role     VARCHAR(20) NOT NULL
        CHECK (Role IN ('Doctor','Patient','Admin','Pharmacist','Radiologist')),
PatientID INT NULL,
DoctorID  INT NULL,
PharmacistID INT NULL,
RadiologistID INT NULL,
IsActive  BIT DEFAULT 1,
CreatedAt DATETIME DEFAULT GETDATE(),
LastLogin DATETIME NULL,
FOREIGN KEY (PatientID) REFERENCES Patients(PatientID) ON DELETE CASCADE,
FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID) ON DELETE CASCADE,
FOREIGN KEY (PharmacistID) REFERENCES Pharmacists(PharmacistID) ON DELETE CASCADE,
FOREIGN KEY (RadiologistID) REFERENCES Radiologists(RadiologistID) ON DELETE CASCADE
);
CREATE INDEX IX_Users_Username ON Users(Username);
CREATE INDEX IX_Users_Email ON Users(Email);
PRINT '✅ Table Users created.';
END ELSE BEGIN
    IF NOT EXISTS (SELECT 1 FROM sys.columns WHERE object_id = OBJECT_ID('Users') AND name =
'RadiologistID')
    BEGIN
        ALTER TABLE Users ADD RadiologistID INT NULL;
        ALTER TABLE Users ADD CONSTRAINT FK_Users_RadiologistID
        FOREIGN KEY (RadiologistID) REFERENCES Radiologists(RadiologistID) ON DELETE CASCADE;
        PRINT '✅ Column RadiologistID added to Users.';
    END ELSE PRINT 'ℹ Column RadiologistID already exists.';
END
GO

```

```
-- =====
```

-- TRANSACTION TABLES

-- =====

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'Visits')

BEGIN

CREATE TABLE Visits (

VisitID INT IDENTITY PRIMARY KEY,

PatientID INT NOT NULL,

DoctorID INT NOT NULL,

VisitDate DATETIME DEFAULT GETDATE(),

ReasonForVisit VARCHAR(500),

VitalSigns VARCHAR(MAX), -- JSON:
{ "bp": "120/80", "temp": "98.6", "pulse": "72", "weight": "70kg", "height": "172cm", "spo2": "98" }

Notes TEXT,

Status VARCHAR(20) DEFAULT 'Scheduled'

CHECK (Status IN ('Scheduled', 'In Progress', 'Completed', 'Cancelled')),

FOREIGN KEY (PatientID) REFERENCES Patients(PatientID),

FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID)

);

CREATE INDEX IX_Visits_PatientID ON Visits(PatientID);

CREATE INDEX IX_Visits_DoctorID ON Visits(DoctorID);

CREATE INDEX IX_Visits_VisitDate ON Visits(VisitDate);

PRINT '✅ Table Visits created.';

END ELSE PRINT '❗ Table Visits already exists.';

GO

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'Diagnoses')

BEGIN

CREATE TABLE Diagnoses (

DiagnosisID INT IDENTITY PRIMARY KEY,

VisitID INT NOT NULL,

```

DiagnosisName VARCHAR(255) NOT NULL,
Description TEXT,
DiagnosisDate DATETIME DEFAULT GETDATE(),
IsChronic BIT DEFAULT 0,
Severity VARCHAR(20) CHECK (Severity IN ('Mild','Moderate','Severe')),
FOREIGN KEY (VisitID) REFERENCES Visits(VisitID) ON DELETE CASCADE
);

CREATE INDEX IX_Diagnoses_VisitID ON Diagnoses(VisitID);

PRINT '✅ Table Diagnoses created.';

END ELSE PRINT '❗ Table Diagnoses already exists.';

GO

-- =====
-- NEW: Chronic Disease Progress Tracking
-- Stores periodic progress entries for chronic conditions.
-- ProgressScore: 0–100 (0 = worst / new, 100 = fully resolved)
-- =====

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'ChronicDiseaseProgress')
BEGIN
    CREATE TABLE ChronicDiseaseProgress (
        ProgressID INT IDENTITY PRIMARY KEY,
        PatientID INT NOT NULL,
        DiagnosisName VARCHAR(255) NOT NULL, -- matches Diagnoses.DiagnosisName
        RecordDate DATETIME DEFAULT GETDATE(),
        ProgressScore FLOAT NOT NULL -- 0-100
            CHECK (ProgressScore BETWEEN 0 AND 100),
        BloodPressure VARCHAR(20) NULL,
        BloodSugar VARCHAR(20) NULL,
        Weight DECIMAL(5,2) NULL,
        Notes TEXT NULL,
        RecordedBy INT NOT NULL, -- DoctorID

```

```

FOREIGN KEY (PatientID) REFERENCES Patients(PatientID) ON DELETE CASCADE,
FOREIGN KEY (RecordedBy) REFERENCES Doctors(DoctorID)
);
CREATE INDEX IX_CDP_PatientID ON ChronicDiseaseProgress(PatientID);
CREATE INDEX IX_CDP_DiagName ON ChronicDiseaseProgress(DiagnosisName);
PRINT '✅ Table ChronicDiseaseProgress created.';
END ELSE PRINT '❗ Table ChronicDiseaseProgress already exists.';
GO

```

```

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'Prescriptions')
BEGIN
CREATE TABLE Prescriptions (
    PrescriptionID INT IDENTITY PRIMARY KEY,
    VisitID INT NOT NULL,
    MedicineName VARCHAR(100) NOT NULL,
    Dosage VARCHAR(50),
    Frequency VARCHAR(50),
    Duration VARCHAR(50),
    Instructions TEXT,
    IsDispensed BIT DEFAULT 0,
    DispensedBy INT NULL,
    DispensedDate DATETIME NULL,
    CreatedAt DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (VisitID) REFERENCES Visits(VisitID) ON DELETE CASCADE,
    FOREIGN KEY (DispensedBy) REFERENCES Pharmacists(PharmacistID)
);
CREATE INDEX IX_Prescriptions_VisitID ON Prescriptions(VisitID);
CREATE INDEX IX_Prescriptions_IsDispensed ON Prescriptions(IsDispensed);
PRINT '✅ Table Prescriptions created.';
END ELSE PRINT '❗ Table Prescriptions already exists.';
GO

```

```

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'LabTests')
BEGIN
    CREATE TABLE LabTests (
        TestID    INT IDENTITY PRIMARY KEY,
        VisitID   INT NOT NULL,
        TestName   VARCHAR(100) NOT NULL,
        TestType   VARCHAR(50),
        Status     VARCHAR(20) DEFAULT 'Pending'
                CHECK (Status IN ('Pending','In Progress','Completed','Cancelled')),
        Results    TEXT,
        ResultDate DATETIME NULL,
        CreatedAt  DATETIME DEFAULT GETDATE(),
        FOREIGN KEY (VisitID) REFERENCES Visits(VisitID) ON DELETE CASCADE
    );
    CREATE INDEX IX_LabTests_VisitID ON LabTests(VisitID);
    PRINT '✅ Table LabTests created.';
END ELSE PRINT '❗ Table LabTests already exists.';
GO

```

```

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'MedicalFiles')
BEGIN
    CREATE TABLE MedicalFiles (
        FileID    INT IDENTITY PRIMARY KEY,
        PatientID  INT NOT NULL,
        VisitID    INT NULL,
        UploadedBy INT NOT NULL,
        FileType   VARCHAR(50),
        FileName   VARCHAR(255) NOT NULL,
        FileExtension VARCHAR(10),
        FilePath   VARCHAR(500) NOT NULL,

```



```

    FileSize    BIGINT,
    Description TEXT,
    UploadedAt  DATETIME DEFAULT GETDATE(),
    FOREIGN KEY (PatientID) REFERENCES Patients(PatientID) ON DELETE CASCADE,
    FOREIGN KEY (VisitID)  REFERENCES Visits(VisitID),
    FOREIGN KEY (UploadedBy) REFERENCES Users(UserID)
);

CREATE INDEX IX_MedicalFiles_PatientID ON MedicalFiles(PatientID);
CREATE INDEX IX_MedicalFiles_VisitID  ON MedicalFiles(VisitID);

PRINT '✅ Table MedicalFiles created.';

END ELSE PRINT '❗ Table MedicalFiles already exists.';

GO

```

```

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'PatientSummaryNotes')
BEGIN
    CREATE TABLE PatientSummaryNotes (
        PatientID INT PRIMARY KEY,
        Notes     NVARCHAR(MAX) NULL,
        UpdatedBy NVARCHAR(100) NULL,
        UpdatedAt DATETIME DEFAULT GETDATE(),
        FOREIGN KEY (PatientID) REFERENCES Patients(PatientID) ON DELETE CASCADE
    );

    PRINT '✅ Table PatientSummaryNotes created.';

END ELSE PRINT '❗ Table PatientSummaryNotes already exists.';

GO

```

```

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'ImportHistory')
BEGIN
    CREATE TABLE ImportHistory (
        ImportID      INT IDENTITY PRIMARY KEY,
        ImportedBy    INT NOT NULL,

```

```

    FileName      VARCHAR(255),

    TotalRecords   INT,

    SuccessfulRecords INT,

    FailedRecords  INT,

    ErrorLog       TEXT,

    ImportDate     DATETIME DEFAULT GETDATE(),

    FOREIGN KEY (ImportedBy) REFERENCES Users(UserID)

);

PRINT '✅ Table ImportHistory created.';

END ELSE PRINT '❗ Table ImportHistory already exists.';

GO

-- =====

-- STORED PROCEDURES

-- =====

CREATE OR ALTER PROCEDURE sp_GetAllRecords
AS BEGIN

    SET NOCOUNT ON;

    SELECT

        v.VisitID, v.PatientID, p.PatientName, p.Email AS PatientEmail,

        p.PhoneNumber AS PatientPhone, p.BloodGroup,

        v.DoctorID, d.DoctorName, d.Specialty,

        v.VisitDate, v.ReasonForVisit, v.Status AS VisitStatus,

        dg.DiagnosisID, dg.DiagnosisName, dg.IsChronic, dg.Severity,

        pr.PrescriptionID, pr.MedicineName, pr.Dosage, pr.Frequency, pr.Duration, pr.IsDispensed

    FROM Visits v

    JOIN Patients p    ON v.PatientID = p.PatientID

    JOIN Doctors d     ON v.DoctorID  = d.DoctorID

    LEFT JOIN Diagnoses dg ON v.VisitID = dg.VisitID

    LEFT JOIN Prescriptions pr ON v.VisitID = pr.VisitID

```

```
WHERE p.IsActive = 1

ORDER BY v.VisitDate DESC;

END;

GO
```

```
CREATE OR ALTER PROCEDURE sp_GetPatientRecords

    @PatientID INT

AS BEGIN

    SET NOCOUNT ON;

    SELECT

        v.VisitID, v.PatientID, p.PatientName, p.Email AS PatientEmail,
        p.PhoneNumber AS PatientPhone, p.BloodGroup,
        v.DoctorID, d.DoctorName, d.Specialty,
        v.VisitDate, v.ReasonForVisit, v.Status AS VisitStatus,
        dg.DiagnosisID, dg.DiagnosisName, dg.IsChronic, dg.Severity,
        pr.PrescriptionID, pr.MedicineName, pr.Dosage, pr.Frequency, pr.Duration, pr.IsDispensed

    FROM Visits v

    JOIN Patients p    ON v.PatientID = p.PatientID

    JOIN Doctors d     ON v.DoctorID  = d.DoctorID

    LEFT JOIN Diagnoses dg  ON v.VisitID = dg.VisitID

    LEFT JOIN Prescriptions pr ON v.VisitID = pr.VisitID

    WHERE p.PatientID = @PatientID AND p.IsActive = 1

    ORDER BY v.VisitDate DESC;

END;

GO
```

-- — Full patient detail procedure (for doctor portal) ———

```
CREATE OR ALTER PROCEDURE sp_GetPatientFullDetails

    @PatientID INT

AS BEGIN

    SET NOCOUNT ON;
```

-- Demographics + summary notes

SELECT

p.PatientID, p.PatientName, p.Email, p.Gender, p.DateOfBirth,
p.PhoneNumber, p.Address, p.BloodGroup,
p.EmergencyContact, p.EmergencyContactName, p.CreatedAt,
ISNULL(n.Notes, '') AS SummaryNotes,
ISNULL(n.UpdatedBy, '') AS NotesUpdatedBy,
n.UpdatedAt AS NotesUpdatedAt

FROM Patients p

LEFT JOIN PatientSummaryNotes n ON n.PatientID = p.PatientID

WHERE p.PatientID = @PatientID AND p.IsActive = 1;

-- All visits with doctor info

SELECT

v.VisitID, v.VisitDate, v.ReasonForVisit, v.VitalSigns,
v.Notes, v.Status, d.DoctorName, d.Specialty

FROM Visits v

JOIN Doctors d ON v.DoctorID = d.DoctorID

WHERE v.PatientID = @PatientID

ORDER BY v.VisitDate DESC;

-- All diagnoses

SELECT

dg.DiagnosisID, dg.DiagnosisName, dg.Description,
dg.Severity, dg.IsChronic, dg.DiagnosisDate, v.VisitDate

FROM Diagnoses dg

JOIN Visits v ON dg.VisitID = v.VisitID

WHERE v.PatientID = @PatientID

ORDER BY v.VisitDate DESC;

-- All prescriptions

SELECT

pr.PrescriptionID, pr.MedicineName, pr.Dosage, pr.Frequency,
pr.Duration, pr.Instructions, pr.IsDispensed, pr.DispensedDate,
d.DoctorName, v.VisitDate

FROM Prescriptions pr

JOIN Visits v ON pr.VisitID = v.VisitID

JOIN Doctors d ON v.DoctorID = d.DoctorID

WHERE v.PatientID = @PatientID

ORDER BY v.VisitDate DESC;

-- Lab tests

SELECT

lt.TestID, lt.TestName, lt.TestType, lt.Status,
lt.Results, lt.ResultDate, v.VisitDate

FROM LabTests lt

JOIN Visits v ON lt.VisitID = v.VisitID

WHERE v.PatientID = @PatientID

ORDER BY v.VisitDate DESC;

-- Chronic disease progress history

SELECT

cdp.ProgressID, cdp.DiagnosisName, cdp.RecordDate,
cdp.ProgressScore, cdp.BloodPressure, cdp.BloodSugar,
cdp.Weight, cdp.Notes, d.DoctorName

FROM ChronicDiseaseProgress cdp

JOIN Doctors d ON cdp.RecordedBy = d.DoctorID

WHERE cdp.PatientID = @PatientID

ORDER BY cdp.RecordDate DESC;

-- Medical files

```

SELECT
    mf.FileID, mf.FileType, mf.FileName, mf.FileSize,
    mf.Description, mf.UploadedAt, u.Username AS UploadedByUsername
FROM MedicalFiles mf
LEFT JOIN Users u ON mf.UploadedBy = u.UserID
WHERE mf.PatientID = @PatientID
ORDER BY mf.UploadedAt DESC;
END;
GO

```

-- — Chronic progress for a specific patient + disease —————

```

CREATE OR ALTER PROCEDURE sp_GetChronicProgress
    @PatientID INT,
    @DiagnosisName VARCHAR(255) = NULL -- NULL = all chronic conditions
AS BEGIN
    SET NOCOUNT ON;
    SELECT
        cdp.ProgressID, cdp.DiagnosisName, cdp.RecordDate,
        cdp.ProgressScore, cdp.BloodPressure, cdp.BloodSugar,
        cdp.Weight, cdp.Notes, d.DoctorName
    FROM ChronicDiseaseProgress cdp
    JOIN Doctors d ON cdp.RecordedBy = d.DoctorID
    WHERE cdp.PatientID = @PatientID
        AND (@DiagnosisName IS NULL OR cdp.DiagnosisName = @DiagnosisName)
    ORDER BY cdp.RecordDate DESC;
END;
GO

```

-- — Latest progress score per chronic disease —————

```

CREATE OR ALTER PROCEDURE sp_GetLatestChronicScores
    @PatientID INT

```

```

AS BEGIN

SET NOCOUNT ON;

SELECT

    cdp.DiagnosisName,
    cdp.ProgressScore AS LatestScore,
    cdp.RecordDate AS LastUpdated,
    d.DoctorName
FROM ChronicDiseaseProgress cdp
JOIN Doctors d ON cdp.RecordedBy = d.DoctorID
WHERE cdp.PatientID = @PatientID
    AND cdp.RecordDate = (
        SELECT MAX(c2.RecordDate)
        FROM ChronicDiseaseProgress c2
        WHERE c2.PatientID = @PatientID AND c2.DiagnosisName = cdp.DiagnosisName
    );
END;
GO

```

```

CREATE OR ALTER PROCEDURE sp_GetPrescriptionsForPharmacy

```

```

    @OnlyPending BIT = 0

```

```

AS BEGIN

SET NOCOUNT ON;

SELECT

    pr.PrescriptionID, pr.VisitID,
    p.PatientID, p.PatientName, p.PhoneNumber,
    v.VisitDate, d.DoctorName,
    pr.MedicineName, pr.Dosage, pr.Frequency, pr.Duration,
    pr.Instructions, pr.IsDispensed, pr.DispensedDate,
    ph.PharmacistName AS DispensedByName
FROM Prescriptions pr
JOIN Visits v ON pr.VisitID = v.VisitID

```

```

JOIN Patients p    ON v.PatientID = p.PatientID
JOIN Doctors d    ON v.DoctorID  = d.DoctorID
LEFT JOIN Pharmacists ph ON pr.DispensedBy = ph.PharmacistID
WHERE (@OnlyPending = 0 OR pr.IsDispensed = 0)
ORDER BY pr.CreatedAt DESC;
END;
GO

```

```

CREATE OR ALTER PROCEDURE sp_GetPatientFiles
    @PatientID INT
AS BEGIN
    SET NOCOUNT ON;
    SELECT
        mf.FileID, mf.PatientID, mf.VisitID,
        mf.FileType, mf.FileName, mf.FileExtension, mf.FilePath,
        mf.FileSize, mf.Description, mf.UploadedAt,
        u.Username AS UploadedByUsername, v.VisitDate
    FROM MedicalFiles mf
    JOIN Users u      ON mf.UploadedBy = u.UserID
    LEFT JOIN Visits v ON mf.VisitID  = v.VisitID
    WHERE mf.PatientID = @PatientID
    ORDER BY mf.UploadedAt DESC;
END;
GO

```

```

CREATE OR ALTER PROCEDURE sp_GetPatientScans
    @PatientID INT
AS BEGIN
    SET NOCOUNT ON;
    SELECT
        mf.FileID, mf.PatientID, mf.FileType, mf.FileName,

```



```

mf.FileSize, mf.Description, mf.UploadedAt,
u.Username AS UploadedByUsername, u.Role AS UploaderRole,
p.PatientName, p.BloodGroup
FROM MedicalFiles mf
JOIN Users u ON mf.UploadedBy = u.UserID
JOIN Patients p ON mf.PatientID = p.PatientID
WHERE mf.PatientID = @PatientID
AND mf.FileType IN ('X-Ray', 'MRI', 'CT Scan', 'Ultrasound', 'PET Scan', 'Mammography', 'Fluoroscopy')
ORDER BY mf.UploadedAt DESC;
END;
GO

```

```

-- =====
-- VIEWS
-- =====

```

```

CREATE OR ALTER VIEW vw_DashboardStats AS
SELECT
    (SELECT COUNT(*) FROM Patients WHERE IsActive = 1) AS TotalPatients,
    (SELECT COUNT(*) FROM Doctors WHERE IsActive = 1) AS TotalDoctors,
    (SELECT COUNT(*) FROM Radiologists WHERE IsActive = 1) AS TotalRadiologists,
    (SELECT COUNT(*) FROM Visits WHERE CAST(VisitDate AS DATE) = CAST(GETDATE() AS DATE)) AS
TodayVisits,
    (SELECT COUNT(*) FROM Prescriptions WHERE IsDispensed = 0) AS PendingPrescriptions,
    (SELECT COUNT(*) FROM LabTests WHERE Status = 'Pending') AS PendingTests;
GO

```

```

CREATE OR ALTER VIEW vw_PharmacistPrescriptionView AS
SELECT
    pr.PrescriptionID, p.PatientID, p.PatientName, p.PhoneNumber,
    v.VisitDate, d.DoctorName, d.Specialty,

```

```

pr.MedicineName, pr.Dosage, pr.Frequency, pr.Duration,
pr.Instructions, pr.IsDispensed, pr.DispensedDate,
ph.PharmacistName AS DispensedBy
FROM Prescriptions pr
JOIN Visits v    ON pr.VisitID = v.VisitID
JOIN Patients p  ON v.PatientID = p.PatientID
JOIN Doctors d   ON v.DoctorID = d.DoctorID
LEFT JOIN Pharmacists ph ON pr.DispensedBy = ph.PharmacistID;
GO

```

-- — Chronic progress trend view —————

```

CREATE OR ALTER VIEW vw_ChronicProgressSummary AS
SELECT
    cdp.PatientID,
    p.PatientName,
    p.BloodGroup,
    cdp.DiagnosisName,
    COUNT(cdp.ProgressID)          AS TotalEntries,
    MIN(cdp.ProgressScore)        AS LowestScore,
    MAX(cdp.ProgressScore)        AS HighestScore,
    AVG(cdp.ProgressScore)        AS AvgScore,
    MAX(cdp.RecordDate)           AS LastRecorded
FROM ChronicDiseaseProgress cdp
JOIN Patients p ON cdp.PatientID = p.PatientID
GROUP BY cdp.PatientID, p.PatientName, p.BloodGroup, cdp.DiagnosisName;
GO

```

```

CREATE OR ALTER VIEW vw_RadiologyScanOverview AS
SELECT
    mf.FileID, mf.FileType AS ScanType, mf.FileName, mf.FileSize,
    mf.Description AS Findings, mf.UploadedAt,

```

```

p.PatientID, p.PatientName, p.BloodGroup,
u.Username AS UploadedByUsername, u.Role AS UploaderRole,
r.RadiologistName, r.Specialization
FROM MedicalFiles mf
JOIN Patients p ON mf.PatientID = p.PatientID
JOIN Users u ON mf.UploadedBy = u.UserID
LEFT JOIN Radiologists r ON u.RadiologistID = r.RadiologistID
WHERE mf.FileType IN ('X-Ray','MRI','CT Scan','Ultrasound','PET Scan','Mammography','Fluoroscopy');
GO

```

```

-- =====
-- SAMPLE DATA
-- =====

```

```

IF NOT EXISTS (SELECT 1 FROM Doctors)
BEGIN
    INSERT INTO Doctors (DoctorName, Email, Specialty, PhoneNumber, LicenseNumber,
YearsOfExperience)
VALUES
    ('Dr. Anil Kumar', 'anil.kumar@hospital.com', 'Cardiology', '9876543210', 'DOC-001', 15),
    ('Dr. Priya Sharma', 'priya.sharma@hospital.com', 'Orthopedics', '9876543221', 'DOC-002', 12),
    ('Dr. Neha Verma', 'neha.verma@hospital.com', 'General Medicine', '9876543243', 'DOC-003',
10),
    ('Dr. Rajesh Singh', 'rajesh.singh@hospital.com', 'Pediatrics', '9876543244', 'DOC-004', 8),
    ('Dr. Kavita Reddy', 'kavita.reddy@hospital.com', 'Dermatology', '9876543245', 'DOC-005', 11);
    PRINT '✅ Sample doctors inserted.';
END ELSE PRINT 'ℹ Doctors already has data.';
GO

```

```

IF NOT EXISTS (SELECT 1 FROM Pharmacists)
BEGIN
    INSERT INTO Pharmacists (PharmacistName, Email, PhoneNumber, LicenseNumber) VALUES

```

```
('Amit Patel', 'amit.patel@hospital.com', '9123456780', 'PHARM-001'),  
('Sunita Mehta', 'sunita.mehta@hospital.com', '9123456781', 'PHARM-002');
```

```
PRINT '✅ Sample pharmacists inserted.';
```

```
END ELSE PRINT '❗ Pharmacists already has data.';
```

```
GO
```

```
IF NOT EXISTS (SELECT 1 FROM Radiologists)
```

```
BEGIN
```

```
INSERT INTO Radiologists (RadiologistName, Email, PhoneNumber, LicenseNumber, Specialization,  
YearsOfExperience)
```

```
VALUES
```

```
('Dr. Priya Nair', 'priya.nair@hospital.com', '9988776601', 'RAD-001', 'Diagnostic Radiology',  
9),
```

```
('Dr. Suresh Iyer', 'suresh.iyer@hospital.com', '9988776602', 'RAD-002', 'Interventional Radiology',  
14),
```

```
('Dr. Anita Bose', 'anita.bose@hospital.com', '9988776603', 'RAD-003', 'Neuroradiology',  
11),
```

```
('Dr. Mohan Das', 'mohan.das@hospital.com', '9988776604', 'RAD-004', 'Musculoskeletal  
Radiology', 7);
```

```
PRINT '✅ Sample radiologists inserted.';
```

```
END ELSE PRINT '❗ Radiologists already has data.';
```

```
GO
```

```
IF NOT EXISTS (SELECT 1 FROM Patients)
```

```
BEGIN
```

```
INSERT INTO Patients (PatientName, Email, Gender, DateOfBirth, PhoneNumber, Address,  
BloodGroup, EmergencyContact, EmergencyContactName)
```

```
VALUES
```

```
('Rahul Mehta', 'rahul.mehta@email.com', 'Male', '1998-05-14', '9123456789', 'Bangalore,  
Karnataka', 'O+', '9123456700', 'Priya Mehta'),
```

```
('Sneha Patel', 'sneha.patel@email.com', 'Female', '2001-09-22', '9234567890', 'Ahmedabad,  
Gujarat', 'A+', '9234567800', 'Rajesh Patel'),
```

```
('Arjun Singh', 'arjun.singh@email.com', 'Male', '1995-12-02', '9345678901', 'Delhi',  
'B+', '9345678900', 'Meera Singh'),
```

('Kiran Reddy', 'kiran.reddy@email.com', 'Male', '1997-02-10', '9789012345', 'Hyderabad, Telangana', 'A+', '9789012300', 'Lakshmi Reddy'),

('Meera Iyer', 'meera.iyer@email.com', 'Female', '1993-11-25', '9890123456', 'Chennai, Tamil Nadu', 'O-', '9890123400', 'Ravi Iyer'),

('Vikram Joshi', 'vikram.joshi@email.com', 'Male', '1988-06-18', '9901234567', 'Pune, Maharashtra', 'B-', '9901234500', 'Anjali Joshi');

PRINT '✅ Sample patients inserted.';

END ELSE PRINT '❗ Patients already has data.';

GO

IF NOT EXISTS (SELECT 1 FROM Users)

BEGIN

INSERT INTO Users (Username, PasswordHash, Email, Role, PatientID, DoctorID, PharmacistID, RadiologistID) VALUES

('rahul_p', 'password123', 'rahul.mehta@email.com', 'Patient', 1, NULL, NULL, NULL),
('sneha_p', 'password123', 'sneha.patel@email.com', 'Patient', 2, NULL, NULL, NULL),
('arjun_p', 'password123', 'arjun.singh@email.com', 'Patient', 3, NULL, NULL, NULL),
('kiran_p', 'password123', 'kiran.reddy@email.com', 'Patient', 4, NULL, NULL, NULL),
('meera_p', 'password123', 'meera.iyer@email.com', 'Patient', 5, NULL, NULL, NULL),
('vikram_p', 'password123', 'vikram.joshi@email.com', 'Patient', 6, NULL, NULL, NULL),
('dr_anil', 'doctor123', 'anil.kumar@hospital.com', 'Doctor', NULL, 1, NULL, NULL),
('dr_priya', 'doctor123', 'priya.sharma@hospital.com', 'Doctor', NULL, 2, NULL, NULL),
('dr_neha', 'doctor123', 'neha.verma@hospital.com', 'Doctor', NULL, 3, NULL, NULL),
('dr_rajesh', 'doctor123', 'rajesh.singh@hospital.com', 'Doctor', NULL, 4, NULL, NULL),
('dr_kavita', 'doctor123', 'kavita.reddy@hospital.com', 'Doctor', NULL, 5, NULL, NULL),
('pharm_amit', 'pharmacy123', 'amit.patel@hospital.com', 'Pharmacist', NULL, NULL, 1, NULL),
('pharm_sunita', 'pharmacy123', 'sunita.mehta@hospital.com', 'Pharmacist', NULL, NULL, 2, NULL),
('rad_priya', 'scan1234', 'priya.nair@hospital.com', 'Radiologist', NULL, NULL, NULL, 1),
('rad_suresh', 'scan1234', 'suresh.iyer@hospital.com', 'Radiologist', NULL, NULL, NULL, 2),
('rad_anita', 'scan1234', 'anita.bose@hospital.com', 'Radiologist', NULL, NULL, NULL, 3),
('rad_mohan', 'scan1234', 'mohan.das@hospital.com', 'Radiologist', NULL, NULL, NULL, 4),
('admin', 'admin123', 'admin@hospital.com', 'Admin', NULL, NULL, NULL, NULL);

```

PRINT '✅ All sample users inserted.';

END ELSE PRINT '❗ Users already has data.';

GO

IF NOT EXISTS (SELECT 1 FROM Visits)

BEGIN

    INSERT INTO Visits (PatientID, DoctorID, VisitDate, ReasonForVisit, VitalSigns, Status) VALUES

        (1, 1, DATEADD(day,-5,GETDATE()), 'Chest pain and discomfort',
        '{"bp":"140/90","temp":"98.6","pulse":"85","weight":"72kg","spo2":"97"}', 'Completed'),

        (1, 1, DATEADD(day,-65,GETDATE()), 'Follow-up hypertension check',
        '{"bp":"138/88","temp":"98.4","pulse":"82","weight":"71kg","spo2":"98"}', 'Completed'),

        (1, 1, DATEADD(day,-130,GETDATE()), 'BP monitoring',
        '{"bp":"142/92","temp":"98.5","pulse":"88","weight":"73kg","spo2":"96"}', 'Completed'),

        (2, 2, DATEADD(day,-10,GETDATE()), 'Knee pain after exercise',
        '{"bp":"120/80","temp":"98.4","pulse":"72","weight":"58kg","spo2":"99"}', 'Completed'),

        (4, 1, DATEADD(day,-8,GETDATE()), 'Frequent headaches',
        '{"bp":"130/85","temp":"98.7","pulse":"78","weight":"68kg","spo2":"98"}', 'Completed'),

        (4, 1, DATEADD(day,-75,GETDATE()), 'Migraine episode',
        '{"bp":"128/84","temp":"98.6","pulse":"76","weight":"68kg","spo2":"98"}', 'Completed'),

        (4, 1, DATEADD(day,-145,GETDATE()), 'Initial migraine consultation',
        '{"bp":"132/86","temp":"98.5","pulse":"80","weight":"67kg","spo2":"99"}', 'Completed'),

        (5, 3, DATEADD(day,-12,GETDATE()), 'Thyroid checkup',
        '{"bp":"118/78","temp":"98.3","pulse":"70","weight":"55kg","spo2":"99"}', 'Completed'),

        (5, 3, DATEADD(day,-100,GETDATE()), 'Follow-up thyroid panel',
        '{"bp":"116/76","temp":"98.4","pulse":"68","weight":"54kg","spo2":"99"}', 'Completed'),

        (6, 2, DATEADD(day,-3,GETDATE()), 'Back pain after lifting weights',
        '{"bp":"122/80","temp":"98.6","pulse":"74","weight":"82kg","spo2":"98"}', 'Completed');

    PRINT '✅ Sample visits inserted.';

END ELSE PRINT '❗ Visits already has data.';

GO

IF NOT EXISTS (SELECT 1 FROM Diagnoses)

BEGIN

    INSERT INTO Diagnoses (VisitID, DiagnosisName, Description, IsChronic, Severity) VALUES

```

```

(1, 'Hypertension', 'Elevated BP requiring monitoring and medication', 1, 'Moderate'),
(4, 'Ligament Strain', 'Minor knee ligament strain; rest and physio recommended', 0, 'Mild'),
(5, 'Migraine', 'Stress-related migraine headaches', 1, 'Moderate'),
(8, 'Hypothyroidism', 'Low thyroid hormone levels requiring medication', 1, 'Moderate'),
(10, 'Muscle Strain', 'Lower back muscle strain from improper lifting', 0, 'Mild');

PRINT '✅ Sample diagnoses inserted.';

END ELSE PRINT 'ℹ Diagnoses already has data.';

GO

```

```

IF NOT EXISTS (SELECT 1 FROM Prescriptions)

BEGIN

    INSERT INTO Prescriptions (VisitID, MedicineName, Dosage, Frequency, Duration, Instructions)
    VALUES

    (1, 'Atenolol', '50mg', 'Once daily', '30 days', 'Take with food in the morning'),
    (1, 'Aspirin', '75mg', 'Once daily', '30 days', 'Take after breakfast'),
    (4, 'Ibuprofen', '400mg', 'Twice daily', '7 days', 'Take after meals'),
    (5, 'Sumatriptan', '25mg', 'As needed', '10 tablets', 'Take at onset of migraine'),
    (8, 'Levothyroxine', '50mcg', 'Once daily', '90 days', 'Take on empty stomach in morning'),
    (10, 'Diclofenac', '50mg', 'Twice daily', '7 days', 'Take after meals');

    PRINT '✅ Sample prescriptions inserted.';

END ELSE PRINT 'ℹ Prescriptions already has data.';

GO

```

-- Seed ChronicDiseaseProgress data

```

IF NOT EXISTS (SELECT 1 FROM ChronicDiseaseProgress)

BEGIN

    -- Rahul Mehta – Hypertension (PatientID=1, DoctorID=1)

    INSERT INTO ChronicDiseaseProgress (PatientID, DiagnosisName, RecordDate, ProgressScore,
    BloodPressure, Weight, Notes, RecordedBy) VALUES

    (1, 'Hypertension', DATEADD(day, -130, GETDATE()), 20, '142/92', 73.0, 'Just diagnosed. Lifestyle
    changes started.', 1),

```

```
(1,'Hypertension', DATEADD(day,-65, GETDATE()), 45, '138/88', 71.0, 'Responding to Atenolol.
Dietary changes in progress.', 1),
```

```
(1,'Hypertension', DATEADD(day,-5, GETDATE()), 62, '140/90', 72.0, 'Good improvement. Maintain
current medication.', 1);
```

```
-- Kiran Reddy – Migraine (PatientID=4, DoctorID=1)
```

```
INSERT INTO ChronicDiseaseProgress (PatientID, DiagnosisName, RecordDate, ProgressScore,
BloodPressure, Notes, RecordedBy) VALUES
```

```
(4,'Migraine', DATEADD(day,-145,GETDATE()), 15, '132/86', 'Frequent episodes. Triggered by
stress.', 1),
```

```
(4,'Migraine', DATEADD(day,-75, GETDATE()), 35, '128/84', 'Sumatriptan effective. Fewer episodes.',
1),
```

```
(4,'Migraine', DATEADD(day,-8, GETDATE()), 45, '130/85', 'Moderate improvement. Sleep hygiene
needs work.', 1);
```

```
-- Meera Iyer – Hypothyroidism (PatientID=5, DoctorID=3)
```

```
INSERT INTO ChronicDiseaseProgress (PatientID, DiagnosisName, RecordDate, ProgressScore,
BloodPressure, Weight, Notes, RecordedBy) VALUES
```

```
(5,'Hypothyroidism', DATEADD(day,-100,GETDATE()), 50, '116/76', 54.0, 'TSH stabilising. Continue
Levothyroxine.', 3),
```

```
(5,'Hypothyroidism', DATEADD(day,-12, GETDATE()), 78, '118/78', 55.0, 'TSH within normal range.
Excellent progress.', 3);
```

```
PRINT '✅ Sample ChronicDiseaseProgress data inserted.;
```

```
END ELSE PRINT '❗ ChronicDiseaseProgress already has data.;
```

```
GO
```

```
IF NOT EXISTS (SELECT 1 FROM PatientSummaryNotes)
```

```
BEGIN
```

```
INSERT INTO PatientSummaryNotes (PatientID, Notes, UpdatedBy) VALUES
```

```
(1, 'Patient has a history of hypertension managed with Atenolol. Regular BP monitoring advised.
No known drug allergies. Last echocardiogram within normal limits.', 'admin'),
```

```
(4, 'Chronic migraine patient — stress trigger identified. Advised lifestyle modification and sleep
hygiene. Sumatriptan prescribed for acute episodes.', 'admin'),
```


(5, 'Hypothyroidism confirmed via elevated TSH levels. Levothyroxine titration ongoing. Follow-up thyroid panel every 3 months.', 'admin');

PRINT '✅ Sample PatientSummaryNotes inserted.';

END ELSE PRINT '❗ PatientSummaryNotes already has data.';

GO

-- =====

-- VERIFICATION

-- =====

```
SELECT 'Patients'          AS TableName, COUNT(*) AS Rows FROM Patients
UNION ALL SELECT 'Doctors',      COUNT(*) FROM Doctors
UNION ALL SELECT 'Pharmacists',  COUNT(*) FROM Pharmacists
UNION ALL SELECT 'Radiologists', COUNT(*) FROM Radiologists
UNION ALL SELECT 'Users',        COUNT(*) FROM Users
UNION ALL SELECT 'Visits',       COUNT(*) FROM Visits
UNION ALL SELECT 'Diagnoses',    COUNT(*) FROM Diagnoses
UNION ALL SELECT 'Prescriptions', COUNT(*) FROM Prescriptions
UNION ALL SELECT 'ChronicDiseaseProgress', COUNT(*) FROM ChronicDiseaseProgress
UNION ALL SELECT 'PatientSummaryNotes', COUNT(*) FROM PatientSummaryNotes;
```

SELECT * FROM vw_DashboardStats;

SELECT * FROM vw_ChronicProgressSummary;

-- — Receptionists master table —————

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'Receptionists')

BEGIN

```
CREATE TABLE Receptionists (
    ReceptionistID INT IDENTITY PRIMARY KEY,
    ReceptionistName VARCHAR(100) NOT NULL,
    Email            VARCHAR(100) UNIQUE NOT NULL,
```

```

    PhoneNumber    VARCHAR(15),
    LicenseNumber   VARCHAR(50),
    CreatedAt       DATETIME DEFAULT GETDATE(),
    IsActive        BIT DEFAULT 1
);

PRINT '✅ Table Receptionists created.';

END ELSE PRINT 'ℹ Table Receptionists already exists.';

GO

-- — Patch Users: add ReceptionistID column + update Role CHECK —

IF NOT EXISTS (SELECT 1 FROM sys.columns WHERE object_id = OBJECT_ID('Users') AND name =
'ReceptionistID')

BEGIN

    ALTER TABLE Users ADD ReceptionistID INT NULL;

    ALTER TABLE Users ADD CONSTRAINT FK_Users_ReceptionistID

        FOREIGN KEY (ReceptionistID) REFERENCES Receptionists(ReceptionistID) ON DELETE CASCADE;

    PRINT '✅ Column ReceptionistID added to Users.';

END ELSE PRINT 'ℹ Column ReceptionistID already exists.';

GO

-- Widen Role CHECK to include Receptionist

IF EXISTS (

    SELECT 1 FROM sys.check_constraints

    WHERE parent_object_id = OBJECT_ID('Users')

        AND name LIKE '%Role%'

        AND definition NOT LIKE '%Receptionist%'

)

BEGIN

    DECLARE @ck NVARCHAR(200);

    SELECT @ck = name FROM sys.check_constraints

        WHERE parent_object_id = OBJECT_ID('Users') AND name LIKE '%Role%';

```

```

EXEC('ALTER TABLE Users DROP CONSTRAINT [' + @ck + ']');

ALTER TABLE Users ADD CONSTRAINT CK_Users_Role

    CHECK (Role IN ('Doctor','Patient','Admin','Pharmacist','Radiologist','Receptionist'));

PRINT '✅ Role CHECK updated to include Receptionist.';

END ELSE PRINT '❗ Role CHECK already includes Receptionist.';

GO

```

-- — Appointments —————

```

IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'Appointments')
BEGIN

    CREATE TABLE Appointments (

        AppointmentID INT IDENTITY PRIMARY KEY,

        PatientID INT NOT NULL,

        DoctorID INT NULL,    -- NULL until receptionist assigns one

        RequestedDate DATE NOT NULL,

        TimeSlot VARCHAR(20), -- e.g. '10:00 AM'

        Status VARCHAR(20) DEFAULT 'Pending'

            CHECK (Status IN ('Pending','Confirmed','Cancelled','Completed')),

        Reason VARCHAR(500),

        PatientNotes TEXT,    -- patient's description of symptoms

        ReceptionistNotes TEXT NULL, -- receptionist internal notes

        HandledBy INT NULL,    -- ReceptionistID who assigned the doctor

        CreatedAt DATETIME DEFAULT GETDATE(),

        UpdatedAt DATETIME DEFAULT GETDATE(),

        FOREIGN KEY (PatientID) REFERENCES Patients(PatientID) ON DELETE CASCADE,

        FOREIGN KEY (DoctorID) REFERENCES Doctors(DoctorID),

        FOREIGN KEY (HandledBy) REFERENCES Receptionists(ReceptionistID)

    );

    CREATE INDEX IX_Appt_PatientID ON Appointments(PatientID);

    CREATE INDEX IX_Appt_DoctorID ON Appointments(DoctorID);

    CREATE INDEX IX_Appt_Status ON Appointments(Status);

```

```
CREATE INDEX IX_Appt_RequestedDate ON Appointments(RequestedDate);

PRINT '✅ Table Appointments created.';

END ELSE PRINT '❗ Table Appointments already exists.';

GO
```

```
-- — Stored procedures —————
```

```
-- All appointments (with patient + doctor info) — for receptionist
```

```
CREATE OR ALTER PROCEDURE sp_GetAllAppointments
    @Status VARCHAR(20) = NULL -- NULL = all statuses
AS BEGIN
    SET NOCOUNT ON;

    SELECT
        a.AppointmentID, a.RequestedDate, a.TimeSlot, a.Status,
        a.Reason, a.PatientNotes, a.ReceptionistNotes, a.CreatedAt,
        p.PatientID, p.PatientName, p.PhoneNumber, p.BloodGroup, p.Gender,
        d.DoctorID, d.DoctorName, d.Specialty,
        r.ReceptionistName AS HandledByName
    FROM Appointments a
    JOIN Patients p ON a.PatientID = p.PatientID
    LEFT JOIN Doctors d ON a.DoctorID = d.DoctorID
    LEFT JOIN Receptionists r ON a.HandledBy = r.ReceptionistID
    WHERE (@Status IS NULL OR a.Status = @Status)
    AND p.IsActive = 1
    ORDER BY a.RequestedDate ASC, a.CreatedAt DESC;

END;

GO
```

```
-- Patient's own appointments
```

```
CREATE OR ALTER PROCEDURE sp_GetPatientAppointments
    @PatientID INT
```

```

AS BEGIN

SET NOCOUNT ON;

SELECT

    a.AppointmentID, a.RequestedDate, a.TimeSlot, a.Status,
    a.Reason, a.PatientNotes, a.ReceptionistNotes, a.CreatedAt,
    d.DoctorID, d.DoctorName, d.Specialty,
    r.ReceptionistName AS HandledByName
FROM Appointments a
LEFT JOIN Doctors    d ON a.DoctorID = d.DoctorID
LEFT JOIN Receptionists r ON a.HandledBy = r.ReceptionistID
WHERE a.PatientID = @PatientID

ORDER BY a.RequestedDate DESC;

END;

GO

```

```

-- Doctor's confirmed upcoming appointments

CREATE OR ALTER PROCEDURE sp_GetDoctorAppointments
    @DoctorID INT
AS BEGIN

SET NOCOUNT ON;

SELECT

    a.AppointmentID, a.RequestedDate, a.TimeSlot, a.Status,
    a.Reason, a.PatientNotes,
    p.PatientID, p.PatientName, p.PhoneNumber, p.BloodGroup
FROM Appointments a
JOIN Patients p ON a.PatientID = p.PatientID
WHERE a.DoctorID = @DoctorID
    AND a.Status IN ('Confirmed', 'Pending')
    AND a.RequestedDate >= CAST(GETDATE() AS DATE)
    AND p.IsActive = 1

ORDER BY a.RequestedDate ASC, a.TimeSlot ASC;

```

END;

GO

-- — View: dashboard stats (refresh to include appointments) —

CREATE OR ALTER VIEW vw_DashboardStats AS

SELECT

(SELECT COUNT(*) FROM Patients WHERE IsActive = 1) AS TotalPatients,

(SELECT COUNT(*) FROM Doctors WHERE IsActive = 1) AS TotalDoctors,

(SELECT COUNT(*) FROM Radiologists WHERE IsActive = 1) AS TotalRadiologists,

(SELECT COUNT(*) FROM Visits WHERE CAST(VisitDate AS DATE) = CAST(GETDATE() AS DATE))
AS TodayVisits,

(SELECT COUNT(*) FROM Prescriptions WHERE IsDispensed = 0) AS PendingPrescriptions,

(SELECT COUNT(*) FROM LabTests WHERE Status = 'Pending') AS PendingTests,

(SELECT COUNT(*) FROM Appointments WHERE Status = 'Pending') AS PendingAppointments;

GO

-- — Sample Receptionists —————

IF NOT EXISTS (SELECT 1 FROM Receptionists)

BEGIN

INSERT INTO Receptionists (ReceptionistName, Email, PhoneNumber, LicenseNumber) VALUES

('Priya Nambiar', 'priya.nambiar@hospital.com', '9911223344', 'REC-001'),

('Suresh Babu', 'suresh.babu@hospital.com', '9911223345', 'REC-002');

PRINT '✅ Sample receptionists inserted.';

END ELSE PRINT 'ℹ Receptionists already has data.';

GO

-- — Sample Receptionist User accounts —————

IF NOT EXISTS (SELECT 1 FROM Users WHERE Role = 'Receptionist')

BEGIN

DECLARE @r1 INT, @r2 INT;

SELECT @r1 = ReceptionistID FROM Receptionists WHERE LicenseNumber = 'REC-001';

```
SELECT @r2 = ReceptionistID FROM Receptionists WHERE LicenseNumber = 'REC-002';
```

```
INSERT INTO Users (Username, PasswordHash, Email, Role, ReceptionistID)
```

```
VALUES
```

```
('rec_priya', 'recept123', 'priya.nambiar@hospital.com', 'Receptionist', @r1),
```

```
('rec_suresh', 'recept123', 'suresh.babu@hospital.com', 'Receptionist', @r2);
```

```
PRINT '✅ Receptionist user accounts inserted.';
```

```
END ELSE PRINT '❗ Receptionist users already exist.';
```

```
GO
```

```
-- — Sample Appointments —————
```

```
IF NOT EXISTS (SELECT 1 FROM Appointments)
```

```
BEGIN
```

```
INSERT INTO Appointments (PatientID, DoctorID, RequestedDate, TimeSlot, Status, Reason, PatientNotes) VALUES
```

```
(1, 1, CAST(DATEADD(day, 1, GETDATE()) AS DATE), '10:00 AM', 'Confirmed', 'BP follow-up', 'BP has been fluctuating. Need a check-up.'),
```

```
(2, NULL, CAST(DATEADD(day, 2, GETDATE()) AS DATE), '11:30 AM', 'Pending', 'Knee pain', 'Persistent knee pain after gym. Need physiotherapy referral.'),
```

```
(5, 3, CAST(DATEADD(day, 3, GETDATE()) AS DATE), '09:00 AM', 'Confirmed', 'Thyroid review', 'Due for 3-month thyroid panel.'),
```

```
(4, NULL, CAST(DATEADD(day, 1, GETDATE()) AS DATE), '02:00 PM', 'Pending', 'Headache / migraine', 'Severe headache for 3 days. Possible migraine.'),
```

```
(6, NULL, CAST(DATEADD(day, 4, GETDATE()) AS DATE), '03:30 PM', 'Pending', 'General check-up', 'Annual health check.');
```

```
PRINT '✅ Sample appointments inserted.';
```

```
END ELSE PRINT '❗ Appointments already has data.';
```

```
GO
```

```
-- — Verification —————
```

```
SELECT 'Receptionists' AS TableName, COUNT(*) AS Rows FROM Receptionists
```

```
UNION ALL SELECT 'Appointments', COUNT(*) FROM Appointments;
```

```
SELECT * FROM vw_DashboardStats;
```

PRINT '  Patch v3.2 complete.';

GO

```
-- =====
-- DATA RETENTION HELPERS (appended to patch v3.2)
-- =====

-- View: shows how many old records exist per patient
CREATE OR ALTER VIEW vw_DataRetentionStatus AS
SELECT
    p.PatientID,
    p.PatientName,
    p.BloodGroup,
    -- total records
    (SELECT COUNT(*) FROM Visits WHERE PatientID = p.PatientID) AS TotalVisits,
    (SELECT COUNT(*) FROM Visits WHERE PatientID = p.PatientID
        AND CAST(VisitDate AS DATE) < DATEADD(year,-1,CAST(GETDATE() AS DATE)))
    AS OldVisits,
    (SELECT COUNT(*) FROM Diagnoses d2
        JOIN Visits v2 ON d2.VisitID = v2.VisitID
        WHERE v2.PatientID = p.PatientID
        AND CAST(v2.VisitDate AS DATE) >= DATEADD(year,-1,CAST(GETDATE() AS DATE))) AS
    DiagnosesThisYear,
    (SELECT COUNT(*) FROM Prescriptions pr
        JOIN Visits v3 ON pr.VisitID = v3.VisitID
        WHERE v3.PatientID = p.PatientID
        AND CAST(v3.VisitDate AS DATE) >= DATEADD(year,-1,CAST(GETDATE() AS DATE))) AS RxThisYear,
    (SELECT COUNT(*) FROM MedicalFiles WHERE PatientID = p.PatientID) AS TotalFiles,
    CASE WHEN EXISTS (
        SELECT 1 FROM Visits WHERE PatientID = p.PatientID
        AND CAST(VisitDate AS DATE) < DATEADD(year,-1,CAST(GETDATE() AS DATE))
    ) THEN 1 ELSE 0 END AS NeedsPurge
```



```
FROM Patients p
WHERE p.IsActive = 1;
GO
```

```
-- Stored procedure: summary of all patients needing purge
CREATE OR ALTER PROCEDURE sp_GetRetentionReport AS
BEGIN
    SET NOCOUNT ON;
    SELECT * FROM vw_DataRetentionStatus WHERE NeedsPurge = 1 ORDER BY OldVisits DESC;
END;
GO
```

```
PRINT '✅ Data retention views and procedures added.';
GO
```

```
--
```

```
-- PATCH v3.3 — VitalSigns table + analytics views + full-text indexes
-- Appended to patch v3.2 | Safe to re-run
```

```
--
```

```
-- — VitalSigns — structured extraction from JSON on Visits —————
```

```
IF NOT EXISTS (SELECT 1 FROM sys.tables WHERE name = 'VitalSigns')
```

```
BEGIN
```

```
CREATE TABLE VitalSigns (
```

```
    VitalID          INT          IDENTITY PRIMARY KEY,
```

```
    VisitID          INT          NOT NULL
```

```
        CONSTRAINT FK_VitalSigns_Visit
```

```
        REFERENCES Visits(VisitID) ON DELETE CASCADE,
```

```

BloodPressureSystolic SMALLINT NULL, -- e.g. 120
BloodPressureDiastolic SMALLINT NULL, -- e.g. 80
HeartRate SMALLINT NULL, -- bpm
Temperature DECIMAL(4,1) NULL, -- °F
SpO2 TINYINT NULL, -- %
Weight DECIMAL(5,1) NULL, -- kg
BloodSugar DECIMAL(6,1) NULL, -- mg/dL (fasting)
RecordedAt DATETIME DEFAULT GETDATE()
);

CREATE INDEX IX_VS_VisitID ON VitalSigns(VisitID);

-- Migrate existing JSON vitals into the new structured table
-- Uses SQL Server 2016+ JSON_VALUE support
INSERT INTO VitalSigns
    (VisitID,
     BloodPressureSystolic, BloodPressureDiastolic,
     HeartRate, Temperature, SpO2)
SELECT
    v.VisitID,
    -- Parse "120/80" style BP stored in $.bp
    TRY_CAST(
        LEFT(JSON_VALUE(v.VitalSigns,'$.bp'),
             CHARINDEX('/',JSON_VALUE(v.VitalSigns,'$.bp')+1)-1)
        AS SMALLINT),
    TRY_CAST(
        SUBSTRING(JSON_VALUE(v.VitalSigns,'$.bp'),
             CHARINDEX('/',JSON_VALUE(v.VitalSigns,'$.bp'))+1, 10)
        AS SMALLINT),
    TRY_CAST(JSON_VALUE(v.VitalSigns,'$.pulse') AS SMALLINT),
    TRY_CAST(JSON_VALUE(v.VitalSigns,'$.temp') AS DECIMAL(4,1)),
    TRY_CAST(

```

```

        REPLACE(JSON_VALUE(v.VitalSigns,'$.spo2'),'%', ''))
        AS TINYINT)
FROM Visits v
WHERE v.VitalSigns IS NOT NULL
      AND v.VitalSigns <> ''
      AND ISJSON(v.VitalSigns) = 1;

PRINT '✅ Table VitalSigns created and migrated from Visits.VitalSigns JSON.';
END ELSE PRINT '❗ Table VitalSigns already exists.';
GO

-- — Analytics view: Patient 360 summary —————
CREATE OR ALTER VIEW vw_Patient360Summary AS
SELECT
    p.PatientID,
    p.PatientName,
    p.BloodGroup,
    p.Gender,
    -- visit metrics
    COUNT(DISTINCT v.VisitID)                AS TotalVisits,
    COUNT(DISTINCT CASE
        WHEN v.VisitDate >= DATEADD(year,-1,GETDATE()) THEN v.VisitID
    END)                AS VisitsLastYear,
    COUNT(DISTINCT CASE
        WHEN v.VisitDate >= DATEADD(month,-6,GETDATE()) THEN v.VisitID
    END)                AS VisitsLast6Mo,
    MAX(CAST(v.VisitDate AS DATE))            AS LastVisitDate,
    -- diagnosis metrics
    COUNT(DISTINCT d.DiagnosisID)            AS TotalDiagnoses,
    SUM(CASE WHEN d.IsChronic = 1 THEN 1 ELSE 0 END)    AS ChronicConditions,
    -- prescription metrics

```

```

COUNT(DISTINCT rx.PrescriptionID)                AS TotalPrescriptions,
SUM(CASE WHEN rx.IsDispensed = 0 THEN 1 ELSE 0 END) AS PendingRx,
-- vitals averages (last year)
AVG(CAST(vs.BloodPressureSystolic AS FLOAT))        AS AvgSystolic,
AVG(CAST(vs.BloodPressureDiastolic AS FLOAT))        AS AvgDiastolic,
AVG(CAST(vs.HeartRate AS FLOAT))                    AS AvgHeartRate,
AVG(CAST(vs.Temperature AS FLOAT))                  AS AvgTemperature,
-- frequent-flyer flag
CASE WHEN COUNT(DISTINCT CASE
  WHEN v.VisitDate >= DATEADD(month,-6,GETDATE()) THEN v.VisitID
END) > 3 THEN 1 ELSE 0 END                          AS IsFrequentFlyer,
-- 1-year retention flag
CASE WHEN EXISTS (
  SELECT 1 FROM Visits v2
  WHERE v2.PatientID = p.PatientID
  AND CAST(v2.VisitDate AS DATE) < DATEADD(year,-1,CAST(GETDATE() AS DATE))
) THEN 1 ELSE 0 END                                AS HasOldRecords
FROM   Patients    p
LEFT JOIN Visits    v ON v.PatientID = p.PatientID
LEFT JOIN Diagnoses d ON d.VisitID   = v.VisitID
LEFT JOIN Prescriptions rx ON rx.VisitID = v.VisitID
LEFT JOIN VitalSigns vs ON vs.VisitID = v.VisitID
      AND v.VisitDate >= DATEADD(year,-1,GETDATE())
WHERE p.IsActive = 1
GROUP BY p.PatientID, p.PatientName, p.BloodGroup, p.Gender;
GO

-- — Analytics view: Frequent Flyers (>3 visits in 6 months) —————
CREATE OR ALTER VIEW vw_FrequentFlyers AS
SELECT *
FROM vw_Patient360Summary

```

```
WHERE IsFrequentFlyer = 1;
```

```
GO
```

```
-- — Analytics view: Most common diagnoses across all patients —————
```

```
CREATE OR ALTER VIEW vw_DiagnosisFrequency AS
```

```
SELECT
```

```
    d.DiagnosisName,
```

```
    COUNT(*)                AS TotalOccurrences,
```

```
    COUNT(DISTINCT v.PatientID) AS UniquePatients,
```

```
    SUM(CASE WHEN d.IsChronic=1 THEN 1 ELSE 0 END) AS ChronicCount,
```

```
    MAX(CAST(v.VisitDate AS DATE)) AS MostRecent,
```

```
    MAX(d.Severity)          AS HighestSeverity
```

```
FROM Diagnoses d
```

```
JOIN Visits v ON d.VisitID = v.VisitID
```

```
WHERE v.VisitDate >= DATEADD(year,-1,GETDATE())
```

```
GROUP BY d.DiagnosisName;
```

```
GO
```

```
-- — Stored procedure: dashboard analytics for Admin —————
```

```
CREATE OR ALTER PROCEDURE sp_GetAdminAnalytics AS
```

```
BEGIN
```

```
    SET NOCOUNT ON;
```

```
-- 1. Frequent flyers
```

```
SELECT TOP 10 * FROM vw_FrequentFlyers ORDER BY VisitsLast6Mo DESC;
```

```
-- 2. Top diagnoses this year
```

```
SELECT TOP 10 * FROM vw_DiagnosisFrequency ORDER BY TotalOccurrences DESC;
```

```
-- 3. Patients needing data purge
```

```
SELECT PatientID, PatientName, BloodGroup, OldVisits
```

```
FROM vw_DataRetentionStatus

WHERE NeedsPurge = 1

ORDER BY OldVisits DESC;

END;

GO
```

-- — Performance indexes for analytics queries —————

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name='IX_Visits_PatientDate')

CREATE INDEX IX_Visits_PatientDate ON Visits(PatientID, VisitDate DESC);

GO
```

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name='IX_Diagnoses_Name')

CREATE INDEX IX_Diagnoses_Name ON Diagnoses(DiagnosisName);

GO
```

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name='IX_Diagnoses_IsChronic')

CREATE INDEX IX_Diagnoses_IsChronic ON Diagnoses(IsChronic) WHERE IsChronic = 1;

GO
```

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE name='IX_Prescriptions_Dispensed')

CREATE INDEX IX_Prescriptions_Dispensed ON Prescriptions(IsDispensed) WHERE IsDispensed = 0;

GO
```

-- — Update vw_DashboardStats to include pending appointments —————

```
CREATE OR ALTER VIEW vw_DashboardStats AS

SELECT

    (SELECT COUNT(*) FROM Patients    WHERE IsActive = 1)      AS TotalPatients,
    (SELECT COUNT(*) FROM Doctors     WHERE IsActive = 1)      AS TotalDoctors,
    (SELECT COUNT(*) FROM Radiologists WHERE IsActive = 1)      AS TotalRadiologists,
    (SELECT COUNT(*) FROM Visits
     WHERE CAST(VisitDate AS DATE) = CAST(GETDATE() AS DATE))  AS TodayVisits,
    (SELECT COUNT(*) FROM Prescriptions WHERE IsDispensed = 0)  AS PendingPrescriptions,
    (SELECT COUNT(*) FROM LabTests     WHERE Status = 'Pending') AS PendingTests,
    (SELECT COUNT(*) FROM Appointments WHERE Status = 'Pending') AS PendingAppointments;
```

GO

-- — Verification —————

SELECT 'VitalSigns' AS TableName, COUNT(*) AS Rows FROM VitalSigns

UNION ALL

SELECT 'Appointments', COUNT(*) FROM Appointments

UNION ALL

SELECT 'Receptionists', COUNT(*) FROM Receptionists;

SELECT * FROM vw_DashboardStats;

PRINT '';

PRINT '  Patch v3.3 complete — VitalSigns, analytics views, performance indexes.';

PRINT ' Run: SELECT * FROM vw_Patient360Summary to see the 360 view.';

PRINT ' Run: EXEC sp_GetAdminAnalytics to see frequent flyers + top diagnoses.';

GO

PRINT '  Schema setup complete — all existing data preserved.';

GO

select*from Doctors;

db.py code –

"""

db.py — Database Connection Module

=====

Centralised database access for the HMS Flask backend.

Configuration priority (highest → lowest):

1. Environment variables (recommended for production)
2. .env file (pip install python-dotenv)
3. Hard-coded defaults (development only)

Environment variables:

HMS_DB_SERVER — SQL Server host, e.g. localhost\\SQLEXPRESS

HMS_DB_NAME — Database name, default HospitalManagementSystem

HMS_DB_DRIVER — ODBC driver string

HMS_DB_USER — SQL Server login (leave empty for Windows auth)

HMS_DB_PASSWORD — SQL Server password (leave empty for Windows auth)

Usage:

```
from db import get_db, row_to_dict
```

```
conn = get_db()
```

```
cur = conn.cursor()
```

```
cur.execute("SELECT * FROM Patients WHERE IsActive = 1")
```

```
rows = [row_to_dict(cur, r) for r in cur.fetchall()]
```

```
cur.close()
```

```
conn.close()
```

```
"""
```

```
import os
```

```
import datetime
```

```
import pyodbc
```

```
# — Try to load .env if python-dotenv is installed —————
```

```
try:
```

```
    from dotenv import load_dotenv
```

```
    load_dotenv()
```

```
except ImportError:
```

```
    pass # python-dotenv not installed — use env vars or defaults
```

```
# — Connection settings —————
```



```

DB_SERVER = os.getenv("HMS_DB_SERVER", r"LAPTOP-OGJ9GR0I\SQLEXPRESS")
DB_NAME   = os.getenv("HMS_DB_NAME", "HospitalManagementSystem")
DB_DRIVER = os.getenv("HMS_DB_DRIVER", "{ODBC Driver 17 for SQL Server}")
DB_USER   = os.getenv("HMS_DB_USER", "") # empty = Windows auth
DB_PASSWORD = os.getenv("HMS_DB_PASSWORD", "")

```

— Build connection string —————

```
def _build_conn_str() -> str:
```

```

    base = (
        f"DRIVER={DB_DRIVER};"
        f"SERVER={DB_SERVER};"
        f"DATABASE={DB_NAME};"
    )

```

```
if DB_USER and DB_PASSWORD:
```

```

    # SQL Server authentication

    return base + f"UID={DB_USER};PWD={DB_PASSWORD};"

```

```
else:
```

```

    # Windows / Trusted Connection authentication

    return base + "Trusted_Connection=yes;"

```

```
def get_db() -> pyodbc.Connection:
```

```
    """
```

Open and return a new pyodbc connection.

autocommit=False — callers must call conn.commit() or conn.rollback().

Always close the connection in a finally block.

```
    """
```

```
    try:
```

```

        conn = pyodbc.connect(_build_conn_str(), autocommit=False)

        return conn

```

```
except pyodbc.Error as exc:
```

```
    raise ConnectionError(
```

```

        f"Cannot connect to SQL Server at '{DB_SERVER}/{DB_NAME}'. "
        f"Check DB_SERVER, ODBC driver, and Windows auth settings.\n"
        f"Original error: {exc}"
    ) from exc

```

```

def row_to_dict(cursor: pyodbc.Cursor, row: pyodbc.Row) -> dict:
    """
    Convert a pyodbc Row to a JSON-serialisable dict.
    Handles: datetime → ISO string, bytes → hex string, everything else as-is.
    """
    result: dict = {}
    for i, col in enumerate(cursor.description):
        val = row[i]
        if isinstance(val, (datetime.datetime, datetime.date)):
            val = val.isoformat()
        elif isinstance(val, bytes):
            val = val.hex()
        result[col[0]] = val
    return result

```

```

def test_connection() -> bool:
    """
    Quick connectivity check. Returns True on success, False on failure.
    Safe to call at startup.
    """
    try:
        conn = get_db()
        cur = conn.cursor()
        cur.execute("SELECT 1")
        cur.close()
        conn.close()
    except:
        return False
    return True

```

```

    print(f"✅ Database connection OK → {DB_SERVER} / {DB_NAME}")
    return True
except Exception as exc:
    print(f"❌ Database connection FAILED: {exc}")
    return False

if __name__ == "__main__":
    test_connection()

```

app.py code –

```

"""

```

Hospital Management System — Flask API (app.py) v3.0

```

=====

```

Run: python app.py

NEW in v3.0:

- Chronic Disease Progress endpoints (GET / POST / DELETE)
- Full Patient Detail endpoint sp_GetPatientFullDetails (doctor portal)
- Theme preference saved per user (stored client-side via JWT; API echoes it)

Install:

```

pip install flask flask-cors PyJWT pyodbc pandas openpyxl
"""

```

```

from flask import Flask, request, jsonify, send_file, g
from flask_cors import CORS
import jwt
import datetime
import os
import math as _math
import hashlib

```

```

from functools import wraps

from werkzeug.utils import secure_filename

import pandas as pd

import pyodbc


# -----
# App setup — serves built React frontend + API
# -----

import sys


# — Static folder: where Vite puts its build output -----
# Vite is configured to build to: hospital-frontend/dist/
# So Flask looks for hospital-frontend/dist/index.html to serve the React SPA.
STATIC_FOLDER = os.path.join(os.path.dirname(__file__), "hospital-frontend", "dist")

app = Flask(
    __name__,
    static_folder=STATIC_FOLDER,
    static_url_path="" # serve static files at root URL, not /static/
)

CORS(app, resources={r"/api/*": {"origins": "*"}})


SECRET_KEY = os.getenv("HMS_SECRET_KEY", "hospital_secret_key_change_in_production")
UPLOAD_FOLDER = os.path.join(os.path.dirname(__file__), "uploads")
ALLOWED_EXT = {"png", "jpg", "jpeg", "pdf", "dcm", "xlsx", "xls", "csv"}
MAX_UPLOAD_BYTES = 50 * 1024 * 1024


os.makedirs(os.path.join(UPLOAD_FOLDER, "medical_files"), exist_ok=True)
os.makedirs(os.path.join(UPLOAD_FOLDER, "patient_imports"), exist_ok=True)


app.config["UPLOAD_FOLDER"] = UPLOAD_FOLDER
app.config["MAX_CONTENT_LENGTH"] = MAX_UPLOAD_BYTES

```

```
# _____  
  
# Database  
  
# _____  
  
DB_SERVER = r"LAPTOP-OGJ9GR0I\SQLEXPRESS" # ← change to your server  
DB_NAME = "HospitalManagementSystem"  
DB_DRIVER = "{ODBC Driver 17 for SQL Server}"
```

```
def get_db() -> pyodbc.Connection:
```

```
    conn_str = (  
        f"DRIVER={DB_DRIVER};"  
        f"SERVER={DB_SERVER};"  
        f"DATABASE={DB_NAME};"  
        "Trusted_Connection=yes;"  
    )  
    return pyodbc.connect(conn_str, autocommit=False)
```

```
def row_to_dict(cursor: pyodbc.Cursor, row: pyodbc.Row) -> dict:
```

```
    result: dict = {}  
    for i, col in enumerate(cursor.description):  
        val = row[i]  
        if isinstance(val, (datetime.datetime, datetime.date)):  
            val = val.isoformat()  
        elif isinstance(val, bytes):  
            val = val.hex()  
        result[col[0]] = val  
    return result
```

```
# _____  
  
# Password helpers  
  
# _____
```

```

def hash_password(plain: str) -> str:
    return hashlib.sha256(plain.encode()).hexdigest()

def verify_password(plain: str, stored: str) -> bool:
    return stored == hash_password(plain) or stored == plain

# -----
# File helper
# -----

def allowed_file(filename: str) -> bool:
    return "." in filename and filename.rsplit(".", 1)[1].lower() in ALLOWED_EXT

# -----
# Auth decorator
# -----

def login_required(f):
    @wraps(f)
    def wrapper(*args, **kwargs):
        auth = request.headers.get("Authorization", "")
        if not auth.startswith("Bearer "):
            return jsonify({"error": "Token missing"}), 401
        try:
            token = auth.split(" ", 1)[1]
            g.user = jwt.decode(token, SECRET_KEY, algorithms=["HS256"])
        except jwt.ExpiredSignatureError:
            return jsonify({"error": "Token expired"}), 401
        except jwt.InvalidTokenError:
            return jsonify({"error": "Invalid token"}), 401
        return f(*args, **kwargs)
    return wrapper

```

```
def current_user() -> dict:
    user: dict = g.get("user")
    if user is None:
        raise RuntimeError("current_user() called outside login_required context")
    return user
```

```
# _____
```

```
# Serve React SPA
```

```
# _____
```

```
from flask import send_from_directory
```

```
# =====
```

```
# AUTHENTICATION
```

```
# =====
```

```
@app.route("/api/auth/login", methods=["POST"])
```

```
def login():
```

```
    data = request.get_json(silent=True) or {}
```

```
    username = str(data.get("username") or "").strip()
```

```
    password = str(data.get("password") or "")
```

```
    if not username or not password:
```

```
        return jsonify({"error": "Username and password are required"}), 400
```

```
conn = get_db(); cur = conn.cursor()
```

```
try:
```

```
    cur.execute("""
```

```
        SELECT UserID, Username, PasswordHash, Email, Role,
```

```
            PatientID, DoctorID, PharmacistID, RadiologistID, ReceptionistID
```

```
        FROM Users
```

```
        WHERE Username = ? AND IsActive = 1
```

```
    """, username)
```

```

row = cur.fetchone()

if row is None or not verify_password(password, row.PasswordHash):
    return jsonify({"error": "Invalid username or password"}), 401

payload = {
    "user_id":    row.UserID,
    "username":   row.Username,
    "email":      row.Email,
    "role":       row.Role,
    "patient_id": row.PatientID,
    "doctor_id":  row.DoctorID,
    "pharmacist_id": row.PharmacistID,
    "radiologist_id": row.RadiologistID,
    "receptionist_id": row.ReceptionistID,
    "exp": datetime.datetime.utcnow() + datetime.timedelta(hours=8),
}

token = jwt.encode(payload, SECRET_KEY, algorithm="HS256")

cur.execute("UPDATE Users SET LastLogin = GETDATE() WHERE UserID = ?", row.UserID)

conn.commit()

return jsonify({"token": token, "role": row.Role, "username": row.Username})

except Exception as exc:
    conn.rollback(); return jsonify({"error": str(exc)}), 500

finally:
    cur.close(); conn.close()

@app.route("/api/auth/register", methods=["POST"])
def register():
    data = request.get_json(silent=True) or {}

    required = ["username", "password", "email", "name", "gender", "dob", "phone", "address",
    "blood_group"]

```



```

missing = [f for f in required if not data.get(f)]
if missing:
    return jsonify({"error": f"Missing fields: {' '.join(missing)}"}, 400

username = str(data["username"]).strip()
password = str(data["password"])
if len(username) < 3:
    return jsonify({"error": "Username must be at least 3 characters"}, 400
if len(password) < 6:
    return jsonify({"error": "Password must be at least 6 characters"}, 400

conn = get_db(); cur = conn.cursor()
try:
    cur.execute("SELECT 1 FROM Users WHERE Username = ?", username)
    if cur.fetchone():
        return jsonify({"error": "Username already taken"}, 409

    cur.execute("SELECT 1 FROM Users WHERE Email = ?", data["email"])
    if cur.fetchone():
        return jsonify({"error": "Email already registered"}, 409

    cur.execute("""
        INSERT INTO Patients
        (PatientName, Email, Gender, DateOfBirth, PhoneNumber,
        Address, BloodGroup, EmergencyContact, EmergencyContactName)
        OUTPUT INSERTED.PatientID
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        data["name"], data["email"], data["gender"], data["dob"],
        data["phone"], data["address"], data["blood_group"],
        data.get("emergency_contact", ""),

```

```

        data.get("emergency_contact_name", ""),
    ))
    patient_row = cur.fetchone()
    if patient_row is None:
        raise RuntimeError("Failed to insert patient record")
    patient_id: int = patient_row[0]

    cur.execute("""
        INSERT INTO Users (Username, PasswordHash, Email, Role, PatientID)
        VALUES (?, ?, ?, 'Patient', ?)
        """, (username, hash_password(password), data["email"], patient_id))
    conn.commit()

    return jsonify({"message": "Registration successful! Please log in.", "username": username,
"patient_id": patient_id}), 201
except Exception as exc:
    conn.rollback(); return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

# =====
# DASHBOARD
# =====

@app.route("/api/dashboard/stats", methods=["GET"])
@login_required
def dashboard_stats():
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("SELECT * FROM vw_DashboardStats")
        row = cur.fetchone()
        return jsonify({
            "total_patients":    int(row[0]) if row else 0,

```

```

        "total_doctors":    int(row[1]) if row else 0,
        "total_radiologists": int(row[2]) if row else 0,
        "today_visits":    int(row[3]) if row else 0,
        "pending_prescriptions": int(row[4]) if row else 0,
        "pending_tests":    int(row[5]) if row else 0,
    })
except Exception as exc:
    return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

# =====
# PATIENTS
# =====

@app.route("/api/patients", methods=["GET"])
@login_required
def get_patients():
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("""
            SELECT PatientID, PatientName, Email, Gender, DateOfBirth,
                PhoneNumber, Address, BloodGroup,
                EmergencyContact, EmergencyContactName, CreatedAt
            FROM  Patients WHERE IsActive = 1 ORDER BY CreatedAt DESC
        """)
        return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])
    except Exception as exc:
        return jsonify({"error": str(exc)}), 500
    finally:
        cur.close(); conn.close()

```

```

@app.route("/api/patients/<int:pid>", methods=["GET"])
@login_required
def get_patient(pid: int):
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("""
            SELECT PatientID, PatientName, Email, Gender, DateOfBirth,
                PhoneNumber, Address, BloodGroup,
                EmergencyContact, EmergencyContactName, CreatedAt
            FROM Patients WHERE PatientID = ? AND IsActive = 1
        """, pid)
        row = cur.fetchone()
        if row is None:
            return jsonify({"error": "Patient not found"}), 404
        return jsonify(row_to_dict(cur, row))
    except Exception as exc:
        return jsonify({"error": str(exc)}), 500
    finally:
        cur.close(); conn.close()

```

```

@app.route("/api/patients/<int:pid>", methods=["PUT"])
@login_required
def update_patient(pid: int):
    data = request.get_json(silent=True) or {}
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("""
            UPDATE Patients
            SET PatientName=?, Gender=?, DateOfBirth=?, PhoneNumber=?,

```

```

        Address=?, BloodGroup=?, EmergencyContact=?, EmergencyContactName=?
        WHERE PatientID = ?
        """, (data.get("name"), data.get("gender"), data.get("dob"), data.get("phone"),
            data.get("address"), data.get("blood_group"),
            data.get("emergency_contact"), data.get("emergency_contact_name"), pid))
    conn.commit()

    return jsonify({"message": "Patient updated successfully"})
except Exception as exc:
    conn.rollback(); return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

```

```

@app.route("/api/patients/<int:pid>", methods=["DELETE"])

```

```

@login_required

```

```

def delete_patient(pid: int):

```

```

    user = current_user()

```

```

    if user.get("role") != "Admin":

```

```

        return jsonify({"error": "Admin access required"}), 403

```

```

    conn = get_db(); cur = conn.cursor()

```

```

    try:

```

```

        cur.execute("SELECT PatientID FROM Patients WHERE PatientID = ? AND IsActive = 1", pid)

```

```

        if cur.fetchone() is None:

```

```

            return jsonify({"error": "Patient not found"}), 404

```

```

        cur.execute("UPDATE Patients SET IsActive = 0 WHERE PatientID = ?", pid)

```

```

        cur.execute("UPDATE Users SET IsActive = 0 WHERE PatientID = ?", pid)

```

```

        conn.commit()

```

```

        return jsonify({"message": "Patient deactivated successfully"})

```

```

    except Exception as exc:

```

```

        conn.rollback(); return jsonify({"error": str(exc)}), 500

```

```

    finally:

```

```

cur.close(); conn.close()

# =====
# PATIENT BULK DELETE (Admin only)
# =====

@app.route("/api/patients/bulk-delete", methods=["POST"])
@login_required
def bulk_delete_patients():
    """
    Bulk deactivate patients by ID list.
    Expects JSON: { "patient_ids": [1,2,3,...] }
    """
    user = current_user()
    if user.get("role") != "Admin":
        return jsonify({"error": "Admin access required"}), 403

    data = request.get_json(silent=True) or {}
    patient_ids = data.get("patient_ids", [])

    if not isinstance(patient_ids, list):
        return jsonify({"error": "patient_ids must be a list"}), 400
    if not patient_ids:
        return jsonify({"error": "No patient IDs provided"}), 400

    # Validate all IDs are integers
    try:
        ids = [int(pid) for pid in patient_ids]
    except (ValueError, TypeError):
        return jsonify({"error": "All patient IDs must be integers"}), 400

    conn = get_db(); cur = conn.cursor()

```

try:

Check which IDs actually exist and are active

placeholders = ','.join('? ' * len(ids))

cur.execute(f"""

SELECT PatientID FROM Patients

WHERE PatientID IN ({placeholders}) AND IsActive = 1

""", ids)

existing = {row[0] for row in cur.fetchall()}

if not existing:

return jsonify({"error": "No active patients found with the provided IDs"}), 404

Update Patients table

cur.execute(f"""

UPDATE Patients SET IsActive = 0

WHERE PatientID IN ({placeholders}) AND IsActive = 1

""", ids)

Update Users table

cur.execute(f"""

UPDATE Users SET IsActive = 0

WHERE PatientID IN ({placeholders})

""", ids)

conn.commit()

return jsonify({

"message": f"Successfully deactivated {len(existing)} patient(s)",

"deactivated_count": len(existing),

"deactivated_ids": list(existing)

})

except Exception as exc:

```

        conn.rollback(); return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

# =====
# PATIENT FULL DETAILS (Doctor Portal)
# =====

@app.route("/api/patients/<int:pid>/full-details", methods=["GET"])
@login_required
def get_patient_full_details(pid: int):
    """
    Returns a multi-result-set from sp_GetPatientFullDetails:
    demographics, visits, diagnoses, prescriptions, lab tests,
    chronic progress history, and medical files — all in one call.
    Accessible by: Doctor, Admin, Radiologist.
    Patients may only access their own record.
    """
    user = current_user()
    role = user.get("role", "")
    if role == "Patient" and user.get("patient_id") != pid:
        return jsonify({"error": "Access denied"}), 403

    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("EXEC sp_GetPatientFullDetails ?", pid)

    def fetch_all():
        rows = cur.fetchall()
        data = [row_to_dict(cur, r) for r in rows]
        return data

```



```

demographics = fetch_all()
cur.nextset()
visits = fetch_all()
cur.nextset()
diagnoses = fetch_all()
cur.nextset()
prescriptions = fetch_all()
cur.nextset()
lab_tests = fetch_all()
cur.nextset()
chronic_progress = fetch_all()
cur.nextset()
files = fetch_all()

if not demographics:
    return jsonify({"error": "Patient not found"}), 404

return jsonify({
    "patient": demographics[0],
    "visits": visits,
    "diagnoses": diagnoses,
    "prescriptions": prescriptions,
    "lab_tests": lab_tests,
    "chronic_progress": chronic_progress,
    "files": files,
})

except Exception as exc:
    return jsonify({"error": str(exc)}), 500

finally:
    cur.close(); conn.close()

```

```

# =====
# CHRONIC DISEASE PROGRESS
# =====

@app.route("/api/chronic-progress/<int:pid>", methods=["GET"])
@login_required
def get_chronic_progress(pid: int):
    """Return all chronic progress entries for a patient."""
    user = current_user()
    role = user.get("role", "")
    if role == "Patient" and user.get("patient_id") != pid:
        return jsonify({"error": "Access denied"}), 403

    diag_name = request.args.get("diagnosis")
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("EXEC sp_GetChronicProgress ?, ?", pid, diag_name)
        rows = cur.fetchall()
        return jsonify([row_to_dict(cur, r) for r in rows])
    except Exception as exc:
        return jsonify({"error": str(exc)}), 500
    finally:
        cur.close(); conn.close()

@app.route("/api/chronic-progress", methods=["POST"])
@login_required
def add_chronic_progress():
    """Doctor records a new progress entry for a chronic condition."""
    user = current_user()
    if user.get("role") not in ("Doctor", "Admin"):

```

```

        return jsonify({"error": "Only doctors can record chronic progress"}), 403

data    = request.get_json(silent=True) or {}
patient_id = data.get("patient_id")
diag_name = data.get("diagnosis_name")
score    = data.get("progress_score")
doctor_id = user.get("doctor_id")

if not all([patient_id, diag_name, score is not None]):
    return jsonify({"error": "patient_id, diagnosis_name, and progress_score are required"}), 400

try:
    score = float(score)
    if not 0 <= score <= 100:
        raise ValueError
except (TypeError, ValueError):
    return jsonify({"error": "progress_score must be a number between 0 and 100"}), 400

conn = get_db(); cur = conn.cursor()
try:
    cur.execute("""
        INSERT INTO ChronicDiseaseProgress
            (PatientID, DiagnosisName, ProgressScore,
            BloodPressure, BloodSugar, Weight, Notes, RecordedBy)
        OUTPUT INSERTED.ProgressID
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
    """, (
        int(patient_id), str(diag_name), score,
        data.get("blood_pressure"),
        data.get("blood_sugar"),
        data.get("weight"),

```

```

        data.get("notes"),
        int(doctor_id) if doctor_id else None,
    ))
    row = cur.fetchone()
    if row is None:
        raise RuntimeError("Insert failed")
    prog_id: int = row[0]
    conn.commit()
    return jsonify({"message": "Progress entry recorded", "progress_id": prog_id}), 201
except Exception as exc:
    conn.rollback(); return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

```

```

@app.route("/api/chronic-progress/<int:progress_id>", methods=["DELETE"])
@login_required
def delete_chronic_progress(progress_id: int):
    """Delete a progress entry (doctor or admin only)."""
    user = current_user()
    if user.get("role") not in ("Doctor", "Admin"):
        return jsonify({"error": "Access denied"}), 403
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("DELETE FROM ChronicDiseaseProgress WHERE ProgressID = ?", progress_id)
        if cur.rowcount == 0:
            return jsonify({"error": "Entry not found"}), 404
        conn.commit()
        return jsonify({"message": "Progress entry deleted"})
    except Exception as exc:
        conn.rollback(); return jsonify({"error": str(exc)}), 500

```

finally:

```
cur.close(); conn.close()
```

```
# =====
```

```
# DOCTORS
```

```
# =====
```

```
@app.route("/api/doctors", methods=["GET"])
```

```
@login_required
```

```
def get_doctors():
```

```
    conn = get_db(); cur = conn.cursor()
```

```
    try:
```

```
        cur.execute("""
```

```
            SELECT DoctorID, DoctorName, Email, Specialty, PhoneNumber, LicenseNumber,
            YearsOfExperience
```

```
            FROM  Doctors WHERE IsActive = 1 ORDER BY DoctorName
```

```
        """)
```

```
        return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])
```

```
    except Exception as exc:
```

```
        return jsonify({"error": str(exc)}), 500
```

```
finally:
```

```
    cur.close(); conn.close()
```

```
@app.route("/api/doctors", methods=["POST"])
```

```
@login_required
```

```
def add_doctor():
```

```
    """Add a new doctor (admin only)."""
```

```
    user = current_user()
```

```
    if user.get("role") != "Admin":
```

```
        return jsonify({"error": "Admin access required"}), 403
```

```

data = request.get_json(silent=True) or {}
required = ["doctor_name", "email", "specialty", "phone_number"]
missing = [f for f in required if not data.get(f)]
if missing:
    return jsonify({"error": f"Missing required fields: {' '.join(missing)}"}), 400

```

```

doctor_name = str(data["doctor_name"]).strip()
email = str(data["email"]).strip()
specialty = str(data["specialty"]).strip()
phone_number = str(data["phone_number"]).strip()
license_number = str(data.get("license_number", "")).strip()
years_of_experience = data.get("years_of_experience")
username = str(data.get("username", "")).strip()
password = str(data.get("password", ""))

```

```

# Validate email format
if "@" not in email:
    return jsonify({"error": "Invalid email format"}), 400

```

```

conn = get_db(); cur = conn.cursor()
try:
    # Check if doctor with same email already exists
    cur.execute("SELECT 1 FROM Doctors WHERE Email = ? AND IsActive = 1", email)
    if cur.fetchone():
        return jsonify({"error": "A doctor with this email already exists"}), 409

```

```

# Insert new doctor
cur.execute("""
    INSERT INTO Doctors (DoctorName, Email, Specialty, PhoneNumber,
        LicenseNumber, YearsOfExperience, IsActive)
    OUTPUT INSERTED.DoctorID

```

```

VALUES (?, ?, ?, ?, ?, ?, 1)
""" , (
    doctor_name, email, specialty, phone_number,
    license_number if license_number else None,
    int(years_of_experience) if years_of_experience else None
))

row = cur.fetchone()
if row is None:
    raise RuntimeError("Failed to insert doctor record")

doctor_id = row[0]

# Also create a user account for the doctor
# Determine username and password
if username: # admin provided a username
    # Check if username already exists
    cur.execute("SELECT 1 FROM Users WHERE Username = ?", username)
    if cur.fetchone():
        return jsonify({"error": "Username already taken"}), 409
else:
    # Auto-generate username from email prefix
    username = email.split('@')[0]
    # Check if username already exists
    cur.execute("SELECT 1 FROM Users WHERE Username = ?", username)
    if cur.fetchone():
        # If username exists, append doctor_id
        username = f"{username}{doctor_id}"

if password:
    # Use provided password

```

```

temp_password = password

# Validate password length
if len(temp_password) < 6:
    return jsonify({"error": "Password must be at least 6 characters"}), 400
else:
    # Generate a temporary password (first 8 chars of email + "123")
    temp_password = email.split('@')[0][:8] + "123"

cur.execute("""
    INSERT INTO Users (Username, PasswordHash, Email, Role, DoctorID)
    VALUES (?, ?, ?, 'Doctor', ?)
    """, (username, hash_password(temp_password), email, doctor_id))

conn.commit()

note = "Doctor user account created"
if not password:
    note += " with auto-generated temporary password"
else:
    note += " with provided password"

return jsonify({
    "message": "Doctor added successfully",
    "doctor_id": doctor_id,
    "username": username,
    "temp_password": temp_password,
    "note": note
}), 201

except Exception as exc:
    conn.rollback()

    return jsonify({"error": str(exc)}), 500

finally:
    cur.close(); conn.close()

```



```

@app.route("/api/doctors/<int:doctor_id>", methods=["DELETE"])
@login_required
def delete_doctor(doctor_id: int):
    """Delete/deactivate a doctor (admin only)."""
    user = current_user()
    if user.get("role") != "Admin":
        return jsonify({"error": "Admin access required"}), 403

    conn = get_db(); cur = conn.cursor()
    try:
        # Check if doctor exists and is active
        cur.execute("SELECT DoctorID FROM Doctors WHERE DoctorID = ? AND IsActive = 1", doctor_id)
        if cur.fetchone() is None:
            return jsonify({"error": "Doctor not found"}), 404

        # Soft delete: set IsActive = 0 in Doctors table
        cur.execute("UPDATE Doctors SET IsActive = 0 WHERE DoctorID = ?", doctor_id)
        # Also deactivate the user account
        cur.execute("UPDATE Users SET IsActive = 0 WHERE DoctorID = ?", doctor_id)

        conn.commit()
        return jsonify({"message": "Doctor deactivated successfully"})
    except Exception as exc:
        conn.rollback()
        return jsonify({"error": str(exc)}), 500
    finally:
        cur.close(); conn.close()

```

#

```

# VISITS

# =====

@app.route("/api/visits", methods=["GET"])
@login_required
def get_visits():
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("""
            SELECT v.VisitID, v.PatientID, p.PatientName,
                   v.DoctorID, d.DoctorName,
                   v.VisitDate, v.ReasonForVisit, v.VitalSigns, v.Notes, v.Status
            FROM   Visits v
            JOIN   Patients p ON v.PatientID = p.PatientID
            JOIN   Doctors d ON v.DoctorID = d.DoctorID
            ORDER BY v.VisitDate DESC
        """)
        return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])
    except Exception as exc:
        return jsonify({"error": str(exc)}), 500
    finally:
        cur.close(); conn.close()

@app.route("/api/visits", methods=["POST"])
@login_required
def create_visit():
    data = request.get_json(silent=True) or {}
    if not data.get("patient_id") or not data.get("doctor_id"):
        return jsonify({"error": "patient_id and doctor_id are required"}), 400
    conn = get_db(); cur = conn.cursor()
    try:

```

```

cur.execute("""
    INSERT INTO Visits (PatientID, DoctorID, ReasonForVisit, VitalSigns, Notes, Status)
    OUTPUT INSERTED.VisitID
    VALUES (?, ?, ?, ?, ?, ?)
""", (int(data["patient_id"]), int(data["doctor_id"]),
      str(data.get("reason", "")), str(data.get("vital_signs", "")),
      str(data.get("notes", "")), str(data.get("status", "Scheduled"))))
row = cur.fetchone()
if row is None:
    raise RuntimeError("Insert failed")
conn.commit()
return jsonify({"message": "Visit created", "visit_id": row[0]}), 201
except Exception as exc:
    conn.rollback(); return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

```

```

# =====

```

```

# RECORDS

```

```

# =====

```

```

@app.route("/api/records/all", methods=["GET"])

```

```

@login_required

```

```

def get_all_records():

```

```

    conn = get_db(); cur = conn.cursor()

```

```

    try:

```

```

        cur.execute("EXEC sp_GetAllRecords")

```

```

        return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])

```

```

    except Exception as exc:

```

```

        return jsonify({"error": str(exc)}), 500

```

```

    finally:

```

```

        cur.close(); conn.close()

```

```

@app.route("/api/records/my", methods=["GET"])
@login_required
def get_my_records():
    user = current_user()
    patient_id = user.get("patient_id")
    if not patient_id:
        return jsonify({"error": "Not a patient account"}), 403
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("EXEC sp_GetPatientRecords ?", int(patient_id))
        return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])
    except Exception as exc:
        return jsonify({"error": str(exc)}), 500
    finally:
        cur.close(); conn.close()

```

```

@app.route("/api/records/doctor", methods=["GET"])
@login_required
def get_doctor_records():
    user = current_user()
    doctor_id = user.get("doctor_id")
    if not doctor_id:
        return jsonify({"error": "Not a doctor account"}), 403
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("""
            SELECT
                p.PatientID, p.PatientName, p.BloodGroup,

```

```

        d.DoctorName, d.Specialty,
        v.VisitID, v.VisitDate, v.ReasonForVisit,
        v.Status AS VisitStatus, v.VitalSigns, v.Notes AS VisitNotes,
        diag.DiagnosisName, diag.Description AS DiagnosisDesc,
        diag.IsChronic, diag.Severity,
        rx.MedicineName, rx.Dosage, rx.Frequency, rx.Duration,
        rx.Instructions, rx.IsDispensed, rx.DispensedDate
    FROM    Visits v
    JOIN    Patients  p  ON v.PatientID = p.PatientID
    JOIN    Doctors   d  ON v.DoctorID  = d.DoctorID
    LEFT JOIN Diagnoses  diag ON diag.VisitID = v.VisitID
    LEFT JOIN Prescriptions rx  ON rx.VisitID = v.VisitID
    WHERE   v.DoctorID = ? AND p.IsActive = 1
    ORDER BY v.VisitDate DESC

    """", int(doctor_id))

    return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])
except Exception as exc:
    return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

```

```

# =====
# DIAGNOSIS
# =====

@app.route("/api/diagnosis", methods=["POST"])
@login_required
def add_diagnosis():
    data = request.get_json(silent=True) or {}
    if not data.get("visit_id") or not data.get("name"):
        return jsonify({"error": "visit_id and name are required"}), 400
    conn = get_db(); cur = conn.cursor()

```

```

try:
    cur.execute("""
        INSERT INTO Diagnoses (VisitID, DiagnosisName, Description, IsChronic, Severity)
        OUTPUT INSERTED.DiagnosisID
        VALUES (?, ?, ?, ?, ?)
    """, (int(data["visit_id"]), str(data["name"]),
        str(data.get("description", "")),
        bool(data.get("is_chronic", False)),
        str(data.get("severity", "Mild"))))
    row = cur.fetchone()
    if row is None:
        raise RuntimeError("Insert failed")
    conn.commit()
    return jsonify({"message": "Diagnosis added", "diagnosis_id": row[0]}), 201
except Exception as exc:
    conn.rollback(); return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

```

PRESCRIPTIONS

```

@app.route("/api/prescriptions", methods=["GET"])
@login_required
def get_prescriptions():
    pending_only = request.args.get("pending", "0") == "1"
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("EXEC sp_GetPrescriptionsForPharmacy ?", int(pending_only))
        return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])
    except Exception as exc:

```

```

        return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

@app.route("/api/prescriptions", methods=["POST"])
@login_required
def add_prescription():
    data = request.get_json(silent=True) or {}
    if not data.get("visit_id") or not data.get("medicine"):
        return jsonify({"error": "visit_id and medicine are required"}), 400
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("""
            INSERT INTO Prescriptions (VisitID, MedicineName, Dosage, Frequency, Duration, Instructions)
            OUTPUT INSERTED.PrescriptionID
            VALUES (?, ?, ?, ?, ?, ?)
            """, (int(data["visit_id"]), str(data["medicine"]),
                str(data.get("dosage", "")), str(data.get("frequency", "")),
                str(data.get("duration", "")), str(data.get("instructions", ""))))
        row = cur.fetchone()
        if row is None:
            raise RuntimeError("Insert failed")
        conn.commit()
        return jsonify({"message": "Prescription added", "prescription_id": row[0]}), 201
    except Exception as exc:
        conn.rollback(); return jsonify({"error": str(exc)}), 500
    finally:
        cur.close(); conn.close()

```

```

@app.route("/api/prescriptions/<int:pid>/dispense", methods=["POST"])
@login_required
def dispense_prescription(pid: int):
    user = current_user()
    pharmacist_id = user.get("pharmacist_id")
    if not pharmacist_id:
        return jsonify({"error": "Only pharmacists can dispense prescriptions"}), 403
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("""
            UPDATE Prescriptions
            SET IsDispensed=1, DispensedBy=?, DispensedDate=GETDATE()
            WHERE PrescriptionID = ? AND IsDispensed = 0
            """, (int(pharmacist_id), pid))
        if cur.rowcount == 0:
            return jsonify({"error": "Prescription not found or already dispensed"}), 404
        conn.commit()
        return jsonify({"message": "Prescription dispensed successfully"})
    except Exception as exc:
        conn.rollback(); return jsonify({"error": str(exc)}), 500
    finally:
        cur.close(); conn.close()

```

```

# =====
# PATIENT SUMMARY SHEET
# =====

```

```

def _ensure_summary_notes_table(cur: pyodbc.Cursor) -> None:
    cur.execute("""
        IF NOT EXISTS (SELECT 1 FROM INFORMATION_SCHEMA.TABLES WHERE TABLE_NAME =
        'PatientSummaryNotes')
        CREATE TABLE PatientSummaryNotes (

```



```

        PatientID INT      PRIMARY KEY,

        Notes    NVARCHAR(MAX) NULL,

        UpdatedBy NVARCHAR(100) NULL,

        UpdatedAt DATETIME  DEFAULT GETDATE()

    )

    """)

```

```
@app.route("/api/patients/<int:pid>/summary", methods=["GET"])
```

```
@login_required
```

```
def get_patient_summary(pid: int):
```

```
    user = current_user()
```

```
    role = user.get("role", "")
```

```
    if role == "Patient" and user.get("patient_id") != pid:
```

```
        return jsonify({"error": "Access denied"}), 403
```

```
    conn = get_db(); cur = conn.cursor()
```

```
    try:
```

```
        _ensure_summary_notes_table(cur); conn.commit()
```

```
        cur.execute("""
```

```
            SELECT p.PatientID, p.PatientName, p.Email, p.Gender, p.DateOfBirth,
```

```
                   p.PhoneNumber, p.Address, p.BloodGroup,
```

```
                   p.EmergencyContact, p.EmergencyContactName,
```

```
                   ISNULL(n.Notes, '') AS SummaryNotes,
```

```
                   ISNULL(n.UpdatedBy, '') AS NotesUpdatedBy, n.UpdatedAt
```

```
            FROM Patients p
```

```
            LEFT JOIN PatientSummaryNotes n ON n.PatientID = p.PatientID
```

```
            WHERE p.PatientID = ? AND p.IsActive = 1
```

```
        """, pid)
```

```
        row = cur.fetchone()
```

```
        if row is None:
```

```
            return jsonify({"error": "Patient not found"}), 404
```

```
patient = row_to_dict(cur, row)
```

```
cur.execute("""
```

```
    SELECT v.VisitID, v.VisitDate, v.ReasonForVisit, v.VitalSigns, v.Notes, v.Status,  
           d.DoctorName, d.Specialty
```

```
    FROM Visits v JOIN Doctors d ON v.DoctorID = d.DoctorID
```

```
    WHERE v.PatientID = ? ORDER BY v.VisitDate DESC
```

```
""", pid)
```

```
visits = [row_to_dict(cur, r) for r in cur.fetchall()]
```

```
cur.execute("""
```

```
    SELECT dg.DiagnosisName, dg.Description, dg.Severity, dg.IsChronic, v.VisitDate  
    FROM Diagnoses dg JOIN Visits v ON dg.VisitID = v.VisitID
```

```
    WHERE v.PatientID = ? ORDER BY v.VisitDate DESC
```

```
""", pid)
```

```
diagnoses = [row_to_dict(cur, r) for r in cur.fetchall()]
```

```
cur.execute("""
```

```
    SELECT rx.MedicineName, rx.Dosage, rx.Frequency, rx.Duration, rx.Instructions,  
           rx.IsDispensed, rx.DispensedDate, d.DoctorName, v.VisitDate
```

```
    FROM Prescriptions rx
```

```
    JOIN Visits v ON rx.VisitID = v.VisitID
```

```
    JOIN Doctors d ON v.DoctorID = d.DoctorID
```

```
    WHERE v.PatientID = ? ORDER BY v.VisitDate DESC
```

```
""", pid)
```

```
prescriptions = [row_to_dict(cur, r) for r in cur.fetchall()]
```

```
cur.execute("""
```

```
    SELECT f.FileID, f.FileType, f.FileName, f.FileSize, f.Description, f.UploadedAt,  
           u.Username AS UploadedByUsername
```

```
    FROM MedicalFiles f LEFT JOIN Users u ON f.UploadedBy = u.UserID
```

```

        WHERE f.PatientID = ? ORDER BY f.UploadedAt DESC

        """ , pid)

    files = [row_to_dict(cur, r) for r in cur.fetchall()]

    # Latest chronic progress scores

    cur.execute("EXEC sp_GetLatestChronicScores ?", pid)

    chronic_scores = [row_to_dict(cur, r) for r in cur.fetchall()]

    return jsonify({"patient": patient, "visits": visits, "diagnoses": diagnoses,
                    "prescriptions": prescriptions, "files": files, "chronic_scores": chronic_scores})

except Exception as exc:

    return jsonify({"error": str(exc)}), 500

finally:

    cur.close(); conn.close()

```

```

@app.route("/api/patients/<int:pid>/summary-notes", methods=["PUT"])
@login_required
def update_summary_notes(pid: int):

    user = current_user()

    if user.get("role") not in ("Admin", "Doctor"):

        return jsonify({"error": "Admin or Doctor access required"}), 403

    data = request.get_json(silent=True) or {}

    notes = str(data.get("notes", ""))

    conn = get_db(); cur = conn.cursor()

    try:

        _ensure_summary_notes_table(cur)

        cur.execute("""

            MERGE PatientSummaryNotes AS target

            USING (SELECT ? AS PatientID) AS src ON target.PatientID = src.PatientID

            WHEN MATCHED THEN

```

```

        UPDATE SET Notes=?, UpdatedBy=?, UpdatedAt=GETDATE()
        WHEN NOT MATCHED THEN

        INSERT (PatientID, Notes, UpdatedBy) VALUES (?, ?, ?);

        """, (pid, notes, user["username"], pid, notes, user["username"]))

    conn.commit()

    return jsonify({"message": "Summary notes updated"})

except Exception as exc:

    conn.rollback(); return jsonify({"error": str(exc)}), 500

finally:

    cur.close(); conn.close()

# =====
# FILE UPLOAD / DOWNLOAD
# =====

@app.route("/api/files/upload", methods=["POST"])
@login_required
def upload_file():

    user = current_user()

    if "file" not in request.files:

        return jsonify({"error": "No file part in request"}), 400

    f = request.files["file"]

    raw_filename: str | None = f.filename

    if not raw_filename or raw_filename == "":

        return jsonify({"error": "No file selected"}), 400

    if not allowed_file(raw_filename):

        return jsonify({"error": "File type not allowed"}), 400

    patient_id_str = request.form.get("patient_id", "")

    if not patient_id_str:

        return jsonify({"error": "patient_id is required"}), 400

    visit_id_str = request.form.get("visit_id") or None

```

```

file_type = request.form.get("file_type", "Other")
description = request.form.get("description", "")

safe_name = secure_filename(raw_filename)
ts = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
stored_name = f"{ts}_{safe_name}"
filepath = os.path.join(UPLOAD_FOLDER, "medical_files", stored_name)
f.save(filepath)
size = os.path.getsize(filepath)
ext = safe_name.rsplit(".", 1)[-1].lower() if "." in safe_name else ""

conn = get_db(); cur = conn.cursor()
try:
    cur.execute("""
        INSERT INTO MedicalFiles
            (PatientID, VisitID, UploadedBy, FileType, FileName, FileExtension, FilePath, FileSize,
Description)
        OUTPUT INSERTED.FileID
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
    """, (int(patient_id_str), int(visit_id_str) if visit_id_str else None,
        int(user["user_id"]), str(file_type), safe_name, ext, filepath, size, description))
    row = cur.fetchone()
    if row is None:
        raise RuntimeError("Insert failed")
    conn.commit()
    return jsonify({"message": "File uploaded successfully", "file_id": row[0]}), 201
except Exception as exc:
    conn.rollback()
    if os.path.exists(filepath):
        os.remove(filepath)
    return jsonify({"error": str(exc)}), 500

```

finally:

cur.close(); conn.close()

@app.route("/api/files/patient/<int:pid>", methods=["GET"])

@login_required

def get_patient_files(pid: int):

conn = get_db(); cur = conn.cursor()

try:

cur.execute("EXEC sp_GetPatientFiles ?", pid)

return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])

except Exception as exc:

return jsonify({"error": str(exc)}), 500

finally:

cur.close(); conn.close()

@app.route("/api/files/download/<int:fid>", methods=["GET"])

@login_required

def download_file(fid: int):

conn = get_db(); cur = conn.cursor()

try:

cur.execute("SELECT FilePath, FileName FROM MedicalFiles WHERE FileID = ?", fid)

row = cur.fetchone()

if row is None:

return jsonify({"error": "File record not found"}), 404

filepath: str = str(row[0])

filename: str = str(row[1])

if not os.path.exists(filepath):

return jsonify({"error": "Physical file missing on server"}), 404

return send_file(filepath, as_attachment=True, download_name=filename)

```
except Exception as exc:

    return jsonify({"error": str(exc)}), 500

finally:

    cur.close(); conn.close()
```

```
# =====
```

```
# SCANS (Radiologist + Doctor)
```

```
# =====
```

```
SCAN_TYPES = {"X-Ray", "MRI", "CT Scan", "Ultrasound", "PET Scan", "Mammography", "Fluoroscopy"}
```

```
@app.route("/api/scans/upload", methods=["POST"])
```

```
@login_required
```

```
def upload_scan():
```

```
    user = current_user()

    if user.get("role") not in ("Radiologist", "Doctor", "Admin"):

        return jsonify({"error": "Access denied"}), 403

    if "file" not in request.files:

        return jsonify({"error": "No file part"}), 400

    f = request.files["file"]

    raw_filename: str | None = f.filename

    if not raw_filename or raw_filename == "":

        return jsonify({"error": "No file selected"}), 400

    if not allowed_file(raw_filename):

        return jsonify({"error": "File type not allowed"}), 400

    patient_id_str = request.form.get("patient_id", "")

    if not patient_id_str:

        return jsonify({"error": "patient_id is required"}), 400

    scan_type = request.form.get("scan_type", "X-Ray")

    if scan_type not in SCAN_TYPES:

        scan_type = "X-Ray"

    description = request.form.get("description", "")
```

```

safe_name = secure_filename(raw_filename)

ts = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")

stored_name = f"scan_{ts}_{safe_name}"

filepath = os.path.join(UPLOAD_FOLDER, "medical_files", stored_name)

f.save(filepath)

size = os.path.getsize(filepath)

ext = safe_name.rsplit(".", 1)[-1].lower() if "." in safe_name else ""

conn = get_db(); cur = conn.cursor()

try:
    cur.execute("""
        INSERT INTO MedicalFiles
            (PatientID, UploadedBy, FileType, FileName, FileExtension, FilePath, FileSize, Description)
        OUTPUT INSERTED.FileID
        VALUES (?, ?, ?, ?, ?, ?, ?, ?)
        """, (int(patient_id_str), int(user["user_id"]),
            scan_type, safe_name, ext, filepath, size, description))
    row = cur.fetchone()

    if row is None:
        raise RuntimeError("Insert failed")

    conn.commit()

    return jsonify({"message": "Scan uploaded successfully", "file_id": row[0]}), 201
except Exception as exc:
    conn.rollback()

    if os.path.exists(filepath):
        os.remove(filepath)

    return jsonify({"error": str(exc)}), 500

finally:
    cur.close(); conn.close()

```

```

@app.route("/api/scans/patient/<int:pid>", methods=["GET"])

```



```

@login_required
def get_patient_scans(pid: int):
    user = current_user()
    if user.get("role") == "Patient" and user.get("patient_id") != pid:
        return jsonify({"error": "Access denied"}), 403
    conn = get_db(); cur = conn.cursor()
    try:
        placeholders = ",".join("?" * len(SCAN_TYPES))
        cur.execute(f"""
            SELECT f.FileID, f.FileType, f.FileName, f.FileSize, f.Description, f.UploadedAt,
                   u.Username AS UploadedByUsername, p.PatientName
            FROM   MedicalFiles f
            LEFT JOIN Users   u ON f.UploadedBy = u.UserID
            LEFT JOIN Patients p ON f.PatientID = p.PatientID
            WHERE  f.PatientID = ? AND f.FileType IN ({placeholders})
            ORDER BY f.UploadedAt DESC
            """, pid, *list(SCAN_TYPES))
        return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])
    except Exception as exc:
        return jsonify({"error": str(exc)}), 500
    finally:
        cur.close(); conn.close()

```

```

@app.route("/api/scans/mine", methods=["GET"])

```

```

@login_required

```

```

def get_my_scans():
    user = current_user()
    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("""

```

```

        SELECT f.FileID, f.FileType, f.FileName, f.FileSize, f.Description, f.UploadedAt,
               p.PatientName, p.BloodGroup
        FROM MedicalFiles f JOIN Patients p ON f.PatientID = p.PatientID
        WHERE f.UploadedBy = ? ORDER BY f.UploadedAt DESC
        """ , int(user["user_id"]))

    return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])

except Exception as exc:
    return jsonify({"error": str(exc)}), 500

finally:
    cur.close(); conn.close()

# =====
# ADMIN — CREDENTIAL MANAGEMENT
# =====

@app.route("/api/admin/patients-without-credentials", methods=["GET"])
@login_required
def patients_without_credentials():
    user = current_user()
    if user.get("role") != "Admin":
        return jsonify({"error": "Admin access required"}), 403

    conn = get_db(); cur = conn.cursor()

    try:
        cur.execute("""
            SELECT p.PatientID, p.PatientName, p.Email, p.Gender, p.PhoneNumber, p.BloodGroup,
            p.CreatedAt

            FROM Patients p

            LEFT JOIN Users u ON u.PatientID = p.PatientID AND u.IsActive = 1

            WHERE p.IsActive = 1 AND u.UserID IS NULL

            ORDER BY p.CreatedAt DESC

            """)

    return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])

```

except Exception as exc:

 return jsonify({"error": str(exc)}), 500

finally:

 cur.close(); conn.close()

@app.route("/api/admin/assign-credentials", methods=["POST"])

@login_required

def assign_credentials():

 user = current_user()

 if user.get("role") != "Admin":

 return jsonify({"error": "Admin access required"}), 403

 data = request.get_json(silent=True) or {}

 patient_id = data.get("patient_id")

 username = str(data.get("username") or "").strip()

 password = str(data.get("password") or "")

 if not patient_id or not username or not password:

 return jsonify({"error": "patient_id, username and password are required"}), 400

 conn = get_db(); cur = conn.cursor()

 try:

 cur.execute("SELECT Email FROM Patients WHERE PatientID = ? AND IsActive = 1",
int(patient_id))

 row = cur.fetchone()

 if row is None:

 return jsonify({"error": "Patient not found"}), 404

 email: str = str(row[0])

 cur.execute("SELECT 1 FROM Users WHERE Username = ?", username)

 if cur.fetchone():

 return jsonify({"error": "Username already taken"}), 409

 cur.execute("""

 INSERT INTO Users (Username, PasswordHash, Email, Role, PatientID)

```

VALUES (?, ?, ?, 'Patient', ?)

"""", (username, hash_password(password), email, int(patient_id)))

conn.commit()

return jsonify({"message": f"Credentials assigned to patient #{patient_id}"}), 201
except Exception as exc:

    conn.rollback(); return jsonify({"error": str(exc)}), 500
finally:

    cur.close(); conn.close()

# =====
# ADMIN — BULK IMPORT
# =====

@app.route("/api/admin/import-patients", methods=["POST"])
@login_required
def import_patients():
    """
    Bulk-import patients from CSV/Excel.

    Required columns : Name, Email, Gender, DOB, Phone, Address, BloodGroup
    Optional columns : Username, Password, EmergencyContactName, EmergencyContact

    • If Username + Password are present in the row the patient gets a login
      account created immediately (no need for a separate "Assign Credentials"
      step).

    • Duplicate emails are skipped.

    • Password is stored as SHA-256 hash (same as register endpoint).
    """
    user = current_user()
    if user.get("role") != "Admin":
        return jsonify({"error": "Admin access required"}), 403
    if "file" not in request.files:

```

```

        return jsonify({"error": "No file provided"}), 400
f = request.files["file"]
raw_filename: str | None = f.filename
if not raw_filename or raw_filename == "":
    return jsonify({"error": "No file selected"}), 400
fname = secure_filename(raw_filename)
if not fname.lower().endswith((".xlsx", ".xls", ".csv")):
    return jsonify({"error": "Only Excel / CSV files are accepted"}), 400

ts = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
filepath = os.path.join(UPLOAD_FOLDER, "patient_imports", f"{ts}_{fname}")
f.save(filepath)
try:
    df = pd.read_csv(filepath) if filepath.endswith(".csv") else pd.read_excel(filepath)
except Exception as exc:
    return jsonify({"error": f"Could not parse file: {exc}"}), 400

required_cols = ["Name", "Email", "Gender", "DOB", "Phone", "Address", "BloodGroup"]
missing_cols = [c for c in required_cols if c not in df.columns]
if missing_cols:
    return jsonify({"error": f"Missing columns: {' '.join(missing_cols)}"}), 400

conn = get_db(); cur = conn.cursor()
total = len(df)
successful = 0; failed = 0; creds_created = 0
errors: list[str] = []

for raw_idx, row in df.iterrows():
    row_num = int(raw_idx) + 2 # type: ignore
    try:
        email_val = str(row.get("Email") or "").strip().lower()

```

```

if not email_val:

    errors.append(f"Row {row_num}: Email is empty — skipped"); failed += 1; continue

# — Skip duplicate patients —————

cur.execute("SELECT PatientID FROM Patients WHERE LOWER(Email) = ? AND IsActive = 1",
email_val)

existing = cur.fetchone()

if existing:

    errors.append(f"Row {row_num}: {email_val} already exists — skipped")

    failed += 1; continue

# — Insert patient —————

cur.execute("""

    INSERT INTO Patients

        (PatientName, Email, Gender, DateOfBirth, PhoneNumber,

        Address, BloodGroup, EmergencyContact, EmergencyContactName)

    OUTPUT INSERTED.PatientID

    VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)

""", (

    str(row.get("Name") or "").strip(),

    email_val,

    str(row.get("Gender") or "").strip(),

    str(row.get("DOB") or "").strip() or None,

    str(row.get("Phone") or "").strip(),

    str(row.get("Address") or "").strip(),

    str(row.get("BloodGroup") or "O+").strip(),

    str(row.get("EmergencyContact") or "").strip() or None,

    str(row.get("EmergencyContactName") or "").strip() or None,

))

pid_row = cur.fetchone()

if pid_row is None:

```

```

        raise RuntimeError("Patient INSERT failed")

patient_id: int = pid_row[0]

# — Optionally create login credentials —————
raw_user = str(row.get("Username") or "").strip()
raw_pass = str(row.get("Password") or "").strip()
if raw_user and raw_pass:
    # Check username not already taken
    cur.execute("SELECT 1 FROM Users WHERE Username = ?", raw_user)
    if cur.fetchone():
        errors.append(
            f"Row {row_num}: Username '{raw_user}' already taken — "
            f"patient imported but no credentials created"
        )
    else:
        cur.execute("""
            INSERT INTO Users (Username, PasswordHash, Email, Role, PatientID)
            VALUES (?, ?, ?, 'Patient', ?)
            """, (raw_user, hash_password(raw_pass), email_val, patient_id))
        creds_created += 1

successful += 1

except Exception as exc:
    errors.append(f"Row {row_num}: {exc}"); failed += 1

try:
    conn.commit()
    cur.execute("""
        INSERT INTO ImportHistory
        (ImportedBy, FileName, TotalRecords, SuccessfulRecords, FailedRecords, ErrorLog)
    """)

```

```

VALUES (?, ?, ?, ?, ?)

"""", (int(user["user_id"]), fname, total, successful, failed, "\n".join(errors)))

conn.commit()

except Exception as exc:

    conn.rollback()

    return jsonify({"error": f"Commit failed: {exc}"}), 500

finally:

    cur.close(); conn.close()

return jsonify({
    "message":    "Import completed",
    "total":      total,
    "successful": successful,
    "failed":     failed,
    "creds_created": creds_created,
    "errors":     errors,
})

```

```

# =====
# LAB TESTS
# =====

```

```

@app.route("/api/lab-tests", methods=["GET"])
@login_required
def get_lab_tests():
    """
    Returns lab tests for the logged-in doctor's patients.
    Doctors see tests for their visits; Admin sees all.
    """

    user = current_user()
    role = user.get("role", "")

```



```

doctor_id = user.get("doctor_id")

conn = get_db(); cur = conn.cursor()

try:
    if role == "Admin":
        cur.execute("""
            SELECT It.TestID, It.VisitID, It.TestName, It.TestType,
                   It.Status, It.Results, It.ResultDate, It.CreatedAt,
                   v.VisitDate, p.PatientName
            FROM   LabTests It
            JOIN   Visits v  ON It.VisitID = v.VisitID
            JOIN   Patients p ON v.PatientID = p.PatientID
            ORDER BY It.CreatedAt DESC
            """)
    elif role == "Doctor" and doctor_id:
        cur.execute("""
            SELECT It.TestID, It.VisitID, It.TestName, It.TestType,
                   It.Status, It.Results, It.ResultDate, It.CreatedAt,
                   v.VisitDate, p.PatientName
            FROM   LabTests It
            JOIN   Visits v  ON It.VisitID = v.VisitID
            JOIN   Patients p ON v.PatientID = p.PatientID
            WHERE  v.DoctorID = ?
            ORDER BY It.CreatedAt DESC
            """, int(doctor_id))
    else:
        return jsonify({"error": "Access denied"}), 403

    rows = cur.fetchall()

    return jsonify([row_to_dict(cur, r) for r in rows])
except Exception as exc:

```

```

        return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

@app.route("/api/lab-tests", methods=["POST"])
@login_required
def add_lab_test():
    """Doctor orders a new lab test for a visit."""
    user = current_user()
    if user.get("role") not in ("Doctor", "Admin"):
        return jsonify({"error": "Only doctors can order lab tests"}), 403

    data = request.get_json(silent=True) or {}
    visit_id = data.get("visit_id")
    test_name = data.get("test_name")
    test_type = data.get("test_type", "Blood Test")

    if not visit_id or not test_name:
        return jsonify({"error": "visit_id and test_name are required"}), 400

    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("""
            INSERT INTO LabTests (VisitID, TestName, TestType, Status)
            OUTPUT INSERTED.TestID
            VALUES (?, ?, ?, 'Pending')
            """, (int(visit_id), str(test_name), str(test_type)))
        row = cur.fetchone()
        if row is None:
            raise RuntimeError("Insert failed")
    
```

```

        conn.commit()

        return jsonify({"message": "Lab test ordered", "test_id": row[0]}), 201
    except Exception as exc:
        conn.rollback()

        return jsonify({"error": str(exc)}), 500
    finally:
        cur.close(); conn.close()

```

```

@app.route("/api/lab-tests/<int:tid>/result", methods=["PUT"])
@login_required
def update_lab_result(tid: int):
    """Doctor enters results for a completed lab test."""
    user = current_user()
    if user.get("role") not in ("Doctor", "Admin"):
        return jsonify({"error": "Only doctors can update lab results"}), 403

```

```

data    = request.get_json(silent=True) or {}
results = data.get("results", "")
result_date = data.get("result_date")

```

```

conn = get_db(); cur = conn.cursor()

```

```

try:

```

```

    cur.execute("""
        UPDATE LabTests
        SET   Results   = ?,
             ResultDate = ?,
             Status    = 'Completed'
        WHERE TestID = ?
    """, (str(results), result_date, tid))

    if cur.rowcount == 0:

```

```

        return jsonify({"error": "Test not found"}), 404

    conn.commit()

    return jsonify({"message": "Lab result updated"})
except Exception as exc:
    conn.rollback()

    return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

```

```

@app.route("/api/lab-tests/<int:tid>", methods=["DELETE"])
@login_required
def delete_lab_test(tid: int):
    """Cancel / delete a lab test (Doctor or Admin only)."""
    user = current_user()
    if user.get("role") not in ("Doctor", "Admin"):
        return jsonify({"error": "Access denied"}), 403

    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("DELETE FROM LabTests WHERE TestID = ?", tid)
        if cur.rowcount == 0:
            return jsonify({"error": "Test not found"}), 404
        conn.commit()
        return jsonify({"message": "Lab test deleted"})
    except Exception as exc:
        conn.rollback()
        return jsonify({"error": str(exc)}), 500
    finally:
        cur.close(); conn.close()

```

```

@app.route("/api/doctors/<int:doctor_id>", methods=["PUT"])
@login_required
def update_doctor(doctor_id: int):
    """Update doctor details (admin only)."""
    user = current_user()
    if user.get("role") != "Admin":
        return jsonify({"error": "Admin access required"}), 403

    data = request.get_json(silent=True) or {}
    # Fields that can be updated
    doctor_name = data.get("doctor_name")
    email = data.get("email")
    specialty = data.get("specialty")
    phone_number = data.get("phone_number")
    license_number = data.get("license_number")
    years_of_experience = data.get("years_of_experience")

    # At least one field must be provided
    if not any([doctor_name, email, specialty, phone_number, license_number is not None,
years_of_experience is not None]):
        return jsonify({"error": "No fields to update"}), 400

    conn = get_db(); cur = conn.cursor()
    try:
        # Check if doctor exists and is active
        cur.execute("SELECT DoctorID, Email FROM Doctors WHERE DoctorID = ? AND IsActive = 1",
doctor_id)
        row = cur.fetchone()
        if row is None:
            return jsonify({"error": "Doctor not found"}), 404

        current_email = row.Email

```

```

# Build dynamic update query

updates = []

params = []

if doctor_name is not None:

    updates.append("DoctorName = ?")

    params.append(str(doctor_name).strip())

if email is not None:

    new_email = str(email).strip()

    if "@" not in new_email:

        return jsonify({"error": "Invalid email format"}), 400

    # Check if email already exists for another doctor

    cur.execute("SELECT 1 FROM Doctors WHERE Email = ? AND DoctorID != ? AND IsActive = 1",
(new_email, doctor_id))

    if cur.fetchone():

        return jsonify({"error": "Email already in use by another doctor"}), 409

    updates.append("Email = ?")

    params.append(new_email)

    # Also update Users table email if it matches the current email

    cur.execute("UPDATE Users SET Email = ? WHERE Email = ? AND DoctorID = ?", (new_email,
current_email, doctor_id))

if specialty is not None:

    updates.append("Specialty = ?")

    params.append(str(specialty).strip())

if phone_number is not None:

    updates.append("PhoneNumber = ?")

    params.append(str(phone_number).strip())

if license_number is not None:

    updates.append("LicenseNumber = ?")

    params.append(str(license_number).strip() if license_number != "" else None)

if years_of_experience is not None:

    updates.append("YearsOfExperience = ?")

```

```
params.append(int(years_of_experience) if years_of_experience != "" else None)
```

```
params.append(doctor_id)
```

```
query = f"UPDATE Doctors SET {' '.join(updates)} WHERE DoctorID = ?"
```

```
cur.execute(query, params)
```

```
conn.commit()
```

```
return jsonify({"message": "Doctor updated successfully"})
```

```
except Exception as exc:
```

```
conn.rollback()
```

```
return jsonify({"error": str(exc)}), 500
```

```
finally:
```

```
cur.close(); conn.close()
```

```
# =====
```

```
# 1-YEAR DATA RETENTION — purge old patient records
```

```
# =====
```

```
@app.route("/api/admin/patients/<int:pid>/purge-old-data", methods=["DELETE"])
```

```
@login_required
```

```
def purge_old_patient_data(pid: int):
```

```
    """
```

```
    Admin-only: delete visits, diagnoses, prescriptions, lab tests, and
```

```
    chronic-progress entries for a patient that are older than 1 year.
```

```
    Medical files are NOT deleted (they may be needed for legal purposes).
```

```
    Returns counts of purged records by type.
```

```
    """
```

```
    user = current_user()
```

```
    if user.get("role") != "Admin":
```

```
        return jsonify({"error": "Admin access required"}), 403
```

```
cutoff = (datetime.datetime.utcnow() - datetime.timedelta(days=365)).strftime("%Y-%m-%d")
```

```
purged: dict = {}
```

```
conn = get_db(); cur = conn.cursor()
```

```
try:
```

```
    # 1. Find old visits for this patient
```

```
    cur.execute("""
```

```
        SELECT VisitID FROM Visits
```

```
        WHERE PatientID = ? AND CAST(VisitDate AS DATE) < ?
```

```
    """, (pid, cutoff))
```

```
    old_visit_ids = [r[0] for r in cur.fetchall()]
```

```
    if old_visit_ids:
```

```
        placeholders = ",".join("?" * len(old_visit_ids))
```

```
        # Delete lab tests tied to old visits
```

```
        cur.execute(f"DELETE FROM LabTests WHERE VisitID IN ({placeholders})", old_visit_ids)
```

```
        purged["lab_tests"] = cur.rowcount
```

```
        # Delete prescriptions tied to old visits
```

```
        cur.execute(f"DELETE FROM Prescriptions WHERE VisitID IN ({placeholders})", old_visit_ids)
```

```
        purged["prescriptions"] = cur.rowcount
```

```
        # Delete diagnoses tied to old visits
```

```
        cur.execute(f"DELETE FROM Diagnoses WHERE VisitID IN ({placeholders})", old_visit_ids)
```

```
        purged["diagnoses"] = cur.rowcount
```

```
        # Delete the visits themselves
```

```
        cur.execute(f"DELETE FROM Visits WHERE VisitID IN ({placeholders})", old_visit_ids)
```

```
        purged["visits"] = cur.rowcount
```

```
    else:
```



```
purged["visits"] = purged["diagnoses"] = purged["prescriptions"] = purged["lab_tests"] = 0
```

2. Delete chronic progress entries older than 1 year

```
cur.execute("""
```

```
    DELETE FROM ChronicDiseaseProgress
```

```
    WHERE PatientID = ? AND CAST(RecordDate AS DATE) < ?
```

```
""", (pid, cutoff))
```

```
purged["chronic_progress"] = cur.rowcount
```

3. Delete completed/cancelled appointments older than 1 year

```
cur.execute("""
```

```
    DELETE FROM Appointments
```

```
    WHERE PatientID = ?
```

```
    AND Status IN ('Completed', 'Cancelled')
```

```
    AND CAST(RequestedDate AS DATE) < ?
```

```
""", (pid, cutoff))
```

```
purged["appointments"] = cur.rowcount
```

```
conn.commit()
```

```
total = sum(purged.values())
```

```
return jsonify({
```

```
    "message": f"Purged {total} records older than 1 year ({cutoff})",
```

```
    "purged": purged,
```

```
    "cutoff": cutoff,
```

```
})
```

except Exception as exc:

```
    conn.rollback()
```

```
    return jsonify({"error": str(exc)}), 500
```

finally:

```
    cur.close(); conn.close()
```

```
# =====  
# APPOINTMENTS  
# =====
```

```
@app.route("/api/appointments", methods=["GET"])  
@login_required  
def get_appointments():  
    """  
    Patient → own appointments only  
    Receptionist / Admin → all appointments, optional ?status=Pending  
    Doctor → own confirmed/upcoming appointments  
    """  
    user = current_user()  
    role = user.get("role", "")  
    status = request.args.get("status") # optional filter  
  
    conn = get_db(); cur = conn.cursor()  
    try:  
        if role == "Patient":  
            pid = user.get("patient_id")  
            if not pid:  
                return jsonify({"error": "Not a patient account"}), 403  
            cur.execute("EXEC sp_GetPatientAppointments ?", int(pid))  
  
        elif role == "Doctor":  
            did = user.get("doctor_id")  
            if not did:  
                return jsonify({"error": "Not a doctor account"}), 403  
            cur.execute("EXEC sp_GetDoctorAppointments ?", int(did))
```

```
elif role in ("Receptionist", "Admin"):
    cur.execute("EXEC sp_GetAllAppointments ?", status)
```

```
else:
    return jsonify({"error": "Access denied"}), 403
```

```
rows = cur.fetchall()
return jsonify([row_to_dict(cur, r) for r in rows])
```

```
except Exception as exc:
    return jsonify({"error": str(exc)}), 500
```

```
finally:
    cur.close(); conn.close()
```

```
@app.route("/api/appointments", methods=["POST"])
```

```
@login_required
```

```
def book_appointment():
```

```
    """Patient books an appointment request (no doctor assigned yet)."""
```

```
    user = current_user()
```

```
    if user.get("role") != "Patient":
```

```
        return jsonify({"error": "Only patients can book appointments"}), 403
```

```
    patient_id = user.get("patient_id")
```

```
    if not patient_id:
```

```
        return jsonify({"error": "Patient ID not found in token"}), 403
```

```
    data = request.get_json(silent=True) or {}
```

```
    requested_date = data.get("requested_date")
```

```
    time_slot = data.get("time_slot", "")
```

```

reason      = data.get("reason", "")
patient_notes = data.get("patient_notes", "")
preferred_doc = data.get("preferred_doctor_id") # optional — patient preference

if not requested_date:
    return jsonify({"error": "requested_date is required"}), 400

conn = get_db(); cur = conn.cursor()
try:
    cur.execute("""
        INSERT INTO Appointments
            (PatientID, DoctorID, RequestedDate, TimeSlot,
             Status, Reason, PatientNotes)
        OUTPUT INSERTED.AppointmentID
        VALUES (?, ?, ?, ?, 'Pending', ?, ?)
    """, (
        int(patient_id),
        int(preferred_doc) if preferred_doc else None,
        requested_date,
        str(time_slot),
        str(reason),
        str(patient_notes),
    ))
    row = cur.fetchone()
    if row is None:
        raise RuntimeError("Insert failed")
    conn.commit()
    return jsonify({
        "message": "Appointment request submitted. Our receptionist will confirm shortly.",
        "appointment_id": row[0]
    }), 201

```

except Exception as exc:

 conn.rollback()

 return jsonify({"error": str(exc)}), 500

finally:

 cur.close(); conn.close()

@app.route("/api/appointments/<int:aid>/assign", methods=["PUT"])

@login_required

def assign_appointment(aid: int):

 """

 Receptionist (or Admin) assigns a doctor and confirms the appointment.

 """

 user = current_user()

 if user.get("role") not in ("Receptionist", "Admin"):

 return jsonify({"error": "Only receptionists can assign doctors"}), 403

 data = request.get_json(silent=True) or {}

 doctor_id = data.get("doctor_id")

 time_slot = data.get("time_slot")

 rec_notes = data.get("receptionist_notes", "")

 status = data.get("status", "Confirmed") # allow setting to Cancelled too

 if status == "Confirmed" and not doctor_id:

 return jsonify({"error": "doctor_id is required when confirming"}), 400

 # Resolve the receptionist's own ID (if role is Receptionist)

 rec_id: int | None = None

 if user.get("role") == "Receptionist":

 conn0 = get_db(); cur0 = conn0.cursor()

 try:

```

cur0.execute("""
    SELECT r.ReceptionistID
    FROM Receptionists r
    JOIN Users u ON u.ReceptionistID = r.ReceptionistID
    WHERE u.UserID = ?
""", int(user["user_id"]))
row0 = cur0.fetchone()
if row0:
    rec_id = row0[0]
finally:
    cur0.close(); conn0.close()

```

```

conn = get_db(); cur = conn.cursor()

```

```

try:

```

```

    cur.execute("""
        UPDATE Appointments
        SET DoctorID      = ?,
            Status        = ?,
            TimeSlot      = COALESCE(?, TimeSlot),
            ReceptionistNotes = ?,
            HandledBy      = COALESCE(?, HandledBy),
            UpdatedAt      = GETDATE()
        WHERE AppointmentID = ?
""", (
    int(doctor_id) if doctor_id else None,
    str(status),
    str(time_slot) if time_slot else None,
    str(rec_notes),
    rec_id,
    aid,
))

```

```

if cur.rowcount == 0:
    return jsonify({"error": "Appointment not found"}), 404
conn.commit()
return jsonify({"message": f"Appointment {status.lower()} successfully"})
except Exception as exc:
    conn.rollback()
    return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

```

```

@app.route("/api/appointments/<int:aid>/cancel", methods=["PUT"])

```

```

@login_required

```

```

def cancel_appointment(aid: int):

```

```

    """Patient cancels their own pending appointment."""

```

```

    user = current_user()

```

```

    conn = get_db(); cur = conn.cursor()

```

```

    try:

```

```

        # Patients can only cancel their own

```

```

        if user.get("role") == "Patient":

```

```

            cur.execute("""

```

```

                UPDATE Appointments

```

```

                SET Status = 'Cancelled', UpdatedAt = GETDATE()

```

```

                WHERE AppointmentID = ? AND PatientID = ? AND Status = 'Pending'

```

```

            """, (aid, int(user.get("patient_id") or 0)))

```

```

        else:

```

```

            cur.execute("""

```

```

                UPDATE Appointments

```

```

                SET Status = 'Cancelled', UpdatedAt = GETDATE()

```

```

                WHERE AppointmentID = ?

```

```

            """, aid)

```

```

if cur.rowcount == 0:
    return jsonify({"error": "Appointment not found or cannot be cancelled"}), 404
conn.commit()
return jsonify({"message": "Appointment cancelled"})
except Exception as exc:
    conn.rollback()
    return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

```

```
@app.route("/api/appointments/<int:aid>/complete", methods=["PUT"])
```

```
@login_required
```

```
def complete_appointment(aid: int):
```

```
    """Doctor marks their appointment as completed."""
```

```
    user = current_user()
```

```
    if user.get("role") not in ("Doctor", "Admin", "Receptionist"):
```

```
        return jsonify({"error": "Access denied"}), 403
```

```
    conn = get_db(); cur = conn.cursor()
```

```
    try:
```

```
        cur.execute("""
```

```
            UPDATE Appointments
```

```
            SET Status = 'Completed', UpdatedAt = GETDATE()
```

```
            WHERE AppointmentID = ?
```

```
        """, aid)
```

```
    if cur.rowcount == 0:
```

```
        return jsonify({"error": "Appointment not found"}), 404
```

```
    conn.commit()
```

```
    return jsonify({"message": "Appointment marked as completed"})
```

```
except Exception as exc:
```



```

        conn.rollback()

        return jsonify({"error": str(exc)}), 500

    finally:

        cur.close(); conn.close()


# -----
# RECEPTIONISTS list (for admin)
# -----

@app.route("/api/receptionists", methods=["GET"])
@login_required
def get_receptionists():
    user = current_user()

    if user.get("role") != "Admin":
        return jsonify({"error": "Admin access required"}), 403

    conn = get_db(); cur = conn.cursor()

    try:
        cur.execute("""
            SELECT ReceptionistID, ReceptionistName, Email, PhoneNumber, LicenseNumber, CreatedAt
            FROM Receptionists WHERE IsActive = 1 ORDER BY ReceptionistName
        """)

        return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])

    except Exception as exc:
        return jsonify({"error": str(exc)}), 500

    finally:
        cur.close(); conn.close()


# =====

```

ANALYTICS — 360° Patient Longitudinal View

#

```
@app.route("/api/analytics/frequent-flyers", methods=["GET"])
@login_required
def frequent_flyers():
    """
    Patients with > threshold visits in the last N months.
    Query params: ?threshold=3&months=6
    Returns visit count, span, most frequent complaint, chronic flag.
    """
    user = current_user()
    if user.get("role") not in ("Doctor", "Admin"):
        return jsonify({"error": "Access denied"}), 403

    threshold = int(request.args.get("threshold", 3))
    months = int(request.args.get("months", 6))

    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("""
            SELECT
                p.PatientID,
                p.PatientName,
                p.BloodGroup,
                p.PhoneNumber,
                p.Gender,
                COUNT(v.VisitID)           AS VisitCount,
                MIN(CAST(v.VisitDate AS DATE)) AS FirstVisit,
                MAX(CAST(v.VisitDate AS DATE)) AS LastVisit,
                DATEDIFF(day,
```

```

        MIN(CAST(v.VisitDate AS DATE)),
        MAX(CAST(v.VisitDate AS DATE)))          AS SpanDays,
    (SELECT TOP 1 v2.ReasonForVisit
     FROM Visits v2
     WHERE v2.PatientID = p.PatientID
        AND v2.VisitDate >= DATEADD(month, -?, GETDATE()))
    GROUP BY v2.ReasonForVisit
    ORDER BY COUNT(*) DESC)          AS MostFrequentComplaint,
    CASE WHEN EXISTS (
        SELECT 1 FROM Diagnoses d2
        JOIN Visits v3 ON d2.VisitID = v3.VisitID
        WHERE v3.PatientID = p.PatientID
            AND d2.IsChronic = 1
    ) THEN 1 ELSE 0 END          AS HasChronic
FROM Patients p
JOIN Visits v ON v.PatientID = p.PatientID
WHERE v.VisitDate >= DATEADD(month, -?, GETDATE())
    AND p.IsActive = 1
GROUP BY p.PatientID, p.PatientName,
        p.BloodGroup, p.PhoneNumber, p.Gender
HAVING COUNT(v.VisitID) > ?
ORDER BY VisitCount DESC
""", (months, months, threshold))
return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])
except Exception as exc:
    return jsonify({"error": str(exc)}), 500
finally:
    cur.close(); conn.close()

```

```
@app.route("/api/analytics/patient-diagnosis-summary/<int:pid>",
```

```

        methods=["GET"])

@login_required
def patient_diagnosis_summary(pid: int):
    """
    Diagnoses ranked by frequency for one patient over the past year.
    Returns DiagnosisName, Frequency, LastSeen, WorstSeverity, IsChronic.
    """
    user = current_user()
    if user.get("role") not in ("Doctor", "Admin"):
        return jsonify({"error": "Access denied"}), 403

    conn = get_db(); cur = conn.cursor()
    try:
        cur.execute("""
            SELECT
                d.DiagnosisName,
                COUNT(*)                AS Frequency,
                MAX(CAST(v.VisitDate AS DATE))    AS LastSeen,
                MAX(d.Severity)                AS WorstSeverity,
                MAX(CAST(d.IsChronic AS TINYINT)) AS IsChronic
            FROM  Diagnoses d
            JOIN  Visits v ON d.VisitID = v.VisitID
            WHERE v.PatientID = ?
            AND  v.VisitDate >= DATEADD(year, -1, GETDATE())
            GROUP BY d.DiagnosisName
            ORDER BY Frequency DESC
        """, pid)

        return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])
    except Exception as exc:
        return jsonify({"error": str(exc)}), 500
    finally:

```

```
cur.close(); conn.close()
```

```
@app.route("/api/analytics/visit-trend/<int:pid>", methods=["GET"])
```

```
@login_required
```

```
def patient_visit_trend(pid: int):
```

```
    """
```

```
    Monthly visit counts for a patient over the past 12 months.
```

```
    Used by the admin patient detail bar chart.
```

```
    """
```

```
    user = current_user()
```

```
    if user.get("role") not in ("Doctor", "Admin"):
```

```
        return jsonify({"error": "Access denied"}), 403
```

```
    conn = get_db(); cur = conn.cursor()
```

```
    try:
```

```
        cur.execute("""
```

```
            SELECT
```

```
                FORMAT(VisitDate, 'yyyy-MM') AS MonthKey,
```

```
                FORMAT(VisitDate, 'MMM yy') AS MonthLabel,
```

```
                COUNT(*) AS VisitCount
```

```
            FROM Visits
```

```
            WHERE PatientID = ?
```

```
            AND VisitDate >= DATEADD(month, -12, GETDATE())
```

```
            GROUP BY FORMAT(VisitDate, 'yyyy-MM'),
```

```
                    FORMAT(VisitDate, 'MMM yy')
```

```
            ORDER BY MonthKey ASC
```

```
        """, pid)
```

```
        return jsonify([row_to_dict(cur, r) for r in cur.fetchall()])
```

```
    except Exception as exc:
```

```
        return jsonify({"error": str(exc)}), 500
```

finally:

```
cur.close(); conn.close()
```

```
# =====
```

```
# BULK VISIT HISTORY IMPORT (with deduplication)
```

```
# =====
```

```
def _find_or_create_patient(cur: pyodbc.Cursor, row: dict) -> tuple:
```

```
    """
```

```
    Match patient by Email first, then (PatientName + DOB) fallback.
```

```
    Returns (PatientID: int, was_created: bool).
```

```
    Never creates a duplicate profile.
```

```
    """
```

```
    email = str(row.get("Email") or "").strip().lower()
```

```
    name = str(row.get("Name") or "").strip()
```

```
    dob = str(row.get("DOB") or "").strip()
```

```
# 1 Primary match: email (unique in DB)
```

```
if email:
```

```
    cur.execute(
```

```
        "SELECT PatientID FROM Patients WHERE LOWER(Email) = ? AND IsActive = 1",
```

```
        email,
```

```
    )
```

```
    r = cur.fetchone()
```

```
    if r:
```

```
        return r[0], False
```

```
# 2 Fallback: name + DOB
```

```
if name and dob:
```

```
    cur.execute("""
```

```

SELECT PatientID FROM Patients

WHERE PatientName = ?

AND CAST(DateOfBirth AS DATE) = TRY_CAST(? AS DATE)

AND IsActive = 1

"", (name, dob))
r = cur.fetchone()

if r:

    return r[0], False

# 3 Not found — create new patient record
placeholder_email = (
    email
    if email
    else f"imported_{datetime.datetime.now().timestamp():.0f}@noemail.local"
)
cur.execute("""
INSERT INTO Patients
(PatientName, Email, Gender, DateOfBirth,
PhoneNumber, Address, BloodGroup)
OUTPUT INSERTED.PatientID
VALUES (?, ?, ?, ?, ?, ?, ?)
"", (
    name or "Unknown",
    placeholder_email,
    str(row.get("Gender") or "").strip() or "Unknown",
    dob or None,
    str(row.get("Phone") or "").strip(),
    str(row.get("Address") or "").strip(),
    str(row.get("BloodGroup") or "O+").strip(),
))
r2 = cur.fetchone()

```

```

if r2 is None:
    raise RuntimeError(f"Failed to insert patient: {name}")
return r2[0], True

```

```
@app.route("/api/admin/import-visits", methods=["POST"])
```

```
@login_required
```

```
def import_visit_history():
```

```
    """
```

Bulk-import historical visits from an Excel/CSV file.

Required columns : Email, VisitDate, ReasonForVisit, DoctorName

Optional columns : Name, DOB, Gender, BloodGroup, Phone, Address,

Username, Password, ← create login on import

DiagnosisName, IsChronic, Severity, Description,

MedicineName, Dosage, Frequency, Duration, Instructions,

BP, HeartRate, Temp, SpO2, Notes

Deduplication

- Patients : Email → (Name+DOB) → create new
- Visits : skip if same PatientID+DoctorID+Date+Reason already exists
- Credentials : only created when Username+Password supplied AND username not already taken

```
    """
```

```
user = current_user()
```

```
if user.get("role") != "Admin":
```

```
    return jsonify({"error": "Admin access required"}), 403
```

```
if "file" not in request.files:
```

```
    return jsonify({"error": "No file provided"}), 400
```



```

f      = request.files["file"]
raw_filename: str | None = f.filename
if not raw_filename or raw_filename == "":
    return jsonify({"error": "No file selected"}), 400

fname = secure_filename(raw_filename)
if not fname.lower().endswith((".xlsx", ".xls", ".csv")):
    return jsonify({"error": "Only Excel / CSV files are accepted"}), 400

ts      = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
filepath = os.path.join(
    UPLOAD_FOLDER, "patient_imports", f"visits_{ts}_{fname}"
)
f.save(filepath)

try:
    df = (
        pd.read_csv(filepath)
        if filepath.endswith(".csv")
        else pd.read_excel(filepath)
    )
except Exception as exc:
    return jsonify({"error": f"Could not parse file: {exc}"}), 400

required_cols = ["Email", "VisitDate", "ReasonForVisit", "DoctorName"]
missing_cols = [c for c in required_cols if c not in df.columns]
if missing_cols:
    return jsonify({"error": f"Missing columns: {' '.join(missing_cols)}"}), 400

import json as _json

```

```

conn = get_db(); cur = conn.cursor()

total      = len(df)
successful  = 0
failed      = 0
patients_created = 0
creds_created  = 0
diagnoses_added = 0
rx_added       = 0
errors: list[str] = []

for raw_idx, row in df.iterrows():
    row_num = int(raw_idx) + 2    # type: ignore
    row_dict = {
        k: (None if (isinstance(v, float) and _math.isnan(v)) else v)
        for k, v in row.items()
    }

    try:
        # — Patient deduplication —————
        patient_id, was_created = _find_or_create_patient(cur, row_dict)
        if was_created:
            patients_created += 1

        # — Optional: create credentials for this patient —————
        raw_user = str(row_dict.get("Username") or "").strip()
        raw_pass = str(row_dict.get("Password") or "").strip()
        if raw_user and raw_pass:
            cur.execute(
                "SELECT 1 FROM Users WHERE PatientID = ? AND IsActive = 1",
                patient_id,

```

```

)
already_has_creds = cur.fetchone()
if not already_has_creds:
    cur.execute("SELECT 1 FROM Users WHERE Username = ?", raw_user)
    username_taken = cur.fetchone()
    if username_taken:
        errors.append(
            f"Row {row_num}: Username '{raw_user}' already taken — "
            f"visit imported but no credentials created"
        )
    else:
        email_val = str(row_dict.get("Email") or "").strip().lower()
        cur.execute("""
            INSERT INTO Users (Username, PasswordHash, Email, Role, PatientID)
            VALUES (?, ?, ?, 'Patient', ?)
            """, (raw_user, hash_password(raw_pass), email_val, patient_id))
        creds_created += 1

# — Doctor lookup —————
doctor_name = str(row_dict.get("DoctorName") or "").strip()
cur.execute(
    "SELECT DoctorID FROM Doctors WHERE DoctorName = ? AND IsActive = 1",
    doctor_name,
)
doc = cur.fetchone()
if not doc:
    errors.append(
        f"Row {row_num}: Doctor '{doctor_name}' not found — skipped"
    )
failed += 1
continue

```

— Visit deduplication —————

```
visit_date = str(row_dict.get("VisitDate") or "").strip()
reason     = str(row_dict.get("ReasonForVisit") or "").strip()
cur.execute("""
    SELECT VisitID FROM Visits
    WHERE PatientID = ? AND DoctorID = ?
    AND CAST(VisitDate AS DATE) = TRY_CAST(? AS DATE)
    AND ReasonForVisit = ?
""", (patient_id, doc[0], visit_date, reason))
if cur.fetchone():
    errors.append(
        f"Row {row_num}: Duplicate visit for "
        f"{row_dict.get('Email')} on {visit_date} — skipped"
    )
    failed += 1
    continue
```

— Build vitals JSON —————

```
vitals: dict[str, str] = {}
if row_dict.get("BP"):     vitals["bp"] = str(row_dict["BP"])
if row_dict.get("HeartRate"): vitals["pulse"] = str(row_dict["HeartRate"])
if row_dict.get("Temp"):   vitals["temp"] = str(row_dict["Temp"])
if row_dict.get("SpO2"):   vitals["spo2"] = str(row_dict["SpO2"])
vital_str = _json.dumps(vitals) if vitals else ""
```

— Insert Visit —————

```
cur.execute("""
    INSERT INTO Visits
    (PatientID, DoctorID, VisitDate,
    ReasonForVisit, VitalSigns, Notes, Status)
```

```

OUTPUT INSERTED.VisitID

VALUES (?, ?, TRY_CAST(? AS DATETIME), ?, ?, ?, 'Completed')

""" , (
    patient_id, doc[0], visit_date, reason,
    vital_str,
    str(row_dict.get("Notes") or ""),
))
visit_row = cur.fetchone()
if visit_row is None:
    raise RuntimeError("Visit INSERT failed — no VisitID returned")
visit_id = visit_row[0]

# — Insert Diagnosis (optional) —————
if row_dict.get("DiagnosisName"):
    is_chronic = str(
        row_dict.get("IsChronic") or "0"
    ).lower() in ("1", "true", "yes")
    cur.execute("""
        INSERT INTO Diagnoses
            (VisitID, DiagnosisName, Description, IsChronic, Severity)
        VALUES (?, ?, ?, ?, ?)
    """, (
        visit_id,
        str(row_dict["DiagnosisName"]),
        str(row_dict.get("Description") or ""),
        is_chronic,
        str(row_dict.get("Severity") or "Mild"),
    ))
    diagnoses_added += 1

# — Insert Prescription (optional) —————

```

```

if row_dict.get("MedicineName"):
    cur.execute("""
        INSERT INTO Prescriptions
            (VisitID, MedicineName, Dosage,
             Frequency, Duration, Instructions)
        VALUES (?, ?, ?, ?, ?, ?)
    """, (
        visit_id,
        str(row_dict["MedicineName"]),
        str(row_dict.get("Dosage") or ""),
        str(row_dict.get("Frequency") or ""),
        str(row_dict.get("Duration") or ""),
        str(row_dict.get("Instructions") or ""),
    ))
    rx_added += 1

```

```

successful += 1

```

```

except Exception as exc:
    errors.append(f"Row {row_num}: {exc}")
    failed += 1

```

— Commit all inserts —————

```

try:
    conn.commit()
    cur.execute("""
        INSERT INTO ImportHistory
            (ImportedBy, FileName, TotalRecords,
             SuccessfulRecords, FailedRecords, ErrorLog)
        VALUES (?, ?, ?, ?, ?, ?)
    """, (

```

```

        int(user["user_id"]), fname, total,
        successful, failed, "\n".join(errors),
    ))
    conn.commit()
except Exception as exc:
    conn.rollback()
    return jsonify({"error": f"Commit failed: {exc}"}), 500
finally:
    cur.close(); conn.close()

return jsonify({
    "message": "Visit history import completed",
    "total": total,
    "successful": successful,
    "failed": failed,
    "patients_created": patients_created,
    "creds_created": creds_created,
    "diagnoses_added": diagnoses_added,
    "rx_added": rx_added,
    "errors": errors,
})

```

```
@app.route("/api/admin/import-template", methods=["GET"])
```

```
@login_required
```

```
def download_import_template():
```

```
    """
```

```
    Serve a ready-to-fill CSV template.
```

```
    ?type=patients → patient profile + optional credentials columns
```

```
    ?type=visits → full visit history + optional credentials columns
```

```
    """
```

```

import io

template_type = request.args.get("type", "patients")

if template_type == "visits":
    headers = [
        # Patient identity (used for deduplication / auto-create)
        "Email", "Name", "DOB", "Gender", "BloodGroup", "Phone", "Address",
        # Optional credentials — fill to create login on import
        "Username", "Password",
        # Visit core
        "VisitDate", "ReasonForVisit", "DoctorName",
        # Diagnosis (optional)
        "DiagnosisName", "IsChronic", "Severity", "Description",
        # Prescription (optional)
        "MedicineName", "Dosage", "Frequency", "Duration", "Instructions",
        # Vitals (optional)
        "BP", "HeartRate", "Temp", "SpO2", "Notes",
    ]
    sample = [
        "rahul@email.com", "Rahul Mehta", "1990-01-15", "Male", "O+",
        "9876543210", "Mumbai",
        "rahul_p", "password123",      # ← credentials columns
        "2024-06-15", "BP follow-up", "Dr. Anil Kumar",
        "Hypertension", "1", "Moderate", "Persistent high BP on 3 readings",
        "Amlodipine", "5mg", "Once daily", "30 days", "Take with food",
        "140/90", "88", "98.4", "98%", "Patient reports mild dizziness",
    ]
else: # patients
    headers = [
        # Core demographics
        "Name", "Email", "Gender", "DOB", "Phone", "Address", "BloodGroup",

```



```

# Emergency contact (optional)

"EmergencyContactName", "EmergencyContact",

# Credentials (optional) — fill to create login on import

"Username", "Password",

]

sample = [

    "John Doe", "john@email.com", "Male", "1990-01-15",

    "9876543210", "Mumbai", "O+",

    "Jane Doe", "9876500000", # emergency contact

    "john_doe", "password123", # ← credentials columns

]

```

```

buf = io.StringIO()
buf.write(",".join(headers) + "\n")
buf.write(",".join(str(s) for s in sample) + "\n")
buf.seek(0)

```

```

return send_file(
    io.BytesIO(buf.getvalue().encode()),
    mimetype="text/csv",
    as_attachment=True,
    download_name=f"hms_{template_type}_template.csv",
)

```

```
@app.route("/", defaults={"path": ""})
```

```
@app.route("/<path:path>")
```

```
def serve_react(path: str):
```

```
    """
```

Serve the built React app for ALL non-API routes.

- If the file exists in hospital-frontend/dist/ (JS, CSS, assets), serve it directly.

- Otherwise serve index.html so React Router handles the URL.

"""

```
dist_dir = os.path.join(os.path.dirname(__file__), "hospital-frontend", "dist")
```

```
# If no build exists yet, return a helpful message
```

```
if not os.path.isdir(dist_dir):
```

```
    return (
```

```
        "<h2>Frontend not built yet.</h2>"
```

```
        "<p>Run: <code>cd hospital-frontend && npm run build</code></p>",
```

```
        503,
```

```
    )
```

```
# Serve actual file if it exists (JS bundles, CSS, images, favicon...)
```

```
target = os.path.join(dist_dir, path)
```

```
if path and os.path.isfile(target):
```

```
    return send_from_directory(dist_dir, path)
```

```
# Fall back to index.html for all React routes
```

```
return send_from_directory(dist_dir, "index.html")
```

```
# _____
```

```
# Entry point
```

```
# _____
```

```
if __name__ == "__main__":
```

```
    port = int(os.getenv("HMS_PORT", "5000"))
```

```
    debug = os.getenv("HMS_DEBUG", "0") == "1"
```

```
    print(f"Hospital Management System v3.2")
```

```
    print(f"DB : {DB_SERVER} / {DB_NAME}")
```

```
    print(f"URL : http://localhost:{port}")
```

```
    print(f"Mode: {'DEBUG' if debug else 'PRODUCTION'}")
```

```
    app.run(debug=debug, host="0.0.0.0", port=port)
```

app.tsx –

/**

* Hospital Management System — App.tsx v3.1

* =====

*

* NEW in v3.1

* • Lab Tests view — doctor can add, view & update test results per visit

* • Chronic Disease Chart — SVG line chart showing score trend over time

* • Chronic chart available from doctor panel AND from summary modal

* • Notification bell — shows pending prescriptions + pending lab tests count

* • .env-aware API_BASE (falls back to localhost:5000 in dev)

*

* v3.0 features

* • 5-theme runtime switching (Midnight / Aurora / Daybreak / Forest / Ember)

* • Doctor: interactive split-panel patient browser with full details

* • Doctor: visit history accordions with vital signs

* • Doctor: chronic disease progress rings (SVG) + trend mini-bars

* • Doctor: progress entry modal — slider 0-100 + vitals logging

* • Doctor: summary sheet modal — printable, editable by Doctor & Admin

* • All roles: theme preference persisted in localStorage

*/

import React, { useState, useEffect, useCallback, useRef } from 'react';

import {

Activity, User, Users, FileText, PlusCircle, LogOut,

Shield, Stethoscope, Heart, Upload, Download, FileImage, UserPlus,

Pill, CheckCircle, Clock, Calendar, Phone, Mail,

Eye, EyeOff, AlertCircle, Edit2, Trash2, Key, X, Save,

ClipboardList, ScanLine, ChevronDown, BookOpen,

Sun, Moon, Palette, TrendingUp, BarChart2, Zap,

Cloud, Trees, Flame,

```

    FlaskConical, Bell, RefreshCw, Plus, ChevronLeft, ChevronRight,
  } from 'lucide-react';

import './App.css';

// _____
// Constants
// _____

// In dev (npm run dev): Vite proxy forwards /api → localhost:5000
// In production (served by Flask): /api hits Flask directly
const API = (import.meta.env.VITE_API_URL as string) || '/api';
const SCAN_TYPES = ['X-Ray', 'MRI', 'CT Scan', 'Ultrasound', 'PET Scan', 'Mammography', 'Fluoroscopy'];

// _____
// Theme System
// _____

export type ThemeKey = 'midnight' | 'aurora' | 'daybreak' | 'forest' | 'ember';

interface ThemeDef {
  key: ThemeKey;
  label: string;
  icon: React.ReactNode;
  swatch: string; // CSS gradient for the swatch circle
}

const THEMES: ThemeDef[] = [
  { key: 'midnight', label: 'Midnight', icon: <Moon size={13}/>, swatch: 'linear-gradient(135deg, #00d4ff, #0099bb)' },
  { key: 'aurora', label: 'Aurora', icon: <Cloud size={13}/>, swatch: 'linear-gradient(135deg, #c471ed, #9b51c0)' },
  { key: 'daybreak', label: 'Daybreak', icon: <Sun size={13}/>, swatch: 'linear-gradient(135deg, #0075d1, #005eaa)' },

```

```

    { key:'forest', label:'Forest', icon:<Trees size={13}/>, swatch:'linear-
gradient(135deg,#00ffaa,#00c882)' },

    { key:'ember', label:'Ember', icon:<Flame size={13}/>, swatch:'linear-
gradient(135deg,#ff9500,#e07800)' },

];

```

```

function useTheme(): [ThemeKey, (k: ThemeKey) => void] {
  const [theme, setThemeState] = useState<ThemeKey>(() => {
    const stored = localStorage.getItem('hms_theme');

    // Migrate old theme keys
    if (stored === 'dark') return 'midnight';
    if (stored === 'light') return 'daybreak';

    if (stored === 'midnight' || stored === 'aurora' || stored === 'daybreak' || stored === 'forest' ||
stored === 'ember') {
      return stored as ThemeKey;
    }

    return 'midnight';
  });

  const setTheme = useCallback((k: ThemeKey) => {
    document.documentElement.setAttribute('data-theme', k);
    localStorage.setItem('hms_theme', k);
    setThemeState(k);
  }, []);

  useEffect(() => { document.documentElement.setAttribute('data-theme', theme); }, [theme]);

  return [theme, setTheme];
}

```

```

// _____
// Types
// _____

```

```

interface AuthUser {
  user_id: number; username: string; email: string; role: string;

```

```
patient_id?: number|null; doctor_id?: number|null;
pharmacist_id?: number|null; radiologist_id?: number|null;
receptionist_id?: number|null;
}
```

```
interface Patient {
  PatientID?: number; PatientName: string; Email: string; Gender: string;
  DateOfBirth: string; PhoneNumber: string; Address: string; BloodGroup: string;
  EmergencyContact?: string; EmergencyContactName?: string;
}
```

```
interface Doctor {
  DoctorID: number;
  DoctorName: string;
  Email: string;
  Specialty: string;
  PhoneNumber: string;
  LicenseNumber?: string;
  YearsOfExperience?: number;
}
```

```
interface Visit {
  VisitID?: number; PatientID: number; DoctorID: number; ReasonForVisit: string;
  VitalSigns?: string; Notes?: string; Status: string;
  PatientName?: string; DoctorName?: string; VisitDate?: string;
}
```

```
interface Prescription {
  PrescriptionID: number; PatientName: string; MedicineName: string; Dosage: string;
  Frequency: string; Duration: string; Instructions?: string;
  IsDispensed: boolean; DispensedDate?: string; DoctorName: string; VisitDate: string;
```

```
}
```

```
interface MedicalFile {
```

```
    FileID: number; FileType: string; FileName: string; FileSize: number;
```

```
    Description: string; UploadedAt: string; UploadedByUsername: string;
```

```
}
```

```
interface DashboardStats {
```

```
    total_patients: number; total_doctors: number; total_radiologists: number;
```

```
    today_visits: number; pending_prescriptions: number; pending_tests: number;
```

```
}
```

```
interface DoctorRecord {
```

```
    PatientID?: number; PatientName: string; BloodGroup: string;
```

```
    DoctorName: string; Specialty: string; VisitID: number; VisitDate: string;
```

```
    ReasonForVisit: string; VisitStatus: string; VitalSigns?: string; VisitNotes?: string;
```

```
    DiagnosisName?: string; DiagnosisDesc?: string; IsChronic?: boolean; Severity?: string;
```

```
    MedicineName?: string; Dosage?: string; Frequency?: string; Duration?: string;
```

```
    Instructions?: string; IsDispensed?: boolean;
```

```
}
```

```
interface PatientNoCreds {
```

```
    PatientID: number; PatientName: string; Email: string; Gender: string;
```

```
    PhoneNumber: string; BloodGroup: string; CreatedAt: string;
```

```
}
```

```
interface ScanFile {
```

```
    FileID: number; FileType: string; FileName: string; FileSize: number;
```

```
    Description: string; UploadedAt: string; UploadedByUsername: string;
```

```
    PatientName?: string; BloodGroup?: string;
```

```
}
```

```

interface PatientSummary {
    patient: Patient & { SummaryNotes: string; NotesUpdatedBy: string; UpdatedAt?: string; };
    visits: Array<{
        VisitID: number; VisitDate: string; ReasonForVisit: string;
        VitalSigns?: string; Notes?: string; Status: string;
        DoctorName: string; Specialty: string;
    }>;
    diagnoses: Array<{
        DiagnosisName: string; Description?: string;
        Severity: string; IsChronic: boolean; VisitDate: string;
    }>;
    prescriptions: Array<{
        MedicineName: string; Dosage: string; Frequency: string; Duration: string;
        Instructions?: string; IsDispensed: boolean; DoctorName: string; VisitDate: string;
    }>;
    files: MedicalFile[];
    chronic_scores?: Array<{ DiagnosisName: string; LatestScore: number; LastUpdated: string;
    DoctorName: string; }>;
}

```

```

interface ChronicEntry {
    ProgressID: number; DiagnosisName: string; RecordDate: string;
    ProgressScore: number; BloodPressure?: string; BloodSugar?: string;
    Weight?: number; Notes?: string; DoctorName: string;
}

```

```

interface LabTest {
    TestID: number; VisitID: number; TestName: string; TestType: string;
    Status: string; Results?: string; ResultDate?: string;
    CreatedAt: string; VisitDate?: string; PatientName?: string;
}

```



```
}
```

```
interface Appointment {
```

```
  AppointmentID: number; RequestedDate: string; TimeSlot: string; Status: string;
```

```
  Reason: string; PatientNotes?: string; ReceptionistNotes?: string; CreatedAt: string;
```

```
  PatientID?: number; PatientName?: string; PhoneNumber?: string; BloodGroup?: string; Gender?:  
string;
```

```
  DoctorID?: number; DoctorName?: string; Specialty?: string;
```

```
  HandledByName?: string;
```

```
}
```

```
interface AdminPatientFull {
```

```
  patient: Record<string,any>;
```

```
  visits: Record<string,any>[];
```

```
  diagnoses: Record<string,any>[];
```

```
  prescriptions: Record<string,any>[];
```

```
  lab_tests: Record<string,any>[];
```

```
  chronic_progress: Record<string,any>[];
```

```
  files: Record<string,any>[];
```

```
}
```

```
interface RegisterForm {
```

```
  username: string; password: string; email: string; name: string;
```

```
  gender: string; dob: string; phone: string; address: string;
```

```
  blood_group: string; emergency_contact: string; emergency_contact_name: string;
```

```
}
```

```
interface RoleEntity {
```

```
  role: string; color: string; glow: string; bg: string; border: string;
```

```
  demoUser: string; demoPass: string; label: string; desc: string;
```

```
  icon: React.ReactNode;
```

```
}
```

```
const ENTITIES: RoleEntity[] = [  
  { role:'Doctor',    color:'#06b6d4', glow:'rgba(6,182,212,.35)', bg:'rgba(6,182,212,.08)',  
    border:'rgba(6,182,212,.3)',  
    demoUser:'dr_anil', demoPass:'doctor123', label:'Clinical Staff', desc:'Manage visits, diagnoses  
& prescriptions',  
    icon:<Stethoscope size={22}/> },  
  { role:'Patient',    color:'#8b5cf6', glow:'rgba(139,92,246,.35)', bg:'rgba(139,92,246,.08)',  
    border:'rgba(139,92,246,.3)',  
    demoUser:'rahul_p', demoPass:'password123', label:'Patient Portal', desc:'View records, reports  
& prescriptions',  
    icon:<User size={22}/> },  
  { role:'Receptionist', color:'#06d6a0', glow:'rgba(6,214,160,.35)', bg:'rgba(6,214,160,.08)',  
    border:'rgba(6,214,160,.3)',  
    demoUser:'rec_priya', demoPass:'recept123', label:'Front Desk', desc:'Manage appointments  
& assign doctors',  
    icon:<Calendar size={22}/> },  
  { role:'Radiologist', color:'#f43f5e', glow:'rgba(244,63,94,.35)', bg:'rgba(244,63,94,.08)',  
    border:'rgba(244,63,94,.3)',  
    demoUser:'rad_priya', demoPass:'scan1234', label:'Radiology Dept', desc:'Upload & manage  
patient scans',  
    icon:<ScanLine size={22}/> },  
  { role:'Pharmacist', color:'#10b981', glow:'rgba(16,185,129,.35)', bg:'rgba(16,185,129,.08)',  
    border:'rgba(16,185,129,.3)',  
    demoUser:'pharm_amit', demoPass:'pharmacy123', label:'Pharmacy', desc:'Dispense & track  
prescriptions',  
    icon:<Pill size={22}/> },  
  { role:'Admin',    color:'#f59e0b', glow:'rgba(245,158,11,.35)', bg:'rgba(245,158,11,.08)',  
    border:'rgba(245,158,11,.3)',  
    demoUser:'admin', demoPass:'admin123', label:'Administration', desc:'Manage staff, patients  
& system data',  
    icon:<Shield size={22}/> },  
];
```

```
// _____
// Helpers
// _____

function decodeJWT(token: string): AuthUser | null {
  try {
    const b64 = token.split('.')[1].replace(/-/g, '+').replace(/_/g, '/');
    return
    JSON.parse(decodeURIComponent(atob(b64).split('').map(c=>'%'+('00'+c.charCodeAt(0).toString(16))
    .slice(-2)).join('')));
  } catch { return null; }
}

function scoreColor(s: number): string {
  return s >= 70 ? '#10b981' : s >= 40 ? '#f59e0b' : '#f43f5e';
}

function sevClass(s?: string): string {
  return s === 'Severe' ? 'sev-badge-severe' : s === 'Moderate' ? 'sev-badge-moderate' : 'sev-badge-mild';
}

function fmtDate(d?: string): string {
  return d ? new Date(d).toLocaleDateString() : '---';
}

function getInitials(name: string): string {
  return name.split(' ').map(w => w[0]).join('').slice(0,2).toUpperCase();
}

function parseVitals(raw?: string): Record<string, string> {
  if (!raw) return {};
  try { return JSON.parse(raw); } catch { return {}; }
}
```

```
}
```

```
// _____
```

```
// ProgressRing SVG Component
```

```
// _____
```

```
function ProgressRing({ score, size = 80, stroke = 8 }: { score: number; size?: number; stroke?: number }) {
```

```
  const r = (size - stroke) / 2;
```

```
  const circ = 2 * Math.PI * r;
```

```
  const off = circ - (score / 100) * circ;
```

```
  const c = scoreColor(score);
```

```
  const fs = Math.round(size * 0.20);
```

```
  return (
```

```
    <div style={{ position: 'relative', width: size, height: size }}>
```

```
      <svg width={size} height={size} style={{ transform: 'rotate(-90deg)' }}>
```

```
        <circle cx={size/2} cy={size/2} r={r} fill="none" stroke="rgba(148,163,184,0.1)"  
strokeWidth={stroke}/>
```

```
        <circle
```

```
          cx={size/2} cy={size/2} r={r} fill="none"
```

```
          stroke={c} strokeWidth={stroke} strokeLinecap="round"
```

```
          strokeDasharray={circ}
```

```
          strokeDashoffset={off}
```

```
          style={{ transition: 'stroke-dashoffset 0.8s cubic-bezier(0.16,1,0.3,1)' }}
```

```
        />
```

```
      </svg>
```

```
    <div style={{ position: 'absolute', inset: 0, display: 'flex', flexDirection: 'column', alignItems: 'center',  
justifyContent: 'center' }}>
```

```
      <span style={{ fontFamily: "'Syne', sans-serif", fontSize: fs, fontWeight: 800, color: c, lineHeight: 1  
}}>{score}%</span>
```

```
    </div>
```

```
  </div>
```

```

);
}

// _____
// ThemeSwitcher Component
// _____

function ThemeSwitcher({ theme, setTheme }: { theme: ThemeKey; setTheme: (k: ThemeKey) => void }) {
  const [open, setOpen] = useState(false);
  const ref = useRef<HTMLDivElement>(null);
  useEffect(() => {
    function h(e: MouseEvent) { if (ref.current && !ref.current.contains(e.target as Node))
setOpen(false); }

    document.addEventListener('mousedown', h);

    return () => document.removeEventListener('mousedown', h);
  }, []);
  const current = THEMES.find(t => t.key === theme)!;

  return (
    <div ref={ref} className="theme-switcher">
      <button className={`theme-toggle-btn ${open ? 'open' : ''}`} onClick={() => setOpen(v => !v)}>
        <Palette size={14}/> {current.label}
        <span className="chevron"><ChevronDown size={12}/></span>
      </button>
      {open && (
        <div className="theme-panel">
          <div className="theme-panel-label">Color Theme</div>
          {THEMES.map(t => (
            <div key={t.key} className={`theme-option ${theme === t.key ? 'active' : ''}`}
              onClick={() => { setTheme(t.key); setOpen(false); }}>
              <span className="theme-swatch" style={{ background: t.swatch }}/>
              {t.icon} {t.label}

```

```

        <span className="checkmark"><CheckCircle size={13}/></span>
      </div>
    )}}
  </div>
}

);
}

// _____
// Modal wrapper
// _____

function Modal({ title, onClose, wide, xlwide, children, extraHeaderActions }:
  { title: string; onClose: ()=>void; wide?: boolean; xlwide?: boolean; children: React.ReactNode;
  extraHeaderActions?: React.ReactNode }) {
  return (
    <div className="modal-overlay" onClick={onClose}>
      <div className={`modal-box ${xlwide ? 'xlwide' : wide ? 'wide' : ''}`} onClick={e =>
        e.stopPropagation()}>
        <div className="modal-header">
          <h3 className="modal-title">{title}</h3>
          <div style={{ display:'flex', alignItems:'center', gap:'0.5rem' }}>
            {extraHeaderActions}
            <button className="modal-close" onClick={onClose}><X size={18}/></button>
          </div>
        </div>
        {children}
      </div>
    </div>
  );
}

```

```
// _____
// Patient Summary Sheet Modal
// _____

function SummaryModal({ patientId, patientName, role, token, onClose }:
  { patientId: number; patientName: string; role: string; token: string; onClose: ()=>void }) {
  const [summary, setSummary] = useState<PatientSummary | null>(null);
  const [loading, setLoading] = useState(true);
  const [err, setErr] = useState("");
  const [notes, setNotes] = useState("");
  const [saving, setSaving] = useState(false);
  const [saveMsg, setSaveMsg] = useState("");
  const [openVisit, setOpenVisit] = useState<number | null>(null);
  const canEdit = role === 'Admin' || role === 'Doctor';

  useEffect(() => {
    fetch(`${API}/patients/${patientId}/summary`, { headers: { Authorization: `Bearer ${token}` } })
      .then(r => r.json())
      .then(d => { setSummary(d); setNotes(d.patient?.SummaryNotes ?? ""); })
      .catch(() => setErr('Failed to load summary'))
      .finally(() => setLoading(false));
  }, [patientId]);

  async function saveNotes() {
    setSaving(true); setSaveMsg("");
    try {
      const res = await fetch(`${API}/patients/${patientId}/summary-notes`, {
        method: 'PUT',
        headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({ notes }),
      });
      const d = await res.json();
    }
  }
}
```

```

    if (!res.ok) throw new Error(d.error);

    setSaveMsg('Notes saved successfully.');
```

if (summary) setSummary({ ...summary, patient: { ...summary.patient, SummaryNotes: notes } });

```

  } catch (e: any) { setSaveMsg(`Error: ${e.message}`); }

  finally { setSaving(false); }
}

```

```

const SectionLabel = ({ text }: { text: string }) => (
  <div className="summary-section-label">{text}</div>
);

```

```

if (loading) return (
  <Modal title="Patient Summary" onClose={onClose} wide>
    <div style={{ textAlign:'center', padding:'3rem', color:'var(--text-dim)' }}>Loading summary...</div>
  </Modal>
);

```

```

if (err || !summary) return (
  <Modal title="Patient Summary" onClose={onClose} wide>
    <div style={{ color:'var(--rose)', padding:'1rem' }}>{err || 'No data'}</div>
  </Modal>
);

```

```

const p = summary.patient;

```

```

return (
  <Modal
    title={`Summary Sheet — ${patientName}`}
    onClose={onClose}
    wide
    extraHeaderActions={

```



```

    <button className="action-btn" style={{ padding:'0.45rem 0.875rem', fontSize:'0.78rem',
marginTop:0 }}
      onClick={() => window.print()}>
         Print
      </button>
    }
  >
  {!canEdit && (
    <div className="summary-readonly-banner">
      <Eye size={14}/> This summary is read-only. Contact your doctor or administrator to update
      clinical notes.
    </div>
  )}

  { /* Demographics */ }
  <div className="summary-section">
    <SectionLabel text="Patient Demographics"/>
    <div className="demographics-grid">
      {[
        ['Name',    p.PatientName],
        ['Blood Group',  p.BloodGroup],
        ['Gender',   p.Gender],
        ['Date of Birth', fmtDate(p.DateOfBirth)],
        ['Phone',    p.PhoneNumber],
        ['Email',    p.Email],
      ].map(([k,v]) => (
        <div key={k} className="demo-cell">
          <div className="demo-cell-label">{k}</div>
          <div className="demo-cell-value" style={{ fontSize: k === 'Email' ? '0.78rem' : undefined
        }}>{v}</div>
        </div>
      ))}
    </div>
  )}

```

```

{p.EmergencyContactName && (
  <div className="demo-cell" style={{ gridColumn: 'span 2' }}>
    <div className="demo-cell-label">Emergency Contact</div>
    <div className="demo-cell-value">{p.EmergencyContactName} —
{p.EmergencyContact}</div>
  </div>
)}

{p.Address && (
  <div className="demo-cell" style={{ gridColumn: 'span 1' }}>
    <div className="demo-cell-label">Address</div>
    <div className="demo-cell-value" style={{ fontSize:'0.82rem' }}>{p.Address}</div>
  </div>
)}
</div>
</div>

{/* Chronic Progress Rings */}
{summary.chronic_scores && summary.chronic_scores.length > 0 && (
  <div className="summary-section">
    <SectionLabel text="Chronic Disease Progress"/>
    <div style={{ background:'var(--raised)', border:'1px solid var(--border)', borderRadius:'var(--r-
md)', padding:'1rem' }}>
      <div className="chronic-rings-container">
        {summary.chronic_scores.map((cs, i) => (
          <div key={i} className="ring-item">
            <ProgressRing score={Math.round(cs.LatestScore)} size={70} stroke={7}/>
            <div className="ring-name">{cs.DiagnosisName}</div>
            <div className="ring-updated">{fmtDate(cs.LastUpdated)}</div>
          </div>
        ))}
      </div>
    </div>
  </div>
)}

```

```

</div>

}}

{ /* Clinical Notes */ }

<div className="summary-section">

  <SectionLabel text="Clinical Summary Notes"/>

  {canEdit ? (

    <div>

      <textarea className="summary-notes-area" rows={5} value={notes}

        onChange={e => setNotes(e.target.value)}

        placeholder="Add clinical summary notes here..."/>

      <div style={{ display:'flex', alignItems:'center', gap:'0.75rem', marginTop:'0.5rem' }}>

        <button onClick={saveNotes} disabled={saving} className="submit-btn"

          style={{ width:'auto', padding:'0.55rem 1.25rem', marginTop:0, display:'flex',
alignItems:'center', gap:'0.4rem' }}>

          <Save size={14}/> {saving ? 'Saving...' : 'Save Notes'}

        </button>

        {saveMsg && <span style={{ fontSize:'0.82rem', color: saveMsg.startsWith('Error') ? 'var(--rose)' : 'var(--emerald)' }}>{saveMsg}</span>}

        {p.NotesUpdatedBy && <span style={{ fontSize:'0.74rem', color:'var(--text-dim)',
marginLeft:'auto' }}>Last by {p.NotesUpdatedBy}</span>}

      </div>

    </div>

  ) : (

    <div style={{ background:'var(--raised)', border:'1px solid var(--border)', borderRadius:'var(--r-md)', padding:'0.875rem 1rem', minHeight:72 }}>

      <p style={{ margin:0, color: notes ? 'var(--text-mid)' : 'var(--text-dim)', fontSize:'0.88rem',
lineHeight:1.6, whiteSpace:'pre-wrap' }}>

        {notes || 'No clinical notes recorded.'}

      </p>

    </div>

  )}

</div>

```

```

{/* Visit Timeline */}

<div className="summary-section">

  <SectionLabel text={`Visit History (${summary.visits.length})`} />

  {summary.visits.length === 0

    ? <p style={{ color:'var(--text-dim)', fontSize:'0.85rem' }}>No visits recorded.</p>

    : summary.visits.map((v, i) => {

      const vitals = parseVitals(v.VitalSigns);

      return (

        <div key={i} className="visit-accordion">

          <div className={`visit-header ${openVisit === i ? 'open' : ''}`}

            onClick={() => setOpenVisit(openVisit === i ? null : i)}>

            <div className="visit-header-left">

              <div className="visit-status-dot"

                style={{ background: v.Status === 'Completed' ? '#10b981' : v.Status === 'In Progress' ?

'#f59e0b' : '#475569' }} />

              <div className="visit-header-info">

                <div className="visit-reason">{v.ReasonForVisit}</div>

                <div className="visit-meta">{v.DoctorName} · {v.Specialty} ·

{fmtDate(v.VisitDate)}</div>

              </div>

            </div>

            <span className={`visit-chevron ${openVisit === i ? 'open' : ''}`}><ChevronDown

size={16}></span>

          </div>

          {openVisit === i && (

            <div className="visit-body">

              {Object.entries(vitals).length > 0 && (

                <>

                  {vitals.bp && <div className="vital-item"><div className="vital-label">BP</div>

<div className="vital-value">{vitals.bp}</div></div>}

                  {vitals.temp && <div className="vital-item"><div className="vital-

label">Temp</div> <div className="vital-value">{vitals.temp}°F</div></div>}

                </>

              )}

            )}

          </div>

        )}

      )}

    )}

  </div>

```

```

        {vitals.pulse && <div className="vital-item"><div className="vital-
label">Pulse</div> <div className="vital-value">{vitals.pulse} bpm</div></div>}

        {vitals.spo2 && <div className="vital-item"><div className="vital-
label">SpO2</div> <div className="vital-value">{vitals.spo2}</div></div>}

        {vitals.weight && <div className="vital-item"><div className="vital-
label">Weight</div> <div className="vital-value">{vitals.weight}</div></div>}

    </>

  )}

  {v.Notes && <div className="visit-notes"><div className="vital-label" style={{
marginBottom:'0.3rem' }}>Clinical Notes</div>{v.Notes}</div>}

</div>

)}

</div>

);

})

}

</div>

```

```

{/* Diagnoses */}

<div className="summary-section">

  <SectionLabel text={`Diagnoses (${summary.diagnoses.length})`} />

  {summary.diagnoses.length === 0

    ? <p style={{ color:'var(--text-dim)', fontSize:'0.85rem' }}>No diagnoses recorded.</p>

    : summary.diagnoses.map((d, i) => (

      <div key={i} className="diagnosis-row">

        <div>

          <div className="diagnosis-name">{d.DiagnosisName}</div>

          {d.Description && <div className="diagnosis-desc">{d.Description}</div>}

          <div className="diagnosis-date">{fmtDate(d.VisitDate)}</div>

        </div>

        <div className="diagnosis-badges">

          <span className={`badge ${sevClass(d.Severity)}`}>{d.Severity}</span>

```

```

    {d.IsChronic && <span className="badge badge-chronic">Chronic</span>}
  </div>
</div>
))
}
</div>

```

```

{/* Prescriptions */}
<div className="summary-section">
  <SectionLabel text={`Prescriptions (${summary.prescriptions.length})`} />
  {summary.prescriptions.length === 0
    ? <p style={{ color:'var(--text-dim)', fontSize:'0.85rem' }}>No prescriptions recorded.</p>
    : (
      <div className="table-container" style={{ marginTop:0 }}>
        <table className="data-table">
          <thead><tr>
            <th>Medicine</th><th>Dosage</th><th>Frequency</th><th>Duration</th><th>Doctor</th><th>Sta
            tus</th>
          </tr></thead>
          <tbody>{summary.prescriptions.map((rx, i) => (
            <tr key={i}>
              <td>
                <strong>{rx.MedicineName}</strong>
                {rx.Instructions && <p style={{ margin:'0.1rem 0 0', fontSize:'0.74rem', color:'var(--text-
                dim)' }}>{rx.Instructions}</p>}
              </td>
              <td>{rx.Dosage}</td><td>{rx.Frequency}</td><td>{rx.Duration}</td>
              <td>{rx.DoctorName}</td>
              <td>
                <span className={`status-badge ${rx.IsDispensed ? 'dispensed' : 'pending'}`}>

```

```
        {rx.IsDispensed ? <><CheckCircle size={12}/> Dispensed</> : <><Clock size={12}/>
Pending</>}
```

```
        </span>
      </td>
    </tr>
  )}</tbody>
</table>
</div>
)
}
</div>
```

```
{/* Files */}

<div className="summary-section">
  <SectionLabel text={`Files & Scans (${summary.files.length})`} />
  {summary.files.length === 0
    ? <p style={{ color:'var(--text-dim)', fontSize:'0.85rem' }}>No files uploaded.</p>
    : (
      <div style={{ display:'grid', gridTemplateColumns:'repeat(2,1fr)', gap:'0.6rem' }}>
        {summary.files.map(f => (
          <div key={f.FileID} style={{ background:'var(--raised)', border:'1px solid var(--border)',
borderRadius:'var(--r-md)', padding:'0.75rem 1rem', display:'flex', alignItems:'center', gap:'0.75rem'
}}>
          <div style={{ width:36, height:36, borderRadius:9, background:'var(--accent-dim)',
border:'1px solid rgba(6,182,212,.2)', display:'flex', alignItems:'center', justifyContent:'center',
color:'var(--accent)', flexShrink:0 }}>
            <FileImage size={16}/>
          </div>
          <div style={{ flex:1, minWidth:0 }}>
            <p style={{ margin:0, fontWeight:700, fontSize:'0.82rem', overflow:'hidden',
textOverflow:'ellipsis', whiteSpace:'nowrap' }}>{f.FileName}</p>
            <p style={{ margin:0, fontSize:'0.72rem', color:'var(--text-dim)' }}>{f.FileType} ·
{f.FileSize/1024/1024}.toFixed(1)} MB · {fmtDate(f.UploadedAt)}</p>
```

```

        </div>

        <a href={`\${API}/files/download/\${f.FileID}`} download style={{ color:'var(--accent)',
display:'flex' }} title="Download"><Download size={15}/></a>

    </div>

    )}
</div>

)
}
</div>
</Modal>

);
}

// _____
// Chronic Progress Modal
// _____

function ChronicProgressModal({ patientId, patientName, token, doctorId, onClose }:
{ patientId: number; patientName: string; token: string; doctorId?: number; onClose: ()=>void }) {
    const [entries, setEntries]    = useState<ChronicEntry[]>([]);
    const [loading, setLoading]    = useState(true);
    const [saving, setSaving]      = useState(false);
    const [msg, setMsg]            = useState("");
    const [selDiag, setSelDiag]    = useState("");
    const [uniqueDiags, setUniqueDiags] = useState<string[]>([]);
    const [form, setForm]          = useState({ diagnosis_name:"", progress_score:50, blood_pressure:"",
notes:"" });

    const load = useCallback(() => {
        fetch(`\${API}/chronic-progress/\${patientId}`, { headers:{ Authorization:`Bearer \${token}` } })
        .then(r => r.json())
        .then((data: ChronicEntry[]) => {
            setEntries(data);

```



```

    const diags = [...new Set(data.map(e => e.DiagnosisName))];
    setUniqueDiags(diags);
    if (diags.length > 0 && !selDiag) setSelDiag(diags[0]);
    setLoading(false);
  })
  .catch(() => setLoading(false));
}, [patientId, token]);

useEffect(() => { load(); }, [load]);

async function save() {
  if (!form.diagnosis_name) return setMsg('Please enter a diagnosis name.');
```

setSaving(true); setMsg('');

```

  try {
    const res = await fetch(`${API}/chronic-progress`, {
      method: 'POST',
      headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ patient_id: patientId, ...form }),
    });
    const d = await res.json();
    if (!res.ok) throw new Error(d.error);
    setMsg('✅ Progress entry saved!');
    setForm(f => ({ ...f, blood_pressure: '', notes: '', progress_score: 50 }));
    load();
  } catch (e: any) { setMsg(`Error: ${e.message}`); }
  finally { setSaving(false); }
}

const filtered = entries.filter(e => !selDiag || e.DiagnosisName === selDiag);

return (
```

```

<Modal title={`  Chronic Progress — ${patientName}`} onClose={onClose} wide>
  <div style={{ display:'grid', gridTemplateColumns:'1fr 1fr', gap:'1.5rem' }}>
    { /* Form */ }

    <div>
      <p style={{ fontSize:'0.72rem', fontWeight:700, textTransform:'uppercase',
letterSpacing:'.08em', color:'var(--text-dim)', marginBottom:'0.875rem' }}>Record New Entry</p>

      <div className="progress-form">
        <div className="form-group">
          <label>Condition / Diagnosis *</label>

          <input type="text" value={form.diagnosis_name} list="diag-list"
            onChange={e => setForm(f => ({...f, diagnosis_name: e.target.value}))}
            placeholder="e.g. Hypertension"/>

          <datalist id="diag-list">{uniqueDiags.map(d => <option key={d} value={d}/>)}</datalist>
        </div>

        <div>
          <label style={{ fontSize:'0.78rem', fontWeight:600, letterSpacing:'.06em',
textTransform:'uppercase', color:'var(--text-mid)', display:'block', marginBottom:'0.5rem' }}>
            Progress Score: <span style={{ color: scoreColor(form.progress_score),
fontFamily:"Syne",sans-serif", fontWeight:800 }}>{form.progress_score}%</span>
          </label>

          <input type="range" className="range-slider" min={0} max={100}
value={form.progress_score}

            onChange={e => setForm(f => ({...f, progress_score: parseInt(e.target.value)}))}/>

          <div className="range-labels"><span>0% — Critical</span><span>100% —
Resolved</span></div>
        </div>

        <div className="progress-score-display">
          <ProgressRing score={form.progress_score} size={80} stroke={8}/>

          <div className="progress-score-label" style={{ marginTop:'0.5rem' }}>
            {form.progress_score >= 70 ? 'Good Progress' : form.progress_score >= 40 ? 'Moderate
Progress' : 'Needs Attention'}
          </div>
        </div>
      </div>
    </div>
  </div>

```

</div>

</div>

<div style={{ display:'grid', gridTemplateColumns:'1fr 1fr', gap:'0.875rem' }}>

<div className="form-group">

<label>Blood Pressure</label>

<input type="text" value={form.blood_pressure} placeholder="e.g. 130/85"

onChange={e => setForm(f => ({...f, blood_pressure: e.target.value}}))}/>

</div>

<div className="form-group">

<label>Score (0–100)</label>

<input type="number" value={form.progress_score} min={0} max={100}

onChange={e => setForm(f => ({...f, progress_score:
Math.min(100,Math.max(0,parseInt(e.target.value) | 0))}}))}/>

</div>

</div>

<div className="form-group">

<label>Clinical Notes</label>

<textarea value={form.notes} rows={3} placeholder="Observations, symptoms,
improvements..."

onChange={e => setForm(f => ({...f, notes: e.target.value}}))}/>

</div>

<button className="submit-btn" onClick={save} disabled={saving}>

<TrendingUp size={15}/> {saving ? 'Saving...' : 'Save Progress Entry'}

</button>

{msg && <p style={{ fontSize:'0.82rem', textAlign:'center', color: msg.startsWith('Error') ? 'var(-
-rose)' : 'var(--emerald)', margin:0 }}>{msg}</p>}

</div>

</div>

```

    { /* History */ }

    <div>

      <div style={{ display:'flex', alignItems:'center', justifyContent:'space-between',
marginBottom:'0.875rem' }}>

        <p style={{ fontSize:'0.72rem', fontWeight:700, textTransform:'uppercase',
letterSpacing:'.08em', color:'var(--text-dim)', margin:0 }}>Progress History</p>

        {uniqueDiags.length > 1 && (

          <select value={selDiag} onChange={e => setSelDiag(e.target.value)}

            style={{ fontSize:'0.78rem', background:'var(--bg-3)', border:'1px solid var(--border)',
borderRadius:'var(--r-sm)', color:'var(--text-mid)', padding:'0.25rem 0.5rem' }}>

              {uniqueDiags.map(d => <option key={d} value={d}>{d}</option>)}

            </select>

          )}

        </div>

      {loading ? (

        <p style={{ color:'var(--text-dim)', fontSize:'0.85rem' }}>Loading...</p>

        ) : filtered.length === 0 ? (

          <div className="empty-state" style={{ padding:'2rem' }}>

            <BarChart2 size={36} style={{ opacity:0.3 }}/>

            <p style={{ color:'var(--text-dim)', fontSize:'0.85rem' }}>No entries yet</p>

          </div>

        ) : (

          <div className="progress-history">

            {filtered.map((e, i) => (

              <div key={e.ProgressID} className="progress-entry">

                <div className="pe-score" style={{ color: scoreColor(e.ProgressScore)
}}>{Math.round(e.ProgressScore)}%</div>

                <div className="pe-info">

                  <div className="pe-note">{e.Notes || 'No notes'}</div>

                  <div className="pe-date">{fmtDate(e.RecordDate)} · {e.DoctorName}</div>

                  {(e.BloodPressure || e.BloodSugar) && (

```

```

        <div className="pe-vitals">

            {e.BloodPressure && `BP: ${e.BloodPressure}`}

            {e.BloodPressure && e.BloodSugar && ' · '}

            {e.BloodSugar && `Sugar: ${e.BloodSugar}`}

        </div>

    )}

</div>

</div>

    )}

</div>

    )}

</div>

</div>

</Modal>

);

}

```

```

// _____

```

```

// Chronic Charts — fetches & renders per-disease line charts

```

```

// _____

```

```

function ChronicChartsInline({ patientId, token }: { patientId: number; token: string }) {

    const [entriesByDiag, setEntriesByDiag] = useState<Record<string, ChronicEntry[]>>({});

    const [loaded, setLoaded] = useState(false);

    useEffect(() => {

        fetch(`${API}/chronic-progress/${patientId}`, { headers: { Authorization: `Bearer ${token}` } })

        .then(r => r.json())

        .then((data: ChronicEntry[]) => {

            const grouped: Record<string, ChronicEntry[]> = {};

            data.forEach(e => {

                if (!grouped[e.DiagnosisName]) grouped[e.DiagnosisName] = [];
            });
        });
    });
}

```

```

        grouped[e.DiagnosisName].push(e);
    });
    setEntriesByDiag(grouped);
    setLoaded(true);
  })
  .catch(() => setLoaded(true));
}, [patientId, token]);

if (!loaded || Object.keys(entriesByDiag).length === 0) return null;

return (
  <div style={{ display: 'flex', flexDirection: 'column', gap: '0.75rem', marginTop: '0.75rem' }}>
    {Object.entries(entriesByDiag).map(([diag, entries]) =>
      entries.length >= 2 ? (
        <ChronicLineChart key={diag} diagName={diag} entries={entries}/>
      ) : null
    )}
  </div>
);
}

// _____
// Doctor Patient Detail Panel
// _____

function PatientDetailPanel({ patient, records, token, role, onOpenSummary, onOpenProgress }:
  { patient: Patient & { PatientID: number }; records: DoctorRecord[]; token: string; role: string;
    onOpenSummary: () => void; onOpenProgress: () => void }) {
  const [openVisit, setOpenVisit] = useState<number | null>(null);

  const patientRecords = records.filter(r => r.PatientID === patient.PatientID || r.PatientName ===
    patient.PatientName);

```

```
const age = new Date().getFullYear() - new Date(patient.DateOfBirth).getFullYear();
```

```
// Derived: unique visits
```

```
const visitMap = new Map<number, DoctorRecord>();
```

```
patientRecords.forEach(r => { if (r.VisitID && !visitMap.has(r.VisitID)) visitMap.set(r.VisitID, r); });
```

```
const visits = [...visitMap.values()];
```

```
// Visit frequency per month
```

```
const freqMap: Record<string, number> = {};
```

```
visits.forEach(v => {
```

```
    const d = new Date(v.VisitDate);
```

```
    const k = `${d.getMonth()+1}/${String(d.getFullYear()).slice(2)}`;
```

```
    freqMap[k] = (freqMap[k] || 0) + 1;
```

```
});
```

```
const freqEntries = Object.entries(freqMap).slice(-6);
```

```
const maxFreq = Math.max(...freqEntries.map(([c]) => c), 1);
```

```
// Unique diagnoses
```

```
const diagMap = new Map<string, DoctorRecord>();
```

```
patientRecords.forEach(r => { if (r.DiagnosisName && !diagMap.has(r.DiagnosisName))  
diagMap.set(r.DiagnosisName, r); });
```

```
const diagnoses = [...diagMap.values()];
```

```
const chronicDiags = diagnoses.filter(d => d.IsChronic);
```

```
// Unique prescriptions
```

```
const rxMap = new Map<string, DoctorRecord>();
```

```
patientRecords.forEach(r => { if (r.MedicineName && !rxMap.has(r.MedicineName))  
rxMap.set(r.MedicineName, r); });
```

```
const prescriptions = [...rxMap.values()];
```

```

return (
  <>
    { /* Header */ }
    <div className="patient-detail-header">
      <div className="patient-detail-identity">
        <div className="patient-avatar">{getInitials(patient.PatientName)}</div>
        <div>
          <div className="patient-detail-name">{patient.PatientName}</div>
          <div className="patient-detail-meta">{patient.Gender} · {age} yrs · DOB:
{fmtDate(patient.DateOfBirth)}</div>
          <div style={{ display:'flex', gap:'0.35rem', marginTop:'0.35rem', flexWrap:'wrap' }}>
            <span className="badge badge-blood">{patient.BloodGroup}</span>
            {chronicDiags.length > 0 && <span className="badge badge-chronic">Ⓢ Chronic</span>}
            <span className="badge badge-accent">{visits.length} Visit{visits.length!==1?'s':''}</span>
          </div>
        </div>
      </div>
    </div>
    <div className="patient-action-btns">
      {chronicDiags.length > 0 && (
        <button className="pab pab-progress" onClick={onOpenProgress}>
          <TrendingUp size={13}/> Record Progress
        </button>
      )}
      <button className="pab pab-summary" onClick={onOpenSummary}>
        <BookOpen size={13}/> Summary Sheet
      </button>
    </div>
  </div>

  { /* Info Grid */ }
  <div className="info-grid-3">

```



```

<div className="info-card">

  <div className="info-card-title">Contact</div>

  <div className="kv-pair"><div className="kv-label">Phone</div><div className="kv-value">{patient.PhoneNumber}</div></div>

  <div className="kv-pair"><div className="kv-label">Email</div><div className="kv-value" style={{ fontSize:'0.78rem' }}>{patient.Email}</div></div>

  <div className="kv-pair"><div className="kv-label">Address</div><div className="kv-value">{patient.Address}</div></div>

</div>

<div className="info-card">

  <div className="info-card-title">Emergency</div>

  <div className="kv-pair"><div className="kv-label">Name</div><div className="kv-value">{patient.EmergencyContactName || '—'}</div></div>

  <div className="kv-pair"><div className="kv-label">Phone</div><div className="kv-value">{patient.EmergencyContact || '—'}</div></div>

  <div className="kv-pair"><div className="kv-label">Blood</div><div className="kv-value" style={{ color:'var(--rose)', fontWeight:700 }}>{patient.BloodGroup}</div></div>

</div>

<div className="info-card">

  <div className="info-card-title">Visit Frequency</div>

  <div className="freq-bars">

    {freqEntries.map(([mo, cnt]) => (

      <div key={mo} className="freq-bar-group">

        <div className="freq-bar" style={{ height: Math.max(6, (cnt/maxFreq)*46) }}/>

        <span className="freq-label">{mo}</span>

      </div>

    ))}

    {freqEntries.length === 0 && <p style={{ fontSize:'0.75rem', color:'var(--text-dim)', margin:'auto' }}>No data</p>}

  </div>

  <p style={{ fontSize:'0.75rem', color:'var(--text-dim)', marginTop:'0.5rem' }}>{visits.length} total visit{visits.length!==1?'s':''}</p>

</div>

</div>

```

```

    { /* Chronic Progress Rings */ }

    { chronicDiags.length > 0 && (
      <div className="section-block">

        <div className="section-header"><div className="section-title">Chronic Disease
        Progress</div></div>

        <div className="info-card" style={{ display:'flex', alignItems:'flex-start', gap:'1.5rem',
        flexWrap:'wrap' }}>

          <div className="chronic-rings-container">

            { chronicDiags.map((d, i) => (
              <div key={i} className="ring-item">

                <ProgressRing score={50} size={72} stroke={7}/>

                <div className="ring-name">{d.DiagnosisName}</div>

                <div className="ring-updated">{fmtDate(d.VisitDate)}</div>

              </div>

            )) }

          </div>

          <button className="pab pab-progress" onClick={onOpenProgress} style={{ alignSelf:'center'
        }}>

            <TrendingUp size={13}/> Record Progress

          </button>

        </div>

        { /* Inline chart per chronic condition */ }

        <ChronicChartsInline patientId={patient.PatientID} token={token}/>

      </div>

    ) }

```

```

    { /* Diagnoses */ }

    <div className="section-block">

      <div className="section-header"><div className="section-title">Diagnoses
      ({diagnoses.length})</div></div>

      {diagnoses.length === 0

```

```

? <p style={{ color:'var(--text-dim)', fontSize:'0.85rem' }}>No diagnoses recorded.</p>
: diagnoses.map((d, i) => (
  <div key={i} className="diagnosis-row">
    <div>
      <div className="diagnosis-name">{d.DiagnosisName}</div>
      {d.DiagnosisDesc && <div className="diagnosis-desc">{d.DiagnosisDesc}</div>}
      <div className="diagnosis-date">{fmtDate(d.VisitDate)}</div>
    </div>
    <div className="diagnosis-badges">
      {d.Severity && <span className={`badge ${sevClass(d.Severity)}`}>{d.Severity}</span>}
      {d.IsChronic && <span className="badge badge-chronic">Chronic</span>}
    </div>
  </div>
))
}
</div>

```

```

{/* Visit History */}
<div className="section-block">
  <div className="section-header"><div className="section-title">Visit History
({visits.length})</div></div>
  {visits.length === 0
    ? <p style={{ color:'var(--text-dim)', fontSize:'0.85rem' }}>No visits recorded.</p>
    : visits.map((v, i) => {
      const vitals = parseVitals(v.VitalSigns);
      return (
        <div key={i} className="visit-accordion">
          <div className={`visit-header ${openVisit === i ? 'open' : ''}`}
            onClick={() => setOpenVisit(openVisit === i ? null : i)}>
            <div className="visit-header-left">
              <div className="visit-status-dot"

```

```

        style={{ background: v.VisitStatus==='Completed' ? '#10b981' : v.VisitStatus==='In
Progress' ? '#f59e0b' : '#475569' }}/>

        <div className="visit-header-info">

            <div className="visit-reason">{v.ReasonForVisit}</div>

            <div className="visit-meta">{v.DoctorName} · {v.Specialty} ·
{fmtDate(v.VisitDate)}</div>

        </div>

    </div>

    <span className={`visit-chevron ${openVisit === i ? 'open' : ''}`}><ChevronDown
size={16}/></span>

    </div>

    {openVisit === i && (

        <div className="visit-body">

            {vitals.bp && <div className="vital-item"><div className="vital-label">Blood
Pressure</div><div className="vital-value">{vitals.bp}</div></div>}

            {vitals.temp && <div className="vital-item"><div className="vital-
label">Temperature</div><div className="vital-value">{vitals.temp}°F</div></div>}

            {vitals.pulse && <div className="vital-item"><div className="vital-
label">Pulse</div><div className="vital-value">{vitals.pulse} bpm</div></div>}

            {vitals.spo2 && <div className="vital-item"><div className="vital-
label">SpO2</div><div className="vital-value">{vitals.spo2}</div></div>}

            {v.VisitNotes && <div className="visit-notes"><div className="vital-label" style={{
marginBottom:'0.3rem' }}>Clinical Notes</div>{v.VisitNotes}</div>}

        </div>

    )}

    </div>

    );

    })

    }

</div>

    { /* Prescriptions */ }

    <div className="section-block">

```

```

    <div className="section-header"><div className="section-title">Prescriptions
    {{prescriptions.length}}</div></div>

    {prescriptions.length === 0
      ? <p style={{ color:'var(--text-dim)', fontSize:'0.85rem' }}>No prescriptions recorded.</p>
      : (
        <div className="table-container">
          <table className="data-table">

            <thead><tr><th>Medicine</th><th>Dosage</th><th>Frequency</th><th>Duration</th><th>Status<
            /th></tr></thead>

            <tbody>{prescriptions.map((rx, i) => (
              <tr key={i}>
                <td><strong>{rx.MedicineName}</strong>{rx.Instructions && <p style={{ margin:'0.1rem
                0 0', fontSize:'0.74rem', color:'var(--text-dim)' }}>{rx.Instructions}</p>}</td>
                <td>{rx.Dosage}</td><td>{rx.Frequency}</td><td>{rx.Duration}</td>
                <td><span className={`status-badge ${rx.IsDispensed ? 'dispensed' : 'pending'}`}>
                  {rx.IsDispensed ? <><CheckCircle size={12}/> Dispensed</> : <><Clock size={12}/>
                  Pending</>}
                </span></td>
              </tr>
            ))}</tbody>
          </table>
        </div>
      )
    }
  </div>
</>
);
}

// _____
// Chronic Disease Line Chart (SVG)
// _____

```

```

function ChronicLineChart({ entries, diagName }: { entries: ChronicEntry[]; diagName: string }) {

  const sorted = [...entries].sort((a, b) => new Date(a.RecordDate).getTime() - new
Date(b.RecordDate).getTime());

  if (sorted.length === 0) return <p style={{ color:'var(--text-dim)', fontSize:'0.85rem' }}>No data to
chart.</p>;

  const W = 480; const H = 160; const PL = 36; const PR = 16; const PT = 12; const PB = 28;

  const iW = W - PL - PR; const iH = H - PT - PB;

  const scores = sorted.map(e => e.ProgressScore);

  const minS = 0; const maxS = 100;

  const toX = (i: number) => PL + (sorted.length < 2 ? iW / 2 : (i / (sorted.length - 1)) * iW);

  const toY = (s: number) => PT + iH - ((s - minS) / (maxS - minS)) * iH;

  const pathD = sorted.map((e, i) => `${i === 0 ? 'M' : 'L'} ${toX(i).toFixed(1)}
${toY(e.ProgressScore).toFixed(1)}`).join(' ');

  const areaD = `${pathD} L ${toX(sorted.length - 1).toFixed(1)} ${toY((PT + iH).toFixed(1))} L
${toX(0).toFixed(1)} ${toY((PT + iH).toFixed(1))} Z`;

  const gridLines = [0, 25, 50, 75, 100];

  return (

    <div style={{ background:'var(--raised)', border:'1px solid var(--border)', borderRadius:'var(--r-md)',
padding:'0.875rem 1rem' }}>

      <p style={{ margin:'0 0 0.75rem', fontSize:'0.72rem', fontWeight:700, textTransform:'uppercase',
letterSpacing:'.08em', color:'var(--text-dim)' }}>

        {diagName} — Progress Over Time

      </p>

      <svg viewBox={`0 0 ${W} ${H}`} style={{ width:'100%', height:'auto', overflow:'visible' }}>

        <defs>

          <linearGradient id="chartArea" x1="0" y1="0" x2="0" y2="1">

            <stop offset="0%" stopColor="var(--accent)" stopOpacity="0.22"/>

            <stop offset="100%" stopColor="var(--accent)" stopOpacity="0.02"/>


```

```

    </linearGradient>

</defs>

{/* Grid lines */}

{gridLines.map(g => (

  <g key={g}>

    <line x1={PL} y1={toY(g)} x2={W - PR} y2={toY(g)} stroke="rgba(148,163,184,0.08)"
strokeWidth="1"/>

    <text x={PL - 4} y={toY(g) + 4} textAnchor="end" fontSize="9" fill="var(--text-dim)">{g}</text>

  </g>

)
)}}

{/* Area fill */}

<path d={areaD} fill="url(#chartArea)"/>

{/* Line */}

<path d={pathD} fill="none" stroke="var(--accent)" strokeWidth="2.5" strokeLinecap="round"
strokeLinejoin="round"/>

{/* Data points */}

{sorted.map((e, i) => (

  <g key={i}>

    <circle cx={toX(i)} cy={toY(e.ProgressScore)} r="5" fill="var(--bg-2)"
stroke={scoreColor(e.ProgressScore)} strokeWidth="2.5"/>

    {/* Tooltip label on hover via title */}

    <title>`${fmtDate(e.RecordDate)}: ${e.ProgressScore}%`</title>

    {/* Date label at bottom */}

    <text x={toX(i)} y={H - 4} textAnchor="middle" fontSize="8" fill="var(--text-dim)">

      {new Date(e.RecordDate).toLocaleDateString('en-GB', { day:'2-digit', month:'short' })}

    </text>

  </g>

)
)}}

```

```

    { /* Score label above last point */}

    {sorted.length > 0 && (

      <text

        x={toX(sorted.length - 1)}

        y={toY(sorted[sorted.length - 1].ProgressScore) - 9}

        textAnchor="middle"

        fontSize="10"

        fontWeight="700"

        fill={scoreColor(sorted[sorted.length - 1].ProgressScore)}

      >

        {sorted[sorted.length - 1].ProgressScore}%

      </text>

    )}

  </svg>

  { /* Legend row */}

  <div style={{ display:'flex', gap:'1rem', marginTop:'0.5rem', flexWrap:'wrap' }}>

    {sorted.map((e, i) => (

      <div key={i} style={{ display:'flex', alignItems:'center', gap:'0.35rem', fontSize:'0.72rem',
color:'var(--text-dim)' }}>

        <div style={{ width:8, height:8, borderRadius:'50%', background: scoreColor(e.ProgressScore),
flexShrink:0 }}/>

        {fmtDate(e.RecordDate)}: <strong style={{ color: scoreColor(e.ProgressScore)
}}>{e.ProgressScore}%</strong>

      </div>

    )})

  </div>

</div>

);
}

```



```
// _____
// Lab Tests Modal — view + add tests for a visit
// _____

function LabTestsModal({ token, visits, onClose }:
  { token: string; visits: Visit[]; onClose: ()=>void }) {
  const [tests, setTests] = useState<LabTest[]>([]);
  const [loading, setLoading] = useState(true);
  const [saving, setSaving] = useState(false);
  const [msg, setMsg] = useState("");
  const [updatingId, setUpdatingId] = useState<number | null>(null);
  const [form, setForm] = useState({ visit_id:"", test_name:"", test_type:'Blood Test' });
  const [resultForm, setResultForm] = useState({ results:"", result_date: new
Date().toISOString().split('T')[0] });

  const TEST_TYPES = ['Blood Test','Urine Test','X-Ray','MRI','CT
Scan','ECG','Echo','Biopsy','Culture','Other'];

  const load = useCallback(() => {
    fetch(`${API}/lab-tests`, { headers:{ Authorization:`Bearer ${token}` } })
      .then(r => r.json())
      .then(d => { setTests(Array.isArray(d) ? d : []); setLoading(false); })
      .catch(() => setLoading(false));
  }, [token]);

  useEffect(() => { load(); }, [load]);

  async function addTest() {
    if (!form.visit_id || !form.test_name) return setMsg('Visit and test name are required. ');
    setSaving(true); setMsg("");
    try {
      const res = await fetch(`${API}/lab-tests`, {
        method:'POST',
        headers:{ Authorization:`Bearer ${token}`, 'Content-Type':'application/json' },

```

```

    body: JSON.stringify({ visit_id: parseInt(form.visit_id), test_name: form.test_name, test_type:
form.test_type })),
  });
  const d = await res.json();
  if (!res.ok) throw new Error(d.error);
  setMsg('✅ Test ordered!');
  setForm(f => ({ ...f, test_name: "", visit_id: "" }));
  load();
} catch(e:any) { setMsg(`Error: ${e.message}`); }
finally { setSaving(false); }
}

```

```

async function saveResult(testId: number) {
  setSaving(true);
  try {
    const res = await fetch(`${API}/lab-tests/${testId}/result`, {
      method: 'PUT',
      headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ results: resultForm.results, result_date: resultForm.result_date }),
    });
    const d = await res.json();
    if (!res.ok) throw new Error(d.error);
    setUpdatingId(null);
    load();
  } catch(e:any) { alert(e.message); }
  finally { setSaving(false); }
}

```

```

const statusColor = (s: string) => s === 'Completed' ? 'var(--emerald)' : s === 'In Progress' ? 'var(--amber)' : s === 'Cancelled' ? 'var(--rose)' : 'var(--text-dim)';

```

```

const statusBg = (s: string) => s === 'Completed' ? 'var(--emerald-dim)' : s === 'In Progress' ? 'var(--amber-dim)' : s === 'Cancelled' ? 'var(--rose-dim)' : 'var(--bg-3)';

```

return (

<Modal title="🧪 Lab Tests Management" onClose={onClose} wide>

 {/* Order new test */}

 <div style={{ background:'var(--raised)', border:'1px solid var(--border)', borderRadius:'var(--r-md)', padding:'1rem', marginBottom:'1.25rem' }}>

 <p style={{ fontSize:'0.72rem', fontWeight:700, textTransform:'uppercase', letterSpacing:'.08em', color:'var(--text-dim)', margin:'0 0 0.875rem' }}>Order New Test</p>

 <div style={{ display:'grid', gridTemplateColumns:'1fr 1fr 1fr auto', gap:'0.75rem', alignItems:'flex-end' }}>

 <div className="form-group" style={{ margin:0 }}>

 <label>Visit *</label>

 <select value={form.visit_id} onChange={e => setForm(f => ({...f, visit_id: e.target.value}))}>

 <option value="">Select Visit</option>

 {visits.map(v => <option key={v.VisitID} value={v.VisitID}>#{v.VisitID} — {v.PatientName} ({fmtDate(v.VisitDate)})</option>)}

 </select>

 </div>

 <div className="form-group" style={{ margin:0 }}>

 <label>Test Name *</label>

 <input type="text" value={form.test_name} onChange={e => setForm(f => ({...f, test_name: e.target.value}))} placeholder="e.g. CBC, TSH, HbA1c"/>

 </div>

 <div className="form-group" style={{ margin:0 }}>

 <label>Test Type</label>

 <select value={form.test_type} onChange={e => setForm(f => ({...f, test_type: e.target.value}))>

 {TEST_TYPES.map(t => <option key={t}>{t}</option>)}

 </select>

 </div>

 <button className="action-btn" onClick={addTest} disabled={saving} style={{ marginTop:0, padding:'0.7rem 1rem', whiteSpace:'nowrap', display:'flex', alignItems:'center', gap:'0.4rem' }}>

 <Plus size={14}/> Order

```

    </button>

  </div>

  {msg && <p style={{ margin:'0.5rem 0 0', fontSize:'0.82rem', color: msg.startsWith('Error') ?
'var(--rose)' : 'var(--emerald)' }}>{msg}</p>}

  </div>

  { /* Tests list */ }

  {loading ? (

    <p style={{ color:'var(--text-dim)', textAlign:'center', padding:'2rem' }}>Loading tests...</p>

    ) : tests.length === 0 ? (

      <div className="empty-state" style={{ padding:'2rem' }}>

        <FlaskConical size={40} style={{ opacity:.3 }}/>

        <p>No lab tests ordered yet.</p>

      </div>

    ) : (

      <div className="table-container">

        <table className="data-table">

          <thead><tr><th>Test</th><th>Type</th><th>Patient</th><th>Ordered</th><th>Status</th><th>Re
sults</th><th>Action</th></tr></thead>

          <tbody>

            {tests.map(t => (

              <React.Fragment key={t.TestID}>

                <tr>

                  <td><strong>{t.TestName}</strong></td>

                  <td><span style={{ fontSize:'0.76rem', padding:'0.15rem 0.5rem', borderRadius:5,
background:'var(--accent-dim)', color:'var(--accent)' }}>{t.TestType}</span></td>

                  <td style={{ fontSize:'0.82rem' }}>{t.PatientName || '—'}</td>

                  <td style={{ fontSize:'0.82rem' }}>{fmtDate(t.CreatedAt)}</td>

                  <td>

                    <span style={{ fontSize:'0.76rem', padding:'0.2rem 0.6rem', borderRadius:5, background:
statusBg(t.Status), color: statusColor(t.Status), fontWeight:600 }}>

                      {t.Status}

```

```

    </span>

  </td>

  <td style={{ fontSize:'0.82rem', maxWidth:200 }}>

    {t.Results ? <span title={t.Results} style={{ color:'var(--text-mid)'
}}>{t.Results.slice(0,40)}{t.Results.length>40?'...':''}</span> : <span style={{ color:'var(--text-dim)'
}}>Pending</span>}

  </td>

  <td>

    {t.Status !== 'Completed' && t.Status !== 'Cancelled' && (

      <button onClick={() => { setUpdatingId(updatingId === t.TestID ? null : t.TestID);
setResultForm({ results:'', result_date: new Date().toISOString().split('T')[0] }); }}

        style={{ padding:'0.3rem 0.65rem', background:'var(--emerald-dim)', border:'1px solid
rgba(16,185,129,.25)', borderRadius:6, color:'var(--emerald)', fontSize:'0.76rem', fontWeight:700,
cursor:'pointer', display:'flex', alignItems:'center', gap:'0.3rem' }}>

          <CheckCircle size={12}/> Add Result

        </button>

      )}

  </td>

</tr>

{updatingId === t.TestID && (

  <tr>

    <td colSpan={7} style={{ padding:'0.75rem 1rem', background:'var(--accent-dim)' }}>

      <div style={{ display:'grid', gridTemplateColumns:'1fr 160px auto', gap:'0.75rem',
alignItems:'flex-end' }}>

        <div className="form-group" style={{ margin:0 }}>

          <label>Results *</label>

          <textarea rows={2} value={resultForm.results} onChange={e => setResultForm(f =>
({...f, results: e.target.value}))} placeholder="Enter test results / findings..." />

        </div>

        <div className="form-group" style={{ margin:0 }}>

          <label>Result Date</label>

          <input type="date" value={resultForm.result_date} onChange={e => setResultForm(f
=> ({...f, result_date: e.target.value}))} />

        </div>

      </div>

    </td>

  </tr>

)

```

```

        <div style={{ display:'flex', gap:'0.5rem' }}>

            <button className="submit-btn" onClick={() => saveResult(t.TestID)}
disabled={saving || !resultForm.results} style={{ width:'auto', padding:'0.6rem 1rem', marginTop:0
}}>

                <Save size={13}/> Save

            </button>

            <button onClick={() => setUpdatingId(null)} style={{ padding:'0.6rem 0.875rem',
background:'var(--bg-3)', border:'1px solid var(--border)', borderRadius:'var(--r-sm)', color:'var(--text-
dim)', cursor:'pointer' }}>

                <X size={13}/>

            </button>

        </div>

    </div>

</td>

</tr>

)}

</React.Fragment>

)}}

</tbody>

</table>

</div>

)}

</Modal>

);

}

```

```

// _____
// BookAppointmentModal — Patient books an appointment
// _____

```

```

function BookAppointmentModal({ token, doctors, onClose, onSave }:
{ token: string; doctors: Doctor[]; onClose: ()=>void; onSave: ()=>void }) {
    const tomorrow = new Date(); tomorrow.setDate(tomorrow.getDate()+1);

```

```
const minDate = tomorrow.toISOString().split('T')[0];
```

```
const [form, setForm] = useState({  
  requested_date: minDate,  
  time_slot: '10:00 AM',  
  reason: '',  
  patient_notes: '',  
  preferred_doctor_id: '',  
});
```

```
const [saving, setSaving] = useState(false);
```

```
const [msg, setMsg] = useState('');
```

```
const TIME_SLOTS = ['09:00 AM','09:30 AM','10:00 AM','10:30 AM','11:00 AM','11:30 AM',  
  '12:00 PM','02:00 PM','02:30 PM','03:00 PM','03:30 PM','04:00 PM','04:30 PM'];
```

```
async function submit(e: React.FormEvent) {  
  e.preventDefault();  
  if (!form.reason.trim()) return setMsg('Please describe your reason for the visit.');
```

```
  setSaving(true); setMsg('');
```

```
  try {  
    const res = await fetch(`${API}/appointments`, {  
      method: 'POST',  
      headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },  
      body: JSON.stringify({  
        requested_date: form.requested_date,  
        time_slot: form.time_slot,  
        reason: form.reason,  
        patient_notes: form.patient_notes,  
        preferred_doctor_id: form.preferred_doctor_id || null,  
      }},  
    );  
  };
```

```

const d = await res.json();

if (!res.ok) throw new Error(d.error);

setMsg('✅ Appointment request sent! Our receptionist will confirm shortly.');
```

setTimeout(() => { onSave(); onClose(); }, 1800);

```

} catch(e:any) { setMsg(`Error: ${e.message}`); }

finally { setSaving(false); }

}

return (

  <Modal title="📅 Book an Appointment" onClose={onClose}>

    <form onSubmit={submit} className="data-form" style={{ padding:0, background:'none',
    boxShadow:'none' }}>

      {/* Info banner */}

      <div style={{ display:'flex', alignItems:'flex-start', gap:'0.625rem', padding:'0.75rem 1rem',
      background:'var(--accent-dim)', border:'1px solid var(--border)', borderRadius:'var(--r-md)',
      marginBottom:'1.25rem', fontSize:'0.82rem', color:'var(--text-mid)', lineHeight:1.5 }}>

        <Calendar size={15} style={{ color:'var(--accent)', flexShrink:0, marginTop:1 }}/>

        Your request will be reviewed by our front desk team who will assign you the best available
        doctor and confirm your appointment.

      </div>

      <div className="form-row">

        <div className="form-group">

          <label>Preferred Date *</label>

          <input type="date" value={form.requested_date} min={minDate}

            onChange={e => setForm(f => ({...f, requested_date: e.target.value}))} required/>

          </div>

          <div className="form-group">

            <label>Preferred Time Slot</label>

            <select value={form.time_slot} onChange={e => setForm(f => ({...f, time_slot:
            e.target.value}))}>

              {TIME_SLOTS.map(t => <option key={t}>{t}</option>)}

            </select>

          </div>

        </div>

      </div>
    </form>
  </Modal>
)

```



```

    </select>

  </div>

</div>

<div className="form-group">
  <label>Reason for Visit *</label>

  <input type="text" value={form.reason} placeholder="e.g. General check-up, chest pain, follow-
up..."

    onChange={e => setForm(f => ({...f, reason: e.target.value}))} required/>

</div>

<div className="form-group">
  <label>Describe Your Symptoms <span style={{ color:'var(--text-dim)', fontWeight:400
}}>(optional)</span></label>

  <textarea rows={3} value={form.patient_notes}

    placeholder="Describe how you've been feeling, duration of symptoms, any medications
you're taking..."

    onChange={e => setForm(f => ({...f, patient_notes: e.target.value}))}/>

</div>

<div className="form-group">
  <label>Preferred Doctor <span style={{ color:'var(--text-dim)', fontWeight:400 }}>(optional —
we'll try to accommodate)</span></label>

  <select value={form.preferred_doctor_id} onChange={e => setForm(f => ({...f,
preferred_doctor_id: e.target.value}))}>

    <option value="">No preference — assign best available</option>

    {doctors.map(d => <option key={d.DoctorID} value={d.DoctorID}>{d.DoctorName} —
{d.Specialty}</option>)}

  </select>

</div>

{msg && (

```

```

    <div style={{ padding:'0.75rem 1rem', borderRadius:'var(--r-md)', marginBottom:'1rem',
fontSize:'0.84rem',

    background: msg.startsWith('Error') ? 'var(--rose-dim)' : 'var(--emerald-dim)',

    color: msg.startsWith('Error') ? 'var(--rose)' : 'var(--emerald)',

    border: `1px solid ${msg.startsWith('Error') ? 'rgba(244,63,94,.25)' : 'rgba(16,185,129,.25)'}`,

    display:'flex', alignItems:'center', gap:'0.5rem' }}>

    {msg.startsWith('Error') ? <AlertCircle size={14}/> : <CheckCircle size={14}/>} {msg}

  </div>

)}

    <button type="submit" className="submit-btn" disabled={saving} style={{ marginTop:'0.5rem',
display:'flex', alignItems:'center', justifyContent:'center', gap:'0.5rem' }}>

    <Calendar size={15}/> {saving ? 'Submitting...' : 'Submit Appointment Request'}

  </button>

</form>

</Modal>

);

}

// _____

// PatientProfileDashboard — Full 360° patient home screen

// Layout: profile hero | appointments | records + prescriptions + files

// _____

function PatientProfileDashboard({ user, token, records, medFiles, doctors, onBookAppointment,
onOpenSummary, onViewRecords, onViewFiles, onViewAppointments }):

{ user: AuthUser; token: string; records: any[]; medFiles: MedicalFile[]; doctors: Doctor[];

  onBookAppointment: ()=>void; onOpenSummary: ()=>void;

  onViewRecords: ()=>void; onViewFiles: ()=>void; onViewAppointments: ()=>void }} {

const [patientDetail, setPatientDetail] = useState<any>(null);

const [appointments, setAppointments] = useState<Appointment[]>([]);

const [loadingDetail, setLoadingDetail] = useState(true);

```

```

useEffect(() => {
  if (!user.patient_id) return;
  fetch(`${API}/patients/${user.patient_id}/summary`, { headers: { Authorization: `Bearer ${token}` } })
    .then(r=>r.json()).then(d=>setPatientDetail(d)).catch(()=>{}).finally(()=>setLoadingDetail(false));
  fetch(`${API}/appointments`, { headers: { Authorization: `Bearer ${token}` } })
    .then(r=>r.json()).then(d=>{ if(Array.isArray(d)) setAppointments(d); }).catch(()=>{});
}, [user.patient_id, token]);

```

```

const p = patientDetail?.patient || {};
const visits = patientDetail?.visits || [];
const diagnoses = patientDetail?.diagnoses || [];
const prescriptions = patientDetail?.prescriptions || [];
const initials = (p.PatientName || user.username || 'P').split('')
  .map((w:string)=>w[0]).join('').slice(0,2).toUpperCase();
const age = p.DateOfBirth ? new Date().getFullYear() - new Date(p.DateOfBirth).getFullYear() : null;

```

```

const statusColor = (s:string) => s==='Confirmed'?`var(--emerald):s==='Cancelled'?`var(--rose):s==='Completed'?`var(--accent):`var(--amber);

```

```

async function downloadFile(fid: number, name: string) {
  const res = await fetch(`${API}/files/download/${fid}`, { headers: { Authorization: `Bearer ${token}` } });
  if (!res.ok) return alert('Download failed');
  const a = document.createElement('a'); a.href = URL.createObjectURL(await res.blob());
  a.download = name;
  document.body.appendChild(a); a.click(); URL.revokeObjectURL(a.href);
  document.body.removeChild(a);
}

```

```

return (
  <div style={{ animation:'gridReveal 0.5s var(--ease-out) both', display:'flex', flexDirection:'column', gap:'1.25rem' }}>

```

```

    { /* — TOP ROW: Profile Hero + Appointments — */ }

    <div style={{ display:'grid', gridTemplateColumns:'1fr 1.4fr', gap:'1.25rem', alignItems:'start' }}>

        { /* Profile Hero Card */ }

        <div style={{ background:'var(--card-bg)', backdropFilter:'blur(20px)', border:'1px solid var(--border-glass)', borderRadius:'var(--r-xl)', overflow:'hidden', boxShadow:'var(--shadow-card)' }}>

            { /* Top gradient banner */ }

            <div style={{ height:72, background:'linear-gradient(135deg, var(--accent-dim) 0%, var(--violet-dim) 100%)', position:'relative', overflow:'hidden' }}>

                <div style={{ position:'absolute', inset:0, background:'linear-gradient(135deg, rgba(0,212,255,0.15), rgba(155,125,255,0.12))' }}/>

                { /* Scanline effect */ }

                <div style={{ position:'absolute', inset:0, backgroundImage:'repeating-linear-gradient(0deg, transparent, transparent 3px, rgba(0,212,255,0.03) 3px, rgba(0,212,255,0.03) 4px)' }}/>

            </div>

            <div style={{ padding:'0 1.5rem 1.5rem' }}>

                { /* Avatar overlapping banner */ }

                <div style={{ marginTop:-36, marginBottom:'0.875rem', display:'flex', alignItems:'flex-end', justifyContent:'space-between' }}>

                    <div style={{ width:72, height:72, borderRadius:'var(--r-lg)', background:'linear-gradient(135deg, var(--accent), var(--violet))', border:'3px solid var(--card-bg)', display:'flex', alignItems:'center', justifyContent:'center', fontFamily:'"Space Grotesk", sans-serif', fontWeight:700, fontSize:'1.4rem', color:'#fff', boxShadow:'0 8px 24px rgba(0,212,255,0.3)', flexShrink:0 }}>

                        {initials}

                    </div>

                    { /* Active badge */ }

                    <div style={{ display:'flex', alignItems:'center', gap:'0.4rem', padding:'0.32rem 0.75rem', background:'var(--emerald-dim)', border:'1px solid rgba(0,229,160,0.28)', borderRadius:999, marginBottom:4 }}>

                        <div style={{ width:7, height:7, borderRadius:'50%', background:'var(--emerald)', animation:'pulse-glow 2s ease-in-out infinite' }}/>

                        <span style={{ fontSize:'0.72rem', fontWeight:700, color:'var(--emerald)', letterSpacing:'0.06em', textTransform:'uppercase' }}>Active</span>
                    </div>
                </div>
            </div>
        </div>

```

```

    </div>
  </div>

  { /* Name + meta */
  <div style={{ marginBottom:'1.25rem' }}>
    <h2 style={{ fontFamily:'"Space Grotesk",sans-serif', fontSize:'1.25rem', fontWeight:700,
    color:'var(--text-bright)', margin:'0 0 0.25rem', letterSpacing:'-0.02em' }}>
      {loadingDetail ? user.username : (p.PatientName || user.username)}
    </h2>
    <div style={{ display:'flex', gap:'0.5rem', flexWrap:'wrap' }}>
      {p.BloodGroup && <span className="badge badge-blood">{p.BloodGroup}</span>}
      {age && <span style={{ fontSize:'0.77rem', color:'var(--text-dim)' }}>Age {age}</span>}
      {p.Gender && <span style={{ fontSize:'0.77rem', color:'var(--text-dim)' }}>·
      {p.Gender}</span>}
    </div>
  </div>

  { /* Info grid */
  <div style={{ display:'grid', gridTemplateColumns:'1fr 1fr', gap:'0.625rem',
  marginBottom:'1.25rem' }}>
    {[
      { label:'Email',    value: p.Email || user.email || '—',    icon:'✉' },
      { label:'Phone',    value: p.PhoneNumber || '—',          icon:'☎' },
      { label:'DOB',      value: p.DateOfBirth ? formatDate(p.DateOfBirth) : '—', icon:'🎂' },
      { label:'Blood Group', value: p.BloodGroup || '—',          icon:'🩸' },
      { label:'Address',   value: p.Address || '—',              icon:'📍' },
      { label:'Emergency', value: p.EmergencyContact || '—',      icon:'🚑' },
    ].map(f=>(
      <div key={f.label} style={{ gridColumn: f.label==='Address' || f.label==='Email'? '1/-1':'auto',
      padding:'0.6rem 0.75rem', background:'var(--glass)', border:'1px solid var(--border-glass)',
      borderRadius:'var(--r-sm)' }}>
        <div style={{ fontSize:'0.62rem', fontWeight:700, letterSpacing:'0.09em',
        textTransform:'uppercase', color:'var(--text-dim)', marginBottom:'0.18rem' }}>{f.icon} {f.label}</div>

```

```
      <div style={{ fontSize:'0.84rem', color:'var(--text-bright)', fontWeight:500,
overflow:'hidden', textOverflow:'ellipsis', whiteSpace:'nowrap' }}>{f.value}</div>
```

```
    </div>
```

```
  )}
```

```
</div>
```

```
  { /* Stat pills */ }
```

```
  <div style={{ display:'grid', gridTemplateColumns:'repeat(3,1fr)', gap:'0.5rem',
marginBottom:'1.25rem' }}>
```

```
    {[
```

```
      { label:'Total Visits', value: visits.length, color:'var(--accent)', bg:'var(--
accent-dim)' },
```

```
      { label:'Diagnoses', value: diagnoses.length, color:'var(--violet)', bg:'var(--
violet-dim)' },
```

```
      { label:'Prescriptions', value: prescriptions.length, color:'var(--emerald)',
bg:'var(--emerald-dim)' },
```

```
    ].map(s=>{
```

```
      <div key={s.label} style={{ padding:'0.75rem 0.5rem', background:s.bg, border:`1px solid
${s.color}25`, borderRadius:'var(--r-md)', textAlign:'center' }}>
```

```
        <div style={{ fontFamily:'"Space Grotesk",sans-serif', fontSize:'1.6rem', fontWeight:700,
color:s.color, lineHeight:1 }}>{s.value}</div>
```

```
        <div style={{ fontSize:'0.64rem', color:'var(--text-dim)', marginTop:'0.28rem',
fontWeight:600, letterSpacing:'0.05em' }}>{s.label}</div>
```

```
      </div>
```

```
    )}
```

```
</div>
```

```
  { /* Actions */ }
```

```
  <div style={{ display:'flex', gap:'0.625rem', flexWrap:'wrap' }}>
```

```
    <button onClick={onBookAppointment} className="action-btn" style={{ flex:1,
justifyContent:'center', padding:'0.7rem 0.875rem', fontSize:'0.82rem' }}>
```

```
      <Calendar size={14}/> Book Appointment
```

```
    </button>
```

```

        <button onClick={onOpenSummary} className="action-btn secondary" style={{ flex:1,
justifyContent:'center', padding:'0.7rem 0.875rem', fontSize:'0.82rem' }}>

        <BookOpen size={14}/> My Summary

    </button>

</div>

</div>

</div>

{ /* Appointments Card */}

<div style={{ background:'var(--card-bg)', backdropFilter:'blur(20px)', border:'1px solid var(--
border-glass)', borderRadius:'var(--r-xl)', overflow:'hidden', boxShadow:'var(--shadow-card)' }}>

    <div style={{ padding:'1.25rem 1.5rem', borderBottom:'1px solid var(--border-glass)',
display:'flex', alignItems:'center', justifyContent:'space-between' }}>

        <div>

            <h3 style={{ fontFamily:"'Space Grotesk',sans-serif", fontWeight:700, fontSize:'1rem',
color:'var(--text-bright)', margin:0 }}> 📅 Appointments</h3>

            <p style={{ fontSize:'0.75rem', color:'var(--text-dim)', margin:'0.2rem 0 0'
}}>{appointments.filter(a=>a.Status==='Pending' || a.Status==='Confirmed').length} upcoming</p>

        </div>

        <button onClick={onViewAppointments} className="action-btn" style={{ padding:'0.45rem
1rem', fontSize:'0.78rem', marginTop:0 }}>Browse All</button>

    </div>

    {appointments.length === 0 ? (

        <div style={{ padding:'3rem', textAlign:'center' }}>

            <Calendar size={36} style={{ color:'var(--border)', marginBottom:'0.75rem' }}/>

            <p style={{ color:'var(--text-dim)', fontSize:'0.84rem' }}>No appointments yet</p>

            <button onClick={onBookAppointment} className="action-btn" style={{
marginTop:'0.875rem', padding:'0.55rem 1.25rem', fontSize:'0.8rem' }}>Book Now</button>

        </div>

    ) : (

        <div style={{ overflowX:'auto' }}>

            <table style={{ width:'100%', borderCollapse:'collapse' }}>

                <thead>

```

```

<tr style={{ background:'linear-
gradient(90deg,rgba(0,212,255,0.05),rgba(155,125,255,0.03))' }}>

  {[ 'Visit Type', 'Date', 'Time', 'Doctor', 'Status' ].map(h=>{

    <th key={h} style={{ padding:'0.75rem 1.25rem', textAlign:'left', fontSize:'0.65rem',
fontFamily:'"Space Grotesk',sans-serif", fontWeight:700, textTransform:'uppercase',
letterSpacing:'0.09em', color:'var(--accent-light)', whiteSpace:'nowrap', borderBottom:'1px solid var(-
-border-glass)' }}>{h}</th>

    )}}

  </tr>

</thead>

<tbody>

  {appointments.slice(0,5).map(a=>{

    <tr key={a.AppointmentID} style={{ borderBottom:'1px solid var(--border-glass)',
transition:'background var(--t-fast)' }}

      onMouseEnter={e=>(e.currentTarget.style.background='var(--glass)')}

      onMouseLeave={e=>(e.currentTarget.style.background='transparent')}>

        <td style={{ padding:'0.85rem 1.25rem', fontSize:'0.86rem', fontWeight:600, color:'var(--
text-bright)' }}>{a.Reason || 'Check-up'}</td>

        <td style={{ padding:'0.85rem 1.25rem', fontSize:'0.82rem', color:'var(--text-mid)',
fontFamily:'"JetBrains Mono',monospace" }}>{fmtDate(a.RequestedDate)}</td>

        <td style={{ padding:'0.85rem 1.25rem', fontSize:'0.82rem', color:'var(--text-dim)'
}}>{a.TimeSlot || '—'}</td>

        <td style={{ padding:'0.85rem 1.25rem', fontSize:'0.82rem', color:'var(--text-mid)'
}}>{a.DoctorName || <span style={{ color:'var(--amber)', fontSize:'0.74rem' }}>Pending
assign</span>}</td>

        <td style={{ padding:'0.85rem 1.25rem' }}>

          <span style={{ padding:'0.2rem 0.6rem', borderRadius:6, fontSize:'0.72rem',
fontWeight:700, background:`${statusColor(a.Status)}18`, color:statusColor(a.Status), border:`1px
solid ${statusColor(a.Status)}30` }}>{a.Status}</span>

        </td>

      </tr>

    )}}

</tbody>

</table>

</div>

```



```

    }}
  </div>
</div>

```

```

{ /* — BOTTOM ROW: Medical Records + Prescriptions + Files — */

```

```

<div style={{ display:'grid', gridTemplateColumns:'1fr 1fr 1fr', gap:'1.25rem' }}>

```

```

  { /* Medical Records */

```

```

    <div style={{ background:'var(--card-bg)', backdropFilter:'blur(20px)', border:'1px solid var(--border-glass)', borderRadius:'var(--r-xl)', overflow:'hidden', boxShadow:'var(--shadow-card)' }}>

```

```

      <div style={{ padding:'1.125rem 1.375rem', borderBottom:'1px solid var(--border-glass)', display:'flex', alignItems:'center', justifyContent:'space-between' }}>

```

```

        <h3 style={{ fontFamily:'"Space Grotesk",sans-serif', fontWeight:700, fontSize:'0.95rem', color:'var(--text-bright)', margin:0 }}> 📁 Medical Records</h3>

```

```

        <button onClick={onViewRecords} className="action-btn" style={{ padding:'0.38rem 0.875rem', fontSize:'0.75rem', marginTop:0 }}>Browse All</button>

```

```

      </div>

```

```

    {records.length === 0 ? (

```

```

      <div style={{ padding:'2rem', textAlign:'center', color:'var(--text-dim)', fontSize:'0.84rem' }}>No records yet</div>

```

```

    ) : (

```

```

      <table style={{ width:'100%', borderCollapse:'collapse' }}>

```

```

        <thead>

```

```

          <tr style={{ background:'rgba(0,212,255,0.03)' }}>

```

```

            {[ 'Date','Diagnosis','Doctor' ].map(h=>(

```

```

              <th key={h} style={{ padding:'0.625rem 1.125rem', textAlign:'left', fontSize:'0.62rem', fontFamily:'"Space Grotesk",sans-serif', fontWeight:700, textTransform:'uppercase', letterSpacing:'0.09em', color:'var(--text-dim)', borderBottom:'1px solid var(--border-glass)' }}>{h}</th>

```

```

            )))

```

```

          </tr>

```

```

        </thead>

```

```

        <tbody>

```

```

          {records.slice(0,5).map((r:any,i:number)=>(

```

```

        <tr key={i} style={{ borderBottom:'1px solid var(--border-glass)', transition:'background
var(--t-fast)' }}

        onMouseEnter={e=>(e.currentTarget.style.background='var(--glass)')}

        onMouseLeave={e=>(e.currentTarget.style.background='transparent')}>

        <td style={{ padding:'0.7rem 1.125rem', fontSize:'0.78rem', color:'var(--text-dim)',
fontFamily:'"JetBrains Mono',monospace', whiteSpace:'nowrap'", whiteSpace:'nowrap'
}}>{fmtDate(r.VisitDate)}</td>

        <td style={{ padding:'0.7rem 1.125rem', fontSize:'0.82rem', color:'var(--text-bright)',
fontWeight:500, maxWidth:120, overflow:'hidden', textOverflow:'ellipsis', whiteSpace:'nowrap'
}}>{r.DiagnosisName | | r.ReasonForVisit | | '—'}</td>

        <td style={{ padding:'0.7rem 1.125rem', fontSize:'0.78rem', color:'var(--text-mid)',
overflow:'hidden', textOverflow:'ellipsis', whiteSpace:'nowrap' }}>{r.DoctorName | | '—'}</td>

        </tr>

    }}}

</tbody>

</table>

}}

</div>

```

```

{/* Prescriptions */}

<div style={{ background:'var(--card-bg)', backdropFilter:'blur(20px)', border:'1px solid var(--
border-glass)', borderRadius:'var(--r-xl)', overflow:'hidden', boxShadow:'var(--shadow-card)' }}>

    <div style={{ padding:'1.125rem 1.375rem', borderBottom:'1px solid var(--border-glass)',
display:'flex', alignItems:'center', justifyContent:'space-between' }}>

        <h3 style={{ fontFamily:'"Space Grotesk',sans-serif", fontWeight:700, fontSize:'0.95rem',
color:'var(--text-bright)', margin:0 }}>📖 Medications</h3>

        <button onClick={onOpenSummary} className="action-btn secondary" style={{
padding:'0.38rem 0.875rem', fontSize:'0.75rem', marginTop:0 }}>View All</button>

    </div>

    {prescriptions.length === 0 ? (

        <div style={{ padding:'2rem', textAlign:'center', color:'var(--text-dim)', fontSize:'0.84rem' }}>No
medications on record</div>

    ) : (

        <table style={{ width:'100%', borderCollapse:'collapse' }}>

            <thead>

```

```

<tr style={{ background:'rgba(0,229,160,0.03)' }}>

  {[ 'Medicine','Dose','Freq','Status'].map(h=>(

    <th key={h} style={{ padding:'0.625rem 1.125rem', textAlign:'left', fontSize:'0.62rem',
fontFamily:'"Space Grotesk',sans-serif", fontWeight:700, textTransform:'uppercase',
letterSpacing:'0.09em', color:'var(--text-dim)', borderBottom:'1px solid var(--border-glass)'
}}>{h}</th>

    )))

</tr>

</thead>

<tbody>

  {prescriptions.slice(0,5).map((rx:any,i:number)=>(

    <tr key={i} style={{ borderBottom:'1px solid var(--border-glass)', transition:'background
var(--t-fast)' }}

      onMouseEnter={e=>(e.currentTarget.style.background='var(--glass)')}

      onMouseLeave={e=>(e.currentTarget.style.background='transparent')}>

        <td style={{ padding:'0.7rem 1.125rem', fontSize:'0.82rem', fontWeight:600, color:'var(--
text-bright)', maxWidth:100, overflow:'hidden', textOverflow:'ellipsis', whiteSpace:'nowrap'
}}>{rx.MedicineName}</td>

        <td style={{ padding:'0.7rem 1.125rem', fontSize:'0.78rem', color:'var(--text-mid)',
fontFamily:'"JetBrains Mono',monospace", whiteSpace:'nowrap' }}>{rx.Dosage || '—'}</td>

        <td style={{ padding:'0.7rem 1.125rem', fontSize:'0.78rem', color:'var(--text-dim)',
whiteSpace:'nowrap' }}>{rx.Frequency || '—'}</td>

        <td style={{ padding:'0.7rem 1.125rem' }}>

          <span className={`status-badge ${rx.IsDispensed?'dispensed':'pending'}`} style={{
fontSize:'0.68rem', padding:'0.18rem 0.5rem' }}>

            {rx.IsDispensed?<><CheckCircle size={10}/> Done</>:<><Clock size={10}/> Active</>}

          </span>

        </td>

      </tr>

    )))

</tbody>

</table>

  )}

</div>

```

```

    { /* Files */ }

    <div style={{ background:'var(--card-bg)', backdropFilter:'blur(20px)', border:'1px solid var(--border-glass)', borderRadius:'var(--r-xl)', overflow:'hidden', boxShadow:'var(--shadow-card)' }}>

      <div style={{ padding:'1.125rem 1.375rem', borderBottom:'1px solid var(--border-glass)', display:'flex', alignItems:'center', justifyContent:'space-between' }}>

        <h3 style={{ fontFamily:'"Space Grotesk',sans-serif", fontWeight:700, fontSize:'0.95rem', color:'var(--text-bright)', margin:0 }}> 📁 My Files</h3>

        <button onClick={onViewFiles} className="action-btn secondary" style={{ padding:'0.38rem 0.875rem', fontSize:'0.75rem', marginTop:0 }}>Browse All</button>

      </div>

      {medFiles.length === 0 ? (

        <div style={{ padding:'2rem 1.375rem', textAlign:'center' }}>

          <FileImage size={32} style={{ color:'var(--border)', marginBottom:'0.625rem' }}/>

          <p style={{ color:'var(--text-dim)', fontSize:'0.82rem', marginBottom:'0.875rem' }}>No files uploaded yet</p>

          <button onClick={onViewFiles} className="action-btn" style={{ padding:'0.5rem 1rem', fontSize:'0.78rem', marginTop:0 }}>

            <Upload size={13}/> Upload File

          </button>

        </div>

      ) : (

        <div style={{ padding:'0.5rem 0.75rem', display:'flex', flexDirection:'column', gap:'0.375rem' }}>

          {medFiles.slice(0,5).map((f:any,i:number)=>(

            <div key={i} style={{ display:'flex', alignItems:'center', gap:'0.75rem', padding:'0.625rem 0.75rem', background:'var(--glass)', border:'1px solid var(--border-glass)', borderRadius:'var(--r-sm)', transition:'border-color var(--t-fast)' }}

              onMouseEnter={e=>(e.currentTarget.style.borderColor='var(--border-hover)')}

              onMouseLeave={e=>(e.currentTarget.style.borderColor='var(--border-glass)')}>

                <div style={{ width:32, height:32, borderRadius:8, background:'var(--accent-dim)', display:'grid', placeItems:'center', flexShrink:0 }}>

                  <FileImage size={15} style={{ color:'var(--accent)' }}/>

                </div>

```

```

    <div style={{ flex:1, minWidth:0 }}>

      <p style={{ margin:0, fontSize:'0.8rem', fontWeight:600, color:'var(--text-bright)',
overflow:'hidden', textOverflow:'ellipsis', whiteSpace:'nowrap' }}>{f.FileName}</p>

      <p style={{ margin:0, fontSize:'0.69rem', color:'var(--text-dim)', fontFamily:'"JetBrains
Mono',monospace" }}>{f.FileType} · {fmtDate(f.UploadedAt)}</p>

    </div>

    <button onClick={()=>downloadFile(f.FileID, f.FileName)} style={{ background:'none',
border:'none', color:'var(--accent)', cursor:'pointer', padding:'0.25rem', borderRadius:4, display:'flex',
transition:'color var(--t-fast)' }}

      onMouseEnter={e=>(e.currentTarget.style.color='var(--accent-light))}

      onMouseLeave={e=>(e.currentTarget.style.color='var(--accent))}>

      <Download size={14}/>

    </button>

  </div>

  )}}

</div>

  )}

</div>

</div>

  </div>

  );
}

```

```
// _____
```

```
// MyAppointments — Patient's appointment history + booking
```

```
// _____
```

```

function MyAppointmentsView({ token, doctors }: { token: string; doctors: Doctor[] }) {

  const [appointments, setAppointments] = useState<Appointment[]>([]);

  const [loading, setLoading] = useState(true);

  const [showBook, setShowBook] = useState(false);

  const load = useCallback(() => {

```

```

fetch(`${API}/appointments`, { headers: { Authorization: `Bearer ${token}` } })
  .then(r => r.json())
  .then(d => { setAppointments(Array.isArray(d) ? d : []); setLoading(false); })
  .catch(() => setLoading(false));
}, [token]);

```

```

useEffect(() => { load(); }, [load]);

```

```

async function cancel(id: number) {
  if (!window.confirm('Cancel this appointment request?')) return;
  await fetch(`${API}/appointments/${id}/cancel`, { method: 'PUT', headers: { Authorization: `Bearer ${token}` } });
  load();
}

```

```

const statusColor = (s: string) => s==='Confirmed'?`var(--emerald)`:s==='Cancelled'?`var(--rose)`:s==='Completed'?`var(--accent)`:`var(--amber)`;

const statusBg = (s: string) => s==='Confirmed'?`var(--emerald-dim)`:s==='Cancelled'?`var(--rose-dim)`:s==='Completed'?`var(--accent-dim)`:`var(--amber-dim)`;

```

```

return (
  <div style={{ animation: 'gridReveal 0.4s var(--ease-out) both' }}>
    {showBook && <BookAppointmentModal token={token} doctors={doctors}
onClose={()=>setShowBook(false)} onSave={load}/>}

    { /* Header row */ }

    <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between',
marginBottom: '1.5rem' }}>

      <div>

        <h2 style={{ fontFamily: '"Syne", sans-serif', fontSize: '1.1rem', fontWeight: 800, margin: '0 0 0.25rem', color: 'var(--text-bright)' }}>My Appointments</h2>

        <p style={{ fontSize: '0.82rem', color: 'var(--text-dim)', margin: 0 }}>Track your appointment requests and upcoming visits</p>

```

```

    </div>

    <button className="action-btn" onClick={()=>setShowBook(true)} style={{ display:'flex',
alignItems:'center', gap:'0.5rem' }}>

        <Plus size={15}/> Book Appointment

    </button>
</div>

{loading ? (

    <div style={{ textAlign:'center', padding:'3rem', color:'var(--text-dim)' }}>Loading
appointments...</div>

) : appointments.length === 0 ? (

    <div className="empty-state">

        <Calendar size={56} style={{ opacity:.25 }}/>

        <h3>No appointments yet</h3>

        <p>Book your first appointment and our front desk team will confirm it for you.</p>

        <button className="action-btn" onClick={()=>setShowBook(true)} style={{ marginTop:'0.5rem',
display:'flex', alignItems:'center', gap:'0.5rem' }}>

            <Plus size={15}/> Book Appointment

        </button>

    </div>

) : (

    <div style={{ display:'flex', flexDirection:'column', gap:'0.75rem' }}>

        {appointments.map(a => (

            <div key={a.AppointmentID} style={{ background:'var(--card-bg)', border:'1px solid var(--
border)', borderRadius:'var(--r-lg)', overflow:'hidden', transition:'border-color var(--t-mid)' }}>

                {/* Top bar colored by status */}

                <div style={{ height:3, background: statusColor(a.Status) }}/>

                <div style={{ padding:'1rem 1.25rem' }}>

                    <div style={{ display:'flex', alignItems:'flex-start', justifyContent:'space-between', gap:'1rem',
flexWrap:'wrap' }}>

                        <div style={{ flex:1 }}>

                            {/* Date + time */}

```

```

<div style={{ display:'flex', alignItems:'center', gap:'0.75rem', marginBottom:'0.5rem',
flexWrap:'wrap' }}>

  <div style={{ display:'flex', alignItems:'center', gap:'0.4rem' }}>

    <Calendar size={14} style={{ color:'var(--accent)' }}/>

    <span style={{ fontFamily:'"Syne",sans-serif', fontWeight:700, fontSize:'0.9rem',
color:'var(--text-bright)' }}>

      {new Date(a.RequestedDate).toLocaleDateString('en-GB',{ weekday:'short', day:'2-
digit', month:'short', year:'numeric' })}

    </span>

  </div>

  {a.TimeSlot && (

    <div style={{ display:'flex', alignItems:'center', gap:'0.3rem' }}>

      <Clock size={13} style={{ color:'var(--text-dim)' }}/>

      <span style={{ fontSize:'0.82rem', color:'var(--text-mid)' }}>{a.TimeSlot}</span>

    </div>

  )}

  <span style={{ padding:'0.2rem 0.65rem', borderRadius:999, fontSize:'0.72rem',
fontWeight:700, background: statusBg(a.Status), color: statusColor(a.Status), border:`1px solid
${statusColor(a.Status)}30` }}>

    {a.Status}

  </span>

</div>

<p style={{ fontWeight:600, color:'var(--text-bright)', margin:'0 0 0.25rem',
fontSize:'0.9rem' }}>{a.Reason}</p>

{a.PatientNotes && <p style={{ fontSize:'0.8rem', color:'var(--text-dim)', margin:'0 0
0.5rem' }}>{a.PatientNotes}</p>}

{ /* Doctor assigned / pending */}

{a.DoctorName ? (

  <div style={{ display:'flex', alignItems:'center', gap:'0.5rem', padding:'0.5rem 0.75rem',
background:'var(--accent-dim)', borderRadius:'var(--r-sm)', border:'1px solid var(--border)', width:'fit-
content' }}>

    <Stethoscope size={13} style={{ color:'var(--accent)' }}/>

```



```

        <span style={{ fontSize:'0.8rem', color:'var(--text-bright)', fontWeight:600
}}>{a.DoctorName}</span>

        <span style={{ fontSize:'0.75rem', color:'var(--text-dim)' }}>· {a.Specialty}</span>

    </div>

    ) : (

        <div style={{ display:'flex', alignItems:'center', gap:'0.4rem', fontSize:'0.78rem',
color:'var(--amber)', padding:'0.4rem 0.75rem', background:'var(--amber-dim)', borderRadius:'var(--
r-sm)', border:'1px solid rgba(217,119,6,.2)', width:'fit-content' }}>

            <Clock size={12}/> Awaiting doctor assignment by receptionist

        </div>

    )}

    {a.ReceptionistNotes && (

        <p style={{ fontSize:'0.78rem', color:'var(--text-dim)', margin:'0.5rem 0 0',
fontStyle:'italic' }}>

            📄 Receptionist: {a.ReceptionistNotes}

        </p>

    )}

</div>

{/* Cancel button — only for Pending */}

{a.Status === 'Pending' && (

    <button onClick={() => cancel(a.AppointmentID)}

        style={{ padding:'0.4rem 0.875rem', background:'var(--rose-dim)', border:'1px solid
rgba(244,63,94,.2)', borderRadius:'var(--r-sm)', color:'var(--rose)', fontSize:'0.78rem', fontWeight:600,
cursor:'pointer', display:'flex', alignItems:'center', gap:'0.3rem', whiteSpace:'nowrap', flexShrink:0 }}>

        <X size={12}/> Cancel

    </button>

)}

</div>

</div>

</div>

)}}

```

```

    </div>

  })
</div>

);
}

// _____
// ReceptionistDashboard — assign doctors to pending appointments
// _____

function ReceptionistDashboard({ token, doctors }: { token: string; doctors: Doctor[] }) {
  const [appointments, setAppointments] = useState<Appointment[]>([]);
  const [loading, setLoading] = useState(true);
  const [statusFilter, setStatusFilter] = useState<string>('Pending');
  const [assigningId, setAssigningId] = useState<number | null>(null);
  const [assignForm, setAssignForm] = useState({ doctor_id: "", time_slot: "", receptionist_notes: "" });
  const [saving, setSaving] = useState(false);
  const [msg, setMsg] = useState("");

  const TIME_SLOTS = ['09:00 AM', '09:30 AM', '10:00 AM', '10:30 AM', '11:00 AM', '11:30 AM',
    '12:00 PM', '02:00 PM', '02:30 PM', '03:00 PM', '03:30 PM', '04:00 PM', '04:30 PM'];

  const load = useCallback(() => {
    const qs = statusFilter === 'All' ? '' : `?status=${statusFilter}`;
    fetch(`${API}/appointments${qs}`, { headers: { Authorization: `Bearer ${token}` } })
      .then(r => r.json())
      .then(d => { setAppointments(Array.isArray(d) ? d : []); setLoading(false); })
      .catch(() => setLoading(false));
  }, [token, statusFilter]);

  useEffect(() => { setLoading(true); load(); }, [load]);

```

```

async function doAssign(aid: number, status: 'Confirmed'|'Cancelled') {
  if (status === 'Confirmed' && !assignForm.doctor_id) { setMsg('Please select a doctor first. ');
return; }

  setSaving(true); setMsg('');

  try {
    const res = await fetch(`${API}/appointments/${aid}/assign`, {
      method: 'PUT',
      headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
      body: JSON.stringify({ doctor_id: assignForm.doctor_id || null, time_slot:
assignForm.time_slot || null, receptionist_notes: assignForm.receptionist_notes, status }),
    });

    const d = await res.json();
    if (!res.ok) throw new Error(d.error);
    setAssigningId(null);
    setAssignForm({ doctor_id: "", time_slot: "", receptionist_notes: "" });
    load();
  } catch (e: any) { setMsg(`Error: ${e.message}`); }
  finally { setSaving(false); }
}

```

```

const statusColor = (s: string) => s === 'Confirmed' ? `var(--emerald)` : s === 'Cancelled' ? `var(--
rose)` : s === 'Completed' ? `var(--accent)` : `var(--amber)`;

```

```

const counts    = { Pending: appointments.filter(a => a.Status === 'Pending').length };

```

```

return (

```

```

  <div style={{ animation: 'gridReveal 0.4s var(--ease-out) both' }}>

```

```

    { /* Header */ }

```

```

    <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between',
marginBottom: '1.5rem', flexWrap: 'wrap', gap: '0.875rem' }}>

```

```

      <div>

```

```

        <h2 style={{ fontFamily: '"Syne', sans-serif", fontWeight: 800, fontSize: '1.1rem', margin: '0 0
0.25rem', color: 'var(--text-bright)' }}>Appointments Queue</h2>

```

```
<p style={{ fontSize:'0.82rem', color:'var(--text-dim)', margin:0 }}>Review requests and assign doctors to patients</p>
```

```
</div>
```

```
<div style={{ display:'flex', gap:'0.5rem', flexWrap:'wrap' }}>
```

```
{['Pending','Confirmed','Cancelled','All'].map(s => (
```

```
<button key={s} onClick={()=>setStatusFilter(s)}
```

```
style={{ padding:'0.45rem 0.875rem', borderRadius:'var(--r-sm)', border:'1px solid var(--border)', background: statusFilter===s ? 'var(--accent-dim)' : 'var(--raised)', color: statusFilter===s ? 'var(--accent)' : 'var(--text-mid)', fontSize:'0.8rem', fontWeight:600, cursor:'pointer', transition:'all var(--t-fast)', display:'flex', alignItems:'center', gap:'0.35rem' }}>
```

```
{s}{s==='Pending'&&counts.Pending>0&&<span style={{ background:'var(--rose)', color:'white', borderRadius:999, fontSize:'0.62rem', fontWeight:800, padding:'0.05rem 0.4rem', minWidth:16, textAlign:'center' }}>{counts.Pending}</span>}
```

```
</button>
```

```
)))
```

```
<button onClick={()=>{setLoading(true);load();}} style={{ padding:'0.45rem 0.75rem', borderRadius:'var(--r-sm)', border:'1px solid var(--border)', background:'var(--raised)', color:'var(--text-dim)', cursor:'pointer' }}>
```

```
<RefreshCw size={14}/>
```

```
</button>
```

```
</div>
```

```
</div>
```

```
{msg && <div style={{ padding:'0.75rem 1rem', background:'var(--rose-dim)', border:'1px solid rgba(244,63,94,.25)', borderRadius:'var(--r-md)', color:'var(--rose)', fontSize:'0.84rem', marginBottom:'1rem', display:'flex', alignItems:'center', gap:'0.5rem' }}><AlertCircle size={14}/>{msg}</div>}
```

```
{loading ? (
```

```
<div style={{ textAlign:'center', padding:'3rem', color:'var(--text-dim)' }}>Loading appointments...</div>
```

```
) : appointments.length === 0 ? (
```

```
<div className="empty-state"><Calendar size={56} style={{ opacity:.25 }}/><h3>No appointments</h3><p>No {statusFilter !== 'All' ? statusFilter.toLowerCase() : ''} appointments at the moment.</p></div>
```

```
) : (
```

```

<div style={{ display:'flex', flexDirection:'column', gap:'0.75rem' }}>

  {appointments.map(a => (

    <div key={a.AppointmentID} style={{ background:'var(--card-bg)', border:'1px solid var(--border)', borderRadius:'var(--r-lg)', overflow:'hidden' }}>

      <div style={{ height:3, background: statusColor(a.Status) }}/>

      <div style={{ padding:'1rem 1.25rem' }}>

        <div style={{ display:'flex', alignItems:'flex-start', gap:'1rem', flexWrap:'wrap' }}>

          { /* Left: patient info */ }

          <div style={{ flex:1, minWidth:220 }}>

            <div style={{ display:'flex', alignItems:'center', gap:'0.75rem', marginBottom:'0.625rem' }}>

              <div style={{ width:40, height:40, borderRadius:10, background:'var(--violet-dim)', border:'1px solid rgba(139,92,246,.2)', display:'flex', alignItems:'center', justifyContent:'center', color:'var(--violet)', flexShrink:0, fontFamily:'"Syne",sans-serif', fontWeight:800, fontSize:'0.85rem' }}>

                {a.PatientName ? a.PatientName.split(' ').map((w:string)=>w[0]).join('').slice(0,2) : '?'}

              </div>

              <div>

                <p style={{ margin:0, fontWeight:700, color:'var(--text-bright)', fontSize:'0.9rem' }}>{a.PatientName || '—'}</p>

                <p style={{ margin:0, fontSize:'0.74rem', color:'var(--text-dim)' }}>{a.Gender} · {a.BloodGroup} · {a.PhoneNumber}</p>

              </div>

            </div>

            <div style={{ display:'flex', alignItems:'center', gap:'0.625rem', flexWrap:'wrap', marginBottom:'0.5rem' }}>

              <div style={{ display:'flex', alignItems:'center', gap:'0.35rem', fontSize:'0.82rem', color:'var(--text-mid)' }}>

                <Calendar size={13} style={{ color:'var(--accent)' }}/>

                {new Date(a.RequestedDate).toLocaleDateString('en-GB',{ weekday:'short', day:'2-digit', month:'short' })} · {a.TimeSlot}

              </div>

              <span style={{ padding:'0.15rem 0.55rem', borderRadius:999, fontSize:'0.7rem', fontWeight:700, background: statusColor(a.Status)+'20', color: statusColor(a.Status) }}>{a.Status}</span>

```

</div>

<p style={{ fontWeight:600, fontSize:'0.85rem', color:'var(--text-bright)', margin:'0 0 0.2rem' }}>{a.Reason}</p>

{a.PatientNotes && <p style={{ fontSize:'0.78rem', color:'var(--text-dim)', margin:0, fontStyle:'italic' }}>"{a.PatientNotes}"</p>}

{a.DoctorName && (

<div style={{ marginTop:'0.5rem', display:'flex', alignItems:'center', gap:'0.4rem', fontSize:'0.78rem', color:'var(--accent)' }}>

<Stethoscope size={12}/> Assigned: {a.DoctorName} ({a.Specialty})

</div>

)}

{a.HandledByName && <p style={{ margin:'0.25rem 0 0', fontSize:'0.72rem', color:'var(--text-dim)' }}>Handled by {a.HandledByName}</p>}

</div>

{/* Right: action */}

{a.Status === 'Pending' && (

<div style={{ flexShrink:0 }}>

{assigningId === a.AppointmentID ? (

<div style={{ display:'flex', flexDirection:'column', gap:'0.5rem', minWidth:240 }}>

<div className="form-group" style={{ margin:0 }}>

<label style={{ fontSize:'0.72rem' }}>Assign Doctor *</label>

<select value={assignForm.doctor_id} onChange={e=>setAssignForm(f=>({...f,doctor_id:e.target.value}})}>

<option value="">Select Doctor</option>

{doctors.map(d => <option key={d.DoctorID} value={d.DoctorID}>{d.DoctorName} — {d.Specialty}</option>)}

</select>

</div>

<div className="form-group" style={{ margin:0 }}>

<label style={{ fontSize:'0.72rem' }}>Confirm Time Slot</label>

```

        <select value={assignForm.time_slot}
onChange={e=>setAssignForm(f=>({...f,time_slot:e.target.value}))}>
            <option value="">Keep patient's preference ({a.TimeSlot})</option>
            {TIME_SLOTS.map(t => <option key={t}>{t}</option>)}
        </select>
    </div>

    <div className="form-group" style={{ margin:0 }}>
        <label style={{ fontSize:'0.72rem' }}>Internal Notes</label>
        <input type="text" value={assignForm.receptionist_notes} placeholder="Optional
notes for doctor..." onChange={e=>setAssignForm(f=>({...f,receptionist_notes:e.target.value}))}/>
    </div>

    <div style={{ display:'flex', gap:'0.5rem' }}>
        <button onClick={()=>doAssign(a.AppointmentID,'Confirmed')} disabled={saving}
className="submit-btn" style={{ flex:1, padding:'0.6rem', marginTop:0, fontSize:'0.82rem',
display:'flex', alignItems:'center', justifyContent:'center', gap:'0.35rem' }}>
            <CheckCircle size={13}/> Confirm
        </button>

        <button onClick={()=>doAssign(a.AppointmentID,'Cancelled')} disabled={saving}
style={{ padding:'0.6rem 0.75rem', background:'var(--rose-dim)', border:'1px solid
rgba(244,63,94,.2)', borderRadius:'var(--r-sm)', color:'var(--rose)', cursor:'pointer', fontSize:'0.82rem',
fontWeight:600 }}>
            <X size={13}/>
        </button>

        <button onClick={()=>setAssigningId(null)} style={{ padding:'0.6rem 0.75rem',
background:'var(--raised)', border:'1px solid var(--border)', borderRadius:'var(--r-sm)', color:'var(--
text-dim)', cursor:'pointer' }}>
            Cancel
        </button>
    </div>
</div>

): (
    <button onClick={()=>{ setAssigningId(a.AppointmentID); setAssignForm({ doctor_id:
a.DoctorID ? String(a.DoctorID):'', time_slot: a.TimeSlot | '', receptionist_notes:'' }); setMsg(''); }}
        style={{ padding:'0.6rem 1.25rem', background:'linear-
gradient(135deg,#06d6a0,#059669)', border:'none', borderRadius:'var(--r-md)', color:'white',

```

```
fontFamily:'Syne',sans-serif",fontWeight:700,fontSize:'0.82rem',cursor:'pointer',boxShadow:'0 3px 12px rgba(6,214,160,.3)',display:'flex',alignItems:'center',gap:'0.4rem',whiteSpace:'nowrap' }}>
```

```
      <Stethoscope size={13}/> Assign Doctor
    </button>
  )}
</div>
)}
</div>
</div>
</div>
)})
</div>
)}
</div>
);
}
```

```
// _____
```

```
function NotificationBell({ pending_rx, pending_tests, role }: { pending_rx: number; pending_tests: number; role: string }) {
```

```
  const [open, setOpen] = useState(false);
```

```
  const ref = useRef<HTMLDivElement>(null);
```

```
  const total = pending_rx + pending_tests;
```

```
  useEffect(() => {
```

```
    function h(e: MouseEvent) { if (ref.current && !ref.current.contains(e.target as Node))
      setOpen(false); }
```

```
    document.addEventListener('mousedown', h);
```

```
    return () => document.removeEventListener('mousedown', h);
```

```
  }, []);
```

```
  return (
```



```

<div ref={ref} style={{ position:'relative' }}>

  <button onClick={() => setOpen(v => !v)}

    style={{ position:'relative', background:'var(--bg-3)', border:'1px solid var(--border)',
borderRadius:'var(--r-md)', width:38, height:38, display:'flex', alignItems:'center',
justifyContent:'center', color:'var(--text-mid)', cursor:'pointer', transition:'all var(--t-mid)' }}>

      <Bell size={16}/>

      {total > 0 && (

        <span style={{ position:'absolute', top:-4, right:-4, background:'var(--rose)', color:'white',
borderRadius:999, fontSize:'0.6rem', fontWeight:800, padding:'0.1rem 0.35rem', minWidth:16,
textAlign:'center', lineHeight:1.4 }}>{total}</span>

      )}

    </button>

    {open && (

      <div style={{ position:'absolute', top:'calc(100% + 8px)', right:0, background:'var(--bg-2)',
border:'1px solid var(--border)', borderRadius:'var(--r-lg)', padding:'0.75rem', minWidth:240,
zIndex:300, boxShadow:'0 16px 48px rgba(0,0,0,.5)', animation:'dropIn .2s var(--ease-out)' }}>

        <p style={{ fontSize:'0.68rem', fontWeight:700, textTransform:'uppercase',
letterSpacing:'.08em', color:'var(--text-dim)', margin:'0 0 0.625rem 0.25rem' }}>Notifications</p>

        {total === 0 ? (

          <p style={{ fontSize:'0.85rem', color:'var(--text-dim)', padding:'0.5rem 0.25rem' }}>All clear! No
pending items.</p>

        ) : (

          <div style={{ display:'flex', flexDirection:'column', gap:'0.4rem' }}>

            {pending_rx > 0 && (

              <div style={{ display:'flex', alignItems:'center', gap:'0.625rem', padding:'0.625rem 0.75rem',
background:'var(--amber-dim)', border:'1px solid rgba(245,158,11,.2)', borderRadius:'var(--r-sm)' }}>

                <Pill size={14} color="var(--amber)"/>

                <div>

                  <p style={{ margin:0, fontSize:'0.82rem', fontWeight:600, color:'var(--text-bright)'
}}>{pending_rx} Pending Prescription{pending_rx>1?'s':''}</p>

                  <p style={{ margin:0, fontSize:'0.72rem', color:'var(--text-dim)' }}>Awaiting dispensing</p>

                </div>

              </div>

            )}

          </div>

        )}

      </div>

    )}

  </div>

```

```

    })
    {pending_tests > 0 && (
      <div style={{ display:'flex', alignItems:'center', gap:'0.625rem', padding:'0.625rem 0.75rem',
background:'var(--violet-dim)', border:'1px solid rgba(139,92,246,.2)', borderRadius:'var(--r-sm)' }}>

        <FlaskConical size={14} color="var(--violet)"/>

        <div>

          <p style={{ margin:0, fontSize:'0.82rem', fontWeight:600, color:'var(--text-bright)'
}}>{pending_tests} Pending Lab Test{pending_tests>1?'s':''}</p>

          <p style={{ margin:0, fontSize:'0.72rem', color:'var(--text-dim)' }}>Results not yet
entered</p>

        </div>

      </div>

    })
  </div>

  })
</div>

}

```

```
// _____
```

```
// EditPatientModal
```

```
// _____
```

```

function EditPatientModal({ patient, token, onClose, onSave }) {
  { patient: Patient; token: string; onClose: ()=>void; onSave: ()=>void } {
  const [form, setForm] = useState({
    name: patient.PatientName, gender: patient.Gender,
    dob: patient.DateOfBirth?.split('T')[0] ?? '', phone: patient.PhoneNumber,
    address: patient.Address, blood_group: patient.BloodGroup,
    emergency_contact: patient.EmergencyContact ?? '',
    emergency_contact_name: patient.EmergencyContactName ?? '',

```

```

});

const [saving, setSaving] = useState(false);

const [err, setErr] = useState("");

async function save(e: React.FormEvent) {
  e.preventDefault(); setSaving(true); setErr("");
  try {
    const res = await fetch(`${API}/patients/${(patient as any).PatientID}`, {
      method: 'PUT', headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
      body: JSON.stringify(form),
    });
    const d = await res.json();
    if (!res.ok) throw new Error(d.error);
    onSave();
  } catch (ex: any) { setErr(ex.message); }
  finally { setSaving(false); }
}

```

```

return (
  <Modal title={`Edit — ${patient.PatientName}`} onClose={onClose}>
    {err && <div className="form-error"><AlertCircle size={14}/> {err}</div>}
    <form onSubmit={save} className="data-form" style={{ padding:0, background:'none',
    boxShadow:'none' }}>
      <div className="form-row">
        <div className="form-group"><label>Full Name *</label><input type="text"
        value={form.name} onChange={e=>setForm({...form,name:e.target.value}} required/></div>
        <div className="form-group"><label>Gender *</label>
          <select value={form.gender} onChange={e=>setForm({...form,gender:e.target.value}}>
            <option>Male</option><option>Female</option><option>Other</option>
          </select>
        </div>
      </div>
    </form>
  </div>
)

```

```

<div className="form-row">

  <div className="form-group"><label>Date of Birth *</label><input type="date"
value={form.dob} onChange={e=>setForm({...form,dob:e.target.value})}/></div>

  <div className="form-group"><label>Blood Group *</label>

    <select value={form.blood_group}
onChange={e=>setForm({...form,blood_group:e.target.value})}>

      {[ 'A+', 'A-', 'B+', 'B-', 'O+', 'O-', 'AB+', 'AB-' ].map(g=><option key={g}>{g}</option>)}

    </select>

  </div>

</div>

<div className="form-row">

  <div className="form-group"><label>Phone *</label><input type="tel" value={form.phone}
onChange={e=>setForm({...form,phone:e.target.value})}/></div>

  <div className="form-group"><label>Address *</label><input type="text"
value={form.address} onChange={e=>setForm({...form,address:e.target.value})}/></div>

</div>

<div className="form-row">

  <div className="form-group"><label>Emergency Contact Name</label><input type="text"
value={form.emergency_contact_name}
onChange={e=>setForm({...form,emergency_contact_name:e.target.value})}/></div>

  <div className="form-group"><label>Emergency Contact Phone</label><input type="tel"
value={form.emergency_contact}
onChange={e=>setForm({...form,emergency_contact:e.target.value})}/></div>

</div>

<button type="submit" className="submit-btn" disabled={saving} style={{ marginTop:'0.5rem'
}}>

  <Save size={15}/> {saving ? 'Saving...' : 'Save Changes'}

</button>

</form>

</Modal>

);
}

```

```
// _____
```

```

// EditDoctorModal
// _____

function EditDoctorModal({ doctor, token, onClose, onSave }:
  { doctor: Doctor; token: string; onClose: ()=>void; onSave: ()=>void }) {
  const [form, setForm] = useState({
    doctor_name: doctor.DoctorName,
    email: doctor.Email,
    specialty: doctor.Specialty,
    phone_number: doctor.PhoneNumber,
    license_number: doctor.LicenseNumber || '',
    years_of_experience: doctor.YearsOfExperience?.toString() || '',
  });
  const [saving, setSaving] = useState(false);
  const [err, setErr] = useState('');

  async function save(e: React.FormEvent) {
    e.preventDefault(); setSaving(true); setErr('');
    try {
      const res = await fetch(`${API}/doctors/${doctor.DoctorID}`, {
        method: 'PUT', headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
        body: JSON.stringify({
          doctor_name: form.doctor_name,
          email: form.email,
          specialty: form.specialty,
          phone_number: form.phone_number,
          license_number: form.license_number || null,
          years_of_experience: form.years_of_experience ? parseInt(form.years_of_experience) : null,
        }),
      });
    };
    const d = await res.json();
    if (!res.ok) throw new Error(d.error);
  }
}

```

```

    onSave();

} catch(ex: any) { setErr(ex.message); }

finally { setSaving(false); }

}

return (

<Modal title={`Edit — ${doctor.DoctorName}`} onClose={onClose}>

  {err && <div className="form-error"><AlertCircle size={14}/> {err}</div>}

  <form onSubmit={save} className="data-form" style={{ padding:0, background:'none',
boxShadow:'none' }}>

    <div className="form-row">

      <div className="form-group"><label>Full Name *</label><input type="text"
value={form.doctor_name} onChange={e=>setForm({...form,doctor_name:e.target.value}}
required/></div>

      <div className="form-group"><label>Email *</label><input type="email" value={form.email}
onChange={e=>setForm({...form,email:e.target.value}}) required/></div>

    </div>

    <div className="form-row">

      <div className="form-group"><label>Specialty *</label><input type="text"
value={form.specialty} onChange={e=>setForm({...form,specialty:e.target.value}}) required/></div>

      <div className="form-group"><label>Phone Number *</label><input type="tel"
value={form.phone_number} onChange={e=>setForm({...form,phone_number:e.target.value}})
required/></div>

    </div>

    <div className="form-row">

      <div className="form-group"><label>License Number</label><input type="text"
value={form.license_number} onChange={e=>setForm({...form,license_number:e.target.value}})
placeholder="Optional"/></div>

      <div className="form-group"><label>Years of Experience</label><input type="number"
min="0" max="50" value={form.years_of_experience}
onChange={e=>setForm({...form,years_of_experience:e.target.value}})
placeholder="Optional"/></div>

    </div>

    <button type="submit" className="submit-btn" disabled={saving} style={{ marginTop:'0.5rem'
}}>

    <Save size={15}/> {saving ? 'Saving...' : 'Save Changes'}

```

```

        </button>
    </form>
</Modal>
);
}

// _____
// AssignCredsModal
// _____

function AssignCredsModal({ patient, token, onClose, onSave }:
{ patient: PatientNoCreds; token: string; onClose: ()=>void; onSave: ()=>void }) {
    const [form, setForm] = useState({ username: "", password: "" });
    const [show, setShow] = useState(false);
    const [saving, setSaving] = useState(false);
    const [err, setErr] = useState("");

    async function save(e: React.FormEvent) {
        e.preventDefault(); setSaving(true); setErr("");
        try {
            const res = await fetch(`${API}/admin/assign-credentials`, {
                method: 'POST', headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },
                body: JSON.stringify({ patient_id: patient.PatientID, ...form }),
            });
            const d = await res.json();
            if (!res.ok) throw new Error(d.error);
            onSave();
        } catch (ex: any) { setErr(ex.message); }
        finally { setSaving(false); }
    }

    return (

```

```

<Modal title={`Assign Credentials — ${patient.PatientName}`} onClose={onClose}>

  <p style={{ color:'var(--text-dim)', fontSize:'0.85rem', marginBottom:'1.25rem' }}>This patient was
  imported via CSV and has no login account yet.</p>

  {err && <div className="form-error"><AlertCircle size={14}/> {err}</div>}

  <form onSubmit={save} className="data-form" style={{ padding:0, background:'none',
  boxShadow:'none' }}>

    <div className="form-group"><label>Username *</label><input type="text"
    value={form.username} minLength={3} required
    onChange={e=>setForm({...form,username:e.target.value})} placeholder="Min 3 characters"/></div>

    <div className="form-group">

      <label>Password *</label>

      <div style={{ position:'relative' }}>

        <input type={show?'text':'password'} value={form.password} minLength={6} required
        onChange={e=>setForm({...form,password:e.target.value})} placeholder="Min 6 characters"
        style={{ paddingRight:'2.5rem' }}/>

        <button type="button" onClick={()=>setShow(v=>!v)}

          style={{ position:'absolute', right:10, top:'50%', transform:'translateY(-50%)',
          background:'none', border:'none', color:'var(--text-dim)', cursor:'pointer', display:'flex' }}>

            {show ? <EyeOff size={15}/> : <Eye size={15}/>}

        </button>

      </div>

    </div>

    <div style={{ background:'var(--accent-dim)', border:'1px solid rgba(6,182,212,.15)',
    borderRadius:'var(--r-md)', padding:'0.75rem 1rem', marginBottom:'1rem', fontSize:'0.82rem',
    color:'var(--accent-light)' }}>

      <strong>{patient.PatientName}</strong> · Blood: {patient.BloodGroup} · {patient.Email}

    </div>

    <button type="submit" className="submit-btn" disabled={saving}>

      <Key size={15}/> {saving ? 'Assigning...' : 'Assign Credentials'}

    </button>

  </form>

</Modal>

);

```



```
}
```

```
// _____
```

```
// ScansModal
```

```
// _____
```

```
function ScansModal({ patientId, patientName, token, onClose }:
```

```
  { patientId: number; patientName: string; token: string; onClose: ()=>void }) {
```

```
  const [scans, setScans] = useState<ScanFile[]>([]);
```

```
  const [loading, setLoading] = useState(true);
```

```
  useEffect(() => {
```

```
    fetch(`${API}/scans/patient/${patientId}`, { headers:{ Authorization: `Bearer ${token}` } })
```

```
      .then(r=>r.json()).then(setScans).catch(console.error).finally(()=>setLoading(false));
```

```
  }, [patientId]);
```

```
  return (
```

```
    <Modal title={`Scans — ${patientName}`} onClose={onClose} wide>
```

```
      {loading ? <div style={{ textAlign:'center', padding:'2rem', color:'var(--text-dim)' }}>Loading  
scans...</div>
```

```
      : scans.length === 0 ? (
```

```
        <div className="empty-state" style={{ padding:'2rem' }}>
```

```
          <ScanLine size={48} style={{ opacity:0.3 }}/>
```

```
          <p>No scans uploaded for this patient yet.</p>
```

```
        </div>
```

```
      ) : (
```

```
        <div style={{ display:'flex', flexDirection:'column', gap:'0.75rem' }}>
```

```
          {scans.map(s => (  
            <div key={s.FileID} style={{ background:'var(--rose-dim)', border:'1px solid  
rgba(244,63,94,.14)', borderRadius:'var(--r-lg)', padding:'0.875rem 1.25rem', display:'flex',  
alignItems:'center', gap:'1rem' }}>
```

```
              <div style={{ width:44, height:44, borderRadius:11, background:'rgba(244,63,94,.1)',  
border:'1px solid rgba(244,63,94,.2)', display:'flex', alignItems:'center', justifyContent:'center',  
color:'var(--rose)', flexShrink:0 }}><ScanLine size={20}/></div>
```

```

<div style={{ flex:1, minWidth:0 }}>

  <p style={{ margin:0, fontWeight:700, fontSize:'0.9rem' }}>{s.FileName}</p>

  <p style={{ margin:'0.15rem 0 0', fontSize:'0.78rem', color:'var(--text-dim)' }}>{s.FileType} ·
  {{s.FileSize/1024/1024).toFixed(2)}} MB · {fmtDate(s.UploadedAt)} ·
  <strong>{s.UploadedByUsername}</strong></p>

  {s.Description && <p style={{ margin:'0.2rem 0 0', fontSize:'0.78rem', color:'var(--text-mid)'
  }}>{s.Description}</p>}

</div>

<a href={`$${API}/files/download/${s.FileID}`} target="_blank" rel="noreferrer"

  style={{ background:'rgba(244,63,94,.12)', border:'1px solid rgba(244,63,94,.25)',
  borderRadius:9, padding:'0.45rem 0.875rem', color:'var(--rose)', display:'flex', alignItems:'center',
  gap:'0.35rem', fontSize:'0.8rem', fontWeight:700, textDecoration:'none' }}>

  <Download size={14}/> Download

</a>

</div>

)}}

</div>

)}

</Modal>

);

}

```

```

// _____
// PatientTimeline — 12-month chronological event feed
// _____

type TLEvent = {
  date: Date; type: 'visit'|'diagnosis'|'rx'|'lab'|'file';
  title: string; subtitle?: string;
  severity?: string; isChronic?: boolean; status?: string;
  color: string; bg: string;
};

```

```

function PatientTimeline({ visits, diagnoses, prescriptions, labTests, files }:
{ visits:any[]; diagnoses:any[]; prescriptions:any[]; labTests:any[]; files:any[] }) {
  const oneYearAgo = new Date(); oneYearAgo.setFullYear(oneYearAgo.getFullYear()-1);

  const events: TLEvent[] = [
    ...visits.map(v=>({ date:new Date(v.VisitDate | | v.visitDate | | ''), type:'visit' as const,
      title:v.ReasonForVisit | | 'Visit', subtitle:`${v.DoctorName | | ''} · ${v.Status | | ''}`,
      status:v.Status, color:'#06b6d4', bg:'var(--accent-dim)' })),
    ...diagnoses.map(d=>({ date:new Date(d.VisitDate | | ''), type:'diagnosis' as const,
      title:d.DiagnosisName | | 'Diagnosis', subtitle:d.Description,
      severity:d.Severity, isChronic:!!d.IsChronic,
      color:d.IsChronic?'#8b5cf6': '#10b981',
      bg:d.IsChronic?'var(--violet-dim)': 'var(--emerald-dim)' })),
    ...prescriptions.map(rx=>({ date:new Date(rx.VisitDate | | ''), type:'rx' as const,
      title:`${rx.MedicineName | | ''} ${rx.Dosage | | ''}`.trim() | | 'Prescription',
      subtitle:`${rx.Frequency | | ''} · ${rx.Duration | | ''}`.replace(/^\cdot | \cdot$/, ""),
      status:rx.IsDispensed?'Dispensed':'Pending',
      color:'#10b981', bg:'var(--emerald-dim)' })),
    ...labTests.map(t=>({ date:new Date(t.CreatedAt | | ''), type:'lab' as const,
      title:t.TestName | | 'Lab Test', subtitle:t.TestType,
      status:t.Status, color:'#06d6a0', bg:'rgba(6,214,160,0.1)' })),
    ...files.map(f=>({ date:new Date(f.UploadedAt | | ''), type:'file' as const,
      title:f.FileName | | 'File', subtitle:`${f.FileType | | ''} · ${((f.FileSize | | 0)/1024/1024).toFixed(1)} MB`,
      color:'#f59e0b', bg:'var(--amber-dim)' })),
  ].filter(e => !isNaN(e.date.getTime()) && e.date >= oneYearAgo)
  .sort((a,b) => b.date.getTime() - a.date.getTime());

  const typeIcon:Record<string,string> = { visit:'🏠', diagnosis:'🩺', rx:'💊', lab:'🧪', file:'📄' };
  const typeLabel:Record<string,string> = { visit:'VISIT', diagnosis:'DIAGNOSIS', rx:'PRESCRIPTION',
lab:'LAB TEST', file:'FILE' };

```

```

const byMonth: Record<string,TLEvent[]> = {};

events.forEach(e => {

  const k = e.date.toLocaleDateString('en-GB',{month:'long',year:'numeric'});

  if (!byMonth[k]) byMonth[k]=[];

  byMonth[k].push(e);

});

if (events.length===0) return (

  <div className="empty-state">

    <div style={{ fontSize:36,opacity:.25,marginBottom:'0.5rem' }}> 📅 </div>

    <h3>No activity in the last 12 months</h3>

    <p>Visits, diagnoses, prescriptions, lab tests and files all appear here chronologically.</p>

  </div>

);

return (

  <div>

    { /* Summary pills */ }

    <div style={{ display:'flex',gap:'0.5rem',flexWrap:'wrap',marginBottom:'1.5rem' }}>

      {[

        { label:'Total Events', count:events.length, color:'var(--text-bright)' },

        { label:'Visits', count:events.filter(e=>e.type==='visit').length, color:'#06b6d4' },

        { label:'Diagnoses', count:events.filter(e=>e.type==='diagnosis').length, color:'#8b5cf6' },

        { label:'Prescriptions',count:events.filter(e=>e.type==='rx').length, color:'#10b981' },

        { label:'Lab Tests', count:events.filter(e=>e.type==='lab').length, color:'#06d6a0' },

        { label:'Files', count:events.filter(e=>e.type==='file').length, color:'#f59e0b' },

      ]}.map(s=>(

        <div key={s.label} style={{ padding:'0.35rem 0.875rem',borderRadius:999,border:'1px solid var(--border)',background:'var(--raised)',fontSize:'0.77rem',color:'var(--text-mid)',display:'flex',gap:'0.4rem',alignItems:'center' }}>

          <strong style={{ color:s.color,fontFamily:"'Syne',sans-serif",fontWeight:800 }}>{s.count}</strong>{' '} {s.label}


```

</div>

)))

</div>

{/* Vertical timeline */}

<div style={{ position:'relative' }}>

<div style={{ position:'absolute',left:19,top:0,bottom:0,width:2;background:'var(--border)',borderRadius:2 }}/>

{Object.entries(byMonth).map(([month,evts])=>{

<div key={month} style={{ marginBottom:'1.75rem' }}>

{/* Month marker */}

<div style={{ display:'flex',alignItems:'center',gap:'0.875rem',marginBottom:'0.875rem',position:'relative' }}>

<div style={{ width:40,height:40,borderRadius:'50%',background:'var(--card-bg)',border:'2px solid var(--accent)',display:'flex',alignItems:'center',justifyContent:'center',fontSize:'0.58rem',fontWeight:800,fontFamily:'"Syne",sans-serif',color:'var(--accent)',flexShrink:0,zIndex:1,lineHeight:1.2,textAlign:'center',padding:'0.25rem' }}>

{month.split(' ')[0].slice(0,3).toUpperCase()}
{month.split(' ')[1].slice(2)}

</div>

{month}

{evts.length} event{evts.length!==1?'s':''}

</div>

{/* Events */}

<div style={{ marginLeft:56,display:'flex',flexDirection:'column',gap:'0.5rem' }}>

{evts.map((e,i)=>{

<div key={i} style={{ background:e.bg,border:`1px solid \${e.color}30`,borderRadius:'var(--r-md)',padding:'0.75rem 1rem',display:'flex',alignItems:'flex-start',gap:'0.75rem' }}>

{typeIcon[e.type]}

<div style={{ flex:1,minWidth:0 }}>

```

        <div style={{
display:'flex',alignItems:'center',gap:'0.5rem',flexWrap:'wrap',marginBottom:'0.2rem' }}>

        <span style={{ fontWeight:600,fontSize:'0.875rem',color:'var(--text-bright)'
}}>{e.title}</span>

        {e.severity && <span className={`badge ${e.severity==='Severe'?'sev-badge-
severe':e.severity==='Moderate'?'sev-badge-moderate':'sev-badge-mild'}`}>{e.severity}</span>}

        {e.isChronic && <span className="badge badge-chronic">Chronic</span>}

        {e.status && <span style={{ fontSize:'0.7rem',padding:'0.15rem
0.5rem',borderRadius:4,fontWeight:700,
background:e.status==='Completed' || e.status==='Dispensed'?'var(--emerald-
dim)':e.status==='Cancelled'?'var(--rose-dim)':'var(--amber-dim)',
color:e.status==='Completed' || e.status==='Dispensed'?'var(--
emerald)':e.status==='Cancelled'?'var(--rose)':'var(--amber)' }}>{e.status}</span>}

        </div>

        {e.subtitle && <p style={{ margin:0,fontSize:'0.78rem',color:'var(--text-
dim)',overflow:'hidden',textOverflow:'ellipsis',whiteSpace:'nowrap' }}>{e.subtitle}</p>}

        <p style={{ margin:'0.2rem 0 0',fontSize:'0.68rem',color:'var(--text-
dim)',fontFamily:"'JetBrains Mono',monospace" }}>

        {e.date.toLocaleDateString('en-GB',{day:'2-digit',month:'short',year:'numeric'})} · <span
style={{ color:e.color,fontWeight:700 }}>{typeLabel[e.type]}</span>

        </p>

        </div>

    </div>

    )}

</div>

</div>

    )}

</div>

</div>

);
}

```

```
// _____
```

```
// AdminPatientDetailPane
```

```

// Full in-depth view of a patient for admin:
// stats summary → visits → diagnoses → prescriptions → lab tests → files
// Plus 1-year data retention management
// _____

function AdminPatientDetailPane({ patient, token, onEdit, onDelete }:
  { patient: Patient & { PatientID: number }; token: string; onEdit:()=>void; onDelete:()=>void }) {

  const [detail, setDetail] = useState<AdminPatientFull| null>(null);
  const [loading, setLoading] = useState(true);
  const [err, setErr] = useState("");
  const [activeTab, setActiveTab] =
    useState<'overview' | 'timeline' | 'visits' | 'diagnoses' | 'prescriptions' | 'labs' | 'files'>('overview');
  const [diagSummary, setDiagSummary] = useState<any[]>([]);
  const [openVisit, setOpenVisit] = useState<number| null>(null);
  const [purging, setPurging] = useState(false);
  const [purgeMsg, setPurgeMsg] = useState("");

  useEffect(() => {
    setLoading(true); setErr(""); setDetail(null); setDiagSummary([]);
    fetch(`${API}/patients/${patient.PatientID}/full-details`, {
      headers: { Authorization: `Bearer ${token}` }
    })
      .then(r => r.json())
      .then(d => { if (d.error) setErr(d.error); else setDetail(d); })
      .catch(() => setErr('Failed to load patient details'))
      .finally(() => setLoading(false));

    // Top diagnoses analytics
    fetch(`${API}/analytics/patient-diagnosis-summary/${patient.PatientID}`, {
      headers: { Authorization: `Bearer ${token}` }
    }).then(r=>r.json()).then(d=>{ if(Array.isArray(d)) setDiagSummary(d); }).catch(()=>{});
  }, [patient.PatientID, token]);

```

```

async function purgeOldData() {
  if (!window.confirm(`This will permanently delete all records for ${patient.PatientName} older than
1 year. Continue?`)) return;

  setPurging(true); setPurgeMsg('');

  try {

    const res = await fetch(`${API}/admin/patients/${patient.PatientID}/purge-old-data`, {
      method: 'DELETE',
      headers: { Authorization: `Bearer ${token}` }
    });

    const d = await res.json();

    if (!res.ok) throw new Error(d.error);

    setPurgeMsg(`✅ ${d.message}`);

    // reload detail

    const r2 = await fetch(`${API}/patients/${patient.PatientID}/full-details`, { headers: {
Authorization: `Bearer ${token}` } });

    const d2 = await r2.json();

    setDetail(d2);

  } catch(e:any) { setPurgeMsg(`Error: ${e.message}`); }

  finally { setPurging(false); }

}

```

```

async function downloadFile(fid: number, name: string) {

  const res = await fetch(`${API}/files/download/${fid}`, { headers: { Authorization: `Bearer ${token}`
} });

  if (!res.ok) return alert('Download failed');

  const a = document.createElement('a'); a.href = URL.createObjectURL(await res.blob());
a.download = name;

  document.body.appendChild(a); a.click(); URL.revokeObjectURL(a.href);
document.body.removeChild(a);

}

```

```

const age = new Date().getFullYear() - new Date(patient.DateOfBirth).getFullYear();

```



```
const oneYearAgo = new Date(); oneYearAgo.setFullYear(oneYearAgo.getFullYear() - 1);
```

```
// Stats
```

```
const thisYearVisits = detail?.visits.filter(v => new Date(v.VisitDate) >= oneYearAgo).length ?? 0;
```

```
const totalVisits = detail?.visits.length ?? 0;
```

```
const chronicCount = detail?.diagnoses.filter(d => d.IsChronic).length ?? 0;
```

```
const pendingRx = detail?.prescriptions.filter(p => !p.IsDispensed).length ?? 0;
```

```
const oldRecords = detail?.visits.filter(v => new Date(v.VisitDate) < oneYearAgo).length ?? 0;
```

```
const TABS = [
```

```
  { key:'overview', label:'Overview', count: null },
```

```
  { key:'timeline', label:'Timeline ♦', count: null },
```

```
  { key:'visits', label:'Visits', count: totalVisits },
```

```
  { key:'diagnoses', label:'Diagnoses', count: detail?.diagnoses.length },
```

```
  { key:'prescriptions', label:'Prescriptions', count: detail?.prescriptions.length },
```

```
  { key:'labs', label:'Lab Tests', count: detail?.lab_tests.length },
```

```
  { key:'files', label:'Files', count: detail?.files.length },
```

```
] as const;
```

```
return (
```

```
  <div style={{ height:'100%', display:'flex', flexDirection:'column', overflow:'hidden' }}>
```

```
    { /* — Patient Header — */ }
```

```
    <div style={{ padding:'1.25rem 1.5rem', background:'var(--card-bg)', borderBottom:'1px solid var(--border)', flexShrink:0 }}>
```

```
      <div style={{ display:'flex', alignItems:'flex-start', justifyContent:'space-between', gap:'1rem', flexWrap:'wrap' }}>
```

```
        <div style={{ display:'flex', alignItems:'center', gap:'1rem' }}>
```

```
          <div style={{ width:52, height:52, borderRadius:'var(--r-lg)', background:'linear-gradient(135deg,var(--accent-dim),var(--violet-dim))', border:'1px solid rgba(6,182,212,.25)', display:'flex', alignItems:'center', justifyContent:'center', fontFamily:'"Syne",sans-serif', fontWeight:800, fontSize:'1.1rem', color:'var(--accent)', flexShrink:0 }}>
```

```
            {getInitials(patient.PatientName)}
```

```
          </div>
```

```

<div>

  <div style={{ fontFamily:'Syne',sans-serif', fontSize:'1.15rem', fontWeight:800, color:'var(--text-bright)', marginBottom:'0.2rem' }}>{patient.PatientName}</div>

  <div style={{ fontSize:'0.8rem', color:'var(--text-dim)' }}>

    {patient.Gender} · {age} yrs · {fmtDate(patient.DateOfBirth)} · {patient.PhoneNumber}

  </div>

  <div style={{ display:'flex', gap:'0.35rem', marginTop:'0.35rem', flexWrap:'wrap' }}>

    <span className="badge badge-blood">{patient.BloodGroup}</span>

    {chronicCount > 0 && <span className="badge badge-chronic">${chronicCount}
Chronic</span>}

    <span className="badge badge-accent">{thisYearVisits} visits this year</span>

  </div>

</div>

</div>

</div>

<div style={{ display:'flex', gap:'0.5rem', flexWrap:'wrap' }}>

  <button onClick={onEdit} className="pab pab-summary"><Edit2 size={13}/> Edit</button>

  <button onClick={()=>onDelete()} className="pab" style={{ background:'var(--rose-dim)',
border:'1px solid rgba(244,63,94,.2)', color:'var(--rose)' }}><Trash2 size={13}/> Deactivate</button>

</div>

</div>

</div>

```

```

{/* — Tabs — */}

```

```

<div style={{ display:'flex', gap:0, borderBottom:'1px solid var(--border)', background:'var(--card-bg)', flexShrink:0, overflowX:'auto' }}>

  {TABS.map(t => (

    <button key={t.key} onClick={()=>setActiveTab(t.key as any)}

      style={{ padding:'0.75rem 1.125rem', border:'none', borderBottom: activeTab===t.key ? '2px solid var(--accent)' : '2px solid transparent', background:'none', color: activeTab===t.key ? 'var(--accent)' : 'var(--text-dim)', fontFamily:'Syne',sans-serif', fontWeight:700, fontSize:'0.8rem',
cursor:'pointer', display:'flex', alignItems:'center', gap:'0.4rem', whiteSpace:'nowrap',
transition:'color var(--t-fast)', letterSpacing:'0.02em' }}>

        {t.label}


```

```

        {t.count != null && <span style={{ padding:'0.1rem 0.4rem', borderRadius:999,
fontSize:'0.65rem', background: activeTab===t.key ? 'var(--accent-dim)' : 'var(--raised)', color:
activeTab===t.key ? 'var(--accent)' : 'var(--text-dim)', fontWeight:700 }}>{t.count}</span>

        </button>

    )}}

</div>

{/* — Tab Body — */}

<div style={{ flex:1, overflowY:'auto', padding:'1.25rem 1.5rem' }}>

    {loading && <div style={{ textAlign:'center', padding:'3rem', color:'var(--text-dim)' }}>Loading
patient details...</div>}

    {err    && <div style={{ color:'var(--rose)', padding:'1rem' }}>{err}</div>}

    {!loading && detail && (<

        {/* == OVERVIEW TAB == */}

        {activeTab === 'overview' && (

            <div>

                {/* Stat cards */}

                <div style={{ display:'grid', gridTemplateColumns:'repeat(auto-fit,minmax(150px,1fr))',
gap:'0.875rem', marginBottom:'1.25rem' }}>

                    {[

                        { label:'Total Visits',    value: totalVisits,          color:'var(--accent)',  bg:'var(--accent-
dim)' },

                        { label:'Visits This Year', value: thisYearVisits,      color:'var(--emerald)', bg:'var(--
emerald-dim)' },

                        { label:'Chronic Conditions', value: chronicCount,      color:'var(--violet)',  bg:'var(--
violet-dim)' },

                        { label:'Pending Rx',      value: pendingRx,          color:'var(--amber)',   bg:'var(--
amber-dim)' },

                        { label:'Lab Tests',       value: detail.lab_tests.length, color:'#06d6a0',
bg:'rgba(6,214,160,.1)' },

                        { label:'Files Uploaded',  value: detail.files.length,  color:'var(--text-mid)', bg:'var(--
raised)' },

```

```

    ].map(s => (
      <div key={s.label} style={{ padding:'0.875rem 1rem', background: s.bg, border:'1px solid
var(--border)', borderRadius:'var(--r-md)' }}>

        <div style={{ fontFamily:'Syne',sans-serif', fontSize:'1.6rem', fontWeight:800, color:
s.color, lineHeight:1 }}>{s.value}</div>

        <div style={{ fontSize:'0.75rem', color:'var(--text-dim)', marginTop:'0.3rem'
}}>{s.label}</div>

      </div>
    )))
  </div>

```

```

  { /* Demographics */ }

  <div style={{ display:'grid', gridTemplateColumns:'1fr 1fr', gap:'0.875rem',
marginBottom:'1.25rem' }}>

    <div className="info-card">

      <div className="info-card-title">Contact Details</div>

      <div className="kv-pair"><div className="kv-label">Email</div><div className="kv-
value" style={{ fontSize:'0.82rem' }}>{patient.Email}</div></div>

      <div className="kv-pair"><div className="kv-label">Phone</div><div className="kv-
value">{patient.PhoneNumber}</div></div>

      <div className="kv-pair"><div className="kv-label">Address</div><div className="kv-
value">{patient.Address}</div></div>

    </div>

    <div className="info-card">

      <div className="info-card-title">Emergency Contact</div>

      <div className="kv-pair"><div className="kv-label">Name</div><div className="kv-
value">{patient.EmergencyContactName || '—'}</div></div>

      <div className="kv-pair"><div className="kv-label">Phone</div><div className="kv-
value">{patient.EmergencyContact || '—'}</div></div>

      <div className="kv-pair"><div className="kv-label">Blood Group</div><div
className="kv-value" style={{ color:'var(--rose)', fontWeight:700
}}>{patient.BloodGroup}</div></div>

    </div>

  </div>

```

```

/* Visit frequency bar chart (last 12 months) */
<div className="info-card" style={{ marginBottom:'1.25rem' }}>

  <div className="info-card-title">Visit Frequency — Last 12 Months</div>

  {{{() => {

    const months: Record<string,number> = {};

    for (let i = 11; i >= 0; i--) {

      const d = new Date(); d.setMonth(d.getMonth() - i);

      const k = d.toLocaleDateString('en-GB',{month:'short',year:'2-digit'});

      months[k] = 0;

    }

    detail.visits.forEach(v => {

      if (new Date(v.VisitDate) >= oneYearAgo) {

        const k = new Date(v.VisitDate).toLocaleDateString('en-GB',{month:'short',year:'2-digit'});

        if (k in months) months[k]++;

      }

    });

    const entries = Object.entries(months);

    const maxC = Math.max(...entries.map(([c])=>c), 1);

    return (

      <div style={{ display:'flex', alignItems:'flex-end', gap:4, height:60, marginTop:'0.5rem' }}>

        {entries.map(([mo,cnt])=>(

          <div key={mo} style={{ display:'flex', flexDirection:'column', alignItems:'center', gap:3,
flex:1 }}>

            <div title={` ${mo}: ${cnt} visit${cnt!==1?'s':''}`} style={{ width:'100%', minHeight:4,
height: Math.max(4,(cnt/maxC)*48), background: cnt>0 ? 'linear-gradient(180deg,var(--accent),var(--violet))' : 'var(--border)', borderRadius:'3px 3px 0 0', transition:'height .5s ease' }}/>

            <span style={{ fontSize:7, color:'var(--text-dim)', fontFamily:'"JetBrains Mono',monospace", whiteSpace:'nowrap' }}>{mo}</span>

          </div>

        ))}

      </div>

    );

  }}}

```

```

    })()
  </div>

  { /* 1-Year Data Retention Policy */

    <div style={{ background: oldRecords>0 ? 'var(--amber-dim)' : 'var(--raised)', border: `1px solid
    ${oldRecords>0?'rgba(245,158,11,.25)': 'var(--border)'}`, borderRadius: 'var(--r-lg)', padding: '1.25rem
    1.5rem' }}>

      <div style={{ display: 'flex', alignItems: 'center', justifyContent: 'space-between', gap: '1rem',
      flexWrap: 'wrap' }}>

        <div>

          <div style={{ display: 'flex', alignItems: 'center', gap: '0.5rem', marginBottom: '0.35rem' }}>

            <Clock size={16} style={{ color: oldRecords>0 ? 'var(--amber)' : 'var(--text-dim)' }}/>

            <span style={{ fontFamily: "'Syne', sans-serif", fontWeight: 800, fontSize: '0.9rem',
            color: 'var(--text-bright)' }}>1-Year Data Retention Policy</span>

          </div>

          <p style={{ fontSize: '0.82rem', color: 'var(--text-dim)', margin: 0 }}>

            {oldRecords > 0

              ? <><strong style={{ color: 'var(--amber)' }}>{oldRecords} visit
              record{oldRecords!==1?'s':''}</strong> older than 1 year found. You can purge them to free storage
              and stay compliant.</>

              : 'All records are within the 1-year retention window. No cleanup needed.'

            }

          </p>

          {purgeMsg && <p style={{ margin: '0.5rem 0 0', fontSize: '0.82rem', color:
          purgeMsg.startsWith('Error') ? 'var(--rose)' : 'var(--emerald)', fontWeight: 600 }}>{purgeMsg}</p>}

        </div>

        {oldRecords > 0 && (

          <button onClick={purgeOldData} disabled={purging}

            style={{ padding: '0.6rem 1.25rem', background: 'var(--amber)', border: 'none',
            borderRadius: 'var(--r-md)', color: '#fff', fontFamily: "'Syne', sans-serif", fontWeight: 700,
            fontSize: '0.82rem', cursor: purging?'wait':'pointer', opacity: purging?0.65:1, whiteSpace: 'nowrap',
            display: 'flex', alignItems: 'center', gap: '0.4rem' }}>

              <Trash2 size={14}/> {purging ? 'Purging...' : `Purge ${oldRecords} Old
              Record${oldRecords!==1?'s':''}`}

            </button>

```

```

    }}
  </div>
</div>
</div>
))

{/* == VISITS TAB == */}
{activeTab === 'visits' && (
  <div>
    <div style={{ display:'flex', alignItems:'center', justifyContent:'space-between',
marginBottom:'0.875rem' }}>
      <p style={{ margin:0, fontSize:'0.82rem', color:'var(--text-dim)' }}>{totalVisits} total · <strong
style={{ color:'var(--accent)' }}>{thisYearVisits}</strong> in last 12 months · <strong style={{
color:'var(--amber)' }}>{oldRecords}</strong> older than 1 year</p>
    </div>
    {detail.visits.length === 0
      ? <div className="empty-state"><Calendar size={40} style={{ opacity:.3 }}/><p>No visits
recorded</p></div>
      : detail.visits.map((v, i) => {
        const vitals = parseVitals(v.VitalSigns);
        const isOld = new Date(v.VisitDate) < oneYearAgo;
        return (
          <div key={i} className="visit-accordion" style={{ opacity: isOld ? 0.7 : 1 }}>
            <div className={`visit-header ${openVisit===i?'open':''}`}
onClick={()=>setOpenVisit(openVisit===i?null:i)}>
              <div className="visit-header-left">
                <div className="visit-status-dot" style={{ background:
v.Status==='Completed'?'#10b981':v.Status==='In Progress'?'#f59e0b': '#475569' }}/>
                <div className="visit-header-info">
                  <div className="visit-reason">{v.ReasonForVisit}</div>
                  <div className="visit-meta">{v.DoctorName} · {fmtDate(v.VisitDate)} {isOld &&
<span style={{ color:'var(--amber)', fontSize:'0.7rem', fontWeight:700 }}>· &gt;1yr old</span></div>
                </div>

```

```

    </div>

    <div style={{ display:'flex', alignItems:'center', gap:'0.5rem' }}>

        <span style={{ fontSize:'0.72rem', padding:'0.15rem 0.5rem', borderRadius:4,
background: v.Status==='Completed'?var(--emerald-dim):'var(--amber-dim)', color:
v.Status==='Completed'?var(--emerald):'var(--amber)', fontWeight:600 }}>{v.Status}</span>

        <span className={`visit-chevron ${openVisit===i?'open':''}`}><ChevronDown
size={16}/></span>

    </div>

</div>

{openVisit===i && (

    <div className="visit-body">

        {vitals.bp && <div className="vital-item"><div className="vital-label">Blood
Pressure</div><div className="vital-value">{vitals.bp}</div></div>}

        {vitals.temp && <div className="vital-item"><div className="vital-
label">Temperature</div><div className="vital-value">{vitals.temp}°F</div></div>}

        {vitals.pulse && <div className="vital-item"><div className="vital-
label">Pulse</div><div className="vital-value">{vitals.pulse} bpm</div></div>}

        {vitals.spo2 && <div className="vital-item"><div className="vital-
label">SpO2</div><div className="vital-value">{vitals.spo2}</div></div>}

        {v.Notes && <div className="visit-notes"><div className="vital-label" style={{
marginBottom:'0.3rem' }}>Notes</div>{v.Notes}</div>}

    </div>

    )}

</div>

);

})

}

</div>

)}

{ /* == DIAGNOSES TAB == */}

{activeTab === 'diagnoses' && (

    detail.diagnoses.length === 0

```



```
    ? <div className="empty-state"><Stethoscope size={40} style={{ opacity:.3 }}/><p>No  
diagnoses recorded</p></div>
```

```
    : detail.diagnoses.map((d, i) => (  
      <div key={i} className="diagnosis-row">  
        <div>  
          <div className="diagnosis-name">{d.DiagnosisName}</div>  
          {d.Description} && <div className="diagnosis-desc">{d.Description}</div>  
          <div className="diagnosis-date">{fmtDate(d.VisitDate)} · {d.DoctorName}</div>  
        </div>  
        <div className="diagnosis-badges">  
          {d.Severity} && <span className={`badge ${sevClass(d.Severity)}`}>{d.Severity}</span>  
          {d.IsChronic} && <span className="badge badge-chronic">Chronic</span>  
        </div>  
      </div>  
    ))  
  )}
```

```
  { /* == PRESCRIPTIONS TAB == */  
  {activeTab === 'prescriptions' && (  
    detail.prescriptions.length === 0  
    ? <div className="empty-state"><Pill size={40} style={{ opacity:.3 }}/><p>No prescriptions  
recorded</p></div>  
    : <div className="table-container">  
      <table className="data-table">  
  
        <thead><tr><th>Medicine</th><th>Dosage</th><th>Frequency</th><th>Duration</th><th>Doctor  
</th><th>Date</th><th>Status</th></tr></thead>  
  
        <tbody>{detail.prescriptions.map((rx, i) => (  
          <tr key={i}>  
            <td><strong>{rx.MedicineName}</strong>{rx.Instructions}&&<p style={{ margin:'0.1rem  
0 0', fontSize:'0.73rem', color:'var(--text-dim)' }}>{rx.Instructions}</p></td>  
            <td>{rx.Dosage}</td><td>{rx.Frequency}</td><td>{rx.Duration}</td>  
            <td>{rx.DoctorName}</td><td>{fmtDate(rx.VisitDate)}</td>
```

```

        <td><span className={`status-badge
${{rx.IsDispensed?'dispensed':'pending'}}`}>{{rx.IsDispensed?'<><CheckCircle size={12}/>
Dispensed</>:<><Clock size={12}/> Pending</>}}</span></td>

```

```

    </tr>
  )}</tbody>
</table>
</div>
}}

```

```

{ /* == LAB TESTS TAB == */
{activeTab === 'labs' && (
  detail.lab_tests.length === 0
  ? <div className="empty-state"><FlaskConical size={40} style={{ opacity:.3 }}/><p>No lab
tests recorded</p></div>

```

```

    : <div className="table-container">
      <table className="data-table">

<thead><tr><th>Test</th><th>Type</th><th>Ordered</th><th>Status</th><th>Results</th></tr></thead>

```

```

    <tbody>{{detail.lab_tests.map((t, i) => (
      <tr key={i}>
        <td><strong>{{t.TestName}}</strong></td>
        <td><span style={{ fontSize:'0.76rem', padding:'0.15rem 0.5rem', borderRadius:5,
background:'var(--accent-dim)', color:'var(--accent)' }}>{{t.TestType}}</span></td>
        <td>{{fmtDate(t.CreatedAt)}}</td>
        <td><span style={{ fontSize:'0.76rem', padding:'0.2rem 0.6rem', borderRadius:5,
fontWeight:600, background: t.Status==='Completed'?'var(--emerald-dim)':'var(--amber-dim)', color:
t.Status==='Completed'?'var(--emerald)':'var(--amber)' }}>{{t.Status}}</span></td>
        <td style={{ fontSize:'0.82rem', color:'var(--text-mid)', maxWidth:200
}}>{{t.Results}}<span style={{ color:'var(--text-dim)', fontStyle:'italic' }}>Pending</span></td>
      </tr>
    )}}</tbody>
</table>
</div>

```

```

    })

    { /* == FILES TAB == */
    {activeTab === 'files' && (
      detail.files.length === 0

      ? <div className="empty-state"><FileImage size={40} style={{ opacity:.3 }}/><p>No files
uploaded</p></div>

      : <div className="files-grid">

        {detail.files.map((f: any) => (
          <div key={f.FileID} className="file-card">

            <div className="file-icon"><FileImage size={28}/></div>

            <div className="file-details">

              <h4>{f.FileType}</h4>

              <p className="file-name">{f.FileName}</p>

              <p className="file-size">{(f.FileSize/1024/1024).toFixed(2)} MB</p>

              <p className="file-date">{fmtDate(f.UploadedAt)} · {f.UploadedByUsername}</p>

              {f.Description && <p className="file-desc">{f.Description}</p>}

            </div>

            <button className="download-btn" onClick={()=>downloadFile(f.FileID,
f.FileName)}><Download size={16}/> Download</button>

          </div>

        )))

      </div>

    })
  }
}

```

```

{ /* == TIMELINE TAB == */
{activeTab === 'timeline' && detail && (
  <PatientTimeline
    visits={detail.visits}
    diagnoses={detail.diagnoses}
    prescriptions={detail.prescriptions}
    labTests={detail.lab_tests}

```

```

        files={detail.files}

    />

  })

  { /* == DIAGNOSIS ANALYTICS — shown at bottom of overview == */

  {activeTab === 'overview' && diagSummary.length > 0 && (

    <div className="info-card" style={{ marginTop:'1.25rem' }}>

      <div className="info-card-title"> 📄 Top Diagnoses — Last 12 Months</div>

      {diagSummary.slice(0,6).map((d:any,i:number)=>{

        <div key={i} style={{ display:'flex',alignItems:'center',gap:'0.75rem',padding:'0.5rem
0',borderBottom:i<Math.min(diagSummary.length,6)-1?'1px solid var(--border)':'none' }}>

          <div style={{ width:30,height:30,borderRadius:7;background:d.IsChronic?'var(--violet-
dim)':'var(--accent-
dim)',display:'flex',alignItems:'center',justifyContent:'center',fontFamily:"'Syne',sans-
serif",fontWeight:800,fontSize:'0.72rem',color:d.IsChronic?'var(--violet)':'var(--accent)',flexShrink:0
}}>

            {d.Frequency}×

          </div>

          <div style={{ flex:1 }}>

            <span style={{ fontWeight:600,fontSize:'0.875rem',color:'var(--text-bright)'
}}>{d.DiagnosisName}</span>

            {d.IsChronic && <span className="badge badge-chronic" style={{
marginLeft:'0.4rem',fontSize:'0.65rem' }}>Chronic</span>}

            {d.WorstSeverity && <span className={`badge ${d.WorstSeverity==='Severe'? 'sev-badge-
severe':d.WorstSeverity==='Moderate'? 'sev-badge-moderate':'sev-badge-mild'}`} style={{
marginLeft:'0.3rem',fontSize:'0.65rem' }}>{d.WorstSeverity}</span>}

          </div>

          <span style={{ fontSize:'0.72rem',color:'var(--text-dim)',fontFamily:"'JetBrains
Mono',monospace' }}>Last: {fmtDate(d.LastSeen)}</span>

        </div>

      )}}

    </div>

  )}

</>

```

```

    </div>

</div>

);
}

// _____
// LoginScreen — Linways-style split layout
// Left: full visual panel | Right: role list → login form
// _____

function LoginScreen({ onLogin, onShowRegister, formErr, loading, theme, setTheme }:
  { onLogin:(u:string,p:string)=>Promise<void>; onShowRegister:()=>void; formErr:string;
  loading:boolean; theme:ThemeKey; setTheme:(k:ThemeKey)=>void }) {

  const [selectedRole, setSelectedRole] = useState<string|null>(null);

  const [username, setUsername]      = useState(" ");
  const [password, setPassword]      = useState(" ");
  const [showPass, setShowPass]      = useState(false);
  const [panel, setPanel]             = useState<'roles' | 'form'>('roles');
  const entity = ENTITIES.find(e => e.role === selectedRole) ?? null;

  function handleRoleClick(role: string) {
    setSelectedRole(role);
    setUsername(""); setPassword("");
    setTimeout(() => setPanel('form'), 80);
  }

  function handleBack() {
    setPanel('roles');
    setTimeout(() => setSelectedRole(null), 300);
  }

  return (

```

```
<div style={{ display:'flex', height:'100vh', overflow:'hidden', fontFamily:'"DM Sans","Segoe UI",sans-serif', background:'var(--bg)' }}>
```

```
  { /* LEFT VISUAL PANEL */ }
```

```
  <div style={{ flex:1, position:'relative', overflow:'hidden', display:'flex', flexDirection:'column', alignItems:'center', justifyContent:'center', minWidth:0 }}>
```

```
    <div style={{ position:'absolute', inset:0, background:'linear-gradient(135deg, var(--bg-1) 0%, var(--bg) 40%, var(--bg-1) 100%)' }}/>
```

```
    <div style={{ position:'absolute', top:'-10%', left:'-5%', width:600, height:600, borderRadius:'50%', background:'radial-gradient(circle, var(--accent-glow) 0%, transparent 65%)', filter:'blur(40px)', animation:'blobFloat1 18s ease-in-out infinite' }}/>
```

```
    <div style={{ position:'absolute', bottom:'-15%', right:'-8%', width:500, height:500, borderRadius:'50%', background:'radial-gradient(circle, var(--violet-dim) 0%, transparent 65%)', filter:'blur(50px)', animation:'blobFloat2 22s ease-in-out infinite 3s' }}/>
```

```
    <div style={{ position:'absolute', top:'40%', left:'55%', width:320, height:320, borderRadius:'50%', background:'radial-gradient(circle, var(--emerald-dim) 0%, transparent 65%)', filter:'blur(40px)', animation:'blobFloat1 15s ease-in-out infinite 6s' }}/>
```

```
    <div style={{ position:'absolute', inset:0, backgroundImage:'linear-gradient(var(--border) 1px, transparent 1px), linear-gradient(90deg, var(--border) 1px, transparent 1px)', backgroundSize:'48px 48px', opacity:.5 }}/>
```

```
  { /* Theme switcher — top-left corner of the page */ }
```

```
  <div style={{ position:'absolute', top:'1rem', left:'1rem', zIndex:10 }}>
```

```
    <ThemeSwitcher theme={theme} setTheme={setTheme}/>
```

```
  </div>
```

```
  <div style={{ position:'relative', zIndex:1, display:'flex', flexDirection:'column', alignItems:'center', gap:'2.5rem', padding:'2rem' }}>
```

```
    { /* Medical cross SVG illustration */ }
```

```
    <svg viewBox="0 0 300 300" style={{ width:'min(260px,32vw)', height:'auto', filter:'drop-shadow(0 0 40px var(--accent-glow))' }}>
```

```
      <circle cx="150" cy="150" r="130" fill="none" stroke="var(--accent)" strokeWidth="1" strokeDasharray="6 4" opacity=".25"/>
```

```
      <circle cx="150" cy="150" r="90" fill="var(--accent-glow)"/>
```

```
      <rect x="118" y="70" width="64" height="160" rx="20" fill="var(--accent)" opacity=".85"/>
```

```
      <rect x="70" y="118" width="160" height="64" rx="20" fill="var(--accent)" opacity=".85"/>
```

```

<rect x="134" y="86" width="32" height="128" rx="10" fill="white" opacity=".2"/>

<rect x="86" y="134" width="128" height="32" rx="10" fill="white" opacity=".2"/>

<polyline points="60,165 85,165 98,138 113,192 128,145 143,165 157,165 170,128 185,196
200,165 240,165" fill="none" stroke="white" strokeWidth="3" strokeLinecap="round"
strokeLinejoin="round" opacity=".65"/>

</svg>

<div style={{ textAlign:'center' }}>

  <div style={{ display:'flex', alignItems:'center', justifyContent:'center', gap:'0.75rem',
marginBottom:'0.5rem' }}>

    <div style={{ width:44, height:44, borderRadius:13, background:'linear-gradient(135deg,var(--
accent),var(--violet))', display:'flex', alignItems:'center', justifyContent:'center', boxShadow:'0 0 24px
var(--accent-glow)' }}>

      <Heart size={22} color="white"/>

    </div>

    <h1 style={{ fontFamily:'Syne',sans-serif', fontSize:'2.1rem', fontWeight:800, letterSpacing:'-
0.04em', margin:0, background:'linear-gradient(135deg, var(--accent-light) 0%, var(--text-bright) 60%,
var(--violet) 100%)', WebkitBackgroundClip:'text', WebkitTextFillColor:'transparent',
backgroundClip:'text' }}>

      MediCare Plus

    </h1>

  </div>

  <p style={{ fontSize:'0.72rem', color:'var(--text-dim)', letterSpacing:'0.18em',
textTransform:'uppercase', fontWeight:700, margin:'0 0 1.75rem' }}>

    Hospital Management System

  </p>

  <div style={{ display:'flex', gap:'0.5rem', justifyContent:'center', flexWrap:'wrap' }}>

    {[ 'Multi-Role Access', 'Chronic Tracking', 'Lab Tests', 'Secure & Private' ].map(f => (

      <span key={f} style={{ padding:'0.3rem 0.875rem', borderRadius:999, background:'var(--
accent-dim)', border:'1px solid var(--border)', fontSize:'0.72rem', color:'var(--text-mid)',
fontWeight:500 }}>{f}</span>

    )))

  </div>

</div>

</div>

```

```
<div style={{ position:'absolute', bottom:'1.25rem', left:0, right:0, textAlign:'center',
fontSize:'0.7rem', color:'var(--text-dim)', letterSpacing:'0.06em' }}>
```

```
    Presidency University · Bengaluru
```

```
</div>
```

```
</div>
```

```
{/* RIGHT PANEL */}
```

```
<div style={{ width:'min(400px,42vw)', minWidth:340, background:'var(--card-bg)',
borderLeft:'1px solid var(--border)', display:'flex', flexDirection:'column', position:'relative',
overflow:'hidden', boxShadow:'-8px 0 40px rgba(0,0,0,0.12)' }}>
```

```
{/* PANEL A: Role selection */}
```

```
<div className={`ls-panel ${panel==='roles'?'ls-panel-in':'ls-panel-out-left'}`
```

```
    style={{ position:'absolute', inset:0, display:'flex', flexDirection:'column', justifyContent:'center',
padding:'2.5rem 2rem', overflowY:'auto' }}>
```

```
<div style={{ marginBottom:'2rem' }}>
```

```
<h2 style={{ fontFamily:'Syne', sans-serif', fontSize:'1.6rem', fontWeight:800, letterSpacing:'-
0.03em', color:'var(--text-bright)', margin:'0 0 0.35rem' }}>Welcome back</h2>
```

```
<p style={{ fontSize:'0.88rem', color:'var(--text-dim)', margin:0 }}>Select your role to sign
in</p>
```

```
</div>
```

```
<div style={{ display:'flex', flexDirection:'column', gap:'0.625rem' }}>
```

```
    {ENTITIES.map((e) => (
```

```
        <button key={e.role} onClick={() => handleRoleClick(e.role)}
```

```
            style={{ all:'unset', display:'flex', alignItems:'center', gap:'1rem', padding:'1rem 1.25rem',
borderRadius:'var(--r-lg)', border:'1px solid var(--border)', background:'var(--raised)', cursor:'pointer',
transition:'all 0.2s cubic-bezier(0.16,1,0.3,1)', position:'relative', overflow:'hidden', boxSizing:'border-
box' }}
```

```
            onMouseEnter={ev => {
```

```
                const b = ev.currentTarget as HTMLButtonElement;
```



```

        b.style.borderColor = e.color; b.style.background = `${e.color}0f`; b.style.transform =
        'translateX(4px)';

    }}

    onMouseLeave={ev => {

        const b = ev.currentTarget as HTMLButtonElement;

        b.style.borderColor = 'var(--border)'; b.style.background = 'var(--raised)'; b.style.transform
        = 'translateX(0)';

    }}>

    <div style={{ position:'absolute', left:0, top:0, bottom:0, width:3, borderRadius:'0 3px 3px 0',
    background:e.color, opacity:0.8 }}/>

    <div style={{ width:40, height:40, borderRadius:10, background:`${e.color}18`, border:`1px
    solid ${e.color}30`, display:'flex', alignItems:'center', justifyContent:'center', color:e.color,
    flexShrink:0 }}>{e.icon}</div>

    <div style={{ flex:1 }}>

        <p style={{ margin:0, fontFamily:'"Syne",sans-serif', fontWeight:800, fontSize:'0.95rem',
        color:'var(--text-bright)' }}>{e.role}</p>

        <p style={{ margin:'0.1rem 0 0', fontSize:'0.75rem', color:'var(--text-dim)' }}>{e.desc}</p>

    </div>

    <ChevronDown size={16} style={{ color:'var(--text-dim)', transform:'rotate(-90deg)',
    flexShrink:0 }}/>

    </button>

    )}}

</div>

<div style={{ marginTop:'2rem', paddingTop:'1.5rem', borderTop:'1px solid var(--border)',
textAlign:'center' }}>

    <p style={{ fontSize:'0.85rem', color:'var(--text-dim)', margin:0 }}>New patient?{' '}

    <button onClick={onShowRegister} style={{ background:'none', border:'none', color:'var(--
    accent)', cursor:'pointer', fontWeight:700, fontFamily:'inherit', fontSize:'0.85rem' }}>Register here
    →</button>

    </p>

</div>

</div>

```

```

{ /* PANEL B: Login form */}

<div className={`ls-panel ${panel==='form'? 'ls-panel-in': 'ls-panel-out-right'}`}

  style={{ position:'absolute', inset:0, display:'flex', flexDirection:'column', justifyContent:'center',
padding:'2.5rem 2rem', overflowY:'auto' }}>

  {entity && (<

    <button onClick={handleBack}

      style={{ all:'unset', display:'inline-flex', alignItems:'center', gap:'0.4rem', fontSize:'0.8rem',
color:'var(--text-dim)', cursor:'pointer', marginBottom:'2rem', fontWeight:600, transition:'color 0.15s'
}}

      onMouseEnter={e => (e.currentTarget.style.color='var(--accent)')}

      onMouseLeave={e => (e.currentTarget.style.color='var(--text-dim)' )}>

        <ChevronDown size={14} style={{ transform:'rotate(90deg)' }}/> All roles

      </button>

      <div style={{ display:'flex', alignItems:'center', gap:'0.875rem', marginBottom:'1.75rem',
paddingBottom:'1.5rem', borderBottom:'1px solid var(--border)' }}>

        <div style={{ width:50, height:50, borderRadius:14, background:`${entity.color}18`,
border:'1px solid ${entity.color}30`, display:'flex', alignItems:'center', justifyContent:'center',
color:entity.color, flexShrink:0 }}>{entity.icon}</div>

        <div>

          <h2 style={{ fontFamily:'Syne', sans-serif", fontSize:'1.4rem', fontWeight:800,
letterSpacing:'-0.02em', color:'var(--text-bright)', margin:'0 0 0.15rem' }}>{entity.role} Login</h2>

          <p style={{ margin:0, fontSize:'0.78rem', color:'var(--text-dim)' }}>{entity.label} ·
{entity.desc}</p>

        </div>

      </div>

    {formErr && (

      <div style={{ padding:'0.75rem 1rem', background:'var(--rose-dim)', border:'1px solid
rgba(220,38,38,0.25)', borderRadius:'var(--r-md)', color:'var(--rose)', fontSize:'0.84rem',
marginBottom:'1.25rem', display:'flex', alignItems:'center', gap:'0.5rem', animation:'shake 0.4s ease'
}}>

        <AlertCircle size={15}/> {formErr}

      </div>

    )}

```

```

<form onSubmit={async e => { e.preventDefault(); await onLogin(username, password); }}>

  <div className="form-group" style={{ marginBottom:'1.1rem' }}>

    <label>Username</label>

    <input type="text" value={username} onChange={e => setUsername(e.target.value)}
required autoComplete="username" placeholder={`Enter ${entity.role.toLowerCase()} username`} />

  </div>

  <div className="form-group" style={{ marginBottom:'1.5rem' }}>

    <label>Password</label>

    <div style={{ position:'relative' }}>

      <input type={showPass?'text':'password'} value={password} onChange={e =>
setPassword(e.target.value)} required autoComplete="current-password" placeholder="Enter your
password" style={{ paddingRight:'2.75rem' }} />

      <button type="button" onClick={() => setShowPass(v => !v)} style={{ position:'absolute',
right:10, top:'50%', transform:'translateY(-50%)', background:'none', border:'none', color:'var(--text-
dim)', cursor:'pointer', display:'flex', alignItems:'center' }}>

        {showPass ? <EyeOff size={15}/> : <Eye size={15}/>}

      </button>

    </div>

  </div>

  <button type="submit" disabled={loading}

    style={{ width:'100%', padding:'0.9rem', background:`linear-
gradient(135deg,$${entity.color},$${entity.color}cc)`, border:'none', borderRadius:'var(--r-md)',
color:'#fff', fontFamily:'"Syne",sans-serif', fontWeight:700, fontSize:'0.9rem', letterSpacing:'0.04em',
cursor:loading?'wait':'pointer', opacity:loading?0.65:1, boxShadow:`0 4px 18px $${entity.glow}`,
display:'flex', alignItems:'center', justifyContent:'center', gap:'0.5rem' }}>

    {loading ? 'Signing in...' : `Sign in as $${entity.role}`}

  </button>

</form>

<div style={{ marginTop:'1.5rem', paddingTop:'1.25rem', borderTop:'1px solid var(--border)',
textAlign:'center' }}>

  <p style={{ fontSize:'0.82rem', color:'var(--text-dim)', margin:0 }}>New patient?{' '}
```

```
        <button onClick={onShowRegister} style={{ background:'none', border:'none', color:'var(--accent)', cursor:'pointer', fontWeight:700, fontFamily:'inherit', fontSize:'0.82rem' }}>Register here
    </button>
```

```
    </p>
```

```
  </div>
```

```
</>}}
```

```
</div>
```

```
</div>
```

```
<style>{'
```

```
    @keyframes blobFloat1 { 0%,100%{transform:translate(0,0) scale(1)}
33%{transform:translate(30px,-20px) scale(1.05)} 66%{transform:translate(-15px,25px) scale(0.97)}} }
```

```
    @keyframes blobFloat2 { 0%,100%{transform:translate(0,0) scale(1)} 40%{transform:translate(-
25px,15px) scale(1.04)} 70%{transform:translate(20px,-18px) scale(0.98)}} }
```

```
    .ls-panel { transition: transform 0.35s cubic-bezier(0.16,1,0.3,1), opacity 0.3s ease; will-change:
transform, opacity; }
```

```
    .ls-panel-in      { transform: translateX(0); opacity: 1; pointer-events: auto; }
```

```
    .ls-panel-out-left { transform: translateX(-110%); opacity: 0; pointer-events: none; }
```

```
    .ls-panel-out-right { transform: translateX(110%); opacity: 0; pointer-events: none; }
```

```
    @media (max-width: 700px) { .ls-left { display:none!important; } .ls-right {
width:100%!important; min-width:0!important; } }
```

```
  `}</style>
```

```
</div>
```

```
);
```

```
}
```

```
// _____
```

```
// RegisterScreen
```

```
// _____
```

```
function RegisterScreen({ onRegister, onBack, formErr, loading }:
```

```
  { onRegister:(e:React.FormEvent,f:RegisterForm)=>Promise<void>; onBack:()=>void; formErr:string;
loading:boolean }) {
```

```
  const [regF, setRegF] = useState<RegisterForm>({
username:'',password:'',email:'',name:'',gender:'Male',dob:'',phone:'',address:'',blood_group:'O+',em
ergency_contact:'',emergency_contact_name:''});
```

```

return (

<div className="login-container"><div className="login-bg"/>

  <div className="login-card">

    <div className="login-header"><div className="logo-container"><UserPlus className="logo-
icon"/></div><h1>Patient Registration</h1><p>Create your account</p></div>

    {formErr && <div className="form-error"><AlertCircle size={15}/> {formErr}</div>}

    <form onSubmit={e=>onRegister(e,regF)} className="register-form">

      <div className="form-row">

        <div className="form-group"><label>Username *</label><input type="text"
value={regF.username} onChange={e=>setRegF({...regF,username:e.target.value})} placeholder="Min
3 chars" minLength={3} required/></div>

        <div className="form-group"><label>Password *</label><input type="password"
value={regF.password} onChange={e=>setRegF({...regF,password:e.target.value})} placeholder="Min
6 chars" minLength={6} required/></div>

      </div>

      <div className="form-row">

        <div className="form-group"><label>Full Name *</label><input type="text"
value={regF.name} onChange={e=>setRegF({...regF,name:e.target.value})} required/></div>

        <div className="form-group"><label>Email *</label><input type="email" value={regF.email}
onChange={e=>setRegF({...regF,email:e.target.value})} required/></div>

      </div>

      <div className="form-row">

        <div className="form-group"><label>Gender *</label>

        <select value={regF.gender}
onChange={e=>setRegF({...regF,gender:e.target.value})}><option>Male</option><option>Female</o
ption><option>Other</option></select>

      </div>

      <div className="form-group"><label>Date of Birth *</label><input type="date"
value={regF.dob} onChange={e=>setRegF({...regF,dob:e.target.value})} max={new
Date().toISOString().split('T')[0]} required/></div>

      </div>

      <div className="form-row">

        <div className="form-group"><label>Phone *</label><input type="tel" value={regF.phone}
onChange={e=>setRegF({...regF,phone:e.target.value})} pattern="[0-9]{10}" required/></div>

        <div className="form-group"><label>Blood Group *</label>

```

```

        <select value={regF.blood_group}
onChange={e=>setRegF({...regF,blood_group:e.target.value})}>
            {[ 'A+', 'A-', 'B+', 'B-', 'O+', 'O-', 'AB+', 'AB-' ].map(bg=><option key={bg}>{bg}</option>)}
        </select>
    </div>
</div>

<div className="form-group"><label>Address *</label><input type="text"
value={regF.address} onChange={e=>setRegF({...regF,address:e.target.value})} required/></div>

<h4 style={{ color:'var(--text-mid)',margin:'1rem 0 0.5rem',fontSize:'0.9rem' }}>Emergency
Contact (Optional)</h4>

<div className="form-row">

    <div className="form-group"><label>Contact Name</label><input type="text"
value={regF.emergency_contact_name}
onChange={e=>setRegF({...regF,emergency_contact_name:e.target.value})}/></div>

    <div className="form-group"><label>Contact Phone</label><input type="tel"
value={regF.emergency_contact}
onChange={e=>setRegF({...regF,emergency_contact:e.target.value})} pattern="[0-9]{10}"/></div>

</div>

<button type="submit" className="login-btn" disabled={loading}>{loading ? 'Creating
Account...' : 'Register'}</button>

</form>

<div className="register-link"><p>Already have an account? <button onClick={onBack}
className="link-btn">Login here</button></p></div>

</div>
</div>
);
}

// _____
// Main App
// _____

export default function App() {
    const [theme, setTheme] = useTheme();

```

```

const [token, setToken] = useState<string | null>(() => localStorage.getItem('token'));

const [user, setUser] = useState<AuthUser | null>(() => { const t=localStorage.getItem('token');
return t?decodeJWT(t):null; });

const [view, setView] = useState('dashboard');

const [navHistory, setNavHistory] = useState<string[]>(['dashboard']);

const [navIndex, setNavIndex] = useState(0);

const [showReg, setShowReg] = useState(false);

const [loading, setLoading] = useState(false);

const [formErr, setFormErr] = useState('');

// Data

const [stats, setStats] = useState<DashboardStats & { pending_appointments?: number }>({
total_patients:0,total_doctors:0,total_radiologists:0,today_visits:0,pending_prescriptions:0,pending_
tests:0 });

const [records, setRecords] = useState<any[]>([]);

const [doctorRecords, setDoctorRecords] = useState<DoctorRecord[]>([]);

const [patients, setPatients] = useState<Patient[]>([]);

const [doctors, setDoctors] = useState<Doctor[]>([]);

const [visits, setVisits] = useState<Visit[]>([]);

const [prescriptions, setPrescriptions] = useState<Prescription[]>([]);

const [medFiles, setMedFiles] = useState<MedicalFile[]>([]);

const [myScans, setMyScans] = useState<ScanFile[]>([]);

const [importResult, setImportResult] = useState<any>(null);

const [pendingOnly, setPendingOnly] = useState(false);

const [noCreds, setNoCreds] = useState<PatientNoCreds[]>([]);

const [patientSearch, setPatientSearch] = useState('');

const [selectedPatient, setSelectedPatient] = useState<(Patient & { PatientID: number }) | null>(null);

const [patientListTab, setPatientListTab] = useState<'mine' | 'other'>('mine');

const [adminSelectedPatient, setAdminSelectedPatient] = useState<(Patient & { PatientID: number
}) | null>(null);

const [adminPatientSearch, setAdminPatientSearch] = useState('');

const [selectedPatientIds, setSelectedPatientIds] = useState<number[]>([]);

```

```

const [labTests, setLabTests] = useState<LabTest[]>([]);
const [showLabTests, setShowLabTests] = useState(false);

// Modal state
const [editPatient, setEditPatient] = useState<Patient | null>(null);
const [editDoctor, setEditDoctor] = useState<Doctor | null>(null);
const [assignPatient, setAssignPatient] = useState<PatientNoCreds | null>(null);
const [deleteConfirm, setDeleteConfirm] = useState<Patient | null>(null);
const [summaryFor, setSummaryFor] = useState<{id:number;name:string} | null>(null);
const [scansFor, setScansFor] = useState<{id:number;name:string} | null>(null);
// Doctor panel modals (lifted out of PatientDetailPanel to avoid overflow:hidden trap)
const [showDoctorSummary, setShowDoctorSummary] = useState(false);
const [showDoctorProgress, setShowDoctorProgress] = useState(false);

// Forms
const [visitF, setVisitF] = useState({
PatientID:0,DoctorID:0,ReasonForVisit:"",VitalSigns:"",Notes:"",Status:'Scheduled' });

const [diagF, setDiagF] = useState({
VisitID:0,DiagnosisName:"",Description:"",IsChronic:false,Severity:'Mild' });

const [rxF, setRxF] = useState({
visit_id:0,medicine:"",dosage:"",frequency:"",duration:"",instructions:" });

const [fileF, setFileF] = useState({ file:null as File | null, file_type:'X-Ray', description:" });

const [scanF, setScanF] = useState({ file:null as File | null, patient_id:"", scan_type:'X-Ray',
description:" });

const [newDoctorF, setNewDoctorF] = useState({
  doctor_name: "",
  email: "",
  specialty: "",
  phone_number: "",
  license_number: "",
  years_of_experience: "",
  username: "",

```



```
password: ""
});
const [impFile, setImpFile] = useState<File | null>(null);
const [impVisitFile, setImpVisitFile] = useState<File | null>(null);
const [importVisitResult, setImportVisitResult] = useState<any>(null);
const [frequentFlyers, setFrequentFlyers] = useState<any[]>([]);
const [ffLoaded, setFfLoaded] = useState(false);
```

```
// Navigation history functions
```

```
const navigateTo = useCallback((newView: string) => {
  setNavHistory(prev => {
    // If we're not at the end of history, truncate future entries
    const truncated = prev.slice(0, navIndex + 1);
    // Add new view
    const updated = [...truncated, newView];
    // Update index
    setNavIndex(updated.length - 1);
    return updated;
  });
  setView(newView);
}, [navIndex]);
```

```
const canGoBack = navIndex > 0;
```

```
const canGoForward = navIndex < navHistory.length - 1;
```

```
const goBack = useCallback(() => {
  if (canGoBack) {
    const newIndex = navIndex - 1;
    setNavIndex(newIndex);
    setView(navHistory[newIndex]);
  }
}
```

```
}, [navIndex, navHistory, canGoBack]);
```

```
const goForward = useCallback(() => {  
  if (canGoForward) {  
    const newIndex = navIndex + 1;  
    setNavIndex(newIndex);  
    setView(navHistory[newIndex]);  
  }  
}, [navIndex, navHistory, canGoForward]);
```

```
const authHdr = useCallback(() => ({ Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' }), [token]);
```

```
async function apiFetch(path: string, opts: RequestInit = {}) {  
  const res = await fetch(`${API}${path}`, { ...opts, headers: { ...authHdr(), ...(opts.headers ?? {}) } });  
  const json = await res.json().catch(() => ({}));  
  if (!res.ok) throw new Error(json.error || `HTTP ${res.status}`);  
  return json;  
}
```

```
const loadStats = useCallback(() =>  
  apiFetch('/dashboard/stats').then(setStats).catch(console.error), [token]);  
  
const loadRecords = useCallback(() =>  
  apiFetch('/records/all').then(setRecords).catch(console.error), [token]);  
  
const loadMyRecords = useCallback(() =>  
  apiFetch('/records/my').then(setRecords).catch(console.error), [token]);  
  
const loadDoctorRecs = useCallback(() =>  
  apiFetch('/records/doctor').then(setDoctorRecords).catch(console.error), [token]);  
  
const loadPatients = useCallback(() => apiFetch('/patients').then(setPatients).catch(console.error),  
[token]);  
  
const loadDoctors = useCallback(() => apiFetch('/doctors').then(setDoctors).catch(console.error),  
[token]);  
  
const loadVisits = useCallback(() => apiFetch('/visits').then(setVisits).catch(console.error),  
[token]);
```

```

const loadRx      = useCallback(() =>
apiFetch(`/prescriptions${pendingOnly?'?pending=1':''}`).then(setPrescriptions).catch(console.error),
[token, pendingOnly]);

const loadFiles   = useCallback(() => { if (user?.patient_id)
apiFetch(`/files/patient/${user.patient_id}`).then(setMedFiles).catch(console.error); }, [token, user]);

const loadMyScans = useCallback(() =>
apiFetch('/scans/mine').then(setMyScans).catch(console.error), [token]);

const loadNoCreds = useCallback(() => apiFetch('/admin/patients-without-
credentials').then(setNoCreds).catch(console.error), [token]);

const loadLabTests = useCallback(() => apiFetch('/lab-
tests').then(setLabTests).catch(console.error), [token]);

const loadAppointments = useCallback(() => apiFetch('/appointments').catch(console.error),
[token]);

useEffect(() => {

  if (!user || !token) return;

  loadStats();

  if (user.role === 'Doctor') { loadDoctorRecs(); loadDoctors(); loadPatients(); loadVisits();
loadRecords(); loadLabTests(); }

  if (user.role === 'Patient') { loadMyRecords(); loadFiles(); loadDoctors(); }

  if (user.role === 'Admin') {

    loadPatients(); loadNoCreds();

    if (!ffLoaded) {

      fetch(`${API}/analytics/frequent-flyers`, { headers:{ Authorization:`Bearer ${token}` } })

        .then(r=>r.json()).then(d=>{ if(Array.isArray(d)){ setFrequentFlyers(d); setFfLoaded(true); }
}).catch(()=>{});

    }

  }

  if (user.role === 'Pharmacist') { loadRx(); }

  if (user.role === 'Radiologist') { loadPatients(); loadMyScans(); }

  if (user.role === 'Receptionist') { loadPatients(); loadDoctors(); }

}, [user, token, view]);

const doLogin = async (username: string, password: string) => {

```

```

    setLoading(true); setFormErr("");

    try {

        const res = await fetch(`${API}/auth/login`, { method:'POST', headers: {'Content-Type': 'application/json'}, body: JSON.stringify({username,password}) });

        const data = await res.json();

        if (!data.token) throw new Error(data.error || 'Login failed');

        localStorage.setItem('token', data.token);

        setToken(data.token);

        setUser(decodeJWT(data.token));

    } catch(e:any) { setFormErr(e.message); }

    finally { setLoading(false); }

};

```

```

const doRegister = async (e: React.FormEvent, regF: RegisterForm) => {

    e.preventDefault(); setLoading(true); setFormErr("");

    try { await apiFetch('/auth/register', { method:'POST', body: JSON.stringify(regF) });
    setShowReg(false); setFormErr(""); }

    catch(e:any) { setFormErr(e.message); }

    finally { setLoading(false); }

};

```

```

const doLogout = () => { localStorage.removeItem('token'); setToken(null); setUser(null);
navigateTo('dashboard'); };

```

```

const submitVisit = async (e: React.FormEvent) => {

    e.preventDefault(); setLoading(true);

    // Always use the logged-in doctor's own ID — never allow substitution

    const doctorId = user?.doctor_id ?? visitF.DoctorID;

    try {

        await apiFetch('/visits', { method:'POST', body: JSON.stringify({ patient_id:visitF.PatientID,
        doctor_id:doctorId, reason:visitF.ReasonForVisit, vital_signs:visitF.VitalSigns, notes:visitF.Notes,
        status:visitF.Status }) });
    }

```

```

    setVisitF({ PatientID:0, DoctorID: doctorId as number, ReasonForVisit:"", VitalSigns:"", Notes:"",
Status:'Scheduled' });

    loadVisits();

  } catch(e:any) { alert(e.message); }

  finally { setLoading(false); }

};

```

```

const submitDiag = async (e: React.FormEvent) => {

  e.preventDefault(); setLoading(true);

  try {

    await apiFetch('/diagnosis', { method:'POST', body: JSON.stringify({ visit_id:diagF.VisitID,
name:diagF.DiagnosisName, description:diagF.Description, is_chronic:diagF.IsChronic,
severity:diagF.Severity }) });

    setDiagF({ VisitID:0,DiagnosisName:"",Description:"",IsChronic:false,Severity:'Mild' });

    loadRecords();

  } catch(e:any) { alert(e.message); }

  finally { setLoading(false); }

};

```

```

const submitRx = async (e: React.FormEvent) => {

  e.preventDefault(); setLoading(true);

  try {

    await apiFetch('/prescriptions', { method:'POST', body: JSON.stringify(rxF) });

    setRxF({ visit_id:0,medicine:"",dosage:"",frequency:"",duration:"",instructions:"" });

    loadRecords();

  } catch(e:any) { alert(e.message); }

  finally { setLoading(false); }

};

```

```

const dispense = async (id: number) => {

  try { await apiFetch(`/prescriptions/${id}/dispense`, { method:'POST' }); loadRx(); }

  catch(e:any) { alert(e.message); }

}

```

```
};
```

```
const submitUpload = async (e: React.FormEvent) => {  
  e.preventDefault(); if (!fileF.file) return; setLoading(true);  
  try {  
    const fd = new FormData(); fd.append('file', fileF.file);  
    fd.append('patient_id', String(user?.patient_id ?? '')); fd.append('file_type', fileF.file_type);  
    fd.append('description', fileF.description);  
    const res = await fetch(`${API}/files/upload`, { method: 'POST', headers: { Authorization: `Bearer ${token}` }, body: fd });  
    const d = await res.json(); if (!res.ok) throw new Error(d.error);  
    setFileF({ file: null, file_type: 'X-Ray', description: '' }); loadFiles();  
  } catch (e: any) { alert(e.message); }  
  finally { setLoading(false); }  
};
```

```
const downloadFile = async (fid: number, name: string) => {  
  const res = await fetch(`${API}/files/download/${fid}`, { headers: { Authorization: `Bearer ${token}` } });  
  if (!res.ok) return alert('Download failed');  
  const a = document.createElement('a'); a.href = URL.createObjectURL(await res.blob());  
  a.download = name;  
  document.body.appendChild(a); a.click(); URL.revokeObjectURL(a.href);  
  document.body.removeChild(a);  
};
```

```
const submitImport = async (e: React.FormEvent) => {  
  e.preventDefault(); if (!impFile) return; setLoading(true);  
  try {  
    const fd = new FormData(); fd.append('file', impFile);  
    const res = await fetch(`${API}/admin/import-patients`, { method: 'POST',  
      headers: { Authorization: `Bearer ${token}` }, body: fd });  
    const d = await res.json(); if (!res.ok) throw new Error(d.error);
```

```

    setImportResult(d); setImpFile(null); loadPatients(); loadNoCreds();
  } catch(e:any) { alert(e.message); }
  finally { setLoading(false); }
};

```

```

const doDeletePatient = async (p: Patient) => {
  try { await apiFetch(`/patients/${(p as any).PatientID}`, { method:'DELETE' });
  setDeleteConfirm(null); loadPatients(); loadNoCreds(); }
  catch(e:any) { alert(e.message); }
};

```

```

const doDeleteDoctor = async (doctorId: number) => {
  if (!confirm(`Are you sure you want to delete doctor #${doctorId}? This will deactivate their
  account.`)) return;
  try {
    await apiFetch(`/doctors/${doctorId}`, { method:'DELETE' });
    loadDoctors();
    alert('Doctor deactivated successfully');
  } catch(e:any) { alert(e.message); }
};

```

```

const submitImportVisits = async (e: React.FormEvent) => {
  e.preventDefault(); if (!impVisitFile) return; setLoading(true);
  try {
    const fd = new FormData(); fd.append('file', impVisitFile);
    const res = await fetch(`${API}/admin/import-visits`, {
      method:'POST', headers:{ Authorization:`Bearer ${token}` }, body:fd
    });
    const d = await res.json();
    if (!res.ok) throw new Error(d.error);
    setImportVisitResult(d); setImpVisitFile(null); loadPatients(); loadNoCreds();
  }
};

```

```
    } catch(e:any) { alert(e.message); }  
    finally { setLoading(false); }  
};
```

```
const submitScanUpload = async (e: React.FormEvent) => {  
    e.preventDefault(); if (!scanF.file || !scanF.patient_id) return; setLoading(true);  
    try {  
        const fd = new FormData(); fd.append('file',scanF.file); fd.append('patient_id',scanF.patient_id);  
        fd.append('scan_type',scanF.scan_type); fd.append('description',scanF.description);  
        const res = await fetch(`${API}/scans/upload`, { method:'POST', headers:{Authorization:`Bearer ${token}`}, body:fd });  
        const d = await res.json(); if (!res.ok) throw new Error(d.error);  
        setScanF({ file:null, patient_id:"", scan_type:'X-Ray', description:" " }); loadMyScans(); alert('Scan  
        uploaded successfully!');  
    } catch(e:any) { alert(e.message); }  
    finally { setLoading(false); }  
};
```

```
const addDoctor = async (e: React.FormEvent) => {  
    e.preventDefault();  
    if (!newDoctorF.doctor_name || !newDoctorF.email || !newDoctorF.specialty ||  
    !newDoctorF.phone_number) {  
        alert('Please fill required fields: Name, Email, Specialty, Phone');  
        return;  
    }  
    setLoading(true);  
    try {  
        const res = await fetch(`${API}/doctors`, {  
            method: 'POST',  
            headers: { Authorization: `Bearer ${token}`, 'Content-Type': 'application/json' },  
            body: JSON.stringify({  
                doctor_name: newDoctorF.doctor_name,
```



```

        email: newDoctorF.email,
        specialty: newDoctorF.specialty,
        phone_number: newDoctorF.phone_number,
        username: newDoctorF.username,
        password: newDoctorF.password,
        license_number: newDoctorF.license_number || "",
        years_of_experience: newDoctorF.years_of_experience ?
parseInt(newDoctorF.years_of_experience) : null
    })
});

const data = await res.json();

if (!res.ok) throw new Error(data.error || 'Failed to add doctor');

alert(`Doctor added successfully! Username: ${data.username}, Temporary password:
${data.temp_password}`);

setNewDoctorF({
    doctor_name: "",
    email: "",
    specialty: "",
    phone_number: "",
    license_number: "",
    years_of_experience: "",
    username: "",
    password: ""
});

loadDoctors();
} catch (err: any) {
    alert(err.message);
} finally {
    setLoading(false);
}
};

```

```

const titles: Record<string,string> = {
  dashboard:'Dashboard', records:'All Medical Records', 'my-records':'My Medical Records',
  'doctor-reports':'Patient Reports', 'create-visit':'Create New Visit',
  'add-diagnosis':'Add Diagnosis', 'add-rx':'Add Prescription',
  'my-files':'My Medical Files', 'upload-file':'Upload Medical File',
  prescriptions:'Prescriptions Management', patients:'Patient Management',
  import:'Import Patients', 'assign-creds':'Assign Credentials',
  'upload-scan':'Upload Scan', 'my-uploads':'My Scan Uploads',
  'my-appointments':'My Appointments', 'appointment-queue':'Appointment Queue',
  'manage-doctors':'Manage Doctors',
};

if (!token || !user) {
  if (showReg) return <RegisterScreen onRegister={doRegister}
onBack={()=>{setShowReg(false);setFormErr("");}} formErr={formErr} loading={loading}/>;

  return <LoginScreen onLogin={doLogin} onShowRegister={()=>{setShowReg(true);setFormErr("");}}
formErr={formErr} loading={loading} theme={theme} setTheme={setTheme}/>;
}

// IDs of patients who have visited THIS doctor
const myPatientIds = new Set(doctorRecords.map(r => r.PatientID).filter(Boolean));

const search = patientSearch.toLowerCase();

const myPatients  = patients.filter(p => myPatientIds.has((p as any).PatientID) && (!search ||
p.PatientName.toLowerCase().includes(search)));

const otherPatients = patients.filter(p => !myPatientIds.has((p as any).PatientID) && (!search ||
p.PatientName.toLowerCase().includes(search)));

// keep legacy for admin/other views

const filteredPatients = patients.filter(p => !search ||
p.PatientName.toLowerCase().includes(search));

return (
  <div className="dashboard">

```

```

    {/* — Global Modals — */}

    {summaryFor && <SummaryModal patientId={summaryFor.id} patientName={summaryFor.name}
role={user.role} token={token} onClose={()=>setSummaryFor(null)}/>}

    {scansFor && <ScansModal patientId={scansFor.id} patientName={scansFor.name}
token={token} onClose={()=>setScansFor(null)}/>}

    {editPatient && <EditPatientModal patient={editPatient} token={token}
onClose={()=>setEditPatient(null)} onSave={()=>{ setEditPatient(null); loadPatients(); }}/>}

    {editDoctor && <EditDoctorModal doctor={editDoctor} token={token}
onClose={()=>setEditDoctor(null)} onSave={()=>{ setEditDoctor(null); loadDoctors(); }}/>}

    {assignPatient && <AssignCredsModal patient={assignPatient} token={token}
onClose={()=>setAssignPatient(null)} onSave={()=>{ setAssignPatient(null); loadNoCreds();
loadPatients(); }}/>}


    {/* Doctor panel modals — rendered at root level to escape overflow:hidden ancestors */}

    {showDoctorSummary && selectedPatient && (
        <SummaryModal patientId={selectedPatient.PatientID}
patientName={selectedPatient.PatientName}
            role={user.role} token={token} onClose={() => setShowDoctorSummary(false)}/>
    )}

    {showDoctorProgress && selectedPatient && (
        <ChronicProgressModal patientId={selectedPatient.PatientID}
patientName={selectedPatient.PatientName}
            token={token} onClose={() => setShowDoctorProgress(false)}/>
    )}

    {showLabTests && (
        <LabTestsModal token={token} visits={visits} onClose={() => { setShowLabTests(false);
loadLabTests(); loadStats(); }}/>
    )}


    {deleteConfirm && (
        <Modal title="Confirm Deletion" onClose={()=>setDeleteConfirm(null)}>
            <p style={{ color:'var(--text-mid)', marginBottom:'1.5rem', lineHeight:1.6 }}>Deactivate <strong
style={{ color:'var(--text-bright)' }}>{deleteConfirm.PatientName}</strong>? Their login will also be
disabled.</p>
            <div style={{ display:'flex', gap:'0.75rem' }}>

```

```
        <button onClick={()=>setDeleteConfirm(null)} className="action-btn secondary" style={{
flex:1 }}>Cancel</button>
```

```
        <button onClick={()=>doDeletePatient(deleteConfirm)} style={{
flex:1,padding:'0.75rem',background:'linear-gradient(135deg,var(--
rose),#e11d48)',border:'none',borderRadius:10,color:'white',fontWeight:700,cursor:'pointer',fontFa
mily:"Syne",sans-serif",display:'flex',alignItems:'center',justifyContent:'center',gap:'0.4rem' }}>
```

```
        <Trash2 size={15}/> Confirm Delete
```

```
    </button>
```

```
  </div>
```

```
</Modal>
```

```
}}
```

```
{/* — Sidebar — */}
```

```
<aside className="sidebar">
```

```
  <div className="sidebar-header">
```

```
    <Heart className="sidebar-logo"/>
```

```
    <div><h2>MediCare</h2><p className="sidebar-subtitle">{user.role}</p></div>
```

```
  </div>
```

```
  <nav className="sidebar-nav">
```

```
    <button className={view==='dashboard'? 'active':''}
onClick={()=>setView('dashboard')}><Activity/> Dashboard</button>
```

```
    {user.role==='Doctor'&&<>
```

```
      <button className={view==='doctor-reports'? 'active':''} onClick={()=>setView('doctor-
reports')}><ClipboardList/> My Patients</button>
```

```
      <button className={view==='records'? 'active':''} onClick={()=>setView('records')}><FileText/>
All Records</button>
```

```
      <button className={view==='create-visit'? 'active':''} onClick={()=>setView('create-
visit')}><PlusCircle/> Create Visit</button>
```

```
      <button className={view==='add-diagnosis'? 'active':''} onClick={()=>setView('add-
diagnosis')}><Stethoscope/> Add Diagnosis</button>
```

```
      <button className={view==='add-rx'? 'active':''} onClick={()=>setView('add-rx')}><Pill/> Add
Prescription</button>
```

```
      <button onClick={()=>setShowLabTests(true)} style={{
display:'flex',alignItems:'center',gap:'0.75rem',padding:'0.75rem
```

```
1rem',background:'transparent',border:'none',borderRadius:'var(--r-md)',color:'var(--text-mid)',fontFamily:'inherit',fontSize:'0.9rem',fontWeight:500,cursor:'pointer',width:'100%',position:'relative',transition:'all var(--t-mid)' }}
```

```
onMouseEnter={e=>(e.currentTarget.style.background='var(--accent-dim)',e.currentTarget.style.color='var(--accent-light)')}
```

```
onMouseLeave={e=>(e.currentTarget.style.background='transparent',e.currentTarget.style.color='var(--text-mid)')}>
```

```
<FlaskConical size={18}/> Lab Tests
```

```
{stats.pending_tests > 0 && <span style={{ marginLeft:'auto',background:'var(--violet)',color:'white',borderRadius:999,fontSize:'0.65rem',fontWeight:700,padding:'0.1rem 0.45rem' }}>{stats.pending_tests}</span>
```

```
</button>
```

```
</>
```

```
{user.role==='Patient'&&<>
```

```
<button className={view==='dashboard'?'active':''}
onClick={()=>setView('dashboard')}><User/> My Profile</button>
```

```
<button className={view==='my-appointments'?'active':''} onClick={()=>setView('my-appointments')}><Calendar/> Appointments</button>
```

```
<button className={view==='my-records'?'active':''} onClick={()=>setView('my-records')}><FileText/> Medical Records</button>
```

```
<button className={view==='my-files'?'active':''} onClick={()=>setView('my-files')}><FileImage/> My Files</button>
```

```
<button className={view==='upload-file'?'active':''} onClick={()=>setView('upload-file')}><Upload/> Upload File</button>
```

```
<button onClick={()=>{ if(user.patient_id)
setSummaryFor({id:user.patient_id,name:user.username}); }} style={{
all:'unset',display:'flex',alignItems:'center',gap:'0.75rem',padding:'0.7rem
0.9rem',borderRadius:'var(--r-md)',cursor:'pointer',color:'var(--text-mid)',width:'100%',boxSizing:'border-box',fontSize:'0.875rem',fontFamily:'Plus Jakarta Sans',sans-serif",fontWeight:500,transition:'all var(--t-mid) var(--ease-out)' }}
onMouseEnter={e=>{(e.currentTarget as HTMLElement).style.background='var(--glass-2)';(e.currentTarget as HTMLElement).style.color='var(--accent)';}}
onMouseLeave={e=>{(e.currentTarget as
HTMLElement).style.background='transparent';(e.currentTarget as HTMLElement).style.color='var(--text-mid)';}}>
```

```
<BookOpen size={17} style={{ flexShrink:0 }}/> Summary Sheet
```

```
</button>
```

```
</>
```

```
{user.role==='Receptionist'&&<>

  <button className={view==='appointment-queue'?'active':''}
onClick={()=>setView('appointment-queue')}>

    <Calendar/> Appointment Queue

    {(stats as any).pending_appointments > 0 && <span style={{
marginLeft:'auto',background:'var(--
rose)',color:'white',borderRadius:999,fontSize:'0.65rem',fontWeight:700,padding:'0.1rem 0.45rem'
}}>{(stats as any).pending_appointments}</span>}

  </button>

  <button className={view==='patients'?'active':''} onClick={()=>setView('patients')}><Users/>
All Patients</button>

</>
```

```
{user.role==='Radiologist'&&<>

  <button className={view==='upload-scan'?'active':''} onClick={()=>setView('upload-
scan')}><Upload/> Upload Scan</button>

  <button className={view==='my-uploads'?'active':''} onClick={()=>setView('my-
uploads')}><ScanLine/> My Uploads</button>

</>
```

```
{user.role==='Pharmacist'&&<button className={view==='prescriptions'?'active':''}
onClick={()=>setView('prescriptions')}><Pill/> Prescriptions</button>}
```

```
{user.role==='Admin'&&<>

  <button className={view==='patients'?'active':''} onClick={()=>setView('patients')}><Users/>
Patients</button>

  <button className={view==='manage-doctors'?'active':''} onClick={()=>setView('manage-
doctors')}><Stethoscope/> Manage Doctors</button>

  <button className={view==='assign-creds'?'active':''} onClick={()=>setView('assign-creds')}>

    <Key/> Assign Credentials

    {noCreds.length>0&&<span style={{ marginLeft:'auto',background:'var(--
rose)',color:'white',borderRadius:999,fontSize:'0.65rem',fontWeight:700,padding:'0.1rem 0.45rem'
}}>{noCreds.length}</span>}

  </button>

</>
```

```

</button>

    <button className={view==='import'? 'active':''} onClick={()=>setView('import')}><Upload/>
    Import Data</button>

  </>
</nav>

<div className="sidebar-footer">

  <div className="user-info"><User size={16}/><span>{user.username}</span></div>

  <button className="logout-btn" onClick={doLogout}><LogOut/> Logout</button>

</div>

</aside>

{ /* — Main — */ }

<main className="main-content">

  <header className="content-header">

    <div style={{ display: 'flex', alignItems: 'center', gap: '1rem' }}>

      <div style={{ display: 'flex', alignItems: 'center', gap: '0.25rem' }}>

        <button

          onClick={goBack}

          disabled={!canGoBack}

          style={{

            background: 'var(--raised)',

            border: '1px solid var(--border)',

            borderRadius: 'var(--r-sm)',

            width: 32,

            height: 32,

            display: 'flex',

            alignItems: 'center',

            justifyContent: 'center',

            color: canGoBack ? 'var(--text-mid)' : 'var(--text-dim)',

            cursor: canGoBack ? 'pointer' : 'not-allowed',

            opacity: canGoBack ? 1 : 0.5,

```

```

    transition: 'all var(--t-fast)'
  }}
  onMouseEnter={e => {
    if (canGoBack) {
      e.currentTarget.style.background = 'var(--accent-dim)';
      e.currentTarget.style.borderColor = 'var(--accent)';
      e.currentTarget.style.color = 'var(--accent)';
    }
  }}
  onMouseLeave={e => {
    if (canGoBack) {
      e.currentTarget.style.background = 'var(--raised)';
      e.currentTarget.style.borderColor = 'var(--border)';
      e.currentTarget.style.color = 'var(--text-mid)';
    }
  }}
>
  <ChevronLeft size={16} />
</button>
<button
  onClick={goForward}
  disabled={!canGoForward}
  style={{
    background: 'var(--raised)',
    border: '1px solid var(--border)',
    borderRadius: 'var(--r-sm)',
    width: 32,
    height: 32,
    display: 'flex',
    alignItems: 'center',
    justifyContent: 'center',
  }}
  >
    <ChevronRight size={16} />
  </button>

```



```

        color: canGoForward ? 'var(--text-mid)' : 'var(--text-dim)',
        cursor: canGoForward ? 'pointer' : 'not-allowed',
        opacity: canGoForward ? 1 : 0.5,
        transition: 'all var(--t-fast)'
    }}
    onMouseEnter={e => {
        if (canGoForward) {
            e.currentTarget.style.background = 'var(--accent-dim)';
            e.currentTarget.style.borderColor = 'var(--accent)';
            e.currentTarget.style.color = 'var(--accent)';
        }
    }}
    onMouseLeave={e => {
        if (canGoForward) {
            e.currentTarget.style.background = 'var(--raised)';
            e.currentTarget.style.borderColor = 'var(--border)';
            e.currentTarget.style.color = 'var(--text-mid)';
        }
    }}
>
    <ChevronRight size={16} />
</button>
</div>
<div>
    <h1>{titles[view] ?? 'Dashboard'}</h1>
    <p className="subtitle">Welcome back, {user.username}</p>
</div>
</div>
<div className="header-right">
    {(user.role === 'Doctor' || user.role === 'Pharmacist') && (
        <NotificationBell

```

```

    pending_rx={stats.pending_prescriptions}

    pending_tests={stats.pending_tests}

    role={user.role}

  />
)}

<ThemeSwitcher theme={theme} setTheme={setTheme}/>

<div className="user-badge">

  {user.role==='Doctor'&&<Stethoscope/>}{user.role==='Patient'&&<User/>}

  {user.role==='Pharmacist'&&<Pill/>}{user.role==='Admin'&&<Shield/>}

  {user.role==='Radiologist'&&<ScanLine/>}{user.role==='Receptionist'&&<Calendar/>}

  <span>{user.role}</span>

</div>

</div>

</header>

<div className="content-body">

  {/* — Dashboard — */}

  {view==='dashboard'&&(

    user.role==='Patient' ? (

      /* — Patient gets the full Profile Dashboard — */

      <PatientProfileDashboard

        user={user} token={token} records={records} medFiles={medFiles}

        doctors={doctors}

        onBookAppointment={()=>setView('my-appointments')}

        onOpenSummary={()=>{ if(user.patient_id)

setSummaryFor({id:user.patient_id,name:user.username}); }}

        onViewRecords={()=>setView('my-records')}

        onViewFiles={()=>setView('my-files')}

        onViewAppointments={()=>setView('my-appointments')}

      />

```

```

): {

  <div className="dashboard-grid">

    <div className="stat-card stat-primary"><div className="stat-icon"><Users/></div><div
className="stat-details"><h3>{stats.total_patients}</h3><p>Total Patients</p></div></div>

    <div className="stat-card stat-success"><div className="stat-
icon"><Stethoscope/></div><div className="stat-details"><h3>{stats.total_doctors}</h3><p>Total
Doctors</p></div></div>

    <div className="stat-card stat-warning"><div className="stat-icon"><Calendar/></div><div
className="stat-details"><h3>{stats.today_visits}</h3><p>Today's Visits</p></div></div>

    <div className="stat-card stat-danger"><div className="stat-icon"><Pill/></div><div
className="stat-details"><h3>{stats.pending_prescriptions}</h3><p>Pending Rx</p></div></div>

    <div className="welcome-card">

      <h2>Welcome to MediCare Plus 🏥 </h2>

      <p>Your comprehensive hospital management solution. Use the sidebar to navigate.</p>

      {user.role==='Admin'&&noCreds.length>0&&(

        <div style={{ background:'var(--rose-dim)',border:'1px solid
rgba(244,63,94,.2)',borderRadius:12,padding:'0.875rem
1rem',marginBottom:'1rem',display:'flex',alignItems:'center',justifyContent:'space-between' }}>

          <div style={{ display:'flex',alignItems:'center',gap:'0.6rem' }}><Key size={18} color="var(--
rose)"/><span style={{ color:'#fda4af',fontSize:'0.88rem',fontWeight:600 }}>{noCreds.length}
patient{noCreds.length>1?'s':''} need login credentials</span></div>

          <button onClick={()=>setView('assign-creds')} className="action-btn" style={{
padding:'0.4rem 0.875rem',fontSize:'0.8rem',marginTop:0 }}>Assign Now</button>

        </div>

      )}

      <div className="quick-actions">

        {user.role==='Doctor'&&<><button onClick={()=>setView('create-visit')}
className="action-btn">Create Visit</button><button onClick={()=>setView('doctor-reports')}
className="action-btn secondary">My Patients</button></>

        {user.role==='Receptionist'&&<>

          <button onClick={()=>setView('appointment-queue')} className="action-btn" style={{
display:'flex', alignItems:'center', gap:'0.5rem' }}>

            <Calendar size={15}/> Appointment Queue

            {(stats as any).pending_appointments > 0 && <span style={{
background:'rgba(255,255,255,0.3)', borderRadius:999, fontSize:'0.7rem', fontWeight:800,
padding:'0.1rem 0.4rem' }}>{(stats as any).pending_appointments}</span>}


```

```

        </button>

        <button onClick={()=>setView('patients')} className="action-btn secondary">All
Patients</button>

    </>

    {user.role==='Pharmacist'&&<button onClick={()=>setView('prescriptions')}
className="action-btn">View Prescriptions</button>}

    {user.role==='Admin'&&<><button onClick={()=>setView('patients')} className="action-
btn">Manage Patients</button><button onClick={()=>setView('import')} className="action-btn
secondary">Import Patients</button></>}

    {user.role==='Radiologist'&&<><button onClick={()=>setView('upload-scan')}
className="action-btn">Upload Scan</button><button onClick={()=>setView('my-uploads')}
className="action-btn secondary">My Uploads</button></>}

</div>

</div>

{/* — Frequent Flyers (Admin) — */}

{user.role==='Admin' && frequentFlyers.length>0 && (

    <div style={{ gridColumn:'1/-1',background:'var(--card-bg)',border:'1px solid var(--border-
glass)',borderRadius:'var(--r-xl)',padding:'1.5rem',boxShadow:'var(--shadow-card)' }}>

        <div style={{ display:'flex',alignItems:'center',justifyContent:'space-
between',marginBottom:'1.25rem',flexWrap:'wrap',gap:'0.5rem' }}>

            <div>

                <h3 style={{ fontFamily:'"Space Grotesk',sans-
serif",fontWeight:700,fontSize:'1rem',margin:'0 0 0.2rem',color:'var(--text-
bright)',display:'flex',alignItems:'center',gap:'0.5rem' }}>

                    🚒 Frequent Flyers

                    <span style={{ background:'var(--rose-dim)',border:'1px solid
rgba(255,77,109,.25)',color:'var(--
rose)',borderRadius:999,fontSize:'0.68rem',fontWeight:700,padding:'0.15rem 0.55rem'
}}>{frequentFlyers.length}</span>

                </h3>

                <p style={{ fontSize:'0.78rem',color:'var(--text-dim)',margin:0 }}>Patients with &gt;3 visits
in 6 months — may need care management review</p>

            </div>

            <button onClick={()=>setView('patients')} className="action-btn secondary" style={{
padding:'0.45rem 0.875rem',fontSize:'0.8rem',marginTop:0 }}>View All Patients →</button>

```

```

</div>

<div style={{ display:'grid',gridTemplateColumns:'repeat(auto-
fill,minmax(280px,1fr))',gap:'0.75rem' }}>

  {frequentFlyers.slice(0,6).map((ff:any,i:number)=>{

    <div key={i} style={{ display:'flex',alignItems:'center',gap:'0.875rem',padding:'0.875rem
1rem',background:'var(--raised)',border:'1px solid var(--border-glass)',borderRadius:'var(--r-
md)',cursor:'pointer',transition:'border-color var(--t-mid)' }}

      onClick={()=>{ setAdminSelectedPatient(ff as any); setView('patients'); }}

      onMouseEnter={e=>(e.currentTarget.style.borderColor='var(--border-hover))}

      onMouseLeave={e=>(e.currentTarget.style.borderColor='var(--border-glass))}>

        <div style={{ width:40,height:40,borderRadius:10,background:'var(--rose-
dim)',border:'1px solid
rgba(255,77,109,.2)',display:'flex',alignItems:'center',justifyContent:'center',fontFamily:'"'Space
Grotesk',sans-serif",fontWeight:700,fontSize:'0.85rem',color:'var(--rose)',flexShrink:0 }}>

          {{ff.PatientName || '?'}.split(' ').map((w:string)=>w[0]).join('').slice(0,2)}

        </div>

        <div style={{ flex:1,minWidth:0 }}>

          <p style={{ margin:0,fontWeight:700,fontSize:'0.875rem',color:'var(--text-
bright)',overflow:'hidden',textOverflow:'ellipsis',whiteSpace:'nowrap' }}>{{ff.PatientName}}</p>

          <p style={{ margin:'0.15rem 0 0',fontSize:'0.75rem',color:'var(--text-dim)'
}}>{{ff.Gender}} · {{ff.BloodGroup}}</p>

          {{ff.MostFrequentComplaint}} && <p style={{ margin:'0.1rem 0
0',fontSize:'0.72rem',color:'var(--
accent)',overflow:'hidden',textOverflow:'ellipsis',whiteSpace:'nowrap' }}>↔
{{ff.MostFrequentComplaint}}</p>

        </div>

        <div style={{ textAlign:'right',flexShrink:0 }}>

          <div style={{ fontFamily:'"'Space Grotesk',sans-
serif",fontWeight:700,fontSize:'1.4rem',color:'var(--rose)',lineHeight:1 }}>{{ff.VisitCount}}</div>

          <div style={{ fontSize:'0.65rem',color:'var(--text-dim)',marginTop:'0.1rem'
}}>visits/6mo</div>

        </div>

      </div>

    )}

  </div>

</div>

```

```

    }}
  </div>
)
}}

{/* — Doctor Reports — Split Panel — */}

{view==='doctor-reports'&&(
  <div style={{ display:'flex', height:'calc(100vh - 140px)', margin:'-2rem -2.5rem',
overflow:'hidden' }}>

    {/* — LEFT COLUMN: My Patients ————— */}

    <div style={{ width:230, flexShrink:0, borderRight:'1px solid var(--border)', display:'flex',
flexDirection:'column', background:'var(--bg-1)' }}>

      {/* Header */}

      <div style={{ padding:'0.875rem 0.875rem 0.5rem', borderBottom:'1px solid var(--border)'
}}>

        <div style={{ display:'flex', alignItems:'center', gap:'0.5rem', marginBottom:'0.625rem' }}>

          <div style={{ width:6, height:6, borderRadius:'50%', background:'var(--accent)',
boxShadow:'0 0 6px var(--accent-glow)', flexShrink:0 }}/>

          <span style={{ fontFamily:'"Syne",sans-serif', fontWeight:800, fontSize:'0.72rem',
textTransform:'uppercase', letterSpacing:'0.08em', color:'var(--accent)' }}>My Patients</span>

          <span style={{ marginLeft:'auto', background:'var(--accent-dim)', border:'1px solid var(--
border)', borderRadius:999, fontSize:'0.65rem', fontWeight:700, padding:'0.1rem 0.45rem',
color:'var(--accent)' }}>{myPatients.length}</span>

        </div>

        <input className="patient-list-search" placeholder="Search..." value={patientSearch}
onChange={e=>{ setPatientSearch(e.target.value); }} style={{ marginBottom:0 }}/>

      </div>

      {/* My patients list */}

      <div style={{ flex:1, overflowY:'auto', padding:'0.625rem' }}>

        {myPatients.length === 0

          ? <p style={{ fontSize:'0.78rem', color:'var(--text-dim)', textAlign:'center', padding:'1.5rem
0.5rem' }}>
            {patientSearch ? 'No matches' : 'No patients assigned yet'}

```

```

    </p>

    : myPatients.map(p => {

        const hasChron = doctorRecords.some(r => r.PatientName === p.PatientName &&
r.IsChronic);

        const visitCount = doctorRecords.filter(r => r.PatientName === p.PatientName).length;

        const isActive = selectedPatient?.PatientID === (p as any).PatientID && patientListTab
=== 'mine';

        return (

            <div key={(p as any).PatientID}

                className={`patient-tile ${isActive ? 'active' : ''}`}

                onClick={() => { setSelectedPatient(p as any); setPatientListTab('mine'); }}>

                    <div className="pt-name">{p.PatientName}</div>

                    <div className="pt-meta">{p.Gender} · {visitCount} visit{visitCount !== 1 ? 's':''}</div>

                    <div className="pt-badges">

                        <span className="badge badge-blood" style={{ fontSize:'0.65rem',padding:'0.1rem
0.4rem' }}>{p.BloodGroup}</span>

                        {hasChron && <span className="badge badge-chronic" style={{
fontSize:'0.65rem',padding:'0.1rem 0.4rem' }}>Chronic</span>}

                    </div>

                </div>

            );

        })

    }

</div>

</div>

{/* — MIDDLE COLUMN: Other Patients ————— */}

<div style={{ width:210, flexShrink:0, borderRight:'1px solid var(--border)', display:'flex',
flexDirection:'column', background:'var(--bg)' }}>

    {/* Header */}

    <div style={{ padding:'0.875rem 0.875rem 0.5rem', borderBottom:'1px solid var(--border)'
}}>

        <div style={{ display:'flex', alignItems:'center', gap:'0.5rem', marginBottom:'0.625rem' }}>

```

```

<div style={{ width:6, height:6, borderRadius:'50%', background:'var(--text-dim)',
flexShrink:0 }}/>

<span style={{ fontFamily:'Syne',sans-serif", fontWeight:800, fontSize:'0.72rem',
textTransform:'uppercase', letterSpacing:'0.08em', color:'var(--text-dim)' }}>Other Patients</span>

<span style={{ marginLeft:'auto', background:'var(--raised)', border:'1px solid var(--
border)', borderRadius:999, fontSize:'0.65rem', fontWeight:700, padding:'0.1rem 0.45rem',
color:'var(--text-dim)' }}>{otherPatients.length}</span>

</div>

<p style={{ fontSize:'0.68rem', color:'var(--text-dim)', margin:0, lineHeight:1.4 }}>All other
registered patients</p>

</div>

{/* Other patients list */}

<div style={{ flex:1, overflowY:'auto', padding:'0.625rem' }}>

  {otherPatients.length === 0

    ? <p style={{ fontSize:'0.78rem', color:'var(--text-dim)', textAlign:'center', padding:'1.5rem
0.5rem' }}>

      {patientSearch ? 'No matches' : 'No other patients'}

    </p>

    : otherPatients.map(p => {

      const isActive = selectedPatient?.PatientID === (p as any).PatientID && patientListTab
=== 'other';

      return (

        <div key={(p as any).PatientID}

          onClick={() => { setSelectedPatient(p as any); setPatientListTab('other'); }}

          style={{ padding:'0.75rem 0.875rem', borderRadius:'var(--r-md)', border:'1px solid
var(--border)', cursor:'pointer', marginBottom:'0.4rem', background: isActive ? 'var(--raised)' :
'transparent', borderColor: isActive ? 'var(--border-hover)' : 'var(--border)', transition:'all 0.18s' }}

          onMouseEnter={e => { if (!isActive) { (e.currentTarget as
HTMLDivElement).style.background='var(--raised)'; (e.currentTarget as
HTMLDivElement).style.borderColor='var(--border-hover)'; } }}

          onMouseLeave={e => { if (!isActive) { (e.currentTarget as
HTMLDivElement).style.background='transparent'; (e.currentTarget as
HTMLDivElement).style.borderColor='var(--border)'; } }}>

          <div style={{ fontWeight:600, fontSize:'0.82rem', color:'var(--text-bright)',
marginBottom:'0.15rem', overflow:'hidden', textOverflow:'ellipsis', whiteSpace:'nowrap'
}}>{p.PatientName}</div>

```



```
        <div style={{ fontSize:'0.72rem', color:'var(--text-dim)' }}>{p.Gender} ·  
{p.BloodGroup}</div>
```

```
        <span style={{ display:'inline-block', marginTop:'0.3rem', fontSize:'0.62rem',  
padding:'0.1rem 0.4rem', borderRadius:4, background:'var(--amber-dim)', color:'var(--amber)',  
border:'1px solid rgba(217,119,6,0.2)', fontWeight:600 }}>
```

```
            View only
```

```
        </span>
```

```
    </div>
```

```
    );
```

```
  })
```

```
  }
```

```
</div>
```

```
</div>
```

```
{/* — RIGHT: Detail pane ————— */}
```

```
<div className="patient-detail-pane" style={{ flex:1, minWidth:0 }}>
```

```
  {selectedPatient ? (
```

```
    <>
```

```
    /* "View only" banner for other patients */
```

```
    {patientListTab === 'other' && (
```

```
      <div style={{ display:'flex', alignItems:'center', gap:'0.625rem', padding:'0.625rem 1rem',  
background:'var(--amber-dim)', borderBottom:'1px solid rgba(217,119,6,0.2)', fontSize:'0.82rem',  
color:'var(--amber)', flexShrink:0 }}>
```

```
        <AlertCircle size={14}/>
```

```
        <span>This patient has not visited your clinic. Viewing public profile only — you can still  
record a visit.</span>
```

```
        <button onClick={()=>setView('create-visit')} style={{ marginLeft:'auto', padding:'0.3rem  
0.75rem', background:'var(--amber)', border:'none', borderRadius:6, color:'white',  
fontSize:'0.75rem', fontWeight:700, cursor:'pointer', fontFamily:'"Syne",sans-serif',  
whiteSpace:'nowrap' }}>
```

```
          + Create Visit
```

```
        </button>
```

```
      </div>
```

```
    )}
```

```

<PatientDetailPanel
  patient={selectedPatient}
  records={patientListTab === 'mine' ? doctorRecords : []}
  token={token}
  role={user.role}
  onOpenSummary={() => setShowDoctorSummary(true)}
  onOpenProgress={() => setShowDoctorProgress(true)}
/>
</>
): (
  <div className="empty-state">
    <div style={{ fontSize:40, opacity:.25, marginBottom:'0.5rem' }}>👉</div>
    <h3>Select a patient</h3>
    <p>Choose from <strong>My Patients</strong> to see full records,<br/>or browse
    <strong>Other Patients</strong> to view their profile.</p>
  </div>
  )}
</div>
</div>
)}

{/* — All / My Records — */}
{({view==='records' | view==='my-records'})&&(
  <div className="records-container">
    {records.length===0
      ? <div className="empty-state"><FileText size={64}/><h3>No records found</h3></div>
      : <div className="table-container"><table className="data-
table"><thead><tr><th>Patient</th><th>Doctor</th><th>Visit
Date</th><th>Diagnosis</th><th>Medicine</th><th>Status</th></tr></thead>
      <tbody>{records.map((r,i)=>{
        <tr key={i}>

```

```

        <td><div className="patient-cell"><strong>{r.PatientName}</strong><span
className="badge badge-blood">{r.BloodGroup}</span></div></td>

        <td><div className="doctor-
cell"><strong>{r.DoctorName}</strong><small>{r.Specialty}</small></div></td>

        <td>{fmtDate(r.VisitDate)}</td>

        <td>{r.DiagnosisName || '---'}</td>

        <td>{r.MedicineName || '---'}</td>

        <td><span className={`status-badge
${r.IsDispensed?'dispensed':'pending'}`}>{r.IsDispensed?'Dispensed':'Pending'}</span></td>

    </tr>

    ))</tbody>

</table></div>

}

</div>

))

```

```

{/* — Create Visit — */}

```

```

{view==='create-visit'&&(

    <div className="form-container"><form onSubmit={submitVisit} className="data-
form"><h3> 📄 Visit Information</h3>

    <div className="form-row">

        <div className="form-group"><label>Patient *</label><select value={visitF.PatientID || ''}
onChange={e=>setVisitF({...visitF, PatientID: Number(e.target.value)})} required><option
value="">Select Patient</option>{patients.map(p=><option key={{p as any}.PatientID} value={{p as
any}.PatientID}>{p.PatientName} — {p.BloodGroup}</option>)}</select></div>

        <div className="form-group">

            <label>Doctor</label>

            {{{() => {

                const myDoc = doctors.find(d => d.DoctorID === user.doctor_id);

                // ensure visitF always has this doctor's ID

                if (myDoc && visitF.DoctorID !== myDoc.DoctorID) {

                    setTimeout(() => setVisitF(f => ({ ...f, DoctorID: myDoc.DoctorID })), 0);

                }

            }}}

```

```

    return (

      <div style={{ display:'flex', alignItems:'center', gap:'0.75rem', padding:'0.875rem 1rem',
background:'var(--accent-dim)', border:'1px solid var(--border)', borderRadius:'var(--r-md)',
cursor:'not-allowed' }}>

        <div style={{ width:32, height:32, borderRadius:8, background:'linear-
gradient(135deg,var(--accent),var(--violet))', display:'flex', alignItems:'center', justifyContent:'center',
flexShrink:0 }}>

          <Stethoscope size={15} color="white"/>

        </div>

        <div style={{ flex:1 }}>

          <p style={{ margin:0, fontWeight:700, fontSize:'0.9rem', color:'var(--text-bright)'
}}>{myDoc ? myDoc.DoctorName : user.username}</p>

          <p style={{ margin:0, fontSize:'0.75rem', color:'var(--text-dim)' }}>{myDoc ?
myDoc.Specialty : 'Doctor'}</p>

        </div>

        <span style={{ fontSize:'0.68rem', padding:'0.2rem 0.5rem', borderRadius:4,
background:'var(--accent)', color:'#fff', fontWeight:700, letterSpacing:'0.06em' }}>YOU</span>

      </div>

    );

  })()

</div>

</div>

  <div className="form-group"><label>Reason for Visit *</label><input type="text"
value={visitF.ReasonForVisit} onChange={e=>setVisitF({...visitF,ReasonForVisit:e.target.value})}
required/></div>

  <div className="form-group"><label>Vital Signs (JSON)</label><textarea
value={visitF.VitalSigns} onChange={e=>setVisitF({...visitF,VitalSigns:e.target.value})}
placeholder='{"bp":"120/80","temp":"98.6","pulse":"72","spo2":"99%","weight":"70kg"}'
rows={3}/></div>

  <div className="form-group"><label>Notes</label><textarea value={visitF.Notes}
onChange={e=>setVisitF({...visitF,Notes:e.target.value})} rows={4}/></div>

  <div className="form-group"><label>Status</label><select value={visitF.Status}
onChange={e=>setVisitF({...visitF,Status:e.target.value})}><option>Scheduled</option><option>In
Progress</option><option>Completed</option></select></div>

  <button type="submit" className="submit-btn"
disabled={loading}>{loading?'Creating...':'Create Visit'}</button>

</form></div>

```

```
}}
```

```
{/* — Add Diagnosis — */}
```

```
{view==='add-diagnosis'&&{
```

```
  <div className="form-container"><form onSubmit={submitDiag} className="data-  
form"><h3>🩺 Diagnosis Information</h3>
```

```
  <div className="form-row">
```

```
    <div className="form-group"><label>Visit *</label><select value={diagF.VisitID || ""}  
onChange={e=>setDiagF({...diagF,VisitID:Number(e.target.value)})} required><option value="">Select  
Visit</option>{visits.map(v=><option key={v.VisitID} value={v.VisitID}>#{v.VisitID} — {v.PatientName}  
{formatDate(v.VisitDate)}</option>)}</select></div>
```

```
    <div className="form-group"><label>Severity *</label><select value={diagF.Severity}  
onChange={e=>setDiagF({...diagF,Severity:e.target.value})}><option>Mild</option><option>Moderat  
e</option><option>Severe</option></select></div>
```

```
  </div>
```

```
  <div className="form-group"><label>Diagnosis Name *</label><input type="text"  
value={diagF.DiagnosisName} onChange={e=>setDiagF({...diagF,DiagnosisName:e.target.value})}  
required/></div>
```

```
  <div className="form-group"><label>Description</label><textarea  
value={diagF.Description} onChange={e=>setDiagF({...diagF,Description:e.target.value})}  
rows={4}/></div>
```

```
  <div className="form-group checkbox-group"><label><input type="checkbox"  
checked={diagF.IsChronic}  
onChange={e=>setDiagF({...diagF,IsChronic:e.target.checked})}/><span>Chronic  
Condition</span></label></div>
```

```
  <button type="submit" className="submit-btn"  
disabled={loading}><loading?'Adding...':'Add Diagnosis'></button>
```

```
  </form></div>
```

```
}}
```

```
{/* — Add Prescription — */}
```

```
{view==='add-rx'&&{
```

```
  <div className="form-container"><form onSubmit={submitRx} className="data-  
form"><h3>💊 Prescription Information</h3>
```

```
  <div className="form-group"><label>Visit *</label><select value={rxF.visit_id || ""}  
onChange={e=>setRxF({...rxF,visit_id:Number(e.target.value)})} required><option value="">Select
```

```
Visit</option>{visits.map(v=><option key={v.VisitID} value={v.VisitID}>#{v.VisitID} — {v.PatientName}
({fmtDate(v.VisitDate)})}</option>)}</select></div>
```

```
<div className="form-row">
```

```
<div className="form-group"><label>Medicine *</label><input type="text"
value={rxF.medicine} onChange={e=>setRxF({...rxF,medicine:e.target.value})} required/></div>
```

```
<div className="form-group"><label>Dosage *</label><input type="text"
value={rxF.dosage} onChange={e=>setRxF({...rxF,dosage:e.target.value})} required/></div>
```

```
</div>
```

```
<div className="form-row">
```

```
<div className="form-group"><label>Frequency *</label><input type="text"
value={rxF.frequency} onChange={e=>setRxF({...rxF,frequency:e.target.value})} required/></div>
```

```
<div className="form-group"><label>Duration *</label><input type="text"
value={rxF.duration} onChange={e=>setRxF({...rxF,duration:e.target.value})} required/></div>
```

```
</div>
```

```
<div className="form-group"><label>Instructions</label><textarea value={rxF.instructions}
onChange={e=>setRxF({...rxF,instructions:e.target.value})} rows={3}/></div>
```

```
<button type="submit" className="submit-btn"
disabled={loading}>{loading?'Adding...':'Add Prescription'}</button>
```

```
</form></div>
```

```
}}
```

```
{/* — My Files (Patient) — */}
```

```
{view==='my-files'&&(
```

```
<div className="files-container">
```

```
{medFiles.length===0
```

```
? <div className="empty-state"><FileImage size={64}/><h3>No files
uploaded</h3><button onClick={()=>setView('upload-file')} className="action-btn">Upload
File</button></div>
```

```
: <div className="files-grid">{medFiles.map(f=>(
```

```
<div key={f.FileID} className="file-card">
```

```
<div className="file-icon"><FileImage size={40}/></div>
```

```
<div className="file-details"><h4>{f.FileType}</h4><p className="file-
name">{f.FileName}</p><p className="file-size">{(f.FileSize/1024/1024).toFixed(2)} MB</p><p
className="file-date">{fmtDate(f.UploadedAt)}</p>{f.Description}&&<p className="file-
desc">{f.Description}</p>}</div>
```

```

        <button className="download-btn"
onClick={()=>downloadFile(f.FileID,f.FileName)}><Download size={20}/> Download</button>

    </div>

    )}</div>

}

</div>

)}

```

```

{ /* — Upload File (Patient) — */

{view==='upload-file'&&(

    <div className="form-container"><form onSubmit={submitUpload} className="data-
form"><h3>  Upload Medical File</h3>

    <div className="form-group"><label>File Type *</label><select value={fileF.file_type}
onChange={e=>setFileF({...fileF,file_type:e.target.value})}>[{'X-Ray','MRI','CT Scan','Blood
Test','Report','Prescription','Other'}.map(t=><option key={t}>{t}</option>)]</select></div>

    <div className="form-group"><label>Select File *</label><div className="file-input-
wrapper"><input type="file" onChange={e=>setFileF({...fileF,file:e.target.files?.[0] || null)}}
accept=".jpg,.jpeg,.png,.pdf,.dcm" required/>{fileF.file}&&<p className="file-selected">Selected:
{fileF.file.name}</p></div><small>JPG, PNG, PDF, DICOM · max 50 MB</small></div>

    <div className="form-group"><label>Description</label><textarea value={fileF.description}
onChange={e=>setFileF({...fileF,description:e.target.value})} rows={3}/></div>

    <button type="submit" className="submit-btn"
disabled={loading || !fileF.file}>{loading?'Uploading...':'Upload File'}</button>

    </form></div>

)}

```

```

{ /* — My Appointments (Patient) — */

{view==='my-appointments'&&(

    <MyAppointmentsView token={token} doctors={doctors}/>

)}

```

```

{ /* — Appointment Queue (Receptionist) — */

{view==='appointment-queue'&&(

    <ReceptionistDashboard token={token} doctors={doctors}/>

```

```

    })

    { /* — Prescriptions (Pharmacist) — */

    {view==='prescriptions'&&(

    <div className="prescriptions-container">

    <div className="prescriptions-header">

    <label className="filter-checkbox"><input type="checkbox" checked={pendingOnly}
onChange={e=>{setPendingOnly(e.target.checked);setTimeout(loadRx,0);}}/><span>Show Pending
Only</span></label>

    </div>

    {prescriptions.length===0

    ? <div className="empty-state"><Pill size={64}/><h3>No prescriptions</h3></div>

    : <div className="table-container"><table className="data-
table"><thead><tr><th>Patient</th><th>Medicine</th><th>Dosage</th><th>Frequency</th><th>D
uration</th><th>Doctor</th><th>Status</th><th>Action</th></tr></thead>

    <tbody>{prescriptions.map(p=>{

    <tr key={p.PrescriptionID}>

    <td><strong>{p.PatientName}</strong></td><td><strong>{p.MedicineName}</strong></td>

    <td>{p.Dosage}</td><td>{p.Frequency}</td><td>{p.Duration}</td><td>{p.DoctorName}</td>

    <td><span className={`status-badge
${p.IsDispensed?'dispensed':'pending'}`}>{p.IsDispensed?<><CheckCircle size={14}/>
Dispensed</>:<><Clock size={14}/> Pending</>}</span></td>

    <td>{!p.IsDispensed&&<button className="dispense-btn"
onClick={()=>dispense(p.PrescriptionID)}>Dispense</button>}</td>

    </tr>

    )}}</tbody>

    </table></div>

    }

    </div>

    })

    { /* — Upload Scan (Radiologist) — */

```



```

{view==='upload-scan'&&(
  <div className="form-container"><form onSubmit={submitScanUpload} className="data-
form"><h3> 🧠 Upload Patient Scan</h3>

  <div className="form-row">

    <div className="form-group"><label>Patient *</label><select value={scanF.patient_id}
onChange={e=>setScanF({...scanF,patient_id:e.target.value})} required><option value="">Select
Patient</option>{patients.map(p=><option key={p as any}.PatientID} value={{p as
any}.PatientID}>{p.PatientName} — {p.BloodGroup}</option>}}</select></div>

    <div className="form-group"><label>Scan Type *</label><select value={scanF.scan_type}
onChange={e=>setScanF({...scanF,scan_type:e.target.value})}>{SCAN_TYPES.map(t=><option
key={t}>{t}</option>}}</select></div>

  </div>

  <div className="form-group"><label>Scan File *</label><div className="file-input-
wrapper"><input type="file" onChange={e=>setScanF({...scanF,file:e.target.files?.[0] || null)}}
accept=".jpg,.jpeg,.png,.pdf,.dcm" required/>{scanF.file}&&<p className="file-selected">Selected:
{scanF.file.name}</p></div><small>JPG, PNG, PDF, DICOM · max 50 MB</small></div>

  <div className="form-group"><label>Description / Findings</label><textarea
value={scanF.description} onChange={e=>setScanF({...scanF,description:e.target.value})} rows={4}
placeholder="e.g. Chest X-Ray — No consolidation seen. Normal cardiac silhouette."/></div>

  <button type="submit" className="submit-btn"
disabled={loading || !scanF.file || !scanF.patient_id}>{loading?'Uploading...':'Upload Scan'}</button>

</form></div>

))

/* — My Uploads (Radiologist) — */

{view==='my-uploads'&&(
  <div className="files-container">

    {myScans.length===0

      ? <div className="empty-state"><ScanLine size={64}/><h3>No scans uploaded
yet</h3><button onClick={()=>setView('upload-scan')} className="action-btn">Upload
Scan</button></div>

      : <div style={{ display:'flex',flexDirection:'column',gap:'0.75rem' }}>{myScans.map(s=>(

        <div key={s.FileID} style={{ background:'var(--card-bg)',border:'1px solid
rgba(244,63,94,.14)',borderRadius:14,padding:'1rem
1.25rem',display:'flex',alignItems:'center',gap:'1rem' }}>

          <div style={{
width:44,height:44,borderRadius:11;background:'rgba(244,63,94,.1)',border:'1px solid

```

```
rgba(244,63,94,.2)',display:'flex',alignItems:'center',justifyContent:'center',color:'var(--rose)',flexShrink:0 }}><ScanLine size={20}/></div>
```

```
<div style={{ flex:1,minWidth:0 }}>
```

```
<div style={{ display:'flex',alignItems:'center',gap:'0.6rem',marginBottom:'0.2rem' }}><p style={{ margin:0,fontWeight:700,fontSize:'0.9rem' }}>{s.FileName}</p><span style={{
fontSize:'0.7rem',padding:'0.12rem
0.5rem',borderRadius:999,background:'rgba(244,63,94,.1)',color:'var(--rose)',fontWeight:700
}}>{s.FileType}</span></div>
```

```
<p style={{ margin:0,fontSize:'0.78rem',color:'var(--text-dim)' }}>Patient: <strong style={{
color:'var(--text-mid)' }}>{s.PatientName}</strong> · {s.BloodGroup} ·
{(s.FileSize/1024/1024).toFixed(2)} MB · {fmtDate(s.UploadedAt)}</p>
```

```
{s.Description}&&<p style={{ margin:'0.25rem 0 0',fontSize:'0.78rem',color:'var(--text-
mid)' }}>{s.Description}</p>
```

```
</div>
```

```
<button onClick={()=>downloadFile(s.FileID,s.FileName)} className="download-btn"
style={{ whiteSpace:'nowrap' }}><Download size={16}/> Download</button>
```

```
</div>
```

```
))/</div>
```

```
}
```

```
</div>
```

```
}}
```

```
{/* — Patients (Admin) — Split Panel — */}
```

```
{view==='patients'&&{
```

```
<div style={{ display:'flex', height:'calc(100vh - 140px)', margin:'-2rem -2.5rem',
overflow:'hidden' }}>
```

```
{/* Left: patient list */}
```

```
<div style={{ width:260, flexShrink:0, borderRight:'1px solid var(--border)', display:'flex',
flexDirection:'column', background:'var(--bg-1)' }}>
```

```
<div style={{ padding:'0.875rem', borderBottom:'1px solid var(--border)' }}>
```

```
<div style={{ display:'flex', alignItems:'center', gap:'0.5rem', marginBottom:'0.5rem' }}>
```

```
<Users size={14} style={{ color:'var(--accent)' }}/>
```

```
<span style={{ fontFamily:'"Syne",sans-serif', fontWeight:800, fontSize:'0.72rem',
textTransform:'uppercase', letterSpacing:'0.08em', color:'var(--text-dim)' }}>All Patients</span>
```

```
    <span style={{ marginLeft:'auto', background:'var(--accent-dim)', border:'1px solid var(--border)', borderRadius:999, fontSize:'0.65rem', fontWeight:700, padding:'0.1rem 0.4rem', color:'var(--accent)' }}>{patients.length}</span>
```

```
</div>
```

```
    <input className="patient-list-search" style={{ marginBottom:0 }} placeholder="Search patients..."
```

```
        value={adminPatientSearch} onChange={e=>setAdminPatientSearch(e.target.value)}/>
```

```
</div>
```

```
{/* Bulk action toolbar */}
```

```
{selectedPatientIds.length > 0 && (
```

```
    <div style={{ padding:'0.625rem', borderBottom:'1px solid var(--border)', background:'var(--amber-dim)', display:'flex', alignItems:'center', justifyContent:'space-between', gap:'0.5rem' }}>
```

```
        <div style={{ display:'flex', alignItems:'center', gap:'0.5rem' }}>
```

```
            <input type="checkbox" checked={selectedPatientIds.length > 0} onChange={(e) => {
```

```
                if (e.target.checked) {
```

```
                    // Select all visible patients
```

```
                    const visiblePatients = patients.filter(p => !adminPatientSearch || p.PatientName.toLowerCase().includes(adminPatientSearch.toLowerCase()));
```

```
                    setSelectedPatientIds(visiblePatients.map(p => (p as any).PatientID));
```

```
                } else {
```

```
                    setSelectedPatientIds([]);
```

```
                }
```

```
            }} style={{ cursor:'pointer' }}/>
```

```
            <span style={{ fontSize:'0.75rem', fontWeight:700, color:'var(--amber)' }}>{selectedPatientIds.length} selected</span>
```

```
        </div>
```

```
    <button onClick={async () => {
```

```
        if (!window.confirm(`Are you sure you want to deactivate ${selectedPatientIds.length} patient(s)? This action cannot be undone.`)) return;
```

```
        try {
```

```
            const res = await fetch(`${API}/patients/bulk-delete`, {
```

```
                method: 'POST',
```

```

headers: {
  'Content-Type': 'application/json',
  Authorization: `Bearer ${token}`
},
body: JSON.stringify({ patient_ids: selectedPatientIds })
});
const data = await res.json();
if (!res.ok) throw new Error(data.error || 'Bulk deletion failed');
alert(`Successfully deactivated ${data.deactivated_count} patient(s)`);
// Refresh patient list
loadPatients();
// Clear selection
setSelectedPatientIds([]);
// Clear selected patient if it was among the deleted ones
if (adminSelectedPatient &&
selectedPatientIds.includes(adminSelectedPatient.PatientID)) {
  setAdminSelectedPatient(null);
}
} catch (err: any) {
  alert(`Error: ${err.message}`);
}

}} style={{ background:'var(--rose)', color:'white', border:'none', borderRadius:'var(--r-sm)', padding:'0.3rem 0.6rem', fontSize:'0.7rem', fontWeight:700, cursor:'pointer', display:'flex', alignItems:'center', gap:'0.3rem' }}>

  <Trash2 size={11}/> Delete Selected

</button>

</div>

)}

<div style={{ flex:1, overflowY:'auto', padding:'0.625rem' }}>

{patients.filter(p=>!adminPatientSearch || p.PatientName.toLowerCase().includes(adminPatientSearch.toLowerCase()))}.map(p => {

```

```

const patientId = (p as any).PatientID;

const isActive = adminSelectedPatient?.PatientID === patientId;

const isSelected = selectedPatientIds.includes(patientId);

return (
  <div key={patientId}
    className={`patient-tile ${isActive?'active':''} ${isSelected?'selected':''}`}
    onClick={(e) => {
      // Don't change selection if clicking on checkbox
      if ((e.target as HTMLInputElement).tagName === 'INPUT') return;
      setAdminSelectedPatient(p as any);
    }}>
    <div style={{ display:'flex', alignItems:'center', gap:'0.5rem', width:'100%' }}>
      <input type="checkbox" checked={isSelected} onChange={(e) => {
        e.stopPropagation();
        if (e.target.checked) {
          setSelectedPatientIds([...selectedPatientIds, patientId]);
        } else {
          setSelectedPatientIds(selectedPatientIds.filter(id => id !== patientId));
        }
      }} style={{ cursor:'pointer', flexShrink:0 }}/>
      <div style={{ flex:1, minWidth:0 }}>
        <div className="pt-name">{p.PatientName}</div>
        <div className="pt-meta">{p.Gender} · {p.BloodGroup}</div>
        <div className="pt-badges">
          <span className="badge badge-blood" style={{ fontSize:'0.65rem',padding:'0.1rem
0.4rem' }}>{p.BloodGroup}</span>
        </div>
      </div>
    </div>
  </div>
);

```

```

    }}}
  </div>

  { /* Assign Credentials shortcut */ }

  {noCreds.length>0&&(

    <div style={{ padding:'0.75rem', borderTop:'1px solid var(--border)', background:'var(--amber-dim)' }}>

      <button onClick={()=>setView('assign-creds')} style={{ width:'100%', padding:'0.6rem',
        background:'var(--amber)', border:'none', borderRadius:'var(--r-md)', color:'white',
        fontFamily:'Syne,sans-serif', fontWeight:700, fontSize:'0.78rem', cursor:'pointer', display:'flex',
        alignItems:'center', justifyContent:'center', gap:'0.4rem' }}>

        <Key size={13}/> {noCreds.length} Need Credentials

      </button>

    </div>

  )}
</div>

{ /* Right: patient detail pane */ }

<div style={{ flex:1, minWidth:0, overflow:'hidden', display:'flex', flexDirection:'column' }}>
  {adminSelectedPatient ? (
    <AdminPatientDetailPane
      key={adminSelectedPatient.PatientID}
      patient={adminSelectedPatient}
      token={token}
      onEdit={()=>setEditPatient(adminSelectedPatient)}
      onDelete={()=>setDeleteConfirm(adminSelectedPatient)}
    />
  ) : (
    <div className="empty-state">

      <div style={{ fontSize:40, opacity:.2, marginBottom:'0.5rem' }}> 🖐️ </div>

      <h3>Select a patient</h3>

      <p>Click any patient from the list to view their complete profile, visit history, diagnoses,
      prescriptions, lab tests, and files.</p>

    </div>
  )}

```

```

    })
  </div>
</div>
})

{/* — Manage Doctors (Admin) — */}

{view==='manage-doctors'&&(
  <div className="patients-container">
    <div style={{ display:'flex', justifyContent:'space-between', alignItems:'center',
marginBottom:'1.5rem' }}>
      <h2 style={{ fontFamily:"Space Grotesk",sans-serif", fontWeight:800, fontSize:'1.4rem',
color:'var(--text-bright)', margin:0 }}>Manage Doctors</h2>
      <span style={{ background:'var(--accent-dim)', border:'1px solid var(--border)',
borderRadius:999, fontSize:'0.75rem', fontWeight:700, padding:'0.2rem 0.75rem', color:'var(--
accent)' }}>{doctors.length} doctors</span>
    </div>

    {/* Add Doctor Form */}

    <div style={{ background:'var(--card-bg)', border:'1px solid var(--border-glass)',
borderRadius:'var(--r-lg)', padding:'1.5rem', marginBottom:'2rem', boxShadow:'var(--shadow-card)'
}}>
      <h3 style={{ fontFamily:"Syne",sans-serif", fontWeight:700, fontSize:'1rem', color:'var(--
accent)', marginTop:0, marginBottom:'1rem', display:'flex', alignItems:'center', gap:'0.5rem'
}}><UserPlus size={18}/> Add New Doctor</h3>

      <form onSubmit={addDoctor} style={{ display:'grid', gridTemplateColumns:'repeat(auto-fit,
minmax(200px, 1fr))', gap:'1rem' }}>
        <div className="form-group">
          <label>Full Name *</label>

          <input type="text" value={newDoctorF.doctor_name}
onChange={e=>setNewDoctorF({...newDoctorF, doctor_name:e.target.value}} placeholder="Dr. John
Doe" required/>
        </div>

        <div className="form-group">
          <label>Email *</label>

```

```

        <input type="email" value={newDoctorF.email}
onChange={e=>setNewDoctorF({...newDoctorF, email:e.target.value}})
placeholder="doctor@hospital.com" required/>

    </div>

    <div className="form-group">

        <label>Specialty *</label>

        <input type="text" value={newDoctorF.specialty}
onChange={e=>setNewDoctorF({...newDoctorF, specialty:e.target.value}}) placeholder="Cardiology"
required/>

    </div>

    <div className="form-group">

        <label>Phone Number *</label>

        <input type="tel" value={newDoctorF.phone_number}
onChange={e=>setNewDoctorF({...newDoctorF, phone_number:e.target.value}})
placeholder="+1234567890" required/>

    </div>

    <div className="form-group">

        <label>License Number</label>

        <input type="text" value={newDoctorF.license_number}
onChange={e=>setNewDoctorF({...newDoctorF, license_number:e.target.value}})
placeholder="Optional"/>

    </div>

    <div className="form-group">

        <label>Years of Experience</label>

        <input type="number" min="0" max="50" value={newDoctorF.years_of_experience}
onChange={e=>setNewDoctorF({...newDoctorF, years_of_experience:e.target.value}})
placeholder="Optional"/>

    </div>

    <div className="form-group">

        <label>Username (optional)</label>

        <input type="text" value={newDoctorF.username}
onChange={e=>setNewDoctorF({...newDoctorF, username:e.target.value}}) placeholder="Leave
empty to auto-generate"/>

    </div>

    <div className="form-group">

```



```

        <label>Password (optional)</label>

        <input type="password" value={newDoctorF.password}
onChange={e=>setNewDoctorF({...newDoctorF, password:e.target.value})} placeholder="Leave
empty to auto-generate"/>

    </div>

    <div style={{ gridColumn:'1/-1', display:'flex', justifyContent:'flex-end', gap:'0.75rem',
marginTop:'0.5rem' }}>

        <button type="button" className="action-btn secondary" onClick={()=>setNewDoctorF({
doctor_name:"", email:"", specialty:"", phone_number:"", license_number:"", years_of_experience:"",
username:"", password:"" })}>Clear</button>

        <button type="submit" className="action-btn" disabled={loading}>{loading ? 'Adding...' :
'Add Doctor'}</button>

    </div>

</form>

</div>

```

```

{/* Doctors Table */}

<div className="table-container">

    <table className="data-table">

        <thead>

            <tr>

                <th>ID</th>

                <th>Name</th>

                <th>Email</th>

                <th>Specialty</th>

                <th>Phone</th>

                <th>License</th>

                <th>Experience</th>

                <th style={{ textAlign:'center' }}>Actions</th>

            </tr>

        </thead>

        <tbody>

            {doctors.length === 0 ? (

```

```
<tr><td colSpan={8} style={{ textAlign:'center', padding:'2rem', color:'var(--text-dim)'
}}>No doctors found. Add a doctor using the form above.</td></tr>
```

```
) : doctors.map(d => (
```

```
<tr key={d.DoctorID}>
```

```
<td><strong>#{d.DoctorID}</strong></td>
```

```
<td><strong>{d.DoctorName}</strong></td>
```

```
<td><div className="contact-cell"><Mail
size={14}/><span>{d.Email}</span></div></td>
```

```
<td><span className="badge badge-blood" style={{ background:'var(--accent-dim)',
color:'var(--accent)' }}>{d.Specialty}</span></td>
```

```
<td><div className="contact-cell"><Phone
size={14}/><span>{d.PhoneNumber}</span></div></td>
```

```
<td>{d.LicenseNumber || '—'}</td>
```

```
<td>{d.YearsOfExperience ? `${d.YearsOfExperience} years` : '—'}</td>
```

```
<td style={{ textAlign:'center' }}>
```

```
<div style={{ display:'flex', gap:'0.5rem', justifyContent:'center' }}>
```

```
<button
```

```
onClick={() => setEditDoctor(d)}
```

```
style={{
```

```
background: 'linear-gradient(135deg, var(--accent), #0ea5e9)',
```

```
border: 'none',
```

```
borderRadius: '6px',
```

```
padding: '0.4rem 0.75rem',
```

```
color: 'white',
```

```
cursor: 'pointer',
```

```
display: 'inline-flex',
```

```
alignItems: 'center',
```

```
gap: '0.35rem',
```

```
fontSize: '0.8rem',
```

```
fontWeight: 700,
```

```
fontFamily: "'Syne', sans-serif"
```

```
}}</div></td></tr>
```

```

        title="Edit doctor"
      >
        <Edit2 size={13}/> Edit
      </button>
      <button
        onClick={() => doDeleteDoctor(d.DoctorID)}
        className="delete-btn"
        style={{
          background: 'linear-gradient(135deg, var(--rose), #dc2626)',
          border: 'none',
          borderRadius: '6px',
          padding: '0.4rem 0.75rem',
          color: 'white',
          cursor: 'pointer',
          display: 'inline-flex',
          alignItems: 'center',
          gap: '0.35rem',
          fontSize: '0.8rem',
          fontWeight: 700,
          fontFamily: "'Syne', sans-serif"
        }}
        title="Delete doctor"
      >
        <Trash2 size={13}/> Delete
      </button>
    </div>
  </td>
</tr>
)}}
</tbody>
</table>

```

```

    </div>
  </div>
})

{/* — Assign Credentials (Admin) — */}
{view==='assign-creds'&&(
  <div className="patients-container">
    {noCreds.length===0
      ? <div className="empty-state"><Key size={64}/><h3>All patients have
credentials</h3></div>
      : <>
        <div style={{ marginBottom:'1rem',padding:'0.875rem 1.25rem',background:'var(--amber-
dim)',border:'1px solid
rgba(245,158,11,.18)',borderRadius:12,display:'flex',alignItems:'center',gap:'0.6rem' }}>
          <AlertCircle size={16} color="var(--amber)"/>
          <p style={{ margin:0,color:'#fbbf24',fontSize:'0.85rem'
}}><strong>{noCreds.length}</strong> patient{noCreds.length>1?'s':''} imported via CSV need login
credentials.</p>
        </div>
        <div className="table-container"><table className="data-
table"><thead><tr><th>Name</th><th>Email</th><th>Gender</th><th>Blood
Group</th><th>Phone</th><th>Imported On</th><th style={{ textAlign:'center'
}}>Action</th></tr></thead>
      <tbody>{noCreds.map(p=>(
        <tr key={p.PatientID}>
          <td><strong>{p.PatientName}</strong></td>
          <td><div className="contact-cell"><Mail
size={14}/><span>{p.Email}</span></div></td>
          <td>{p.Gender}</td>
          <td><span className="badge badge-blood">{p.BloodGroup}</span></td>
          <td><div className="contact-cell"><Phone
size={14}/><span>{p.PhoneNumber}</span></div></td>
          <td>{fmtDate(p.CreatedAt)}</td>
          <td style={{ textAlign:'center' }}>

```

```

        <button onClick={()=>setAssignPatient(p)} style={{ background:'linear-
gradient(135deg,var(--amber),#d97706)',border:'none',borderRadius:8,padding:'0.4rem
0.875rem',color:'white',cursor:'pointer',display:'inline-
flex',alignItems:'center',gap:'0.35rem',fontSize:'0.8rem',fontWeight:700,fontFamily:'"Syne',sans-serif"
}}>

```

```

        <Key size={13}/> Assign
    </button>
</td>
</tr>
</tbody>
</table></div>
</>
}
</div>
})

```

```

{ /* — Import (Admin) — */}
{view==='import'&&(
    <div className="import-container" style={{ display:'grid', gridTemplateColumns:'1fr 1fr',
gap:'1.5rem', alignItems:'start' }}>

```

```

{ /* == PANEL 1: Import New Patients == */}
<div className="import-section">
    <h3> 📄 Import New Patients</h3>

    <div className="import-instructions">
        <h4>Required Columns:</h4>
        <p>Name, Email, Gender, DOB, Phone, Address, BloodGroup</p>
        <div style={{ marginTop:'0.625rem', paddingTop:'0.625rem', borderTop:'1px solid var(--
border-glass)' }}>
            <p style={{ fontSize:'0.8rem', color:'var(--emerald)', fontWeight:600,
marginBottom:'0.22rem', display:'flex', alignItems:'center', gap:'0.4rem' }}> 🔑 Optional — auto-
create login credentials:</p>
            <p style={{ fontSize:'0.78rem', color:'var(--text-dim)' }}>

```

Username, Password — if provided, a Patient login is created immediately (no separate "Assign Credentials" step).

</p>

<p style={{ fontSize:'0.78rem', color:'var(--text-dim)', marginTop:'0.25rem' }}>Also optional: EmergencyContactName, EmergencyContact</p>

</div>

<Download size={15}/> Download Patient Template

</div>

<form onSubmit={submitImport} className="import-form">

<div className="form-group">

<label>Select Excel / CSV File</label>

<div className="file-input-wrapper">

<input type="file" onChange={e=>setImpFile(e.target.files?.[0] || null)} accept=".xlsx,.xls,.csv" required/>

{impFile && <p className="file-selected"> 📄 {impFile.name}</p>}

</div>

<small>Supports .xlsx, .xls, .csv · max 50 MB</small>

</div>

<button type="submit" className="submit-btn" disabled={loading || !impFile} style={{ display:'flex', alignItems:'center', justifyContent:'center', gap:'0.5rem' }}>

{loading ? 'Importing...' : <><Upload size={15}/> Import Patients</>}

</button>

</form>

{importResult && (

<div style={{ marginTop:'1.25rem', padding:'1.25rem', background:'var(--glass)', border:'1px solid var(--border-glass)', borderRadius:'var(--r-lg)' }}>

<p style={{ fontFamily:'"Space Grotesk",sans-serif', fontWeight:700, fontSize:'0.78rem', textTransform:'uppercase', letterSpacing:'0.07em', color:'var(--text-mid)', marginBottom:'0.875rem' }}>Import Results</p>

```

    <div className="result-stats">

      <div className="result-stat
success"><strong>{importResult.successful}</strong><span>Patients Imported</span></div>

      <div className="result-stat
error"><strong>{importResult.failed}</strong><span>Failed</span></div>

      <div className="result-stat total"><strong>{importResult.total}</strong><span>Total
Rows</span></div>

    </div>

    {(importResult.creds_created??0) > 0 && (

      <div style={{ marginTop:'0.875rem', padding:'0.7rem 1rem', background:'var(--emerald-
dim)', border:'1px solid rgba(0,229,160,0.22)', borderRadius:'var(--r-md)', fontSize:'0.82rem',
color:'var(--emerald)', display:'flex', alignItems:'center', gap:'0.5rem' }}>

        <CheckCircle size={14}/> {importResult.creds_created} login
account{importResult.creds_created!==1?'s':''} created from CSV credentials.

      </div>

    )}

    {importResult.successful > 0 && (importResult.creds_created??0) <
importResult.successful && (

      <div style={{ marginTop:'0.875rem', padding:'0.7rem 1rem', background:'var(--amber-
dim)', border:'1px solid rgba(255,189,46,0.2)', borderRadius:'var(--r-md)', display:'flex',
alignItems:'center', justifyContent:'space-between', gap:'0.75rem', flexWrap:'wrap' }}>

        <div style={{ display:'flex', alignItems:'center', gap:'0.5rem', fontSize:'0.82rem',
color:'var(--amber)' }}>

          <Key size={14}/> {importResult.successful-(importResult.creds_created??0)}
patient{(importResult.successful-(importResult.creds_created??0))!==1?'s':''} still need login
credentials.

        </div>

        <button onClick={()=>setView('assign-creds')} className="action-btn" style={{
padding:'0.4rem 0.875rem', fontSize:'0.78rem', marginTop:0 }}>Assign Credentials →</button>

      </div>

    )}

    {importResult.errors?.length > 0 && (

      <div className="error-log" style={{ marginTop:'0.75rem' }}><h5>Errors
({importResult.errors.length}):</h5><ul>{importResult.errors.map((e:string,i:number)=><li
key={i}>{e}</li>)}</ul></div>

    )}

```

```
</div>

}}

</div>
```

```
{/* == PANEL 2: Import Visit History == */}

<div className="import-section">

  <h3> 📄 Import Visit History</h3>

  <div className="import-instructions">

    <h4>Required Columns:</h4>

    <p>Email, VisitDate, ReasonForVisit, DoctorName</p>

    <div style={{ marginTop:'0.625rem', paddingTop:'0.625rem', borderTop:'1px solid var(--border-glass)' }}>

      <p style={{ fontSize:'0.8rem', color:'var(--emerald)', fontWeight:600,
marginBottom:'0.22rem', display:'flex', alignItems:'center', gap:'0.4rem' }}> 🔑 Optional — import
credentials alongside visits:</p>

      <p style={{ fontSize:'0.78rem', color:'var(--text-dim)', marginBottom:'0.25rem'
}}><strong>Username, Password</strong> — creates a login for new patients during visit
import.</p>

      <p style={{ fontSize:'0.78rem', color:'var(--text-dim)' }}>Also optional: Name, DOB,
DiagnosisName, MedicineName, BP, HeartRate, Temp, SpO2, Notes, IsChronic, Severity, Dosage,
Frequency, Duration</p>

    </div>

    <a href={` ${API}/admin/import-template?type=visits`} className="download-template-
btn" style={{ marginTop:'0.5rem' }}>download</a>

    <Download size={15}/> Download Visit Template

  </a>

</div>

  <div style={{ padding:'0.75rem 1rem', background:'var(--accent-dim)', border:'1px solid var(-
-border-glass)', borderRadius:'var(--r-md)', marginBottom:'1rem', fontSize:'0.8rem', color:'var(--text-
mid)', lineHeight:1.5 }}>

    <strong style={{ color:'var(--accent)', display:'flex', alignItems:'center', gap:'0.4rem',
marginBottom:'0.25rem' }}><Shield size={12}/> Deduplication enabled</strong>
```


Patients matched by Email → Name+DOB fallback. Duplicate visits (same patient+doctor+date+reason) and duplicate usernames are skipped automatically.

</div>

<form onSubmit={submitImportVisits} className="import-form">

<div className="form-group">

<label>Select Excel / CSV File</label>

<div className="file-input-wrapper">

<input type="file" onChange={e=>setImpVisitFile(e.target.files?.[0] || null)}
accept=".xlsx,.xls,.csv" required/>

{impVisitFile && <p className="file-selected"> 📎 {impVisitFile.name}</p>}

</div>

<small>Supports .xlsx, .xls, .csv · max 50 MB</small>

</div>

<button type="submit" className="submit-btn" disabled={loading || !impVisitFile} style={{
display:'flex', alignItems:'center', justifyContent:'center', gap:'0.5rem' }}>

{loading ? 'Importing...' : <><Upload size={15}/> Import Visit History</>}

</button>

</form>

{importVisitResult && (

<div style={{ marginTop:'1.25rem', padding:'1.25rem', background:'var(--glass)',
border:'1px solid var(--border-glass)', borderRadius:'var(--r-lg)' }}>

<p style={{ fontFamily:'"Space Grotesk",sans-serif', fontWeight:700, fontSize:'0.78rem',
textTransform:'uppercase', letterSpacing:'0.07em', color:'var(--text-mid)', marginBottom:'0.875rem'
}}>Import Results</p>

<div className="result-stats">

<div className="result-stat
success">{importVisitResult.successful}Visits Imported</div>

<div className="result-stat
error">{importVisitResult.failed}Failed</div>

<div className="result-stat
total">{importVisitResult.total}Total Rows</div>

</div>

```

<div style={{ display:'flex', gap:'0.5rem', flexWrap:'wrap', marginTop:'0.875rem' }}>

  {{importVisitResult.patients_created??0} > 0 && (

    <div style={{ padding:'0.38rem 0.75rem', borderRadius:999, background:'var(--violet-
dim)', border:'1px solid rgba(155,125,255,0.22)', fontSize:'0.78rem', color:'var(--violet)',
fontWeight:600 }}>

       {importVisitResult.patients_created} new patient
      profile{importVisitResult.patients_created!==1?'s':''} created

    </div>

  }}

  {{importVisitResult.creds_created??0} > 0 && (

    <div style={{ padding:'0.38rem 0.75rem', borderRadius:999, background:'var(--emerald-
dim)', border:'1px solid rgba(0,229,160,0.22)', fontSize:'0.78rem', color:'var(--emerald)',
fontWeight:600 }}>

       {importVisitResult.creds_created} login
      account{importVisitResult.creds_created!==1?'s':''} created

    </div>

  }}

  {{importVisitResult.diagnoses_added??0} > 0 && (

    <div style={{ padding:'0.38rem 0.75rem', borderRadius:999, background:'var(--amber-
dim)', border:'1px solid rgba(255,189,46,0.22)', fontSize:'0.78rem', color:'var(--amber)',
fontWeight:600 }}>

       {importVisitResult.diagnoses_added} diagnoses added

    </div>

  }}

  {{importVisitResult.rx_added??0} > 0 && (

    <div style={{ padding:'0.38rem 0.75rem', borderRadius:999, background:'var(--accent-
dim)', border:'1px solid rgba(0,212,255,0.22)', fontSize:'0.78rem', color:'var(--accent)',
fontWeight:600 }}>

       {importVisitResult.rx_added} prescriptions added

    </div>

  }}

</div>

{importVisitResult.errors?.length > 0 && (

```

```

    <div className="error-log" style={{ marginTop:'0.875rem' }}><h5>Errors
    ({importVisitResult.errors.length}):</h5><ul>{importVisitResult.errors.map((e:string,i:number)=><li
    key={i}>{e}</li>)}</ul></div>

```

```
    })
```

```
</div>
```

```
  })
```

```
</div>
```

```
</div>
```

```
  })
```

```
</div>
```

```
</main>
```

```
</div>
```

```
);
```

```
}
```

```
app.css -
```

```
@import
```

```
url('https://fonts.googleapis.com/css2?family=Space+Grotesk:wght@300;400;500;600;700&family=Plus+Jakarta+Sans:ital,wght@0,300;0,400;0,500;0,600;0,700;0,800;1,400&family=JetBrains+Mono:wght@400;500;600&display=swap');
```

```
*,*::before,*::after{margin:0;padding:0;box-sizing:border-box;}
```

```
/* — MIDNIGHT ————— */
```

```
:root,[data-theme="midnight"]{
```

```
  --bg:#03050d;--bg-1:#060b18;--bg-2:#0a1020;--bg-3:#0e1628;--bg-4:#141e35;--bg-5:#1a2741;
```

```
  --accent:#00d4ff;--accent-b:#0099bb;--accent-light:#7aefff;--accent-glow:rgba(0,212,255,0.18);--
  accent-dim:rgba(0,212,255,0.08);
```

```
  --violet:#9b7dff;--violet-b:#7c5ce8;--violet-dim:rgba(155,125,255,0.12);
```

```
  --emerald:#00e5a0;--emerald-dim:rgba(0,229,160,0.12);
```

```
  --amber:#ffbd2e;--amber-dim:rgba(255,189,46,0.12);
```

```
  --rose:#ff4d6d;--rose-dim:rgba(255,77,109,0.12);
```

```
  --text-bright:#e8f4ff;--text-mid:#7a9ab8;--text-dim:#3d5a78;--text-faint:#1a2d42;
```

```

--border:rgba(0,212,255,0.08);--border-hover:rgba(0,212,255,0.30);--border-
active:rgba(0,212,255,0.60);--border-glass:rgba(255,255,255,0.06);

--card-bg:rgba(10,16,32,0.85);--raised:rgba(14,22,40,0.90);--glass:rgba(255,255,255,0.03);--glass-
2:rgba(255,255,255,0.06);--sidebar-bg:rgba(6,11,24,0.95);

--shadow-card:0 0 0 1px var(--border-glass),0 8px 32px rgba(0,0,0,0.6);

--shadow-float:0 16px 64px rgba(0,0,0,0.7),0 0 0 1px rgba(0,212,255,0.1);

--shadow-glow:0 0 40px rgba(0,212,255,0.22);--shadow-btn:0 4px 24px rgba(0,212,255,0.40);--
shadow-violet:0 4px 24px rgba(155,125,255,0.35);

--r-xs:3px;--r-sm:8px;--r-md:14px;--r-lg:20px;--r-xl:28px;--r-2xl:36px;

--ease-out:cubic-bezier(0.16,1,0.3,1);--ease-spring:cubic-bezier(0.34,1.56,0.64,1);

--t-fast:120ms;--t-mid:220ms;--t-slow:380ms;

}

/* — AURORA ————— */

[data-theme="aurora"]{

--bg:#050210;--bg-1:#080418;--bg-2:#0e0822;--bg-3:#140d2e;--bg-4:#1c1340;--bg-5:#241952;

--accent:#c471ed;--accent-b:#9b51c0;--accent-light:#e8b4ff;--accent-glow:rgba(196,113,237,0.20);--
accent-dim:rgba(196,113,237,0.08);

--violet:#f48fb1;--violet-b:#e91e8c;--violet-dim:rgba(244,143,177,0.12);

--emerald:#26d0ce;--emerald-dim:rgba(38,208,206,0.12);--amber:#ffd54f;--amber-
dim:rgba(255,213,79,0.12);--rose:#ff6b8a;--rose-dim:rgba(255,107,138,0.12);

--text-bright:#f5e9ff;--text-mid:#b48fd4;--text-dim:#6b3a8a;--text-faint:#2d1050;

--border:rgba(196,113,237,0.10);--border-hover:rgba(196,113,237,0.35);--border-
active:rgba(196,113,237,0.65);--border-glass:rgba(255,255,255,0.06);

--card-bg:rgba(14,8,34,0.88);--raised:rgba(20,13,46,0.92);--glass:rgba(255,255,255,0.03);--glass-
2:rgba(255,255,255,0.06);--sidebar-bg:rgba(8,4,24,0.95);

--shadow-card:0 0 0 1px var(--border-glass),0 8px 32px rgba(0,0,0,0.65);--shadow-float:0 16px 64px
rgba(0,0,0,0.75),0 0 0 1px rgba(196,113,237,0.12);

--shadow-glow:0 0 40px rgba(196,113,237,0.25);--shadow-btn:0 4px 24px rgba(196,113,237,0.45);--
shadow-violet:0 4px 24px rgba(244,143,177,0.35);

}

/* — DAYBREAK ————— */

[data-theme="daybreak"]{

--bg:#f0f6ff;--bg-1:#e4effc;--bg-2:#ffffff;--bg-3:#f0f6ff;--bg-4:#daeaf8;--bg-5:#c5ddf5;

```

```

--accent:#0075d1;--accent-b:#005eaa;--accent-light:#1d6fa3;--accent-glow:rgba(0,117,209,0.15);--
accent-dim:rgba(0,117,209,0.08);

--violet:#6b21e8;--violet-b:#5b18c5;--violet-dim:rgba(107,33,232,0.10);

--emerald:#057a55;--emerald-dim:rgba(5,122,85,0.10);--amber:#b45309;--amber-
dim:rgba(180,83,9,0.10);--rose:#be123c;--rose-dim:rgba(190,18,60,0.10);

--text-bright:#060f1e;--text-mid:#374151;--text-dim:#6b7280;--text-faint:#9ca3af;

--border:rgba(0,117,209,0.14);--border-hover:rgba(0,117,209,0.40);--border-
active:rgba(0,117,209,0.70);--border-glass:rgba(0,0,0,0.07);

--card-bg:#ffffff;--raised:#f0f6ff;--glass:rgba(255,255,255,0.7);--glass-2:rgba(255,255,255,0.9);--
sidebar-bg:#ffffff;

--shadow-card:0 0 0 1px rgba(0,117,209,0.12),0 4px 20px rgba(0,0,0,0.08);--shadow-float:0 8px 40px
rgba(0,0,0,0.12),0 0 0 1px rgba(0,117,209,0.10);

--shadow-glow:0 0 30px rgba(0,117,209,0.18);--shadow-btn:0 4px 20px rgba(0,117,209,0.28);--
shadow-violet:0 4px 20px rgba(107,33,232,0.25);
}

/* — FOREST ————— */

[data-theme="forest"]{

--bg:#020a06;--bg-1:#040e0a;--bg-2:#071410;--bg-3:#0c1e18;--bg-4:#122b22;--bg-5:#183a2e;

--accent:#00ffaa;--accent-b:#00c882;--accent-light:#80ffd0;--accent-glow:rgba(0,255,170,0.18);--
accent-dim:rgba(0,255,170,0.08);

--violet:#f0b429;--violet-b:#d48f00;--violet-dim:rgba(240,180,41,0.12);

--emerald:#4aeabc;--emerald-dim:rgba(74,234,188,0.12);--amber:#ffd700;--amber-
dim:rgba(255,215,0,0.12);--rose:#ff6b6b;--rose-dim:rgba(255,107,107,0.12);

--text-bright:#e8fff4;--text-mid:#5eaf88;--text-dim:#2d6648;--text-faint:#0f2d1e;

--border:rgba(0,255,170,0.09);--border-hover:rgba(0,255,170,0.32);--border-
active:rgba(0,255,170,0.62);--border-glass:rgba(255,255,255,0.05);

--card-bg:rgba(7,20,16,0.88);--raised:rgba(12,30,24,0.92);--glass:rgba(255,255,255,0.03);--glass-
2:rgba(255,255,255,0.06);--sidebar-bg:rgba(4,14,10,0.95);

--shadow-card:0 0 0 1px var(--border-glass),0 8px 32px rgba(0,0,0,0.65);--shadow-float:0 16px 64px
rgba(0,0,0,0.72),0 0 0 1px rgba(0,255,170,0.10);

--shadow-glow:0 0 40px rgba(0,255,170,0.22);--shadow-btn:0 4px 24px rgba(0,255,170,0.40);--
shadow-violet:0 4px 24px rgba(240,180,41,0.35);
}

/* — EMBER ————— */

[data-theme="ember"]{

```

```

--bg:#080400;--bg-1:#100700;--bg-2:#160a00;--bg-3:#1e1000;--bg-4:#281600;--bg-5:#341d00;

--accent:#ff9500;--accent-b:#e07800;--accent-light:#ffd080;--accent-glow:rgba(255,149,0,0.20);--
accent-dim:rgba(255,149,0,0.08);

--violet:#ff6b35;--violet-b:#e0501a;--violet-dim:rgba(255,107,53,0.12);

--emerald:#22c55e;--emerald-dim:rgba(34,197,94,0.12);--amber:#ffd700;--amber-
dim:rgba(255,215,0,0.12);--rose:#ff3b3b;--rose-dim:rgba(255,59,59,0.12);

--text-bright:#fff8f0;--text-mid:#b88060;--text-dim:#7a4a20;--text-faint:#3d1f00;

--border:rgba(255,149,0,0.10);--border-hover:rgba(255,149,0,0.35);--border-
active:rgba(255,149,0,0.65);--border-glass:rgba(255,255,255,0.05);

--card-bg:rgba(22,10,0,0.88);--raised:rgba(30,16,0,0.92);--glass:rgba(255,255,255,0.03);--glass-
2:rgba(255,255,255,0.06);--sidebar-bg:rgba(16,7,0,0.95);

--shadow-card:0 0 0 1px var(--border-glass),0 8px 32px rgba(0,0,0,0.65);--shadow-float:0 16px 64px
rgba(0,0,0,0.72),0 0 0 1px rgba(255,149,0,0.10);

--shadow-glow:0 0 40px rgba(255,149,0,0.25);--shadow-btn:0 4px 24px rgba(255,149,0,0.42);--
shadow-violet:0 4px 24px rgba(255,107,53,0.35);
}

```

```

/* ===== BASE

```

```

html{scroll-behavior:smooth;}

```

```

body{font-family:'Plus Jakarta Sans','Segoe UI',sans-serif;background:var(--bg);color:var(--text-
bright);line-height:1.6;overflow-x:hidden;-webkit-font-smoothing:antialiased;-moz-osx-font-
smoothing:grayscale;transition:background var(--t-slow) var(--ease-out),color var(--t-mid);}

```

```

body::after{content:"";position:fixed;inset:0;background-image:url("data:image/svg+xml,%3Csvg
viewBox='0 0 256 256' xmlns='http://www.w3.org/2000/svg'%3E%3Cfilter
id='n'%3E%3CfeTurbulence type='fractalNoise' baseFrequency='0.85' numOctaves='4'
stitchTiles='stitch'%3E%3C/filter%3E%3Crect width='100%25' height='100%25' filter='url(%23n)'
opacity='0.04'%3E%3C/svg%3E");pointer-events:none;z-index:9999;opacity:0.25;}

```

```

::-webkit-scrollbar{width:5px;height:5px};::-webkit-scrollbar-track{background:transparent};::-
webkit-scrollbar-thumb{background:var(--bg-5);border-radius:4px;}

```

```

.font-display{font-family:'Space Grotesk',sans-serif;}.mono{font-family:'JetBrains Mono',monospace;}

```

```

/* ===== ANIMATIONS =====

```

```

*/

```

```

@keyframes

```

```

fadeUp{from{opacity:0;transform:translateY(20px)}to{opacity:1;transform:translateY(0)}}

```

```

@keyframes fadeIn{from{opacity:0}to{opacity:1}}

```

```

@keyframes scaleIn{from{opacity:0;transform:scale(0.95)}to{opacity:1;transform:scale(1)}}

@keyframes shimmer{0%{background-position:-200% center}100%{background-position:200% center}}

@keyframes float{0%,100%{transform:translateY(0px)}50%{transform:translateY(-8px)}}

@keyframes blobFloat1{0%,100%{transform:translate(0,0) scale(1)}33%{transform:translate(30px,-20px) scale(1.05)}66%{transform:translate(-15px,25px) scale(0.97)}}

@keyframes blobFloat2{0%,100%{transform:translate(0,0) scale(1)}40%{transform:translate(-25px,15px) scale(1.04)}70%{transform:translate(20px,-18px) scale(0.98)}}

@keyframes dropIn{from{opacity:0;transform:translateY(-8px) scale(0.97)}to{opacity:1;transform:translateY(0) scale(1)}}

@keyframes shake{0%,100%{transform:translateX(0)}20%{transform:translateX(-6px)}40%{transform:translateX(6px)}60%{transform:translateX(-4px)}80%{transform:translateX(4px)}}

@keyframes cardReveal{from{opacity:0;transform:translateY(24px) scale(0.97)}to{opacity:1;transform:translateY(0) scale(1)}}

@keyframes
gridReveal{from{opacity:0;transform:translateY(16px)}to{opacity:1;transform:translateY(0)}}

@keyframes pulse-glow{0%,100%{box-shadow:0 0 20px var(--accent-glow)}50%{box-shadow:0 0 40px var(--accent-glow),0 0 60px var(--accent-glow)}}

@keyframes spin{to{transform:rotate(360deg)}}

@keyframes scanline{0%{transform:translateY(-100%)}100%{transform:translateY(100%)}}

/* ===== THEME SWITCHER ===== */

.theme-switcher{position:relative;display:inline-flex;}

.theme-toggle-btn{display:flex;align-items:center;gap:0.45rem;padding:0.45rem 0.875rem;background:var(--glass-2);border:1px solid var(--border-glass);border-radius:var(--r-md);color:var(--text-mid);font-family:'Space Grotesk',sans-serif;font-size:0.78rem;font-weight:600;letter-spacing:0.04em;cursor:pointer;backdrop-filter:blur(12px);transition:all var(--t-mid) var(--ease-out);}

.theme-toggle-btn:hover{border-color:var(--border-hover);color:var(--text-bright);}

.theme-toggle-btn.open .chevron{transform:rotate(180deg);}

.theme-toggle-btn .chevron{transition:transform var(--t-mid) var(--ease-out);display:flex;}

.theme-panel{position:absolute;top:calc(100% + 8px);right:0;background:var(--card-bg);backdrop-filter:blur(24px);border:1px solid var(--border-glass);border-radius:var(--r-lg);padding:0.5rem;min-width:200px;z-index:300;box-shadow:0 20px 60px rgba(0,0,0,0.6);animation:dropIn 0.2s var(--ease-out) both;}

```

```

.theme-panel-label{font-size:0.63rem;font-weight:700;letter-spacing:0.12em;text-
transform:uppercase;color:var(--text-dim);padding:0.3rem 0.75rem 0.6rem;}

.theme-option{display:flex;align-items:center;gap:0.75rem;padding:0.6rem 0.75rem;border-
radius:var(--r-sm);cursor:pointer;color:var(--text-mid);font-size:0.84rem;font-
weight:500;transition:all var(--t-fast);}

.theme-option:hover{background:var(--glass-2);color:var(--text-bright);}

.theme-option.active{background:var(--accent-dim);color:var(--accent-light);}

.theme-swatch{width:22px;height:22px;border-radius:50%;flex-shrink:0;border:2px solid
rgba(255,255,255,0.15);}

.theme-option .checkmark{margin-left:auto;color:var(--accent);display:none;}

.theme-option.active .checkmark{display:flex;}

/* ===== BADGES =====
*/

.badge{display:inline-flex;align-items:center;gap:0.25rem;padding:0.2rem 0.625rem;border-
radius:6px;font-family:'JetBrains Mono',monospace;font-size:0.73rem;font-weight:600;}

.badge-blood{background:var(--rose-dim);border:1px solid rgba(255,77,109,0.25);color:#ff8fa0;font-
weight:700;}

.badge-chronic{background:var(--violet-dim);border:1px solid
rgba(155,125,255,0.25);color:#b8a8ff;font-weight:700;}

.badge-accent{background:var(--accent-dim);border:1px solid rgba(0,212,255,0.20);color:var(--
accent-light);}

.sev-badge-mild{background:var(--emerald-dim);color:var(--emerald);border:1px solid
rgba(0,229,160,0.2);}

.sev-badge-moderate{background:var(--amber-dim);color:var(--amber);border:1px solid
rgba(255,189,46,0.2);}

.sev-badge-severe{background:var(--rose-dim);color:var(--rose);border:1px solid
rgba(255,77,109,0.2);}

.status-badge{display:inline-flex;align-items:center;gap:0.35rem;padding:0.28rem 0.7rem;border-
radius:6px;font-size:0.77rem;font-weight:600;white-space:nowrap;}

.status-badge svg{width:12px;height:12px;}

.status-badge.pending{background:var(--amber-dim);color:var(--amber);border:1px solid
rgba(255,189,46,0.22);}

.status-badge.dispensed{background:var(--emerald-dim);color:var(--emerald);border:1px solid
rgba(0,229,160,0.22);}

```



```
/* ===== BUTTONS =====  
*/  
  
.action-btn{padding:0.68rem 1.5rem;background:linear-gradient(135deg,var(--accent) 0%,var(--  
accent-b) 100%);border:none;border-radius:var(--r-md);color:#fff;font-family:'Space Grotesk',sans-  
serif;font-weight:700;font-size:0.84rem;letter-spacing:0.03em;cursor:pointer;display:inline-flex;align-  
items:center;gap:0.5rem;box-shadow:var(--shadow-  
btn);position:relative;overflow:hidden;transition:transform var(--t-mid) var(--ease-out),box-shadow  
var(--t-mid),filter var(--t-mid);}  
  
.action-btn::before{content:"";position:absolute;inset:0;background:linear-  
gradient(180deg,rgba(255,255,255,0.13) 0%,transparent 60%);pointer-events:none;}  
  
.action-btn:hover:not(:disabled){transform:translateY(-2px);box-shadow:0 8px 28px var(--accent-  
glow),var(--shadow-btn);filter:brightness(1.08);}  
  
.action-btn:disabled{opacity:0.45;cursor:not-allowed;}  
  
.action-btn.secondary{background:var(--glass-2);border:1px solid var(--border-glass);color:var(--text-  
bright);box-shadow:none;}  
  
.action-btn.secondary::before{display:none;}  
  
.action-btn.secondary:hover{background:var(--glass);border-color:var(--border-hover);box-shadow:0  
0 0 3px var(--accent-glow);}  
  
.submit-btn{width:100%;padding:0.875rem 1.5rem;background:linear-gradient(135deg,var(--accent)  
0%,var(--accent-b) 100%);border:none;border-radius:var(--r-md);color:#fff;font-family:'Space  
Grotesk',sans-serif;font-weight:700;font-size:0.88rem;letter-spacing:0.04em;cursor:pointer;box-  
shadow:var(--shadow-btn);position:relative;overflow:hidden;display:flex;align-items:center;justify-  
content:center;gap:0.5rem;margin-top:0.5rem;transition:transform var(--t-mid) var(--ease-out),box-  
shadow var(--t-mid),filter var(--t-mid);}  
  
.submit-btn::before{content:"";position:absolute;inset:0;background:linear-  
gradient(180deg,rgba(255,255,255,0.11) 0%,transparent 60%);}  
  
.submit-btn:hover:not(:disabled){transform:translateY(-2px);box-shadow:0 8px 28px var(--accent-  
glow);filter:brightness(1.08);}  
  
.submit-btn:disabled{opacity:0.45;cursor:not-allowed;}  
  
.icon-btn{width:34px;height:34px;display:grid;place-items:center;border:none;border-radius:var(--r-  
sm);cursor:pointer;transition:all var(--t-fast);}  
  
.icon-btn svg{width:15px;height:15px;}  
  
.icon-btn.edit{background:var(--accent-dim);color:var(--accent);}  
  
.icon-btn.delete{background:var(--rose-dim);color:var(--rose);}  
  
.icon-btn.edit:hover{background:rgba(0,212,255,0.18);transform:scale(1.1);}  
  
.icon-btn.delete:hover{background:rgba(255,77,109,0.18);transform:scale(1.1);}
```

```
.pab{padding:0.5rem 1rem;border-radius:var(--r-sm);font-family:'Space Grotesk',sans-serif;font-size:0.77rem;font-weight:700;letter-spacing:0.03em;cursor:pointer;display:flex;align-items:center;gap:0.4rem;transition:all var(--t-mid) var(--ease-out);border:none;}

.pab-summary{background:var(--accent-dim);border:1px solid rgba(0,212,255,0.22);color:var(--accent);}

.pab-summary:hover{background:rgba(0,212,255,0.16);transform:translateY(-1px);}

.pab-progress{background:var(--violet-dim);border:1px solid rgba(155,125,255,0.22);color:var(--violet);}

.pab-progress:hover{background:rgba(155,125,255,0.18);transform:translateY(-1px);}
```

```
/* ===== FORMS
```

```
===== */
```

```
.form-group{display:flex;flex-direction:column;gap:0.4rem;}

.form-group label{font-size:0.72rem;font-weight:700;letter-spacing:0.09em;text-transform:uppercase;color:var(--text-mid);}

.form-group input,.form-group select,.form-group textarea{padding:0.8rem 1rem;background:var(--glass);border:1px solid var(--border-glass);border-radius:var(--r-md);color:var(--text-bright);font-family:'Plus Jakarta Sans',sans-serif;font-size:0.91rem;backdrop-filter:blur(8px);transition:border-color var(--t-mid),box-shadow var(--t-mid),background var(--t-mid);}

.form-group select option{background:var(--bg-2);color:var(--text-bright);}

.form-group input:focus,.form-group select:focus,.form-group textarea:focus{outline:none;border-color:var(--accent);background:var(--accent-dim);box-shadow:0 0 3px var(--accent-glow);}

.form-group input::placeholder,.form-group textarea::placeholder{color:var(--text-dim);}

.form-group small{font-size:0.75rem;color:var(--text-dim);font-style:italic;}

.form-row{display:grid;grid-template-columns:repeat(auto-fit,minmax(220px,1fr));gap:1.25rem;}

.form-error{padding:0.8rem 1rem;background:var(--rose-dim);border:1px solid rgba(255,77,109,0.28);border-radius:var(--r-md);color:#ff8fa0;font-size:0.88rem;display:flex;align-items:center;gap:0.5rem;animation:shake 0.4s var(--ease-out);}

.data-form{background:var(--card-bg);backdrop-filter:blur(20px);border:1px solid var(--border-glass);border-radius:var(--r-xl);padding:2rem 2.25rem;box-shadow:var(--shadow-card);display:flex;flex-direction:column;gap:1.25rem;}

.data-form h3{font-family:'Space Grotesk',sans-serif;font-size:1.05rem;font-weight:700;color:var(--text-bright);padding-bottom:1rem;border-bottom:1px solid var(--border-glass);}

.form-container{max-width:820px;animation:gridReveal 0.4s var(--ease-out) both;}

.range-slider{width:100%;appearance:none;height:5px;border-radius:3px;background:var(--border-glass);outline:none;cursor:pointer;}
```

```
.range-slider::-webkit-slider-thumb{appearance:none;width:20px;height:20px;border-radius:50%;background:var(--accent);cursor:pointer;box-shadow:0 0 3px var(--accent-glow);transition:transform var(--t-fast);}
```

```
.range-slider::-webkit-slider-thumb:hover{transform:scale(1.2);}
```

```
.range-labels{display:flex;justify-content:space-between;font-size:0.68rem;color:var(--text-dim);margin-top:0.3rem;}
```

```
.checkbox-group{display:flex;align-items:center;}
```

```
.checkbox-group label{display:flex;align-items:center;gap:0.75rem;cursor:pointer;text-transform:none;letter-spacing:0;font-size:0.9rem;color:var(--text-mid);}
```

```
.checkbox-group input[type="checkbox"]{width:18px;height:18px;accent-color:var(--accent);cursor:pointer;}
```

```
/* ===== MODALS
```

```
===== */
```

```
.modal-overlay{position:fixed;inset:0;background:rgba(0,0,0,0.80);backdrop-filter:blur(8px);display:flex;align-items:center;justify-content:center;z-index:1000;padding:1rem;animation:fadeIn 0.2s var(--ease-out);}
```

```
.modal-box{background:var(--card-bg);backdrop-filter:blur(32px) saturate(1.8);border:1px solid var(--border-glass);border-radius:var(--r-xl);padding:1.75rem;width:100%;max-width:560px;max-height:92vh;overflow-y:auto;box-shadow:var(--shadow-float);animation:scaleIn 0.3s var(--ease-out) both;}
```

```
.modal-box.wide{max-width:860px;}.modal-box.xlwide{max-width:1020px;}
```

```
.modal-header{display:flex;align-items:center;justify-content:space-between;margin-bottom:1.25rem;}
```

```
.modal-title{font-family:'Space Grotesk',sans-serif;font-size:1.1rem;font-weight:700;color:var(--text-bright);}
```

```
.modal-close{background:none;border:none;color:var(--text-dim);cursor:pointer;display:flex;padding:0.25rem;border-radius:var(--r-sm);transition:color var(--t-fast),background var(--t-fast);}
```

```
.modal-close:hover{color:var(--text-bright);background:var(--glass-2);}
```

```
/* ===== DASHBOARD SHELL ===== */
```

```
.dashboard{display:flex;height:100vh;overflow:hidden;background:var(--bg);}
```

```
/* SIDEBAR */
```

```

.sidebar{width:268px;background:var(--sidebar-bg);border-right:1px solid var(--border-glass);display:flex;flex-direction:column;flex-shrink:0;position:relative;z-index:10;backdrop-filter:blur(20px);transition:width var(--t-slow) var(--ease-out);}

.sidebar::after{content:"";position:absolute;top:0;right:-1px;bottom:0;width:1px;background:linear-gradient(180deg,transparent 0%,var(--accent) 40%,var(--violet) 70%,transparent 100%);opacity:0.2;}

.sidebar-header{padding:1.5rem 1.25rem;display:flex;align-items:center;gap:0.875rem;border-bottom:1px solid var(--border-glass);}

.sidebar-logo{width:36px;height:36px;color:var(--accent);background:var(--accent-dim);padding:7px;border-radius:10px;flex-shrink:0;}

.sidebar-header h2{font-family:'Space Grotesk',sans-serif;font-size:1.2rem;font-weight:700;background:linear-gradient(135deg,var(--accent-light),var(--violet));-webkit-background-clip:text;-webkit-text-fill-color:transparent;background-clip:text;}

.sidebar-subtitle{font-size:0.68rem;color:var(--text-dim);text-transform:uppercase;letter-spacing:0.1em;font-weight:600;}

.sidebar-nav{flex:1;padding:1rem 0.75rem;display:flex;flex-direction:column;gap:0.2rem;overflow-y:auto;scrollbar-width:none;}

.sidebar-nav::-webkit-scrollbar{display:none;}

.sidebar-nav button{display:flex;align-items:center;gap:0.75rem;padding:0.7rem 0.9rem;background:transparent;border:none;border-radius:var(--r-md);color:var(--text-mid);font-family:'Plus Jakarta Sans',sans-serif;font-size:0.875rem;font-weight:500;cursor:pointer;width:100%;position:relative;text-align:left;transition:all var(--t-mid) var(--ease-out);}

.sidebar-nav button svg{width:17px;height:17px;flex-shrink:0;}

.sidebar-nav button:hover{background:var(--glass-2);color:var(--accent);transform:translateX(3px);}

.sidebar-nav button.active{background:var(--accent-dim);color:var(--accent-light);box-shadow:inset 0 0 1px var(--border-hover);}

.sidebar-nav
button.active::before{content:"";position:absolute;left:0;top:20%;bottom:20%;width:3px;background:linear-gradient(180deg,var(--accent),var(--violet));border-radius:0 3px 3px 0;}

.sidebar-footer{padding:0.875rem 0.75rem;border-top:1px solid var(--border-glass);}

.user-info{display:flex;align-items:center;gap:0.6rem;padding:0.65rem 0.9rem;background:var(--accent-dim);border:1px solid var(--border-glass);border-radius:var(--r-md);margin-bottom:0.65rem;font-size:0.85rem;color:var(--accent);font-weight:600;}

.user-info svg{width:15px;height:15px;}

.logout-btn{width:100%;display:flex;align-items:center;justify-content:center;gap:0.6rem;padding:0.7rem;background:var(--rose-dim);border:1px solid rgba(255,77,109,0.18);border-radius:var(--r-md);color:var(--rose);font-family:'Plus Jakarta Sans',sans-serif;font-size:0.85rem;font-weight:600;cursor:pointer;transition:all var(--t-mid);}

```

```

.logout-btn svg{width:15px;height:15px;}

.logout-btn:hover{background:rgba(255,77,109,0.16);border-
color:rgba(255,77,109,0.40);transform:translateY(-1px);}

/* MAIN */

.main-content{flex:1;display:flex;flex-direction:column;overflow:hidden;background:var(--bg);}

.content-header{padding:1.5rem 2.25rem;background:var(--card-bg);backdrop-
filter:blur(16px);border-bottom:1px solid var(--border-glass);display:flex;align-items:center;justify-
content:space-between;gap:1rem;position:sticky;top:0;z-index:5;}

.content-header h1{font-family:'Space Grotesk',sans-serif;font-size:1.55rem;font-weight:700;letter-
spacing:-0.03em;color:var(--text-bright);margin-bottom:0.12rem;}

.subtitle{color:var(--text-dim);font-size:0.82rem;}

.header-right{display:flex;align-items:center;gap:0.75rem;}

.user-badge{display:flex;align-items:center;gap:0.6rem;padding:0.55rem 1rem;background:var(--
accent-dim);border:1px solid var(--border-glass);border-radius:var(--r-md);flex-shrink:0;white-
space:nowrap;}

.user-badge svg{width:15px;height:15px;color:var(--accent);}

.user-badge span{font-family:'Space Grotesk',sans-serif;font-size:0.78rem;font-weight:700;letter-
spacing:0.06em;text-transform:uppercase;color:var(--accent);}

.content-body{flex:1;padding:2rem 2.25rem;overflow-y:auto;scrollbar-width:thin;scrollbar-color:var(--
bg-5) transparent;}

/* ===== STAT CARDS ===== */

.dashboard-grid{display:grid;grid-template-columns:repeat(auto-
fit,minmax(240px,1fr));gap:1.25rem;animation:gridReveal 0.5s var(--ease-out) both;}

.stat-card{background:var(--card-bg);backdrop-filter:blur(20px) saturate(1.6);border:1px solid var(--
border-glass);border-radius:var(--r-lg);padding:1.5rem 1.75rem;display:flex;align-
items:center;gap:1.25rem;position:relative;overflow:hidden;transition:transform var(--t-mid) var(--
ease-out),border-color var(--t-mid),box-shadow var(--t-mid);}

.stat-card::before{content:"";position:absolute;top:0;left:0;right:0;height:1.5px;background:var(--bar-
color,linear-gradient(90deg,var(--accent),var(--violet)));opacity:0.9;}

.stat-card::after{content:"";position:absolute;top:-40px;right:-
40px;width:130px;height:130px;background:var(--orb-color,var(--accent-glow));border-
radius:50%;filter:blur(35px);opacity:0;transition:opacity var(--t-slow);}

.stat-card:hover{transform:translateY(-4px);border-color:var(--border-hover);box-shadow:var(--
shadow-card),0 12px 40px rgba(0,0,0,0.4);}

```

```

.stat-card:hover::after{opacity:1;}

.stat-primary{--bar-color:linear-gradient(90deg,var(--accent),var(--accent-light));--orb-color:var(--accent-glow);}

.stat-success{--bar-color:linear-gradient(90deg,var(--emerald),#80ffd0);--orb-color:var(--emerald-dim);}

.stat-warning{--bar-color:linear-gradient(90deg,var(--amber),#ffe080);--orb-color:var(--amber-dim);}

.stat-danger {--bar-color:linear-gradient(90deg,var(--rose),#ff9aa8);--orb-color:var(--rose-dim);}

.stat-icon{width:52px;height:52px;border-radius:var(--r-md);display:grid;place-items:center;flex-shrink:0;}

.stat-primary .stat-icon{background:var(--accent-dim);color:var(--accent);}

.stat-success .stat-icon{background:var(--emerald-dim);color:var(--emerald);}

.stat-warning .stat-icon{background:var(--amber-dim);color:var(--amber);}

.stat-danger .stat-icon{background:var(--rose-dim);color:var(--rose);}

.stat-icon svg{width:24px;height:24px;}

.stat-details h3{font-family:'Space Grotesk',sans-serif;font-size:2rem;font-weight:700;letter-spacing:-0.04em;line-height:1;margin-bottom:0.28rem;}

.stat-details p{color:var(--text-mid);font-size:0.84rem;font-weight:500;}

.welcome-card{grid-column:1/-1;background:linear-gradient(135deg,rgba(0,212,255,0.06) 0%,var(--card-bg) 40%,rgba(155,125,255,0.05) 100%);backdrop-filter:blur(20px);border:1px solid var(--border-glass);border-radius:var(--r-xl);padding:2.25rem;position:relative;overflow:hidden;}

.welcome-card::before{content:'';position:absolute;top:0;left:0;right:0;height:1px;background:linear-gradient(90deg,transparent,var(--accent),var(--violet),transparent);opacity:0.5;}

.welcome-card h2{font-family:'Space Grotesk',sans-serif;font-size:1.55rem;font-weight:700;letter-spacing:-0.02em;margin-bottom:0.5rem;}

.welcome-card>p{color:var(--text-mid);font-size:0.92rem;margin-bottom:1.75rem;max-width:500px;}

.quick-actions{display:flex;gap:0.875rem;flex-wrap:wrap;}

/* ===== INFO CARDS =====
*/

.info-item{display:flex;align-items:center;gap:0.875rem;padding:0.875rem 1.25rem;background:var(--glass);border:1px solid var(--border-glass);border-radius:var(--r-md);backdrop-filter:blur(12px);transition:border-color var(--t-mid);}

.info-item:hover{border-color:var(--border-hover);}

.info-item svg{color:var(--accent);flex-shrink:0;}

```

```
.info-item strong{font-family:'Space Grotesk',sans-serif;font-size:1.4rem;font-weight:700;color:var(--accent-light);line-height:1;}
```

```
.info-item span{font-size:0.76rem;color:var(--text-dim);margin-top:0.15rem;}
```

```
.info-card{background:var(--glass);backdrop-filter:blur(14px);border:1px solid var(--border-glass);border-radius:var(--r-md);padding:1rem 1.125rem;}
```

```
.info-card-title{font-size:0.65rem;font-weight:700;letter-spacing:0.12em;text-transform:uppercase;color:var(--text-dim);margin-bottom:0.75rem;}
```

```
.kv-pair{display:flex;flex-direction:column;gap:0.12rem;margin-bottom:0.5rem;}
```

```
.kv-pair:last-child{margin-bottom:0;}
```

```
.kv-label{font-size:0.65rem;color:var(--text-dim);text-transform:uppercase;letter-spacing:0.08em;}
```

```
.kv-value{font-size:0.875rem;color:var(--text-bright);font-weight:500;}
```

```
.info-grid-2{display:grid;grid-template-columns:1fr 1fr;gap:0.875rem;margin-bottom:1.25rem;}
```

```
.info-grid-3{display:grid;grid-template-columns:1fr 1fr 1fr;gap:0.875rem;margin-bottom:1.25rem;}
```

```
/* ===== DOCTOR SPLIT PANEL ===== */
```

```
.patient-list-pane{width:280px;flex-shrink:0;border-right:1px solid var(--border-glass);overflow-y:auto;padding:0.875rem;background:var(--sidebar-bg);backdrop-filter:blur(20px);}
```

```
.patient-list-search{width:100%;padding:0.6rem 0.875rem;background:var(--glass-2);border:1px solid var(--border-glass);border-radius:var(--r-sm);color:var(--text-bright);font-family:'Plus Jakarta Sans',sans-serif;font-size:0.84rem;margin-bottom:0.875rem;backdrop-filter:blur(8px);}
```

```
.patient-list-search:focus{outline:none;border-color:var(--accent);}
```

```
.patient-list-search::placeholder{color:var(--text-dim);}
```

```
.patient-tile{padding:0.825rem 0.95rem;border-radius:var(--r-md);border:1px solid var(--border-glass);cursor:pointer;transition:all var(--t-mid) var(--ease-out);margin-bottom:0.4rem;background:var(--glass);position:relative;overflow:hidden;}
```

```
.patient-tile:hover{border-color:var(--border-hover);background:var(--glass-2);}
```

```
.patient-tile.active{border-color:var(--border-hover);background:var(--accent-dim);box-shadow:0 0 0 1px var(--border-hover);}
```

```
.patient-tile.selected{border-color:var(--amber);background:var(--amber-dim);box-shadow:0 0 0 1px var(--amber);}
```

```
.patient-tile::before{content:'';position:absolute;left:0;top:0;bottom:0;width:3px;background:linear-gradient(180deg,var(--accent),var(--violet));border-radius:0 3px 3px 0;opacity:0;transition:opacity var(--t-mid);}
```

```
.patient-tile.active::before{opacity:1;}
```

```
.patient-tile.selected::before{background:linear-gradient(180deg,var(--amber),var(--rose));opacity:1;}
```

```

.pt-name{font-weight:600;font-size:0.86rem;color:var(--text-bright);}

.pt-meta{font-size:0.74rem;color:var(--text-dim);margin-top:0.18rem;}

.pt-badges{display:flex;gap:0.3rem;flex-wrap:wrap;margin-top:0.4rem;}

.patient-detail-pane{flex:1;overflow-y:auto;padding:1.5rem 2rem;}

.patient-detail-header{display:flex;align-items:flex-start;justify-content:space-between;margin-bottom:1.5rem;gap:1rem;flex-wrap:wrap;}

.patient-detail-identity{display:flex;align-items:center;gap:1rem;}

.patient-avatar{width:54px;height:54px;border-radius:var(--r-lg);background:linear-gradient(135deg,var(--accent-dim),var(--violet-dim));border:1px solid rgba(0,212,255,0.2);display:flex;align-items:center;justify-content:center;font-family:'Space Grotesk',sans-serif;font-weight:700;font-size:1.1rem;color:var(--accent);flex-shrink:0;}

.patient-detail-name{font-family:'Space Grotesk',sans-serif;font-size:1.2rem;font-weight:700;color:var(--text-bright);}

.patient-detail-meta{font-size:0.8rem;color:var(--text-dim);margin-top:0.22rem;}

.patient-action-btns{display:flex;gap:0.5rem;flex-wrap:wrap;}

/* — Freq bars — */

.freq-bars{display:flex;align-items:flex-end;gap:5px;height:54px;margin-top:0.25rem;}

.freq-bar-group{display:flex;flex-direction:column;align-items:center;gap:3px;flex:1;}

.freq-bar{width:100%;border-radius:3px 3px 0 0;min-height:5px;background:linear-gradient(180deg,var(--accent),var(--violet));transition:height 0.6s var(--ease-out);}

.freq-label{font-size:7px;color:var(--text-dim);font-family:'JetBrains Mono',monospace;}

.section-block{margin-bottom:1.25rem;}

.section-header{display:flex;align-items:center;justify-content:space-between;margin-bottom:0.75rem;}

.section-title{font-size:0.64rem;font-weight:700;letter-spacing:0.12em;text-transform:uppercase;color:var(--text-dim);display:flex;align-items:center;gap:0.5rem;}

.section-title::before,.section-title::after{content:"";height:1px;background:var(--border-glass);flex:1;}

/* — Visit accordion — */

.visit-accordion{border:1px solid var(--border-glass);border-radius:var(--r-md);overflow:hidden;margin-bottom:0.5rem;}

.visit-header{padding:0.875rem 1.125rem;display:flex;align-items:center;justify-content:space-between;cursor:pointer;background:var(--glass);transition:background var(--t-fast);gap:0.75rem;}

```



```

.visit-header:hover{background:var(--glass-2);}
.visit-header.open{background:var(--accent-dim);}
.visit-header-left{display:flex;align-items:center;gap:0.75rem;flex:1;}
.visit-status-dot{width:8px;height:8px;border-radius:50%;flex-shrink:0;}
.visit-reason{font-weight:600;font-size:0.875rem;color:var(--text-bright);}
.visit-meta{font-size:0.74rem;color:var(--text-dim);margin-top:0.15rem;}
.visit-chevron{color:var(--text-dim);display:flex;transition:transform var(--t-mid) var(--ease-out);}
.visit-chevron.open{transform:rotate(180deg);}
.visit-body{padding:0.875rem 1.125rem;border-top:1px solid var(--border-glass);display:grid;grid-template-columns:repeat(4,1fr) 2fr;gap:0.875rem;background:var(--card-bg);}
.vital-item{display:flex;flex-direction:column;gap:0.18rem;}
.vital-label{font-size:0.63rem;color:var(--text-dim);text-transform:uppercase;letter-spacing:0.08em;}
.vital-value{font-size:0.88rem;font-weight:600;color:var(--text-bright);font-family:'JetBrains Mono',monospace;}
.visit-notes{font-size:0.82rem;color:var(--text-mid);line-height:1.6;grid-column:5;}
.diagnosis-row{display:flex;align-items:flex-start;justify-content:space-between;padding:0.875rem 1rem;background:var(--glass);border:1px solid var(--border-glass);border-radius:var(--r-md);margin-bottom:0.4rem;gap:0.875rem;transition:border-color var(--t-mid);}
.diagnosis-row:hover{border-color:var(--border-hover);}
.diagnosis-name{font-weight:600;font-size:0.875rem;color:var(--text-bright);}
.diagnosis-desc{font-size:0.78rem;color:var(--text-mid);margin-top:0.18rem;}
.diagnosis-date{font-size:0.7rem;color:var(--text-dim);margin-top:0.28rem;}
.diagnosis-badges{display:flex;flex-direction:column;gap:0.28rem;align-items:flex-end;flex-shrink:0;}

```

/* ===== TABLES

===== */

```

.records-container,.patients-container,.prescriptions-container,.files-container,.import-container{animation:gridReveal 0.4s var(--ease-out) both;}
.table-container{background:var(--card-bg);backdrop-filter:blur(20px);border:1px solid var(--border-glass);border-radius:var(--r-xl);overflow:hidden;box-shadow:var(--shadow-card);}
.data-table{width:100%;border-collapse:collapse;}
.data-table thead{background:linear-gradient(90deg,rgba(0,212,255,0.05) 0%,rgba(155,125,255,0.03) 100%);border-bottom:1px solid var(--border-glass);}

```

```
.data-table th{padding:0.95rem 1.5rem;text-align:left;font-family:'Space Grotesk',sans-serif;font-size:0.7rem;font-weight:700;text-transform:uppercase;letter-spacing:0.1em;color:var(--accent-light);white-space:nowrap;}
```

```
.data-table td{padding:0.95rem 1.5rem;border-bottom:1px solid var(--border-glass);font-size:0.88rem;vertical-align:middle;}
```

```
.data-table tbody tr{transition:background var(--t-fast);}
```

```
.data-table tbody tr:hover{background:var(--glass);}
```

```
.data-table tbody tr:last-child td{border-bottom:none;}
```

```
.patient-cell,.doctor-cell{display:flex;flex-direction:column;gap:0.18rem;}
```

```
.patient-cell strong,.doctor-cell strong{font-weight:600;color:var(--text-bright);}
```

```
.doctor-cell small{color:var(--text-mid);font-size:0.78rem;}
```

```
.contact-cell{display:flex;align-items:center;gap:0.5rem;}
```

```
.contact-cell svg{color:var(--accent);flex-shrink:0;width:13px;height:13px;}
```

```
/* ===== PROGRESS RINGS ===== */
```

```
.ring-item{display:flex;flex-direction:column;align-items:center;gap:0.5rem;}
```

```
.ring-wrapper{position:relative;display:flex;align-items:center;justify-content:center;}
```

```
.ring-center{position:absolute;inset:0;display:flex;flex-direction:column;align-items:center;justify-content:center;}
```

```
.ring-score{font-family:'Space Grotesk',sans-serif;font-weight:700;line-height:1;}
```

```
.ring-name{font-size:0.78rem;font-weight:600;color:var(--text-bright);text-align:center;max-width:90px;}
```

```
.ring-updated{font-size:0.67rem;color:var(--text-dim);text-align:center;}
```

```
.chronic-rings-container{display:flex;gap:1.5rem;flex-wrap:wrap;align-items:flex-start;}
```

```
.progress-form{display:flex;flex-direction:column;gap:1rem;}
```

```
.progress-score-display{text-align:center;padding:1rem;background:var(--glass);border-radius:var(--r-md);border:1px solid var(--border-glass);}
```

```
.progress-score-label{font-size:0.74rem;color:var(--text-dim);margin-top:0.3rem;}
```

```
.progress-history{display:flex;flex-direction:column;gap:0.5rem;}
```

```
.progress-entry{display:flex;align-items:center;gap:0.875rem;padding:0.75rem 1rem;background:var(--glass);border:1px solid var(--border-glass);border-radius:var(--r-md);transition:border-color var(--t-mid);}
```

```
.progress-entry:hover{border-color:var(--border-hover);}
```

```
.pe-score{font-family:'Space Grotesk',sans-serif;font-size:1.2rem;font-weight:700;min-width:48px;text-align:center;}

.pe-info{flex:1;min-width:0;}.pe-note{font-size:0.82rem;color:var(--text-mid);}

.pe-date{font-size:0.7rem;color:var(--text-dim);margin-top:0.2rem;}

.pe-vitals{font-size:0.7rem;color:var(--text-dim);font-family:'JetBrains Mono',monospace;}
```

```
/* ===== PRESCRIPTIONS
```

```
===== */
```

```
.prescriptions-header{margin-bottom:1.5rem;display:flex;align-items:center;justify-content:space-between;}

.filter-checkbox{display:flex;align-items:center;gap:0.65rem;cursor:pointer;font-size:0.9rem;font-weight:500;color:var(--text-mid);user-select:none;transition:color var(--t-fast);}

.filter-checkbox:hover{color:var(--accent-light);}

.filter-checkbox input[type="checkbox"]{width:17px;height:17px;accent-color:var(--accent);cursor:pointer;}

.dispense-btn{padding:0.42rem 0.875rem;background:linear-gradient(135deg,var(--emerald) 0%,#00b880 100%);border:none;border-radius:var(--r-sm);color:#fff;font-family:'Space Grotesk',sans-serif;font-size:0.76rem;font-weight:700;letter-spacing:0.04em;cursor:pointer;transition:transform var(--t-mid),box-shadow var(--t-mid);white-space:nowrap;}

.dispense-btn:hover{transform:translateY(-1px);box-shadow:0 4px 14px rgba(0,229,160,0.4);}
```

```
/* ===== FILES
```

```
===== */
```

```
.files-grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(280px,1fr));gap:1.25rem;animation:gridReveal 0.4s var(--ease-out) both;}

.file-card{background:var(--glass);backdrop-filter:blur(14px);border:1px solid var(--border-glass);border-radius:var(--r-lg);padding:1.5rem;display:flex;flex-direction:column;gap:0.875rem;position:relative;overflow:hidden;transition:transform var(--t-mid),border-color var(--t-mid),box-shadow var(--t-mid);}

.file-card::before{content:"";position:absolute;top:0;left:0;right:0;height:1px;background:linear-gradient(90deg,var(--accent),var(--violet));opacity:0;transition:opacity var(--t-mid);}

.file-card:hover{transform:translateY(-3px);border-color:var(--border-hover);box-shadow:var(--shadow-card);}

.file-card:hover::before{opacity:0.8;}

.file-icon{width:48px;height:48px;background:var(--accent-dim);border-radius:var(--r-md);display:grid;place-items:center;color:var(--accent);}

.file-icon svg{width:24px;height:24px;}
```

```

.file-details h4{font-family:'Space Grotesk',sans-serif;font-size:0.78rem;font-weight:700;color:var(--accent-light);text-transform:uppercase;letter-spacing:0.06em;margin-bottom:0.28rem;}

.file-name{font-size:0.86rem;color:var(--text-bright);word-break:break-all;}

.file-size,.file-date{font-size:0.78rem;color:var(--text-dim);font-family:'JetBrains Mono',monospace;}

.file-desc{font-size:0.82rem;color:var(--text-mid);padding-top:0.5rem;border-top:1px solid var(--border-glass);}

.download-btn{display:flex;align-items:center;justify-content:center;gap:0.5rem;padding:0.625rem;background:var(--accent-dim);border:1px solid rgba(0,212,255,0.18);border-radius:var(--r-md);color:var(--accent-light);font-family:'Space Grotesk',sans-serif;font-size:0.8rem;font-weight:700;cursor:pointer;transition:all var(--t-mid);}

.download-btn:hover{background:var(--accent);border-color:var(--accent);color:#fff;transform:translateY(-1px);box-shadow:0 4px 14px var(--accent-glow);}

.file-input-wrapper input[type="file"]{width:100%;padding:1rem;background:var(--glass);border:2px dashed var(--border-glass);border-radius:var(--r-md);cursor:pointer;color:var(--text-mid);transition:border-color var(--t-mid),background var(--t-mid);}

.file-input-wrapper input[type="file"]:hover{border-color:var(--accent);background:var(--accent-dim);}

.file-selected{margin-top:0.5rem;font-size:0.84rem;color:var(--accent-light);font-family:'JetBrains Mono',monospace;}

```

```

/* ===== SUMMARY MODAL

```

```

===== */

```

```

.summary-section{margin-bottom:1.5rem;}

.summary-section-label{display:flex;align-items:center;font-size:0.63rem;font-weight:800;letter-spacing:0.12em;text-transform:uppercase;color:var(--text-dim);margin-bottom:0.75rem;gap:0.5rem;}

.summary-section-label::before,.summary-section-label::after{content:"";flex:1;height:1px;background:var(--border-glass);}

.summary-readonly-banner{display:flex;align-items:center;gap:0.5rem;background:var(--accent-dim);border:1px solid var(--border-glass);border-radius:var(--r-md);padding:0.6rem 1rem;margin-bottom:1.25rem;font-size:0.8rem;color:var(--accent-light);}

.demographics-grid{display:grid;grid-template-columns:repeat(3,1fr);gap:0.75rem;}

.demo-cell{background:var(--glass);border:1px solid var(--border-glass);border-radius:var(--r-md);padding:0.625rem 0.875rem;}

.demo-cell-label{font-size:0.63rem;font-weight:700;letter-spacing:0.09em;text-transform:uppercase;color:var(--text-dim);margin-bottom:0.18rem;}

.demo-cell-value{font-size:0.875rem;color:var(--text-bright);font-weight:600;}

```

```
.summary-notes-area{width:100%;box-sizing:border-box;background:var(--glass);border:1px solid var(--border-glass);border-radius:var(--r-md);color:var(--text-bright);font-family:'Plus Jakarta Sans',sans-serif;font-size:0.86rem;padding:0.75rem 1rem;outline:none;resize:vertical;transition:border-color var(--t-mid);}
```

```
.summary-notes-area:focus{border-color:var(--accent);}
```

```
/* ===== NOTIFICATION BELL ===== */
```

```
.bell-btn{position:relative;background:var(--glass-2);border:1px solid var(--border-glass);border-radius:var(--r-md);width:38px;height:38px;display:flex;align-items:center;justify-content:center;color:var(--text-mid);cursor:pointer;transition:all var(--t-mid);backdrop-filter:blur(12px);}
```

```
.bell-btn:hover{border-color:var(--border-hover);color:var(--accent);}
```

```
.bell-badge{position:absolute;top:-4px;right:-4px;background:var(--rose);color:#fff;border-radius:999px;font-size:0.58rem;font-weight:800;padding:0.1rem 0.35rem;min-width:16px;text-align:center;line-height:1.4;}
```

```
/* ===== IMPORT SECTION ===== */
```

```
.import-container{animation:gridReveal 0.4s var(--ease-out) both;}
```

```
.import-section{background:var(--card-bg);backdrop-filter:blur(20px);border:1px solid var(--border-glass);border-radius:var(--r-xl);padding:2rem 2.25rem;box-shadow:var(--shadow-card);}
```

```
.import-section h3{font-family:'Space Grotesk',sans-serif;font-size:1rem;font-weight:700;color:var(--text-bright);margin-bottom:1.25rem;display:flex;align-items:center;gap:0.5rem;}
```

```
.import-instructions{background:var(--accent-dim);border:1px solid var(--border-glass);border-radius:var(--r-md);padding:1.25rem 1.5rem;margin-bottom:1.75rem;}
```

```
.import-instructions h4{font-family:'Space Grotesk',sans-serif;font-size:0.78rem;font-weight:700;letter-spacing:0.08em;text-transform:uppercase;color:var(--accent-light);margin-bottom:0.3rem;}
```

```
.import-instructions p{color:var(--text-mid);font-size:0.86rem;}
```

```
.download-template-btn{display:inline-flex;align-items:center;gap:0.5rem;padding:0.6rem 1.25rem;background:var(--glass-2);border:1px solid var(--border-glass);border-radius:var(--r-md);color:var(--text-bright);text-decoration:none;font-family:'Space Grotesk',sans-serif;font-size:0.8rem;font-weight:700;margin-top:0.875rem;transition:all var(--t-mid);}
```

```
.download-template-btn:hover{background:var(--accent);border-color:var(--accent);color:#fff;transform:translateY(-1px);}
```

```
.result-stats{display:grid;grid-template-columns:repeat(3,1fr);gap:1rem;}
```

```

.result-stat{display:flex;flex-direction:column;align-items:center;padding:1rem;border-radius:var(--r-md);text-align:center;}

.result-stat.success{background:var(--emerald-dim);border:1px solid rgba(0,229,160,0.22);}

.result-stat.error{background:var(--rose-dim);border:1px solid rgba(255,77,109,0.22);}

.result-stat.total{background:var(--accent-glow);border:1px solid rgba(0,212,255,0.22);}

.result-stat strong{font-family:'Space Grotesk',sans-serif;font-size:1.9rem;font-weight:700;letter-spacing:-0.04em;line-height:1;margin-bottom:0.28rem;}

.result-stat.success strong{color:var(--emerald);}

.result-stat.error strong{color:var(--rose);}

.result-stat.total strong{color:var(--accent);}

.result-stat span{font-size:0.76rem;color:var(--text-mid);font-weight:500;}

.error-log{margin-top:1rem;padding:1rem;background:var(--rose-dim);border:1px solid rgba(255,77,109,0.15);border-radius:var(--r-md);max-height:240px;overflow-y:auto;scrollbar-width:thin;}

.error-log h5{font-family:'Space Grotesk',sans-serif;font-size:0.76rem;font-weight:700;letter-spacing:0.08em;text-transform:uppercase;color:var(--rose);margin-bottom:0.75rem;}

.error-log ul{list-style:none;}

.error-log li{padding:0.38rem 0.6rem;margin-bottom:0.32rem;background:var(--bg-2);border-radius:6px;font-family:'JetBrains Mono',monospace;font-size:0.76rem;color:var(--text-mid);}

.import-form{display:flex;flex-direction:column;gap:1.1rem;}

```

```

/* ===== EMPTY STATE =====
*/

```

```

.empty-state{display:flex;flex-direction:column;align-items:center;justify-content:center;padding:5rem 2rem;gap:1rem;text-align:center;}

.empty-state svg{color:var(--bg-5);opacity:0.5;margin-bottom:0.5rem;}

.empty-state h3{font-family:'Space Grotesk',sans-serif;font-size:1.15rem;font-weight:700;color:var(--text-bright);}

.empty-state p{color:var(--text-dim);font-size:0.88rem;max-width:320px;}

```

```

/* ===== LOGIN SCREEN =====
*/

```

```

.ls-panel{transition:transform 0.35s cubic-bezier(0.16,1,0.3,1),opacity 0.3s ease;will-change:transform,opacity;}

.ls-panel-in{transform:translateX(0);opacity:1;pointer-events:auto;}

```

```

.ls-panel-out-left{transform:translateX(-110%);opacity:0;pointer-events:none;}

.ls-panel-out-right{transform:translateX(110%);opacity:0;pointer-events:none;}

.role-tile{all:unset;position:relative;display:flex;flex-direction:column;align-items:center;justify-content:center;gap:0.7rem;padding:1.4rem 0.75rem;cursor:pointer;box-sizing:border-box;background:var(--glass);border:1px solid var(--border-glass);border-radius:18px;backdrop-filter:blur(14px);transition:all 0.25s var(--ease-out);}

.role-tile:hover{background:var(--glass-2);border-color:var(--border-hover);transform:translateY(-2px);}

.role-tile .top-bar{position:absolute;top:0;left:14%;right:14%;height:2px;border-radius:999px;opacity:0;transition:opacity 0.25s;}

.role-tile.selected .top-bar{opacity:1;}

.role-tile .check-badge{position:absolute;top:9px;right:9px;width:20px;height:20px;border-radius:50%;display:flex;align-items:center;justify-content:center;font-size:10px;color:#fff;opacity:0;transition:opacity 0.2s;}

.role-tile.selected .check-badge{opacity:1;}

.role-icon-wrap{width:50px;height:50px;border-radius:13px;display:flex;align-items:center;justify-content:center;border:1px solid;transition:all 0.25s;}

.role-name{font-family:'Space Grotesk',sans-serif;font-size:0.88rem;font-weight:700;transition:color 0.25s;}

.role-label{font-size:0.58rem;font-weight:700;letter-spacing:0.08em;text-transform:uppercase;color:var(--text-mid);}

.role-desc{font-size:0.67rem;text-align:center;line-height:1.4;color:var(--text-mid);transition:color 0.25s;}

.role-grid{display:grid;grid-template-columns:repeat(5,1fr);gap:0.75rem;width:100%;}

.login-container{min-height:100vh;display:grid;place-items:center;padding:2rem;position:relative;overflow:hidden;background:var(--bg);}

.login-bg{position:absolute;inset:0;}

.login-card{background:var(--card-bg);backdrop-filter:blur(24px) saturate(1.5);border:1px solid var(--border-glass);border-radius:var(--r-xl);padding:3rem;width:100%;max-width:460px;box-shadow:var(--shadow-float);position:relative;z-index:1;animation:cardReveal 0.7s var(--ease-out) both;}

.login-header{text-align:center;margin-bottom:2.5rem;}

.logo-container{width:72px;height:72px;margin:0 auto 1.5rem;background:linear-gradient(135deg,var(--accent),var(--violet));border-radius:var(--r-lg);display:grid;place-items:center;box-shadow:var(--shadow-glow);}

.logo-icon{width:34px;height:34px;color:#fff;}

.login-header h1{font-family:'Space Grotesk',sans-serif;font-size:1.9rem;font-weight:700;background:linear-gradient(135deg,var(--accent-light) 0%,var(--text-bright) 50%,var(--

```

```

violet) 100%);-webkit-background-clip:text;-webkit-text-fill-color:transparent;background-
clip:text;margin-bottom:0.4rem;}

.login-header p{color:var(--text-mid);font-size:0.88rem;}

.login-btn{width:100%;padding:0.875rem;background:linear-gradient(135deg,var(--accent),var(--
accent-b));border:none;border-radius:var(--r-md);color:#fff;font-family:'Space Grotesk',sans-
serif;font-weight:700;font-size:0.9rem;letter-spacing:0.04em;cursor:pointer;box-shadow:var(--
shadow-btn);transition:transform var(--t-mid),box-shadow var(--t-mid),filter var(--t-mid);}

.login-btn:hover:not(:disabled){transform:translateY(-2px);filter:brightness(1.08);}

.login-btn:disabled{opacity:0.5;cursor:not-allowed;}

.register-form{max-height:68vh;overflow-y:auto;padding-right:0.5rem;scrollbar-width:thin;scrollbar-
color:var(--accent) transparent;display:flex;flex-direction:column;gap:1.1rem;}

.register-link{text-align:center;margin-top:1.5rem;color:var(--text-mid);font-size:0.88rem;}

.link-btn{background:none;border:none;color:var(--accent);cursor:pointer;font-weight:600;font-
family:inherit;font-size:inherit;transition:color var(--t-fast);}

.link-btn:hover{color:var(--accent-light);text-decoration:underline;}

```

```

/* ===== RESPONSIVE =====
*/

```

```

@media(max-width:1280px){.role-grid{grid-template-columns:repeat(3,1fr);}.info-grid-3{grid-
template-columns:1fr 1fr;}.visit-body{grid-template-columns:repeat(2,1fr);}.visit-notes{grid-
column:1/-1;}}

```

```

@media(max-width:1024px){.sidebar{width:240px;}.content-body{padding:1.5rem;}}

```

```

@media(max-width:768px){.dashboard{flex-direction:column;}.sidebar{width:100%;flex-
direction:row;border-right:none;border-bottom:1px solid var(--border-glass);overflow-
x:auto;}.sidebar::after{display:none;}.sidebar-header{padding:1rem;border-bottom:none;border-
right:1px solid var(--border-glass);flex-shrink:0;}.sidebar-nav{flex-
direction:row;padding:0.75rem;gap:0.35rem;}.sidebar-nav button{padding:0.55rem 0.8rem;flex-
shrink:0;font-size:0.8rem;}.sidebar-nav button.active::before{display:none;}.sidebar-
footer{display:none;}.content-header{padding:1.25rem 1.5rem;flex-direction:column;align-
items:flex-start;gap:0.75rem;}.content-header h1{font-size:1.25rem;}.content-
body{padding:1.25rem;}.dashboard-grid{grid-template-columns:1fr 1fr;}.info-grid-2{grid-template-
columns:1fr;}.demographics-grid{grid-template-columns:1fr 1fr;}.role-grid{grid-template-
columns:repeat(2,1fr);}.result-stats{grid-template-columns:1fr;}.data-table{font-size:0.8rem;}.data-
table th,.data-table td{padding:0.75rem 0.875rem;}}

```

```

@media(max-width:560px){.login-card{padding:2rem 1.5rem;}.form-row{grid-template-
columns:1fr;}.quick-actions{flex-direction:column;}.action-btn{width:100%;text-align:center;justify-
content:center;}.dashboard-grid{grid-template-columns:1fr;}.demographics-grid{grid-template-
columns:1fr;}.visit-body{grid-template-columns:1fr 1fr;}}

```